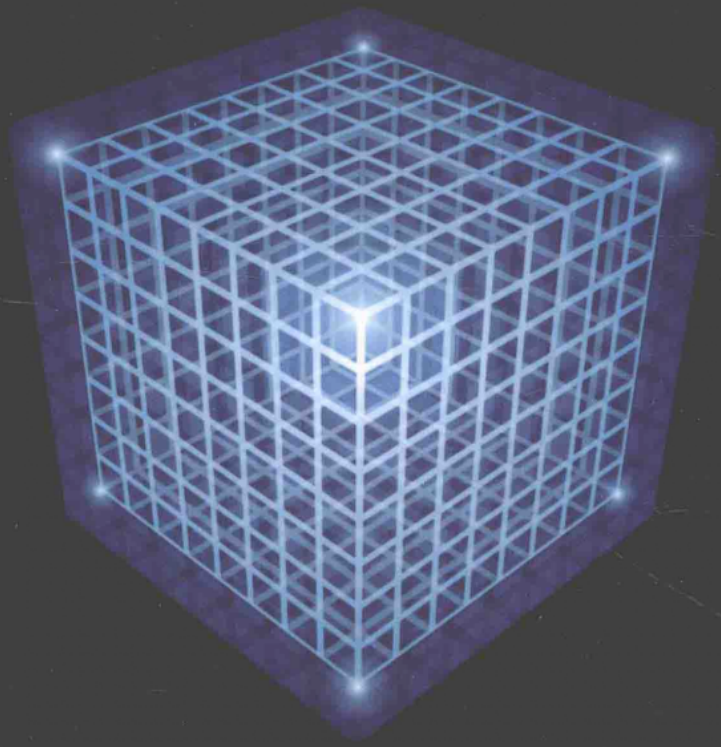
HZ BOOKS
华章IT

Pearson

高性能计算技术丛书



CUDA for Engineers

An Introduction to High-Performance Parallel Computing

CUDA

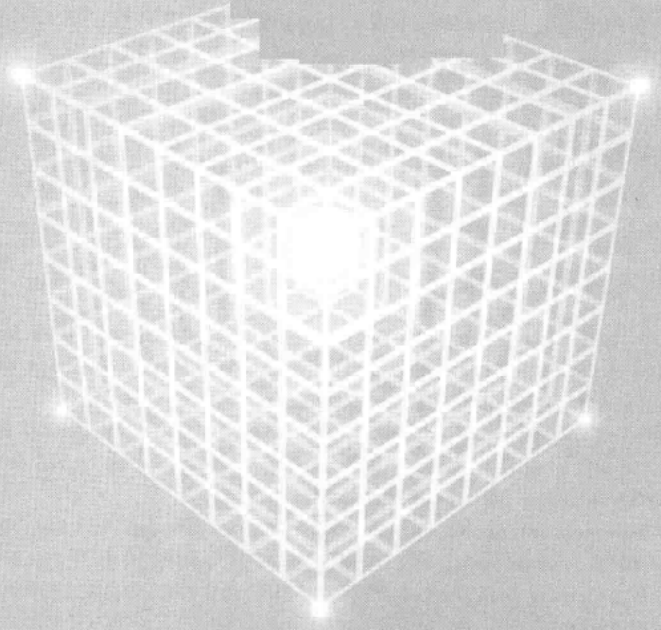
高性能并行计算

[美] 杜安·斯托尔蒂（Duane Storti）
[土耳其] 梅特·尤尔托卢（Mete Yurtoglu） 著

苏统华 项文成 李松泽 姚宇鹏 孙博文 译

机械工业出版社
China Machine Press

高性能计算技术丛书



CUDA for Engineers

An Introduction to High-Performance Parallel Computing

CUDA

高性能并行计算

[美] 杜安·斯托尔蒂（Duane Storti）
[土耳其] 梅特·尤尔托卢（Mete Yurtoglu） 著

苏统华 项文成 李松泽 姚宇鹏 孙博文 译

机械工业出版社
China Machine Press

图书在版编目（CIP）数据

CUDA高性能并行计算 /（美）杜安·斯托尔蒂（Duane Storti），（土）梅特·尤尔托卢（Mete Yurtoglu）著；苏统华等译．—北京：机械工业出版社，2016.12
（高性能计算技术丛书）
书名原文：CUDA for Engineers: An Introduction to High-Performance Parallel Computing

ISBN 978-7-111-55749-4

I. C… II. ①杜… ②梅… ③苏… III. 计算机图形学 IV. TP391.411

中国版本图书馆CIP数据核字（2017）第005436号

本书版权登记号：图字：01-2016-0766

Authorized translation from the English language edition, entitled CUDA for Engineers: An Introduction to High-Performance Parallel Computing, 9780134177410 by Duane Storti, Mete Yurtoglu, published by Pearson Education, Inc., Copyright © 2016.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by anymeans, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

Chinese simplified language edition published by Pearson Education Asia Ltd., and China Machine Press Copyright © 2017.

本书中文简体字版由Pearson Education（培生教育出版集团）授权机械工业出版社在中华人民共和国境内（不包括香港、澳门特别行政区及台湾地区）独家出版发行。未经出版者书面许可，不得以任何方式抄袭、复制或节录本书中的任何部分。

本书封底贴有Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

CUDA高性能并行计算

出版发行：机械工业出版社（北京市西城区百万庄大街22号　邮政编码：100037）
责任编辑：陈佳媛　　责任校对：董纪丽
印　　刷：三河市宏图印务有限公司　　版　　次：2017年2月第1版第1次印刷
开　　本：186mm×240mm　1/16　　印　　张：16.5
书　　号：ISBN 978-7-111-55749-4　　定　　价：79.00元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：（010）88379426　88361066　　投稿热线：（010）88379604
购书热线：（010）68326294　88379649　68995259　　读者信箱：hzit@hzbook.com

版权所有·侵权必究
封底无防伪标均为盗版
本书法律顾问：北京大成律师事务所　韩光/邹晓东

本书赞誉

FORTRAN（盛行于 20 世纪 60 年代）是最早允许我们在大型机上编程的语言。之后是 BASIC（流行于 20 世纪 80 年代）赋予我们为第一批微型计算机编写程序的能力。现在轮到 CUDA，让我们可以为超级微型计算机编写程序。

本书介绍的技术能够帮助工程和数学等领域的研究者以超越微机 100 倍的速度执行计算任务。这使新的计算任务得以完成，也使本书得以成为颠覆传统规则的教程。

——Richard H. Rand，康奈尔大学机械与航空航天工程系教授、数学系教授

本书结构合理，内容实用，能够帮助读者快速体验 CUDA 并行编程并即时得到结果。本书围绕不同科学和工程问题，展示了 GPU 编程的魅力。书中提供了优秀的示例程序和项目练习，让人读之愉悦。

——Mark Staveley 博士，微软 Azure 高性能计算高级项目经理

本书名副其实，手把手教授读者基本概念、核心策略、关键术语和典型示例。这些内容有机构成的教学体系以老道而深入的方式介绍高性能计算。本书同时适用于专家和普通读者。

——Joseph M. Iaquinto 博士，VA Puget Sound 公司研究专家

本书体现的实用性与我为工程师讲授数值方法课程的教学方法具有惊人的一致性。本书将为工程专业的学生以及程序员补充新的数值计算工具箱，使他们能够基于 CUDA 进行高性能科学计算。对于有一定编程基础的 CUDA 初学者，本书堪称完美。建议读者遵从作者的建议，尽早好好练习实践项目。践行本书的理论，你将可以熟练应对 GPU 计算方面的项目，进入 CUDA 开发者行列。

——Lorena A. Barba，乔治华盛顿大学机械与航空航天工程系副教授

致 谢 *Acknowledgements*

我们需要向很多人致谢，没有他们也就没有这本书。

感谢我们的家人，在写作本书的过程中，我们对他们时有忽略。谢谢 Laura Lewin 和 Pearson 公司的所有人，他们为本书的编辑、出版和营销做了大量努力。多谢《CUDA 专家手册：GPU 编程权威指南》[⊖]一书的作者 Nicholas Wilt 先生（曾任职于英伟达公司，现任职于亚马逊公司），是他建议我们与 Laura 联系，最终促成了本书的出版。同样需要感谢英伟达公司的 Thomas Bradley、微软公司的 Mark Staveley 和康奈尔大学的 Richard Rand，作为技术评审专家，他们提供了颇具建设性的评论、校正和见解。

谢谢华盛顿大学西雅图校区机械工程系的同事们。他们与我们上聊科技大势下聊底层技术细节，为本书贡献了素材。这些同事包括却不仅限于 Mark Ganter、Di Zhang、Ben Weiss 等人。他们帮本书绘制了多幅图片并提供了多个好用的软件帮我们制作标签和自动排版代码。我们也要感谢机械工程系系主任 Per Reinhall，是他批准了我们的课程申请，正是这门课促使我们编写了本书的很多内容。另外要感谢同校放射科的 David Haynor 和西雅图 VA 医院的 William Ledoux，他们的研究项目持续围绕着 CUDA 相关的技术探索。

特别感谢英伟达公司的员工们。包括首席执行官黄仁勋，他一直坚信并热情坚持基于 GPU 的并行计算；学术项目经理 Chandra Cheij；高等教育与医疗行业主任 Kimberly Powell；给我们帮助和启迪的 CUDA 大师 Jon Saposhnik 和 Bob Crovella。最后同样的感谢送给战略营销主任 Jay White，他长期支持西雅图地区的 GPU 计算交流会并是我们聚会的召集人。

⊖ 此书已由机械工业出版社出版，书号为 978-7-111-47265-0。

我们还要感谢所有的学生。他们敢于参加华盛顿大学早期的CUDA课程，颇有冒险精神。特别是已毕业的硕士生Grant Marchelli（现在他已拿到了博士学位并受聘在Envitrum公司任职CTO），他参与了课程的很多关键环节，包括建立实验环境、提供示例代码并受邀讲课。特别感谢Gerald Barnett，当本书进入第一稿编辑环节时，他立即放下自己的事情，全情参与。

最后，真心感谢本书的读者。分享对象的价值取决于有没有合适的人去共享。谢谢你们对本书感兴趣。我们真诚地希望本书能为你们带来有用的经验和丰厚的回报。

目 录 *Contents*

本书赞誉
致谢

第 0 章 本书导读 ····· 1

0.1 什么是 CUDA ····· 1
0.2 学习 CUDA 的“须知” ····· 2
0.3 本书的读者对象 ····· 3
0.4 学习 CUDA 的必备 ····· 3
0.5 本书的组织结构 ····· 4
0.6 本书体例 ····· 5
0.7 本书代码 ····· 6
0.8 用户指南 ····· 7
0.9 历史沿革 ····· 8
参考文献 ····· 9

第 1 章 起步 ····· 10

1.1 运行 CUDA 样例程序 ····· 10
1.1.1 在 Windows 下运行 CUDA 样例程序 ····· 11
1.1.2 在 Linux 下运行 CUDA 样例程序 ····· 13
1.1.3 估计“加速效果” ····· 13
1.2 运行我们自己的串行程序 ····· 14
1.2.1 dist_v1 应用 ····· 15
1.2.2 dist_v2 应用 ····· 16
1.3 本章小结 ····· 18
1.4 推荐项目 ····· 18

第 2 章 CUDA 基础知识 ····· 19

2.1 CUDA 并行模式 ····· 19
2.2 需要知道的 CUDA API 和 C 语言拓展 ····· 21
2.3 本章小结 ····· 23
2.4 推荐项目 ····· 23
参考文献 ····· 24

第 3 章 从循环到网格 ····· 25

3.1 并行化 dist_v1 ····· 25
3.2 并行化 dist_v2 ····· 29
3.3 标准操作流程 ····· 33
3.4 简化操作流程 ····· 33
3.4.1 统一内存和托管数组 ····· 34

3.4.2 使用 cudaMallocManaged() 实现的距离应用 ……34
3.5 本章小结……36
3.6 推荐项目……37
参考文献……38

第 4 章 二维网格与交互式图形……39

4.1 启动二维计算网格……40
4.1.1 二维内核启动的语法……41
4.1.2 定义二维内核……41
4.1.3 dist_2d……43
4.2 通过图形交互实时显示……45
4.3 stability 应用程序……54
4.4 本章小结……62
4.5 推荐项目……62
参考文献……62

第 5 章 模板与共享内存……64

5.1 线程间依赖……64
5.2 一维网格上的导数计算……66
5.2.1 实现 dd_ld_global……66
5.2.2 实现 dd_ld_shared……69
5.2.3 解决二维拉普拉斯方程：heat_2d……72
5.2.4 图像边缘锐化：sharpen……83
5.3 本章小结……96
5.4 推荐项目……97
参考文献……98

第 6 章 归约与原子操作……99

6.1 全局交互的线程……99
6.2 实现 parallel_dot……100
6.3 计算整体属性：centroid_2d……106
6.4 本章小结……113
6.5 推荐项目……113
参考文献……114

第 7 章 三维数据交互……115

7.1 计算三维网格数据：dist_3d……117
7.2 查看三维数据并与之交互：vis_3d……119
7.2.1 切片法……121
7.2.2 体绘制法……124
7.2.3 光线投射法……124
7.2.4 创建 vis_3d 应用……126
7.3 本章小结……139
7.4 推荐项目……139
参考文献……140

第 8 章 CUDA 函数库实践……141

8.1 自定义的与现有的……141
8.2 Thrust 库……143
8.2.1 使用 inner_product() 计算向量的模……144
8.2.2 使用 transform() 计算距离……147
8.2.3 使用 generate()、transform() 以及 reduce() 对 π 的值进行估计……151
8.3 cuRAND 库……156
8.4 NPP 库……158
8.4.1 sharpen_npp……159

8.4.2 更多使用 NPP 进行的图像处理操作 ……163
8.5 线性代数中的 cuSOLVER 和 cuBLAS 实践 ……166
8.6 cuDNN 库 ……170
8.7 ArrayFire 库 ……171
8.8 本章小结 ……171
8.9 推荐项目 ……171
参考文献 ……172

第 9 章 探索 CUDA 生态系统 …… 174

9.1 主要资源的权威列表 ……174
9.1.1 CUDA 空间 ……174
9.1.2 其他的主要网络资源 ……175
9.1.3 在线课程 ……176
9.1.4 CUDA 书籍 ……176
9.2 更多资源 ……179
9.2.1 CUDA 样例 ……179
9.2.2 CUDA 语言和库 ……179
9.2.3 更多的 CUDA 书籍 ……179
9.3 本章小结 ……180
9.4 推荐项目 ……180

附录 A 硬件设置 …… 182

附录 B 软件设置 …… 189

附录 C C 语言编程须知 …… 201

附录 D CUDA 实践技巧：计时、性能分析、错误处理与调试 …… 229

第 0 章 Chapter 0

本书导读

欢迎阅读本书。本书的目标是让你亲身参与个人高性能计算（Personal High-Performance Computing，PHPC）。如果你跟随我们的 CUDA 世界之旅，只需一台基本的游戏级计算机，你就可以执行大规模并行计算并从中获益。这种规模的计算任务在几年之前是需要超级计算机才能完成的。本书包含的内容能够帮助你去阅读更高深的 CUDA 著作并开发自己的 CUDA 项目。

首先，介绍一下 CUDA 以及我们的教学策略。

0.1 什么是 CUDA

CUDA 是英伟达公司为促进高性能并行计算的普及所创建的支持并行计算的软硬件平台。CUDA 的硬件方面涉及显卡上配备的一个或多个兼容 CUDA 的图形处理器（Graphics Processing Units，GPU）。英伟达 CUDA 工具箱软件则提供了基于 C/C++ 编程语言的开发环境 [1]。

CUDA 使用的基于 GPU 方式的大规模并行计算也是很多最快和最节能超级计算机所采用的关键技术。核心的度量准则已经从每秒浮点计算次数（FLOPS）转变为每瓦电能的每秒浮点计算次数（FLOPS/watt，即计算的总量除以消耗的能量）。而 GPU 并行方式在每瓦电能的每秒浮点计算次数上具有优势。实际上，在 2012 年 6 月到

2013 年 12 月期间，全世界最节能的 10 大超级计算机从刚开始的完全基于 IBM 的绿色基因系统（配有 PowerPC CPU）转变成基于英伟达公司的 GPU 系统[2]。在这个快速转换为 GPU 计算的过程中，计算能力与电能消耗的比率已经翻了两番，并在持续增长。

0.2 学习 CUDA 的“须知”

基于 GPU 的并行计算是真正改变行业面貌的技术。你需要知道基于 GPU 的并行计算以保持不被如下工程领域抛下：应用计算、工程设计和分析、计算机仿真、机器学习、视觉和成像系统或任何其他一些计算密集型领域。基于 GPU 的并行计算对一些计算任务可以减少数个数量级的时间消耗，所以那些本来需要持续运行若干星期才能完成的大型计算任务（如在一个大的数据集训练机器学习系统），现在可以在数小时执行结束。对于中等规模的计算任务（像产生三维轮廓图），本来要等待几分钟的，现在却能进行实时的交互了。而这些收益只需你付出可以接受的成本，不论是在精力付出上，还是在硬件投入上。你需要知道 CUDA，因为它是目前在榨取 GPU 并行计算能力上支持力度最好的，也是最方便使用的平台。

为了让你与 CUDA 第一次亲密接触，我们也将尽最大努力提供应该让你知道的一切（尽可能删去你不需要知道的内容！）。这本书并不打算成为一个百科全书式的指南，这类的优秀 CUDA 书籍已经上市。我们将提供这类参考资源的链接。我们希望你在学习高级 CUDA 实用知识时可以阅读它们。对于这类指南，它们的最大不足在于假定读者已经具有并行计算和 CUDA 的背景知识，这也是它们的专业行话和语境可以成立的基础。

我们的目标是以清晰、简明的方式介绍 CUDA 最为核心的内容。在此过程中，不要求任何专业背景作为先决条件也不要迷失于烦琐的细节之中。我们意在提供直接通往更有意义的动手实践的路径。你可以很快进入 CUDA 世界，根本无需阅读过分冗长的背景材料。如果你需要搭建一个支持 CUDA 的基本系统或者补充 C 语言编程知识，你都可以在本书的简明附录中找到相关的指导。在本书的第 1 章中，你就可以亲手运行标准示例程序，亲身体验 CUDA。到了本书的第 3 章，你应该就可以运行完全由你自己编写的 CUDA 小程序了。随后的章节讲解多个完整应用实例（你可以生成、运行、修改它们），同时也推荐了一些 CUDA 项目作为练手用的作业。请准备好迎接

一段快节奏的令人兴奋的CUDA旅程！请专注于那些你需要知道的可以帮助你借助CUDA实现加速的内容！

0.3 本书的读者对象

我们的目标读者是懂技术的工程师。如果你是一个在职工程师或是学习了一年工程课程的大学生，那么本书就是为你准备的。本书提供的实例期望以清晰明了的方式帮助工程师利用CUDA驱动的计算，这些实例包括：

- 可视化二维（2D）和三维（3D）数据；
- 求解微分方程并不断改变初始值或边界条件；
- 显示/处理图像或图像栈；
- 计算内积和质心；
- 求解线性代数方程组；
- 蒙特卡罗计算。

我们假设你仅有少量开展计算的经验。学习过一门使用C或C++进行计算的入门级课程就足够了。如果你之前使用的是其他编程语言，那么附录C可以为你快速上手提供帮助。当你的应用程序涉及创建数组和循环结构，你就可以欣赏到基于CPU系统的串行计算典范与基于GPU和CUDA方式的并行计算范式的鲜明对比。

至于所需要的数学基础，接触过微分方程、有限差分逼近和线性代数将有助于理解某几个实例。即便没有这方面的储备也没有大碍，我们同时会提供必备的基础知识。当一个例子涉及专业背景时，会在讲述例子之前提供相关数学和工程概念的扼要说明，确保你甚至可以欣赏和领会在你专业之外的示例应用。

与背景知识同样不做要求的还有一些。你不需要和我们一起完全进入CUDA；你不需要成为一个计算机科学家或经验丰富的专业程序员；你不需要任何特定技术领域的背景知识；你也不需要具备高端的或者稀奇的计算系统。

0.4 学习CUDA的必备

你需要一台支持CUDA的计算机。这台计算机不需要特别花哨，相当于一台网吧里玩游戏的计算机的配置即可。你还需要某些很容易获得的免费软件。如果你所在的机

构已经为你准备好了使用 CUDA 的全部计算资源，那你就太幸运了，可以马上开工。但是，我们一定要考虑那些将使用自己的个人计算机作为 CUDA 计算平台的读者。附录 A 和附录 B 将手把手教你配置自己的系统。我们的目标读者所使用的操作系统可能横跨多种类型，因此针对 Windows 和类 UNIX 系统（包括 Linux 和 OS X）上的程序生成与运行会分别提供指导。

0.5 本书的组织结构

除了本章之外，本书包含有九章正文和四个附录。正文章节提供了大多数读者需要掌握的核心知识，而附录用于查缺补漏，补充背景知识。我们的呈现方式鼓励你积极参与到 CUDA 实践之中。为了充分利用本书，请在你阅读这些章节时，亲自创建、测试和修改书中的应用程序（也称为 app）。

第 1 章启动了新奇的 CUDA 世界之旅，引导你检查你的 CUDA 系统。你将运行 CUDA 工具箱附带的 CUDA 示例应用程序，以确保你有一个支持 CUDA 的系统；你也将运行几个 C 语言编写的简单程序，以确保你可以随时创建、编译和执行 C 程序。第一批 C 程序计算一组输入值与参考点的距离：先考虑每次输入一个数，而后扩展为每次输入一个数组。这些应用程序是后面采用 CUDA 并行化的待对比代码。

如果你在尝试 CUDA 的简单应用时碰到问题，请转入附录补充必要的基础知识。附录 A 包括如何检查你的计算机是否配有支持 CUDA 的 GPU 以及在不支持的情况下如何购买并安装一个。附录 B 展示了如何安装 CUDA 软件。随后的附录 C 涵盖了 C 语言编程的核心要素。

在你完成第 1 章并根据需要读完了附录部分，你就可以进入第 2 章。该章讲解用于并行化的基本 CUDA 模型和为支持 CUDA 编程所扩展的 C 语言（即 CUDA C）。

该书的其余部分围绕一系列实例，讲解并实现了 CUDA 的重要知识点。

第 3 章展示了如何并行化第 1 章介绍的距离函数。这个过程揭示了使用 CUDA 进行并行计算的最基本要素，也给出了在 CPU 和 GPU 之间传输数据的典型模式。我们还简要介绍了统一内存，它可以让你的开发过程变得更简单。到了该章的最后，你将基于我们介绍的实例应用程序获得 CUDA 实际操作经验，进而用来创建你自己的应用程序。该章首次引用附录 D，讨论了 CUDA 开发工具。

第 4 章对前一章距离计算函数实例做了升级，扩展到更高维度。一旦我们创建了二

维数组的数据（它可以作为一个图像进行处理），我们将借此机会来介绍有关 OpenGL 互操作性的技术，使显示的图形可以与键盘 / 鼠标进行实时交互。我们也将利用微分方程介绍仿真的基础知识，为多种不同的初始条件同时启动仿真代码，达到并行稳定性分析之目的。

在第 5 章中，我们开始正视现实中的挑战，考虑处于不同线程的计算并非相互独立的问题。最简单同时也是最常见的情况之一，每个线程与计算网格中邻近的其他线程相互依赖，这就是所说的模板模式（stencil pattern）。我们在直观的图像滤波语境下介绍基本知识，然后引进一个不同的模板来求解稳态温度分布问题（steady-state temperature distribution）。该章的讨论带领我们学习了如何使用分块（tiling）和共享内存，这些技巧适用于各式各样的应用场景。

第 6 章用于解决计算过程中所有线程存在依赖的挑战性场景。我们先从计算两个向量的点积这一简单例子出发，识别出其中的挑战（以及一些解决方案），然后继续解决一个与质心有关的有趣应用。

第 7 章探讨需要使用三维网格的并行计算场景。这一扩展直接建立在之前一维和二维网格的基础之上。然后，我们探索切片法、体绘制法和光线投射法，采用二维网格达到交互式显示三维网格数据的目的。

第 8 章简要介绍 CUDA 函数库。这样你将了解存在哪些可复用的代码，并知道什么时候以及以什么方式利用它们。我们使用 CUDA 库不仅重新实现前面章节里的一些应用，也创建了两个新的应用程序。

当你抵临本书的结尾，仍然有大量需要学习的东西和需要探索的材料。第 9 章为你提供额外的 CUDA 资源，包括书籍、博客、网站、视频和示例，它们能够进一步加强你的 CUDA 经验。

综上所述，图 0.1 给出了学习本书的推荐流程。

0.6 本书体例

本书使用以下约定：

- 为了跟正常文字区分，代码清单使用等宽（`monospace`）字体排版。
- 我们经常把类 UNIX 系统，如 Linux 和 OS X，统一称为 Linux。
- 我们把完整的示例程序称为应用程序（简写为 app）。

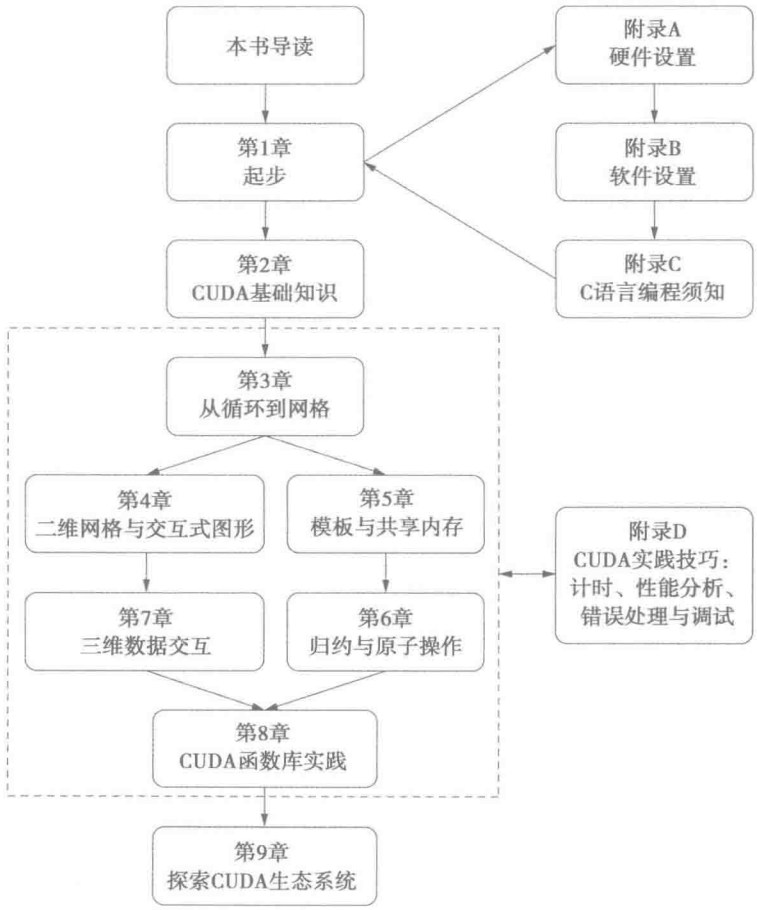


图 0.1 章节间的依赖关系

- 我们使用箭头（⇒）来表示嵌套的菜单选择。
- 遵从常规用法，“一维”、“二维”和“三维”分别缩写为 1D、2D 和 3D。
- 一些图形为了增强灰度对比强度做了修改。
- 每章以建议项目清单结束，为读者提供附加实践 CUDA 的机会。
- 方括号里放一个数字，并采用普通文本格式表达，代表引用的所在章节或附录末尾的一个参考文献编号。

0.7 本书代码

本书应用程序的代码可以通过 www.cudaforengineers.com 获取。虽然书中采用了

一些代码片断和“骨架”代码（skeleton code），但标记为“代码清单”的代码（包含行号）是真实可运行代码的一部分。我们努力确保代码经过测试，可以成功运行于多个系统，包括 Windows 7、Windows 8.1、Linux 操作系统（Ubuntu 14.04 LTS）和 OS X 10.10（除了第 3 章最后有一个确实会发生异常的例子）。

本书介绍的代码在创建阶段非常在意简洁性、可读性、可解释性和易懂性。这一指导思想允许我们在很多时候并不需要考虑很多商业代码需要的功能，如错误检测和性能评价等。有关这些功能的主题统一放到了附录 D 中，所以你可以根据需要添加它们。

CUDA 应用程序既可以用 C 语言也可以用 C++ 语言的编程风格进行开发。我们主要用 C 语言编程风格，以尽量减少对读者所需编程背景的要求。

0.8 用户指南

我们编写本书的目的就是让广大的技术型读者积极参与到使用 CUDA 进行 GPU 并行计算的洪流之中。作为比喻，我们邀请你进行一次基于 GPU 并行计算的旅程，而本书即是本旅程的导游手册。比喻为导游手册在许多方面都是恰当的，其中包括：

- 如果你只阅读本教程，却并没有动手实践，那么本书的预期使命还没有真正完成。请亲自加入实践的旅程！
- 多数导游手册帮助你规划出国旅行之前需要做的事情，比如办理接种疫苗和护照。而我们这里不涉及这些，而是分别在附录 A 和附录 B 中介绍所需的硬件配置和软件配置。
- 到国外旅行往往涉及简单学习一个新的语言，甚至是特殊的方言。对于此次旅行，语言是 C，而方言是 CUDA。如果你熟悉 C（或 C++），你可以应付自如。如果你之前没有 C 语言的基础，也不要恐惧，附录 C 为你准备了本次旅程所需的基本知识点。
- 导游手册一般提供一系列必去观赏的圣地和必去参加的活动。我们的 CUDA 世界之旅必须看的地方是本书的第 1 章和第 2 章，这些地方让你尝试一下 CUDA 示例代码的威力，并获得接触 CUDA 方言的初体验。必须做的活动是第 3 章，让你学习如何把串行代码转换成并行 CUDA 代码。
- 对于那些希望进入更广泛和更深入的 CUDA 之旅的读者，只需继续学习后续章节。

- 浓缩的和有针对性的旅行是可能的。出发点是第 3 章，在该章我们创建了第一个并行应用程序。如果你急于到达某个特定的目的地，请直接进入第 3 章。如果你没有碰到麻烦，可以从这一章开始往下推进。否则，还请返回到第 1 章和第 2 章（甚至附录），以填补必要的基础知识。从那里开始，你可以选择不同的行进方向：
  - 如果你的目标围绕交互式图形学、科学可视化、游戏等，请转到第 4 章和第 7 章继续学习。
  - 如果你的目标是科学计算、偏微分方程或是图像处理等，则请继续学习第 5 章和第 6 章。
  - 如果你的主要目标是利用现有 CUDA 函数库，那么只需直接跳到第 8 章进行阅读。
- 不论是长途还是短途，我们都喜欢看到新奇的事物，而实际尝试做新的东西会提升我们经历的长期影响。所以，不要只是单纯阅读。碰到代码，就去创建、编译并执行它们！还要修改它们并测试你的修改！建议你做一做每章末尾给出的推荐项目，或者创建你自己的项目。
- 在某些阶段的旅程，需要你独自去经历，还有一些旅程你是和一群游客及一名导游一起出行。我们尽最大努力来编写一本很适合你独自阅读的书。但是如果有条件在一个课程中使用本书，请多多与老师和同学进行讨论和交流。提出问题和回答问题是开展学习的好方法，另一种学习方法是使用最合适的调试工具。
- 任何旅途都可能出现差错，所以需要额外的指引或调度信息。解决这些问题的材料安排在附录 D 和第 9 章，它们提供了额外的资源链接，帮助你探索“ CUDA 生态系统”。

0.9 历史沿革

本书是作者在 CUDA 疆域探险数年的经验总结。最初开展的是容积医学影像（volumetric medical imaging）、2D 和 3D 成像（即透视和 CT 扫描）的配准以及计算机辅助设计（computer-aided design）和增材制造（additive manufacturing）的新方法研究。随着 CUDA 的效用和重要性变得愈加明显，分享对 CUDA 的欣赏和经验无疑是非常正确的。我们创建了一门专题性的 CUDA 应用课程，课程面向专业分布很广泛的学生，只需他们具备基本的计算经验即可。本书的写作是在三次课程讲授基础上进行的，课程历经完善，

已发展到包括系统配置、CUDA 样例、C 语言编程基础速成、CUDA 的并行化模型（同时包含了实现它们所需的超出 C 语言基础知识之外的东西），另外对一些实用的 CUDA 特性予以全面讲解，包括原子功能、共享内存和交互式图形学。所有这些内容都安排在为期 10 周一个学期的前半部分，其余时间专供项目实践和学生演示之用。每个学生在第 5 周进行项目选题的演讲，在最后的第 10 周，对所做项目进行展示。在两者之间的课上，安排为答疑时间和（教师或客座专家）分享创建和使用 CUDA 应用的经验。与本书的逻辑结构对应，课程的执行流程大致如下：

- 系统配置和初始 CUDA 样例运行：附录 A、附录 B 和第 1 章。
- C 语言基础：附录 C。
- CUDA 基础，包括并行模型和关键语言扩展：第 2 章和第 3 章。
- 交互式图形学：第 4 章。
- 共享内存：第 5 章的第一部分。
- 原子功能：第 6 章的第一部分。
- 项目：让学生自由探索剩下的第 5～9 章，寻找项目实践的灵感，鼓励他们积极编写代码，并尽可能多地帮助他们去克服其中碰到的障碍。

虽然本书的组织跟课程有一定的关系，但涉及的内容大多是无向导的 CUDA 世界，如此偌大的疆域，很难不迷路。我们搜集并查阅了很多材料，包括书籍、示例代码、研究类论文、研讨会讲座，以精选出学习 CUDA 必需的那些知识，能够让外行工程师也知道利用如 CUDA 这样的强大工具能做什么以及如何做到。我们真诚希望，本书可以同时支持个人学习和课堂配套教学；我们真诚地希望本书能帮助你避开拦路虎，让你的 CUDA 之旅更高效、更愉快、更有收获。

参考文献

[1] NVIDIA Corporation, "CUDA Toolkit," NVIDIA CUDA Zone, 2015, https://developer.nvidia.com/cuda-toolkit.

[2] CompuGreen, LLC, "The Green 500 List—June 2014," The Green 500, June 2014, Accessed July 1, 2015, http://www.green500.org/lists/green201406.

Chapter 1 第 1 章

起　步

本章带领读者进入CUDA并行计算的世界。我们将从运行一个CUDA工具箱（CUDA Toolkit）提供的样例程序开始。这个样例程序包含串行和并行两个版本，因此读者可以通过运行性能上的对比来建立CUDA是如何增强运算能力的第一印象。需要注意的是，搭建和执行应用需要依赖读者使用的操作系统，我们会对在Windows、Linux和OS X下运行样例程序提供详细的指导。在本章的结尾，我们将给出后文用以讲解CUDA并行化的两个简单C语言程序。

在本章中，如果出现了需要读者注意的事项或者相关知识，我们会用提示（checkpoint）来指引读者阅读附录的相关内容。当提示读者此处需要去查询附录内容时，请相信这样做会比勉强继续阅读而不去弄清相关的关键知识更加节省时间。

在这里我们要介绍第一个提示：读者需要在一个能够使用CUDA的系统上工作，其中包括一块支持CUDA的图形显卡并且安装了CUDA开发套件。如果有任何关于开发环境的问题请参见附录A和附录B来确认硬件状态和软件状态。

1.1　运行CUDA样例程序

让我们从运行第一个样例程序nbody开始，以此获得一个有意义的CUDA初体验。这个程序模拟了大量受到万有引力影响的粒子群的运动情况，并且进行了可视化。

1.1.1 在 Windows 下运行 CUDA 样例程序

如果你在一个支持 CUDA 的 Windows 系统下工作，你需要拥有一块支持 CUDA 的 GPU 和相关的驱动程序，CUDA 工具箱以及添加 NVIDIA's Nsight 插件的 Microsoft Visual Studio。本书中的代码和讨论都是基于 CUDA7.5、Nsight Visual Studio Edition 4.7 和 Visual Studio Community 2013 with 64-bit compilation（注意将解决方案设置为 x64 而不是 Win32）。

这里我们假设读者知道如何找到并且运行 CUDA 样例程序。这里是另一个提示：如果你不知道如何找到并且运行一个 CUDA 样例程序，请到附录 B 中找到相关的说明，然后再回来进行下一步学习。

在 CUDA Samples 文件夹中（参见图 1.1）你可以找到很多子文件夹，保存着多种类型的 CUDA 样例程序。另外还有图中高亮显示的 bin 文件夹。

图 1.1 CUDA 样例文件夹

找到并进入 bin/win64/Release 文件夹，这里存放着所有的可执行程序，参见图 1.2。提示：如果读者在 Release 文件夹中没有找到这些可执行程序，请到附录 B 中找到关于将 CUDA 样例代码编译成可执行程序的说明。

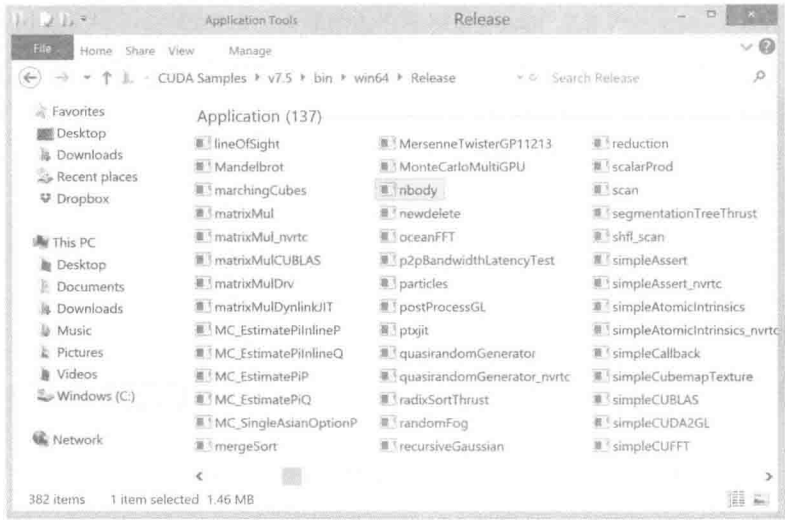


图 1.2 Release 文件夹及 CUDA 样例的可执行程序

在 Release 文件夹下，找到 nbody 样例程序的可执行程序（见图 1.2），双击打开这个有趣且有意义的样例程序。程序会打开两个窗口，一个是如图 1.3 所示的图形化窗口，另一个是控制台窗口（其中最后一行应显示了读者所使用的 GPU 型号及计算能力，如图 1.4 所示）。（读者也许需要移动一下图形化窗口来查看控制台窗口的内容。）nbody 模拟了上千个受到万有引力作用的粒子的运动并且对运动进行了可视化处理。图形化窗口的标题栏中包含了一些相关信息，其中包括模拟的粒子数目，每秒进行的相互作用次数，每秒进行的总浮点运算次数（GFLOPS）及其精度。

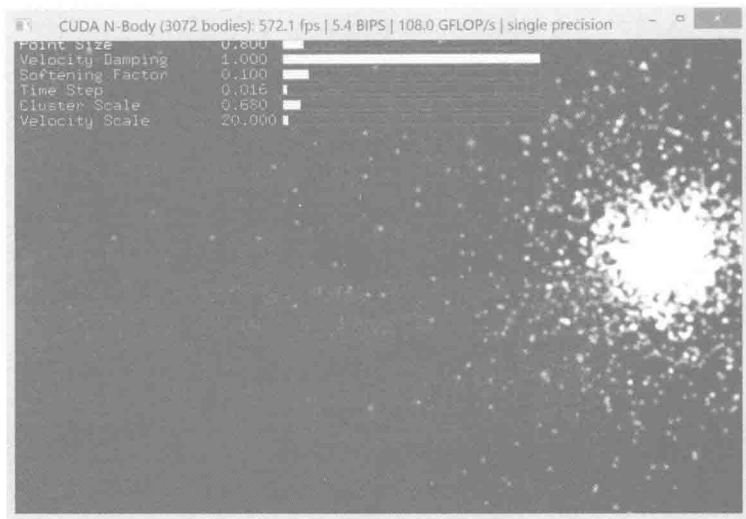


图 1.3 nbody 模拟程序的图形化输出窗口

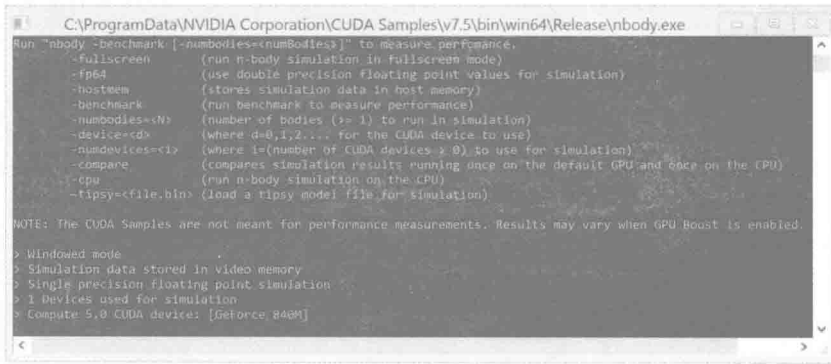


图 1.4 nbody 的控制台窗口及可选命令行参数

通过控制台命令行指令执行 CUDA 样例也是十分有用的，读者可以参考以下步骤。

首先，让我们先给出另一个提示：如果在执行下面建议的命令中遇到了任何问题，请到附录 B 寻找相关解决办法。

启动 Visual Studio 并且打开 Visual Studio 命令行界面（可以在 TOOLS 菜单中找到）使用资源管理器，复制 Release 文件夹地址（其中包含了 CUDA 样例程序的可执行程序）。输入 cd 命令并粘贴文件夹路径可以将当前路径设置到 Release 文件夹，然后执行 nbody 命令来运行这个模拟程序。

1.1.2 在 Linux 下运行 CUDA 样例程序

这里让我们从一个提示入手：如果在执行下面建议的命令中遇到了任何问题，请到附录 B 寻找相关解决办法。

打开终端并使用 cd 命令进入到 NVIDIA_CUDA-7.5_Samples/5_Simulations/nbody 目录下使用 make 命令编译 nbody 程序，然后输入 ./nbody 命令来运行这个模拟程序。通过类似的流程，读者可以运行任何感兴趣的 CUDA 样例程序。

1.1.3 估计“加速效果”

在 GPU 并行的术语中，我们将把一个计算任务以原来 N 倍的速度运行称为“ N_X 加速”（这里 X 是“倍”的意思）。（这里的“加速”一词并不准确，使用它可能会让一些学习过动力学的读者感到不适，但是我们要适应并行计算的传统做法，毕竟术语是不太可能改变的。）

这是一个初步评估加速效果的好时机，因为 nbody 在这一点上很有代表性。nbody 的命令行窗口提供了一系列的可选参数，这些参数可以在通过命令行执行 nbody 时设置。我们将用到这其中的三个参数，如果感兴趣，读者可以尝试使用更多的参数：

- -benchmark 命令可以让这个模拟过程进入适合评估性能的模式，尽可能减少可视化输出所耗费的时间。
- -cpu 命令可以在 CPU 下运行这个程序而不是在 GPU 下。
- -fp64 命令可以在这个模拟中进行 64 位双精度浮点运算而不是 32 位单精度运算。

nbody 命令行窗口也显示了“这些 CUDA 样例不是为了性能评估之用”的声明，因此我们在 nbody 上进行一些基准测试只是为了获取一些粗糙的加速估计结果。但是需要对这些数字保持一些应有的怀疑，不要过分迷信。

首先运行 nbody -benchmark（Linux/OS X 下运行 ./nbody -benchmark）并记录下

GPU 下运行的输出结果，其中包含每秒进行浮点运算数的报告。下一步，运行带有 -benchmark -cpu 参数的 nbody 程序并且记录下对应的 CPU 执行结果。然后加入 -fp64 参数重复上面的测试。

在我们使用 GeForce 840M 的笔记本上，估算的 GPU 的每秒单精度浮点运算次数大约是 CPU 的 120 倍。在双精度下，GPU 大约加速 6 倍左右。这里，请注意另一个提示：在继续阅读下文前，读者应该在自己的系统上实际运行一下上述测试来获取一个实在的加速体验。

需要注意，正如其声明中所言，CPU 的实现可能没有被全面优化过，而 GPU 端的程序可能已经做了全面的优化。事实上，nbody 下 docs 子文件夹包含了一个全面而有价值的 GPU 优化实现的论文。先不考虑那个声明，我们可以从这些初次的尝试中学到一些东西：1. 基于 GPU 的并行可以在计算吞吐上带来明显收益。2. 使用一个 GeForce 显卡在单精度运算上的提升高出双精度计算很多倍。

读者所获得的加速体验可以很好地反映 CUDA 开发的经验法则。当你第一次在 GPU 上并行化你的计算任务时，你可以获得初步的性能提升；通常通过选择正确的内存模式（我们将对可选的模式进行讨论）抑或购买一块更加高端的 GPU（特别是当双精度计算对你十分重要并且你能负担得起一块 Tesla 显卡），你可以获得第二次性能上的提升；性能的第三次大幅提升往往需要进一步对硬件细节参数进行调优。我们的讨论会针对前两个加速情况，然后给出一些关于第三次性能提升的参考资料。

以上就是我们对 nbody 样例的观察，但是在继续之前，让我们运行一个更加简单的样例。在 2_Graphics 文件夹中运行 volumeRender 样例（如图 1.1 所示），通过移动鼠标实时改变视角与一个 3D 数据集进行交互。我们将在第 7 章中建立我们自己的 3D 可视化应用。

读者可以随时继续探索 CUDA 的样例程序，这里需要准备好阅读下一节。

1.2 运行我们自己的串行程序

是时候把我们的目光从 CUDA 样例程序移开，构建并运行我们自己的程序。在本节我们将给出完成相同功能的两个应用的代码 dist_v1 和 dist_v2。每个应用都计算了从一个参考点到 N 个在直线上均匀分布的点之间的一组距离。这一任务精心设计，力求简单，但是对于工程师而言仍然具有明确的指导意义。读者也很容易考虑到如何将这里的实现

推广到更加通用的场合。

值得注意的是，虽然 dist_v1 与 dist_v2 实现的是同一个任务，但是它们通过不同的方法来完成。

- dist_v1 使用一个 for 循环，循环内先对循环变量进行归一化得到一个输入位置然后计算并存储该位与参考位置的距离。这里我们建立一个 distance() 函数计算一个参考位置和某个点间的距离，该函数调用了 N 次。
- dist_v2 以建立一个输入位置的数组作为开始，然后将这个数组指针传给 distance-Array() 函数，该函数在一次调用中计算并存储整个数组距离值。

dist_v1 和 dist_v2 将作为我们在第 3 章中讲解并行化的初始实验对象。这里给出它们的代码，每个程序的创建、编译以及运行的指南和详细解释参见附录 C。这里给出另一个提示：你需要能够编译并运行 dist_v1 和 dist_v2 这两个程序。如果你对此有任何问题，请参考附录 C 来找到解决这些问题的方法。

1.2.1 dist_v1 应用

dist_v1 应用只包含一个文件 main.cpp，内容如代码清单 1.1 所示。对应的 Linux 下的 Makefile 文件参见代码清单 1.2。

代码清单1.1 dist_v1/main.cpp

```
#include <math.h> // Include standard math library containing sqrt
#define N 64 // Specify a constant value for array length

// A scaling function to convert integers 0,1,...,N-1
// to evenly spaced floats ranging from 0 to 1
float scale(int i, int n)
{
  return ((float)i)/(n - 1);
}

// Compute the distance between 2 points on a line
float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

int main()
{
  // Create an array of N floats (initialized to 0.0)
  // We will overwrite these values to store our results.
  float out[N] = {0.0f};

  // Choose a reference value from which distances are measured
  float ref = 0.5f;

  /* for loop to scale the index to obtain coordinate value,
```

```
27    *  compute the distance from the reference point,
28    *  and store the result in the corresponding entry of out */
29    for (int i = 0; i < N; ++i)
30    {
31      float x = scale(i, N);
32      out[i] = distance(x, ref);
33    }
34
35    return 0;
36  }
```

代码清单1.2 dist_v1/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall
main.exe: main.cpp
    $(NVCC) $(NVCC_FLAGS) $< -o $@
```

1.2.2 dist_v2 应用

dist_v2 应用的代码包含了三个文件：main.cpp（参见代码清单 1.3）、aux_functions.cpp（参见代码清单 1.4）以及头文件 aux_functions.h（参见代码清单 1.5）。Linux 下的 Makefile 文件参见代码清单 1.6。

代码清单1.3 dist_v2/main.cpp

```
 1 #include "aux_functions.h"
 2 #include <stdlib.h> // supports dynamic memory management
 3
 4 #define N 20000000 // A large array size
 5
 6 int main()
 7 {
 8   float *in = (float*)calloc(N, sizeof(float));
 9   float *out = (float*)calloc(N, sizeof(float));
10   const float ref = 0.5f;
11
12   for (int i = 0; i < N; ++i)
13   {
14     in[i] = scale(i, N);
15   }
16
17   distanceArray(out, in, ref, N);
18
19   // Release the heap memory after we are done using it
20   free(in);
21   free(out);
22   return 0;
23 }
```

代码清单1.4 dist_v1/aux_functions.cpp

```
1 #include "aux_functions.h"
2 #include <math.h>
```

```
3
4 float scale(int i, int n)
5 {
6   return ((float)i) / (n - 1);
7 }
8
9 float distance(float x1, float x2)
10 {
11   return sqrt((x2 - x1)*(x2 - x1));
12 }
13
14 void distanceArray(float *out, float *in, float ref, int n)
15 {
16   for (int i = 0; i < n; ++i)
17   {
18     out[i] = distance(in[i], ref);
19   }
20 }
```

代码清单1.5 dist_v2/aux_functions.h

```
1 #ifndef AUX_FUNCTIONS_H
2 #define AUX_FUNCTIONS_H
3
4 // Function to scale input on interval [0,1]
5 float scale(int i, int n);
6 // Compute the distance between 2 points on a line
7 float distance(float x1, float x2);
8 // Compute scaled distance for an array of input values
9 void distanceArray(float *out, float *in, float ref, int n);
10
11 #endif
```

代码清单1.6 dist_v2/Makefile

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3
4 all: main.exe
5
6 main.exe: main.o aux_functions.o
7   $(NVCC) $^ -o $@
8
9 main.o: main.cpp aux_functions.h
10   $(NVCC) $(NVCC_FLAGS) -c $< -o $@
11
12 aux_functions .o: aux_functions.cpp aux_functions.h
13   $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

这里我们到达了本章的最后一个提示：在继续之前，读者需要运行 dist_v1 和 dist_v2 并且使用一个调试器证实计算的距离的值从 0.5 开始（归一化得到的值 0 与参考点 0.5 的距离），在接近数组中间的时候递减到 0，继续到末尾时增长到 0.5（对应于从参考点 0.5 到归一化值 1.0 之间的距离）。如果读者遇到了任何问题，请阅读附录 C “C 语言编程需

知”中的相关内容。

1.3 本章小结

在本章，我们运行了一些 CUDA 样例程序并且获得了 GPU 并行计算的直观体验。同时我们介绍了两个串行的程序，将在后文作为并行化的实验函数。其中 dist_v1 提供了一个最简单的并行化实验例子，而 dist_v2 则描述了一个实用的结构和更典型的数据流。

现在要准备好进入第 2 章，开展 CUDA 并行计算模型的讨论，之后的第 3 章将展示使用 CUDA 进行并行化的实现细节。

1.4 推荐项目

项目 1～5 是关于运行其他 CUDA 样例程序的练习。

1. 运行在 1_Utilities 文件夹下的 deviceQuery 样例程序，获取读者计算机系统中支持 CUDA 的 GPU 信息。
2. 运行在 2_Graphics 文件夹下的 Mandelbrot 样例程序。模拟选项将会出现在命令行窗口中。你可以通过 ESC 键随时退出模拟。
3. 运行在 2_Graphics 文件夹下的 volumeFilter 样例程序。模拟选项将会出现在命令行窗口中。
4. 运行在 5_Simulations 文件夹下的 smokeParticles 样例程序。在模拟中点击鼠标右键可以看到交互选项。
5. 运行在 5_Simulations 文件夹下的 fluidsGL 样例程序。在模拟中使用鼠标左键点击和拖动来操纵液态粒子（r 键重置模拟）。
6. 我们定义了 distance() 函数返回 sqrt((x2-x1)*(x2-x1)) 的计算结果。另一种替代计算欧氏距离的替代方式是使用绝对值距离。请创建一个替代现有方法的基于绝对值的 distance() 函数。参考 http://en.cppreference.com/w/c/numeric/math 来查看更多常用数学函数。

第 2 章 Chapter 2

CUDA 基础知识

在第 1 章中我们的讨论以计算从一个参考点到一组输入位置距离的函数 distanceArray() 结束。这个计算完全是串行的，距离数值是根据一个 for 循环中的计数 i 和输入数组的范围顺序计算的。但是，任何一个距离的计算相对于其他计算都是独立的。在串行实现中，我们不能发挥计算独立的优势反而会在计算中等待，直到一个计算完成再进行下一个计算。如果读者所使用的系统同一时间只能进行一个计算，那么串行的实现无可苛责。然而，在一个通用的 GPU 计算中，我们拥有成百上千的可以同时进行运算的硬件单元。为了让我们拥有的众多处理器发挥优势，我们将串行模式（同一时间内只进行一个计算任务，其他的依次等待）转换为并行模式（大量的计算同时执行）。本章中描述了 CUDA 模式中的并行、CUDA 的基本编程语言扩展以及应用程序编程接口（API）[1, 2]。

2.1 CUDA 并行模式

从串行到 CUDA 并行同时涉及硬件和软件两方面。硬件的转换涉及包含了多个运算单元以及运算规划和数据传输机制的芯片。软件的转换涉及 API 以及对编程语言的扩展。

主机和设备

明确一下我们的术语，主机指代 CPU 和内存，设备指代 GPU 和显存。

GPU 能够进行并行化的关键属性是其并不是只有一个或几个计算单元（像现代多核 CPU 一样）而是其具有成百上千计算单元。如果读者能够将计算分成大量的独立子任务，这些大量的计算单元为并行执行这些任务提供了可行性，换言之，同时执行这些任务而不是串行进行。值得注意的是，这样的并行涉及许多规划方面的问题：如何让一个特定的计算单元知道其需要执行哪个子任务？如何让大量的计算单元进行指令和数据的访问而不会造成巨大的通信阻塞？

CUDA 引用了单指令多线程（SIMT）的并行模式。CUDA GPU 包含了大量的基础计算单元，这些单元被称为核（core）每一个核都包含了一个逻辑计算单元（ALU）和一个浮点计算单元（FPU）。多个核集成在一起被称为多流处理器（SM）。

我们将一个计算任务分解为多个子任务，并将其称为线程，多个线程被组织为线程块。线程块被分解为大小与一个 SM 中核数量相同的线程束（warp）。每个线程束由一个特定的多流处理器执行。这些多流处理器的控制单元指挥其所有核同时在一个线程束的每个线程中执行同一个指令，因此这个术语称为单指令多线程（single instruction multiple thread）。

执行同一指令并不是仅仅重复一种运算，因为每个线程使用由 CUDA 提供的单独的索引值进行了不同的运算[3]。这种 SIMT 的方法是可拓展的，因为可以通过使用更多的多流处理器分担计算负载来增加计算的吞吐量。图 2.1 显示了一个 GPU 和一个 CPU 之间的架构对比。CPU 拥有较少的核，占据芯片的小部分，而更多的区域被用来加速那些少量核的控制器和缓存占据。通常而言，访问数据所需的时间会随着计算核和存贮数据的内存间位置的距离而增加。核等待数据的时间被称为延迟（latency），并且一个 CPU 通过设计大量的可以快速访问的空间存储数据来缩小延迟。

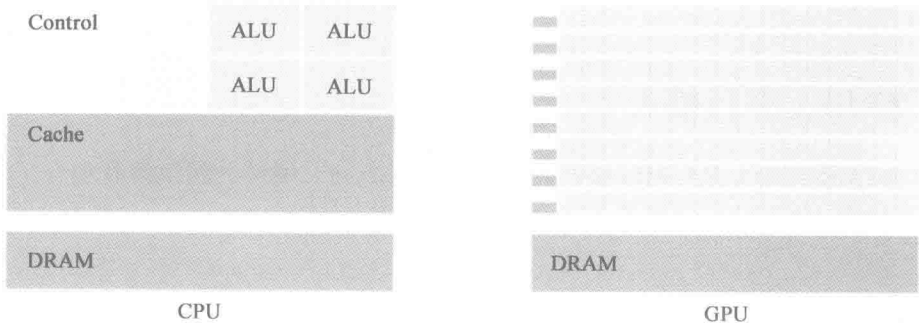


图 2.1 CPU 与 GPU 的架构对比

（图片来源于 CUDA Programming Guide，已获得 NVIDIA 公司版权）

GPU 的空间分配则十分不同。芯片上的大多数地方被分配给了大量组织成多流处理器的运算单元和共享的控制单元。与其缩小延迟（需要为每个核配备大量缓存）GPU 更倾向于隐藏延迟。当一个线程束所需要的数据不可获得时，多流处理器会转向执行另一个可获得数据的线程束。所关注的重点是整体的运算吞吐量而不是单个核心的执行速度。

现在我们已经准备好讨论 CUDA 的 SIMT 软件方面的实现。关键的软件结构是一种叫作核函数（kernel）的特殊形式的函数，这个函数中产生大量的组织成可以分配给多流处理器的计算线程。用 CUDA 术语来讲，我们加载一个核函数来创建一个由多个线程块组成的线程网格，线程块由多个线程组成。为了替换串行的计算，我们需要一种方法来告诉每个线程去执行哪一部分运算，换言之，访问哪一个输入的入口或者哪一个输出的出口，以及选择哪个计算或存储。CUDA 通过为每个线程提供内建的索引变量来进行这样的编址。这些 CUDA 的索引变量会替换串行代码里的循环索引。加载核函数、创建计算网格、索引线程块和线程的具体过程将在 2.2 节中进行讨论。

2.2　需要知道的 CUDA API 和 C 语言拓展

CUDA 并行所需要的基本任务包含以下几点：

- 使用特定的网格维度加载核函数（线程块和线程的数目）。
- 明确哪些函数编译后运行在设备（GPU）上、主机（CPU）上，或者两者之上。
- 访问和运用线程块和线程的计算索引值。
- 分配内存和传输数据。

让我们从介绍核函数的加载开始。正如上面讨论的，核函数是一种特殊的函数，加载核函数与常规的函数调用看起来很像，详细来说，加载核函数从一个函数名开始，比如 aKernel，然后以一个包含了以逗号分开的参数列表的括号结尾。现在我们来看一下编程语言拓展：为了自然地表达并行并且声明计算网格，在函数名和含有参数的括号中间加入了网格的维度和线程块的维度（被放在三个尖括号中）：

```
aKernel<<<Dg, Db>>>(args)
```

需要注意的是，Dg、网格中的线程块数和 Db、线程块中的线程数目，一起组成了加载核函数中的执行配置和维度的声明。

这构成了加载一个和函数的语法，但是如何声明一个从主机端调用但是在设备端执

行的函数仍有一些疑问。CUDA 使用在函数前面添加下面的一个函数标识符来作为区分：

- __global__ 是标志着和函数的标识符（可以在主机端调用并在设备端执行）。
- __host__ 函数从主机端调用在主机端执行。（这是一个默认的限定符，通常被省略。）
- __device__ 函数从设备端调用并在设备端执行。（从核函数中调用的函数需要有 __device__ 限定符。）
- 在函数头添加 __host____device__ 函数会让系统分别编译这个函数的主机版本和设备版本。

动态并行

在 CUDA 5.0 以后，在一个计算能力在 3.5 或以上的设备中，从另外的核函数中加载核函数成为了可能[1, 4, 5, 6]。在设备上调用 __global__ 函数被称为动态并行，这超出了本书的范围。

核函数有几个值得注意的权限和限制：

- 核函数不能带有返回值，因此返回类型通常为 void。并且核函数需如下声明：

```
__global__ void aKernel(typedArgs)
```

- 核函数提供了对于每一个线程块和线程的维度数和索引变量。

  维度数目变量：

  - gridDim 声明了网格中的线程块数目。
  - blockDim 声明了每个线程块中的线程数目。

  索引变量：

  - blockIdx 给出了这个线程块在网格中的索引。
  - threadIdx 给出了这个线程在线程块中的索引。
- 在 GPU 上执行的核函数通常不能访问主机端 CPU 可以访问的内存中的数据。

内存访问的发展

最近的发展旨在统一 CPU 和一个或多个 GPU 可以访问的内存。我们将会讨论到在主机端和设备端进行内存传输的管理所需要的知识。我们会在第 3 章介绍一种可以使用的统一内存访问。

CUDA 运行时 API 提供了一些可以将输入数据传输到设备端和将结果传回到主机端的函数，如下所示：

- cudaMalloc() 函数可以分配设备端内存。
- cudaMemcpy() 将数据传入或传出设备。
- cudaFree() 释放掉设备中不再使用的内存。

核函数并行进行多次运算，但是它们也放弃了对执行顺序的控制。CUDA 为需要同步和并发执行时提供了相应的函数：

- __syncThreads() 可以在一个线程块中进行线程同步。
- cudaDeviceSynchronize() 函数可以有效地同步一个网格中的所有线程。
- 原子操作，例如 atomicAdd()，可以防止多线程并发访问一个变量时造成的冲突。

除了上文介绍的函数和标识符以外，CUDA 同时提供了一些额外的有用的数据类型：

- size_t：代表内存大小的专用变量类型。
- cudaError_t：错误处理的专用变量。
- 向量类型：CUDA 将标准 C 的向量数据类型拓展到了 4 个。独立的组件通过后缀 .x、.y、.z 和 .w 进行访问。

向量类型的关键重要用法

CUDA 在 blockIdx 和 threadIdx 上使用 uint3 向量类型。uint3 变量是一个包含了三个整型数的向量。

CUDA 在 gridDim 和 blockDim 上使用了 dim3 向量类型。dim3 类型与 uint3 一样，但将为声明的变量设置为 1。我们在声明执行设置时将使用 dim3 类型。

2.3　本章小结

现在你已经掌握了开始创建一个拥有使用了 CUDA 进行 GPU 并行计算的优点的应用所必需的工具，我们将在第 3 章中开始创建 CUDA 应用。

2.4　推荐项目

1. 去 CUDA Zone 注册并加入到 CUDA 开发者中（如果读者还没有这样做的话）。
2. 观看 www.nvidia.com/object/nvision08_gpu_v_cpu.html 的视频，体会关于并行和串行执行的有趣的对比。
3. 阅读《CUDA Programming Guide》的导论部分，详见 http://docs.nvidia.com/cuda/pdf/

CUDA_C_Programming_Guide.pdf。

4. 观看 David Luebke 在 2011 年 GPU 技术大会的视频“GPU Computing: Past, Present, and Future”体会对 GPU 并行计算起源和方向的有趣的透视。(不要担心那些偶尔不熟悉的术语，将观看这个视频作为一个获得 CUDA 术语的机会。) 这个视频详见：http://on-demand-gtc.gputechconf.com/content/includes/gtc/video/stream-david-luebke-june2011.html。

参考文献

[1] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[2] Jason Sanders and Edward Kandrot, *CUDA by Example* (Boston, MA: Addison-Wesley, 2011).

[3] Rob Farber, "The CUDA Execution Model," Chap. 4 in *CUDA Application Design and Development* (Waltham, MA: Morgan Kaufmann, 2011), 85–108.

[4] NVIDIA Corporation, *Dynamic Parallelism in CUDA*, 2012, http://developer.download.nvidia.com/assets/cuda/files/CUDADownloads/TechBrief_Dynamic_Parallelism_in_CUDA.pdf.

[5] Stephen Jones, *Introduction to Dynamic Parallelism* (2012), http://on-demand.gputechconf.com/gtc/2012/presentations/S0338-GTC2012-CUDA-Programming-Model.pdf.

[6] Andrew Adinetz, "Adaptive Parallel Computation with CUDA Dynamic Parallelism," *Parallel Forall* (blog), May 2014, http://devblogs.nvidia.com/parallelforall/introduction-cuda-dynamic-parallelism/.

第 3 章 Chapter 3

从循环到网格

现在我们要使用第 2 章所学知识将第 1 章和附录 C 中的 C 语言代码进行并行化。回顾我们创建的两个版本的距离计算应用。在 dist_v1 中我们使用一个 for 循环来计算一个数组的距离值。在 dist_v2 中我们为输出值创建了一个数组然后调用 distanceArray() 方法来计算全部距离值的数组（同样是使用一个 for 循环串行的计算）。本章会使用 CUDA 来并行化这个距离应用，并行化的方法是使用一个可以同时执行的线程网格来替换循环的串行计算。

我们将会创建、编译并执行新的应用，但是重点关注使用 CUDA 进行并行化。如果读者在编译并执行没有 CUDA 声明的 C 语言程序中存在问题，请参考附录 C。

3.1 并行化 dist_v1

这是首个探索并行化的示例，在展示完整的并行代码之前，我们要一步步地讲述如何从 dist_v1 的代码变化到并行化版本 dist_v1_cuda，完整的代码参见代码清单 3.1。读者需要执行以下的基本步骤。

1. 创建一个包含 kernel.cu 文件的 dist_v1_cuda 工程。

a. 在 Linux 下，创建一个 dist_v1_cuda 文件夹并在其中新建一个 kernel.cu 文件。

b. 在 Visual Studio 中，创建一个新的 CUDA 7.5 运行时项目，命名为 dist_v1_cuda，项目中包含一个 kernel.cu 文件。删除 kernel.cu 文件中的样例代码。

2. 将 dist_v1/main.cpp 中的代码复制并粘贴到 kernel.cu 文件中。按照如下过程修改 kernel.cu 代码：

a. 删除 #include <math.h>，因为 CUDA 内联文件已经包含了 math.h。加入 #include <stdio.h> 使程序能够在控制台输出。

b. 在目前的 #define 字段下，加入另一个 #define TPB 32，来声明在执行我们的核函数中将作为核函数执行配置中线程块维度的数大小。

c. 拷贝整个 for 循环并粘贴在 main() 函数上方。我们将把这个循环编写到一个核函数的定义中。

d. 使用 distanceKernel<<<N/TPB, TPB>>>(d_out, ref, N) 核函数调用替换 main() 函数中的 for 循环，调用中包含了三个要素：

i. 函数名称 distanceKernel。

ii. 三对 <> 符号，其中包含了执行的配置参数。此时，我们设定 N/TPB 个线程块（2 个）其中每个线程块包含 TPB 个线程（32 个）。

iii. 代码清单中还包含了指向输出数组的指针 d_out，参考地址 ref 和数组长度 N。

e. 将 main() 函数之前的 for 循环按照如下步骤转换成一个核函数：

i. 使用由如下内容组成的核函数声明替换第一行的 for 循环（{} 中的所有内容）：

1. 标识符 __global__

2. 返回类型 void

3. 函数名 distanceKernel

4. 括号中包含了使用逗号分隔的带有类型的形参列表 (float *d_out, float ref, int len)

ii. 在 {} 的第一行中输入下面的计算公式，式中使用核函数的内建线程索引和维度变量计算索引 i（用来代替循环中现在被删除了的同名索引）。

```
const int i = blockIdx.x*blockDim.x + threadIdx.x;
```

iii. 将左边的数组 out[i] 修改为 d_out[i]。

iv. 在 {} 的最后一行中输入下面的 printf() 语句：

```
printf("i = %2d: dist from %f to %f is %f.\n", i, ref, x,
d_out[i]);
```

f. 在 scale() 和 distance() 的定义前添加标识符 __device__.

g. 管理如下的存储：

i. 在 main() 开始前，删除定义 out 数组的语句（此处不需要的语句），使用下面的代

码创建一个位于设备上的可以存储 N 个浮点数的数组 d_out。

```
float *d_out;
cudaMalloc(&d_out, N*sizeof(float));
```

ii. 在 main() 函数的 return 语句上方，使用如下语句释放内存：

```
cudaFree(d_out);
```

为了确保所有的语句段能够正确组合在一起，dist_v1_cua/kernel.cu 文件的完整代码如代码清单 3.1 所示。

代码清单3.1　dist_v1_cuda/kernel.cu

```
#include <stdio.h>
#define N 64
#define TPB 32

__device__ float scale(int i, int n)
{
  return ((float)i)/(n - 1);
}

__device__ float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

__global__ void distanceKernel(float *d_out, float ref, int len)
{
  const int i = blockIdx.x*blockDim.x + threadIdx.x;
  const float x = scale(i, len);
  d_out[i] = distance(x, ref);
  printf("i = %2d: dist from %f to %f is %f.\n", i, ref, x, d_out[i]);
}

int main()
{
  const float ref = 0.5f;

  // Declare a pointer for an array of floats
  float *d_out = 0;

  // Allocate device memory to store the output array
  cudaMalloc(&d_out, N*sizeof(float));

  // Launch kernel to compute and store distance values
  distanceKernel<<<N/TPB, TPB>>>(d_out, ref, N);

  cudaFree(d_out); // Free the memory
  return 0;
}
```

在执行这个并行的应用之前，有几个需要依次给出的评论。希望通过我们的努力能够让读者在阅读完整的代码时对代码的所有组件的目的和意图有一个正确的理解。

我们也想要提醒读者一些核函数的特殊性，因为它是我们使用 CUDA 并行化的主要工具。一个需要注意的事实是核函数（比如 distanceKernel）在设备上执行，所以它不能向主机端返回变量，因此使用 void 类型作为返回值。核函数可以访问设备内存，但是不能访问主机内存，因此我们需要使用 cudaMalloc() 函数（而不是 malloc 函数）为设备上输出数组分配内存，同时使用 d_out 命名，d_ 作为一个便于记忆的标志（显然不是强制性的标志）来提醒读者 d_out 是一个设备上的数组。最后，注意核函数的定义就如同我们只在单线程编程一样进行书写。在之前串行代码的循环中发生的所有事情现在被每个线程所对应的 i 所接管，它根据 CUDA 为每个线程提供的索引和维度变量来获取所有计算网格中的数据索引。

选择执行配置

注意，无论是把它看作艺术还是科学，选择合适的执行配置会为给定的系统提供最好的性能。就现在来说，让我们不要纠结于细节，只要注意在选择每个线程块的线程数目时保证是 32 的整数倍即可，这样可以对应每个 SM 中的 CUDA 核心数目。系统支持的线程块和线程网格中也有数量的限制。在《 CUDA C Programming Guide》[1] 中可以找到记录这个限制的表格。一个特别重要的限制是每个线程块不能包含超过 1024 的线程。因为线程网格可能包括超过 1024 的总线程，读者应该尽量在启动核函数时设置大量的线程块，并且制定一些执行配置的实验计划，来探究在读者的应用和设备上的最优执行方案。对于较大的问题，测试每个线程块的线程数目的合理值包括 128、256 和 512。

执行 dist_v1_cuda

当编码完成后，是时候来编译和执行这个应用了。由于代码中已经包含了核函数加载（不是 C/C++ 的一部分），因此我们需要调用 NVIDIA C 编译器——nvcc。在 Windows 下，Visual Studio 识别这种扩展并且自动调用 nvcc 编译器，直接按下 F7 来构建这个应用。在 Linux 下，使用代码清单 3.2 中的 Makefile 文件指示编译器使用 nvcc 编译器来构建这个程序。

代码清单3.2 dist_v1_cuda/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall

main.exe: kernel.cu
    $(NVCC) $(NVCC_FLAGS) $^ -o $@
```

注意在代码清单 3.1 中，我们在第 1 行里包括了标准的输入和输出，并且在第 20 行加入了一个 printf() 语句来将结果打印在屏幕上，因此现在你可以执行这个应用然后在控制台中观察输出结果。检验输出结果和那些由串行应用 dist_v1 和 dist_v2 运算的结果是否一致。

关于 printf() 函数

需要注意的是，计算能力小于 2.0 的 GPU 是不支持在核函数中使用 prinf() 函数的。如果你对 printf() 函数不熟悉（或者对输出流中加入字符串更感兴趣，更倾向于使用 C++ 中的 cout），请使用你所偏好的 C/C++ 语法。我们这么做是为了方便，但是值得注意的是 printf() 不应被考虑作为一个普遍使用的 debug 工具。

关于 debugging 的更多信息

Visual Studio 或者 gdb 提供的常见的 C/C++ 调试工具不适用于显示设备内存中的数值，有一些 CUDA 专用的调试工具可以用来达到这样的目的。关于 CUDA 调试工具的讨论，包括观察存储在设备内存中的数值的方法请参见附录 D。

可能你注意到的第一件事是输出不一定是以索引顺序从 0 到 63 依次输出的。这是串行和并行应用的一个根本区别。在串行应用中，计算在一个循环中按照一定先后顺序执行。在 CUDA 实现的并行中，我们放弃了对计算顺序的部分控制而获得由成百上千个处理器并行计算提供的计算加速。

第二件事（也是最重要的事）是检查利用并行计算得到的距离值是否与应用 dist_v1 和 dist_v2 计算得到的值一致。总而言之，如果产生了不正确的结果，那么加速再多也是毫无意义的。

3.2　并行化 dist_v2

第一个并行化的距离应用 dist_v1_cuda 不具有充分的代表性，因为它并没有涉及大量输入数据。现在我们已经准备好了并行化第二个距离应用 dist_v2，这是一个更加典型的例子，涉及对一个输入数组的操作。将一个数组数据传递到 GPU 的策略并不复杂：

1. 在设备内存（GPU）上创建一个数组，类型和大小与主机内存（CPU）上的输入数组一致。

2. 在设备内存上创建一个内存用来存储输出的数据。（你可以选择使用复用输入数组

的办法存储输出，但是目前我们先将输入和输出分开考虑。）

3. 将主机端的输入数组复制到设备内存上对应的数组上。

4. 启动核函数进行计算并将输出值存储在设备内存上的输出数组中。

5. 将设备内存中的输出数组复制回主机内存中的对应数组上。

6. 释放设备上的数组内存。

上面列出的步骤很适合 dist_v2 的组织方式，其中包括了一个 main.cpp 文件（包含了 main() 函数的代码），一个单独的文件（名为 aux_functions.cpp）来包含定义辅助功能的代码，一个头文件（名为 aux_functions.h）来包含像 distanceArray() 函数一样的原型函数，使其可以在其他文件中被“看见”并调用（例如 main.cpp 文件）。

因为步骤 4 中涉及 CUDA 核函数调用，我们将 aux_functions.cpp（及其对应的头文件）替换为 kernel.cu（及其对应的头文件——kernel.h）。完整的代码在代码清单 3.3、代码清单 3.4 以及代码清单 3.5 中显示，代码清单 3.6 是 Makefile 文件。

代码清单3.3 dist_v2_cuda/main.cpp

```
#include "kernel.h"
#include <stdlib.h>
#define N 64

float scale(int i, int n)
{
  return ((float)i)/(n - 1);
}

int main()
{
  const float ref = 0.5f;

  float *in = (float*)calloc(N, sizeof(float));
  float *out = (float*)calloc(N, sizeof(float));

  // Compute scaled input values
  for (int i = 0; i < N; ++i)
  {
    in[i] = scale(i, N);
  }

  // Compute distances for the entire array
  distanceArray(out, in, ref, N);

  free(in);
  free(out);
  return 0;
}
```

Dist_v2_cuda/main.cpp 的代码与 dist_v2/main.cpp 几乎一样。不同之处包括把 scale() 移到 main.cpp 中，并引用 kernel.h 来代替原来的 aux_functions.h 文件。最大的变化发生

在 kernel.cu 文件中新定义的 distanceArray() 函数中。

代码清单3.4　dist_v2_cuda/kernel.cu

```
1 #include "kernel.h"
2 #include <stdio.h>
3 #define TPB 32
4
5 __device__
6 float distance(float x1, float x2)
7 {
8   return sqrt((x2 - x1)*(x2 - x1));
9 }
10
11 __global__
12 void distanceKernel(float *d_out, float *d_in, float ref)
13 {
14   const int i = blockIdx.x*blockDim.x + threadIdx.x;
15   const float x = d_in[i];
16   d_out[i] = distance(x, ref);
17   printf("i = %2d: dist from %f to %f is %f.\n", i, ref, x, d_out[i]);
18 }
19
20 void distanceArray(float *out, float *in, float ref, int len)
21 {
22   // Declare pointers to device arrays
23   float *d_in = 0;
24   float *d_out = 0;
25
26   // Allocate memory for device arrays
27   cudaMalloc(&d_in, len*sizeof(float));
28   cudaMalloc(&d_out, len*sizeof(float));
29
30   // Copy input data from host to device
31   cudaMemcpy(d_in, in, len*sizeof(float), cudaMemcpyHostToDevice);
32
33   // Launch kernel to compute and store distance values
34   distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
35
36   // Copy results from device to host
37   cudaMemcpy(out, d_out, len*sizeof(float), cudaMemcpyDeviceToHost);
38
39   // Free the memory allocated for device arrays
40   cudaFree(d_in);
41   cudaFree(d_out);
42 }
```

这里需要强调几个要点：

1. distanceKernel() 函数与 dist_v1_cuda/kernel.cu 中的版本有明显的不同。这里，形参包含了一个指针 d_in，指向一个已经被构建好的输入数组。

2. 对于 distanceArray() 新的定义执行了上面所列出的全部 5 个步骤。设备端的输入输出数组在第 23～28 行代码中定义；输入数据从主机端复制到设备端在第 31 行执行；从核函数调用并获取 GPU 并行化的执行结果在第 43 行；输出的距离值从设备端复制到

主机端在第 37 行；设备端的内存释放在第 40 行和第 41 行。

3. 函数 distanceArray() 调用核函数 distance-Kernel()。这样的函数被称为封装（warpper）或者启动函数（launcher）。

执行配置与整数运算

明确核函数的执行设置：每个线程块拥有 TPB 个线程，然后一共有 N/TPB 个线程块（N 的值实际是在进行函数调用时通过长度 len 传入的）。我们已经使用了 #define 来设置 TPB=32 和 N=64，因此本次一共需要两个包含了 32 个线程的线程块，就像我们之前使用的一样，但是记住存储这些维度的变量是一个整型的类型，使用整型数的相关运算。如果你设置 N=65，那么你仍然会得到 2 个（65/32）包含 32 个线程的线程块，数组的最后一个输入不会被计算到，因为没有与它对应的线程及数据索引。有一个简单的办法可以保证线程网格覆盖到整个数组长度，就是声明线程块数目从 N/TPB 到 (N+TPB-1)/TPB，以此来保证线程块数目被取整。这也意味着网格的索引可以超出最大的数组索引，那么我们就需要在核函数中加入一个控制语句来防止执行中可能超出数组范围并引发段冲突错误。在这里，我们仅继续考虑一个简单的版本，但是我们会在后面的应用中加入更加可靠的机制。

注意线程网格和线程块的维度，实际是三维的类型 dim3。未指明的分量默认设为 1。执行配置 <<<len/TPB, TPB>>> 将被解释为 <<dim3(len/TPB, 1, 1), dim3(TPB, 1, 1)>>>。当我们使用更大维度的网格时我们将会使用一些有意义的 .y 的值和 .z 的值。

代码清单3.5 dist_v2_cuda/kernel.h

```
1 #ifndef KERNEL_H
2 #define KERNEL_H
3
4 // Kernel wrapper for computing array of distance values
5 void distanceArray(float *out, float *in, float ref, int len);
6
7 #endif
```

代码清单3.6 dist_v2_cuda/Makefile

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3
4 all: main.exe
5
6 main.exe: main.o kernel.o
7   $(NVCC) $^ -o $@
8
9 main.o: main.cpp kernel.h
```

```
10    $(NVCC) $(NVCC_FLAGS) -c $< -o $@
11
12 kernel.o: kernel.cu kernel.h
13    $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

读者可以用这些现成的代码编译并执行 dist_v2_cuda。验证这些距离的值是否被正确计算，并且留意一下所有关键的 CUDA 代码都包含在 kernel.cu 文件中。对比 distanceArray() 函数在 dist_v2/aux_functions.h 和 dist_v2_cuda/kernel.h 中的原型。注意这些原型是完全相同的，但是函数的定义却被改变了。用软件开发的术语来说，我们改变了实现但是保持接口不变。这种方式为已有程序开发支持 CUDA 的版本提供了解决思路。

3.3　标准操作流程

看过以上典型的程序流程，我们来分析一下流程哪些部分是 CUDA 带来的开销，哪些能带来收益。开销应该是十分明显的：创建镜像数组并且在设备端和主机端传输数据，这些都是在串行计算中不需要进行的额外工作。为了抵消这些内存操作的“额外开销”，我们能从 GPU 成百上千的处理器核心上得到运算上的收益。这些讨论直接引出了一些使用 CUDA 的推荐策略：

- 一次性将你的数据复制到设备端。
- 启动一个执行了大量工作的核函数（因此从大量并行化中获得的收益将大大超出内存传输的代价）。
- 只将结果复制回主机端一次。

这些并不是一成不变的，但是它们提供了一些有用的规则。在这里，你拥有执行这个典型工作流的所有工具。然而，在我们接触更复杂问题之前，让我们快速浏览一些可以简化开发流程的其他可选方案。

3.4　简化操作流程

上面所述的标准操作流是主流的工作方式，然而其中的部分过于死板和烦琐，因此一些 NVIDIA 专家一起努力提供了一个可替代的流式方案，叫作统一内存（unified memory）。这个方法打破了主机内存和设备内存的围墙，因此你可以只用一个可以从主

机端和设备端共同访问的数组（至少看起来是这样的）。

3.4.1 统一内存和托管数组

有一个好消息是统一内存把你从创建一个数组的不同拷贝（在主机和设备中）和显式的 CPU 和 GPU 间传输数据中解放出来。你可以建立一个设备和主机都能访问的托管数组来代替以前的工作。实际上，数组中的数据仍然需要在主机和设备间传输，但是 CUDA 系统会调度并执行这些操作，而不需要你去管理。注意托管内存数组会带来一个开发效率和执行效率的折衷。让系统去管理数据传输可能会简化并加速你的应用开发进程，但是自动管理的数据传输不能保证会像你手工管理得一样好。在开发过程中，你可能会发现编码的瓶颈在数据传输部分，这时决定进行显式的分配内存和传输数据是明智的。关于这个话题的更多内容我们会在后面详细讨论性能时提及，但现在我们只关心最基本的统一内存知识并实现一个托管内存版本的距离计算应用。

关于统一内存的一些重要事项：

- 统一内存对系统有特殊的要求。包括计算能力 3.0 以上的 GPU 和 64 位的 Linux 或者 Windows 系统 [1]（目前 OS X 系统不支持统一内存）。
- 另一个可能的巨大收益是对于那些喜欢面向对象 C++ 编程的读者，统一内存更适合于处理传输结构数据并有助于避免深拷贝（deep copy）问题 [2]。
- 目前统一内存访问还是从软件层面进行模拟，当 GPU 转成硬件层面实现时，托管数组的性能损耗可能会降低。

3.4.2 使用 cudaMallocManaged() 实现的距离应用

当我们并行化 dist_v2 来创建 dist_v2_cuda 时，我们遵循了主流的范式：创建一些额外的数组并显式地使用 cudaMemcpy() 函数在主机和设备间通过 PCIe 总线进行数据的输入和输出。

这里我们来看一下流式实现的细节。它可以使得一个数组在设备和主机上都能访问统一内存。CUDA 的 C 语言拓展中最关键的部分是函数 cudaMallocManaged()，我们将使用这个函数来为托管内存分配空间（在定义了指针以后，与 cudaMalloc() 函数使用方法类似）。

我们再次简化讨论的内容，直接浏览一个使用统一内存优化的距离应用的代码，代码参见代码清单 3.7，其中有一些修改之处值得注意：

- distanceArray() 函数的代码量被大大减少了，因为我们不需要再分配内存并显式地进行数据传输，所有必须做的事情就是启动一个计算用的核函数（并且调用 cudaDeviceSynchronize() 保证在返回之前计算已经完成）。
- 只在一个地方创建了所有相关的数组。在 main() 函数中，定义 in 和 out 指针。使用 cudaMallocManaged() 函数为数组分配统一内存，然后在计算完成后使用 cudaFree() 函数来释放内存。
- 为了简化介绍，我们选择将所有的代码放在一个文件中。这一次我们创建一个 .cu 文件，而不是像第 1 章中那样使用不包含 CUDA 语言拓展的只包含纯 C 代码的 .cpp 文件。
- 注意每个函数的修饰符：
  - scale() 在 CPU 执行的 for 循环内调用，来生成输入数据。合适的修饰符是 __host__ 但是我们可以不添加它，因为它是默认的标识符。
  - distance() 函数将从核函数中调用并在 GPU 上执行，因此需要添加 __device__ 标识。
  - distanceKernel() 如所有的核函数一样，是从主机上调用而在设备上执行，因此需要使用修饰符 __global__，并且设置返回类型是 void。

代码清单3.7　dist_v2_cuda_unified/kernel.cu

```
#include <stdio.h>
#define N 64
#define TPB 32

float scale(int i, int n)
{
  return ((float)i)/(n - 1);
}

__device__
float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

__global__
void distanceKernel(float *d_out, float *d_in, float ref)
{
  const int i = blockIdx.x*blockDim.x + threadIdx.x;
  const float x = d_in[i];
  d_out[i] = distance(x, ref);
  printf("i = %2d: dist from %f to %f is %f.\n", i, ref, x, d_out[i]);
}

int main()
{
```

```
27   const float ref = 0.5f;
28   // Declare pointers for input and output arrays
29   float *in = 0;
30   float *out = 0;
31
32   // Allocate managed memory for input and output arrays
33   cudaMallocManaged(&in, N*sizeof(float));
34   cudaMallocManaged(&out, N*sizeof(float));
35
36   // Compute scaled input values
37   for (int i = 0; i < N; ++i)
38   {
39     in[i] = scale(i, N);
40   }
41
42   // Launch kernel to compute and store distance values
43   distanceKernel <<<N/TPB, TPB>>>(out, in, ref);
44   cudaDeviceSynchronize();
45
46   // Free the allocated memory
47   cudaFree(in);
48   cudaFree(out);
49
50   return 0;
51 }
```

又到了需要编译并运行应用的时候，我们可以检查运算结果是否正确。（Linux 下使用的 Makefile 文件参见代码清单 3.2，也可以用来构建这个项目。）我们在第 22 行中包含一个 printf() 语句来在控制台上打印结果，你也可以通过 debug 工具来观察结果。

在 Linux 下，也可以使用 cuda-gbd 命令来观察托管数组中的值，就像观察任何一个变量的值一样（使用 print out [0]@64 ）。

使用 Windows 下的 Visual Studio，你可以在 Locals 窗口中看到，但是在那里只能查看一个值。你可以使用如下的步骤查看其他的值：

1. 在第 47 行设置一个断点，正好处于释放内存之前。
2. 开始调试（或按 F5 功能键）。
3. 选择 DEBUG ⇒ QuickWatch（或按下 Shift+F9 组合键）。
4. 当 QuickWatch 窗口打开时，输入 out, 64。
5. 点击名称旁边的三角形按钮来打开 out 数组中的成员列表。

关于 CUDA 专用的调试工具，请参见附录 D。

3.5 本章小结

在本章中，我们使用 CUDA 依次创建了 dist_v1 与 dist_v2 的并行化版本——dist_

v1_cuda 与 dist_v2_cuda。

dist_v2_cuda 为 CUDA 应用标准工作流程提供范例。下面是一些简明的内容回顾（CUDA 应用包括的大致步骤）：

- 创建主机端输入和输出的数组，为输入数据和结果提供存储空间。
- 在设备端上为类似的输入和输出数组声明指针并分配内存。
- 将输入数据从主机端复制到设备端对应的数组上。
- 启动一个核函数在设备端上进行计算并将结果写入设备端的输出数组。
- 将结果从设备端的数组复制到主机端对应的数组上。
- 释放为数组分配的内存。

恭喜你，你已经迈入了大规模并行计算的大门。现在你应该能够开始修改一些样例应用和 CUDA 样例来创建你自己的 CUDA 应用了。注意我们用“需知”（Need-to-know）的理念来尽量快速和简明地完成 CUDA 的入门部分。我们实现了标准的 CUDA 操作流程（其中包括在主机端和设备端分别存储一份数据以及显式的数据传输的函数调用），也实现了使用统一内存这一简便方法来让开发方法变得尽可能简单。

虽然我们快速实现了最初希望利用 CUDA 能力的目标，但是值得注意的是我们跳过了一些应该考虑的重要问题（错误处理、CUDA 调试、计时以及性能分析等），当你开发较大的 CUDA 项目时。这些是你需要（至少是想要）知道的。这些主题将在附录 D 中讨论，读者现在已经掌握了阅读它们的背景知识。在这里，我们推荐读者去进行下面章节的学习，并在需要掌握关于错误处理、调试、计时或者性能分析的相关细节知识时，再去参阅附录 D 中一些实践技巧的内容。

3.6　推荐项目

1. 改变距离数组中的元素数目并进行实验。当你将数目 N 定义成 128、1024、63、65 的时候是否遇到了一些问题？
2. 计算包含 4096 个距离的距离数组并尝试改变 TPB。你可以在系统上运行的最大（和最小）线程块大小是多大？注意，这个问题的答案依赖于你的 GPU 设备的计算能力。参考《 CUDA C Programming Guide 》[1] 来检查你的结果是否和其中的“每个线程块最大线程数目”的表格相同。

参考文献

[1] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[2] Mark Harris, "Unified Memory in CUDA 6," *Parallel Forall* (blog), November 2013. http://devblogs.nvidia.com/parallelforall/unified-memory-in-cuda-6/.

第 4 章 Chapter 4

二维网格与交互式图形

本章关注如何将 CUDA 的并行模型自然地扩展到二维空间（2D）。我们首先来学习关于启动二维计算网格的基础知识，并且创造一个内核骨架。你可以向这个内核骨架加入要想的功能来计算一个二维网格中的数据。然后，为了详细说明内核程序，我们创建一个应用程序 dist_2d，这个程序用来计算平面上一个参考点到二维网格点阵（点阵中的成员规格一致）中每个成员的距离。通过将网格中的点对应为图像中的像素，我们可以计算出一幅新图像，图像中每个像素的灰度值以距离值为基础。

生成图像数据时，最好利用 CUDA 的图形互操作能力（简称为图形互操作）。图形互操作支持与标准图形应用程序接口（API）间的协作，包括 Direct3D[1] 和 OpenGL[2]。我们将使用 OpenGL，并且坚持够用就好的原则，只介绍我们需要知道的方法。因此，这里将只提供 OpenGL 的一些必要的东西，用来以实时交互方式在屏幕上显示你的结果。

到本章结束时，你将会成功运行一个 flashlight（手电筒）应用程序，你可以交互式地展示一张阴影图像，可以通过移动鼠标或者键盘输入来移动参考点，而阴影由到参考点的距离控制。除此之外，你还会成功运行一个 stability（稳定性）应用程序，交互式地显示成百上千个振子的动态运动的数值模拟结果。这些实验可以引领你开始创造属于你自己的 CUDA 交互式应用程序，并帮助你在实践中抓住要点，把握方向。

4.1 启动二维计算网格

这里，我们扩展之前涉及的一维数组的例子（这些点规则地分布在一条直线段上），并且进一步考虑点均匀地分布在二维平面的一块空间中的应用程序。我们会遇到其他一些适用于这种方案的应用程序（例如，模拟热传导），其中，数字图像处理是最常见（可能也是最直观）的示例。利用这个直观的联系，我们将通过图像处理术语来介绍这些概念，所有的概念都可以直接迁移到其他应用程序中。

一幅数字栅格图像由一组图像元素或像素组成。这些像素被安放在一个统一的二维矩阵网格中，其中，每个像素都具有一个量化的强度值。具体来说，我们将宽度、高度分别用 x、y 坐标表示，并且将图像定义为宽 W 像素，高 H 像素。如果在每个像素中存储的量化强度值是一个数字，那么可以将图像数据用一个 W×H 大小的矩阵来表示。

在 CUDA 中从一维问题过渡到二维问题时，我想你一定会感到很惊讶，因为仅仅需要很少的调整就可以做到这种转变。在一维中，我们指定线程块和网格的大小为整数，并且根据公式，利用 blockDim.x、blockIdx.x 和 threadIdx.x 计算出索引值 i。

```
int i = blockIdx.x*blockDim.x + threadIdx.x;
```

这里，我们重新诠释表达式的右边并定义为一个新索引 c。c 用来标记每个像素所在列的索引。（当我们从左向右遍历一个像素行时，c 从最小值 0，增大到最大值 W–1。）我们还引入了一个索引 r 来标记像素的行索引（范围从 0 到 H–1）。行索引的计算和列索引一样，但是使用“.y”（代替了列索引中的“.x”），所以列索引和行索引的计算如下：

```
int c = blockIdx.x*blockDim.x + threadIdx.x;
int r = blockIdx.y*blockDim.y + threadIdx.y;
```

为了使数据存储和传输更加简单，我们继续在一个“扁平的”一维数组中存储和传输数据。因此，我们需要多用一个新整型变量来索引一维数组。我们将这个变量叫作 i。注意，i 在这里的作用与一维数组中一致，但是在其他地方（包括 CUDA 样例中），你将看到一些命名为 idx、flatIdx 和 offset 的变量索引一维数组。我们在一维数组中是以行为主序来存储矩阵元素，也就是说，从矩阵第 0 行开始存储数据，然后存储第 1 行的数据，以此类推——因此，一维数组中索引 i 的计算如下：

```
int i = r*w + c;
```

为了描述一个可以直观对应一个图像（或矩阵，或者其他常规二维离散化表示）的二维计算网格，我们用含有两个特殊成员的 dim3 变量指定线程块和网格的大小。回忆

一下，在一次内核调用中，包含在 3 对尖括号内的 1 个数值会看成是一个 dim3 变量中的 .x 成员，而未指定的 .y 和 .z 成员取默认值 1。在此处的二维空间中，我们指定 .x 和 .y 成员。dim3 中的 .z 成员（在这里取默认值 1）将在我们开始学习三维网格时发挥作用（参见第 7 章）。

话不多讲，我们将安排一些必要的语法学习，然后直接开始使用一个二维网格进行像素值的并行计算。

4.1.1　二维内核启动的语法

二维内核的启动仅在执行配置方面与一维内核有所不同。计算一幅图像的数据包含 W 列和 H 行，并且我们可以在二维线程块中用 x 方向的 TX 个线程和 y 方向上的 TY 个线程进行计算。你可以选择将二维网格组织成一维网格，但是你将遇到最大线程块维度的限制和单个线程块中线程数的限制。（详情请见《CUDA C Programming Guide》[3]。）

我们用一个声明来指定二维线程块的大小：

```
dim3 blockSize(TX, TY); // Equivalent to dim3 blockSize(TX, TY, 1);
```

然后，像一维那样，我们计算 x、y 方向上的线程块数（bx 和 by）：

```
int bx = (W + blockSize.x - 1)/blockSize.x ;
int by = (H + blockSize.y - 1)/blockSize.y ;
```

指定网格大小（在线程块中）的语法是：

```
dim3 gridSize = dim3(bx, by);
```

掌握了这些细节，我们就可以开始启动内核程序了：

```
kernelName<<<gridSize, blockSize>>>(args)
```

4.1.2　定义二维内核

一个二维网格内核的原型或者声明看起来和以前的一维网格内核完全一样：以限定符 __global__ 开始，然后是返回类型 void，接着是一个合法的函数名，例如 kernel2D，最后以用逗号分隔的有类型参数列表结束（最好包含一个指向存储输出图像数据数组 d_out 的指针，以及图像的宽度、高度和其他必要输入）。kernel2D 函数首先计算行、列和平面索引，并且测试行和列的索引值是否与图像中的像素点相对应。剩下的就是计算像素值。

把这些代码段组合到一起，在代码清单 4.1 中给出了典型的二维内核。

代码清单4.1 二维网格的内核启动“骨架”代码。用你自己的代码代替INSERT_CODE_HERE来计算输出值

```
__global__
void kernel2D(float *d_out, int w, int h, … )
{
  // Compute column and row indices.
  const int c = blockIdx.x * blockDim.x + threadIdx.x;
  const int r = blockIdx.y * blockDim.y + threadIdx.y;
  const int i = r * w + c; // 1D flat index

  // Check if within image bounds.
  if ((c >= w) || (r >= h))
    return;

  d_out[i] = INSERT_CODE_HERE; // Compute/store pixel in device array.
}
```

变量名大写的注意事项

我们需要引用参数值，例如图像的宽度和高度，这些参数值在函数定义中被认为是输入变量。但是，函数调用的参数值通常用 #define 将其指定为一个常量（恒定值）。我们将按照普遍的习惯，用大写字母表示常量，用相同的小写名称表示输入变量。例如，代码清单 4.1 中 kernel2D() 函数的原型如下所示：

```
void kernel2D(uchar4 *d_out, int w, int h, … )
```

而对该函数的调用如下所示：

```
#define W 500
#define H 500
kernel2D<<<gridSize, blockSize>>>(d_out, W, H, … )
```

表明宽度和高度的输入值为常数，这里均为 500。

在这里，有一个值得关注处理的细节，即图像的常见数据类型。每个像素的量化值存储都是 uchar4 型的。uchar4 型是一种存储四个无符号字符值的向量（每个字符占一个字节的存储空间）。实际上，可以把 uchar4 的四个成员（通常用后缀 .x、.y、.z 和 .w 表示）分别看作红、绿、蓝和 alpha（透明度）四个显示通道，每个通道的指定整数范围为 0～255。这种描述图像像素值的格式通常简称为 RGBA。

把代码段整合到一起，在代码清单 4.2 中给出了计算图像的典型二维内核结构。

代码清单 4.2 计算图像数据的“骨架”代码。计算时，RED_FORMULA、GREEN_FORMULA和BLUE_FORMULA应该用你自己的代码替换，每个颜色通道的取值范围为0~255

```
__global__
void kernel2D(uchar4 *d_output, int w, int h, … )
```

```
{
  // Compute column and row indices.
  int c = blockIdx.x*blockDim.x + threadIdx.x;
  int r = blockIdx.y*blockDim.y + threadIdx.y;
  int i = r * w + c; // 1D flat index

  // Check if within image bounds.
  if ((r >= h) || (c >= w)) {
      return;
  }

  d_output[i].x = RED_FORMULA;     //Compute red
  d_output[i].y = GREEN_ FORMULA; //Compute green
  d_output[i].z = BLUE_ FORMULA;  //Compute blue
  d_output[i].w = 255; // Fully opaque
}
```

4.1.3　dist_2d

让我们将二维网格的综合讨论与我们之前的那个距离应用程序结合起来，编写一个产生二维数组的应用程序，其中，二维数组中的数据是距参考点的距离。然后，我们将这个应用程序改写为一个存储 RGBA 图像数据的数组。代码清单 4.3 提供了在二维网格中计算距离的所有代码。

代码清单4.3　在二维网格中计算距离

```
#define W 500
#define H 500
#define TX 32 // number of threads per block along x-axis
#define TY 32 // number of threads per block along y-axis

__global__
void distanceKernel(float *d_out, int w, int h, float2 pos)
{
  const int c = blockIdx.x*blockDim.x + threadIdx.x;
  const int r = blockIdx.y*blockDim.y + threadIdx.y;
  const int i = r*w + c;
  if ((c >= w) || (r >= h)) return;

  // Compute the distance and set d_out[i]
  d_out[i] = sqrtf((c - pos.x)*(c - pos.x) +
                   (r - pos.y)*(r - pos.y));
}

int main()
{
  float *out = (float*)calloc(W*H, sizeof(float));
  float *d_out; // pointer for device array
  cudaMalloc(&d_out, W*H*sizeof(float));

  const float2 pos = {0.0f, 0.0f}; // set reference position
  const dim3 blockSize(TX, TY);
  const int bx = (W + TX - 1)/TX;
  const int by = (W + TY - 1)/TY;
```

```
29    const dim3 gridSize = dim3(bx, by);
30
31    distanceKernel<<<gridSize, blockSize>>>(d_out, W, H, pos);
32
33    // Copy results to host.
34    cudaMemcpy(out, d_out, W*H*sizeof(float), cudaMemcpyDeviceToHost);
35
36    cudaFree(d_out);
37    free(out);
38    return 0;
39 }
```

在上述代码中，第 6～17 行是内核函数，它和代码清单 4.1 的代码很相似，但它的结果是用勾股定理计算位置 {c, r} 和参考位置 pos 的距离而得出的。(注意，我们已经把 pos 定义成 float2 类型，因此，pos 可以存储参考位置的两个坐标 {pos.x, pos.y}。) 代码清单中其余的代码解释如下：第 19～39 行给出了 main() 函数的详细内容，main() 函数开始时，把一个合适大小的输出数组初始化为 0。第 22 行和第 23 行声明了一个指向设备端数组 d_out 的指针，并且用 cudaMalloc() 函数申请了内存空间。第 25 行设置了参考位置，第 26～29 行设置了内核启动参数：一个 bx×by 个线程块规格的二维网格，其中每个线程块有 TX×TY 个线程。第 31 行启动内核计算距离值，并在第 34 行复制到主机端的 out 数组。第 36 行和第 37 行释放申请的设备端和主机端的内存，接下来，main() 函数返回 0，标志着函数结束。

下面，我们做一些小的改变。根据距离图像产生一个计算 RGBA 值数组的应用程序。完整代码见代码清单 4.4。

代码清单4.4　基于二维参考点距离的图像数据的并行计算

```
 1 #define W  500
 2 #define H  500
 3 #define TX 32 // number of threads per block along x-axis
 4 #define TY 32 // number of threads per block along y-axis
 5
 6 __device__
 7 unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }
 8
 9 __global__
10 void distanceKernel(uchar4 *d_out, int w, int h, int2 pos)
11 {
12   const int c = blockIdx.x*blockDim.x + threadIdx.x;
13   const int r = blockIdx.y*blockDim.y + threadIdx.y;
14   const int i = r*w + c;
15   if ((c >= w) || (r >= h)) return;
16
17   // Compute the distance (in pixel spacings)
18   const int d = sqrtf((c - pos.x) * (c - pos.x) +
19                       (r - pos.y) * (r - pos.y));
20   // Convert distance to intensity value on interval [0, 255]
21   const unsigned char intensity = clip(255 - d);
```

```
22
23    d_out[i].x = intensity; // red channel
24    d_out[i].y = intensity; // green channel
25    d_out[i].z = 0; // blue channel
26    d_out[i].z = 255; // fully opaque
27 }
28
29 int main()
30 {
31    uchar4 *out = (uchar4*)calloc(W*H, sizeof(uchar4));
32    uchar4 *d_out; // pointer for device array
33    cudaMalloc(&d_out, W*H*sizeof(uchar4));
34
35    const int2 pos = {0, 0}; // set reference position
36    const dim3 blockSize(TX, TY);
37    const int bx = (W + TX - 1)/TX;
38    const int by = (W + TY - 1)/TY;
39    const dim3 gridSize = dim3(bx, by);
40
41    distanceKernel<<<gridSize, blockSize>>>(d_out, W, H, pos);
42
43    // Copy results to host.
44    cudaMemcpy(out, d_out, W*H*sizeof(uchar4), cudaMemcpyDeviceToHost);
45
46    cudaFree(d_out);
47    free(out);
48    return 0;
49 }
```

这里的距离是用像素间的间距计算的，所以这里参考位置 pos 的类型是 int2 型，距离 d 的类型是 int 型。将距离值转变为无符号字符类型的强度值，用 clip() 函数将这些强度值限制在允许的范围 0~255。输出数组 out 和 d_out 与之前的一致，都是 uchar4 类型的。d_out[i].x = intensity 和 d_out[i].y = intensity 的任务是在红色和绿色通道中存储强度值，来产生一个黄色的距离图像。（我们把蓝色成员设置为 0，把 alpha 设置为 255，使其完全不透明。但你还是应该用其他颜色规格试验一下。）

4.2　通过图形交互实时显示

既然能构造产生图像数据的应用程序，那么我们就能显示那些图像并探索实时进行 CUDA 的最大并行化可以让我们做到什么，这是非常有意义的事。

实时图像交互涉及 CUDA 为交互提供的一个标准图形包。我们将使用 OpenGL，它可能是（或者说已经是）许多书籍的主题 [2, 4, 5]，因此我们会采用“需知”的处理思路。我们仅仅介绍使用 OpenGL 来显示单一纹理矩形的必备知识，并且提供少量代码实例，这些实例通过使用 OpenGL 实用工具包（GLUT）来显示如何通过键盘和鼠标控制交互。基本思想是，通过矩形提供了窥探应用程序世界的一个窗口，并且，你可以使用 CUDA，

根据你想让用户看到的场景来计算像素的阴影值。CUDA/OpenGL 的互操作提供了交互式的控制，并且，像显示的矩形纹理一样，实时显示变化的场景（或者，更准确地说，其刷新速率可以和 ~60Hz 刷新速率的典型现代视觉显示系统相比）。

这里我们给出一个示例应用程序的代码。这个应用程序可以实现打开一个图像窗口并且交互式地显示图像，其中图像元素值源自到参考点的距离。我们可以使用鼠标或者键盘交互式地改变参考点位置。我们把这个应用程序命名为 flashlight（手电筒），因为它产生了一个可移动的光圈，并且越远离“光点”中心，光强度就越弱。图 4.1 展示了这个应用程序运行后的截屏图。

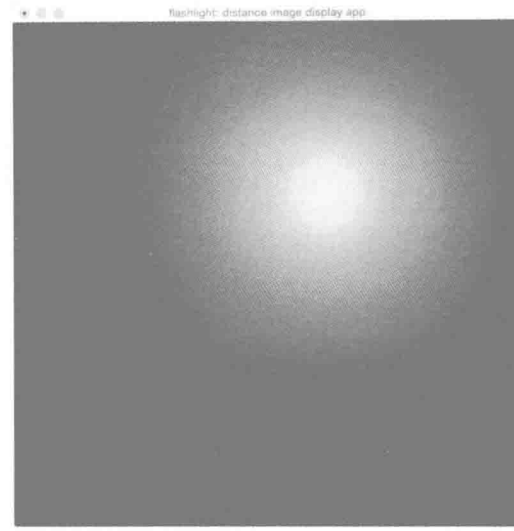


图 4.1 应用程序运行后的交互式光点

整个应用程序少于 200 行代码，我们把代码组织成三个文件：

- main.cpp 包含 CUDA/OpenGL 建立和互操作的基本要素。包含大概 100 行代码（约为总数的一半），当我们对其内容进行简要解释后，你应该有能力以 flashlight（手电筒）为模板，对 main.cpp 做少量改动后创造出属于自己的应用程序。
- kernel.cu 包含必要的 CUDA 代码，包括上面描述过的 clip() 函数，kernelLauncher() 函数的定义和实际内核函数（这里是 distanceKernel()）的定义，这个内核函数必须将输入写入到一个 uchar4 的数组中。
- interactions.h 定义了回调函数 keyboard()、mouseMove() 和 mouseDrag() 来指定系统如何应答输入。当我们浏览整个代码时，最重要的是你能够以 flashlight（手电筒）为模板来快速创造出你自己的应用程序，仅仅需要以下几步：

1. 通过对 Linux 下代码目录的复制，或者通过在 Windows 下的 Visual Studio 创建一个以 flashlight（手电筒）为模板的新工程。

2. 编辑内核函数来产生你想要显示的数据。

3. 在 interactions.h 中编辑回调函数来指定你的应用程序应该如何应答键盘和鼠标的输入，并且编辑 printInstructions() 来为用户的交互定制指令。

4. 除了以上这些步骤，你还可以在 interactions.h 中编辑 #define TITLE_STRING 声明来在图像窗口的标题栏中定制应用程序的名字。

代码清单 4.5 至代码清单 4.8 展示了用 CUDA/OpenGL 的互操作在你的屏幕上显示距离图像的所有必需代码，我们还将带你熟悉一些必要的代码而不过分在意过多的细节。

代码清单4.5　flashlight/main.cpp

```
#include "kernel.h"
#include <stdio.h>
#include <stdlib.h>
#ifdef _WIN32
#define WINDOWS_LEAN_AND_MEAN
#define NOMINMAX
#include <windows.h>
#endif
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/freeglut.h>
#endif
#include <cuda_runtime.h>
#include <cuda_gl_interop.h>
#include "interactions.h"

// texture and pixel objects
GLuint pbo = 0;     // OpenGL pixel buffer object
GLuint tex = 0;     // OpenGL texture object
struct cudaGraphicsResource *cuda_pbo_resource;

void render() {
  uchar4 *d_out = 0;
  cudaGraphicsMapResources(1, &cuda_pbo_resource, 0);
  cudaGraphicsResourceGetMappedPointer((void **)&d_out, NULL,
                                       cuda_pbo_resource);
  kernelLauncher(d_out, W, H, loc);
  cudaGraphicsUnmapResources(1, &cuda_pbo_resource, 0);
}

void drawTexture() {
  glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, W, H, 0, GL_RGBA,
               GL_UNSIGNED_BYTE, NULL);
  glEnable(GL_TEXTURE_2D);
  glBegin(GL_QUADS);
  glTexCoord2f(0.0f, 0.0f); glVertex2f(0, 0);
  glTexCoord2f(0.0f, 1.0f); glVertex2f(0, H);
  glTexCoord2f(1.0f, 1.0f); glVertex2f(W, H);
  glTexCoord2f(1.0f, 0.0f); glVertex2f(W, 0);
  glEnd();
  glDisable(GL_TEXTURE_2D);
}

void display() {
  render();
  drawTexture();
  glutSwapBuffers();
}
```

```
51
52 void initGLUT(int *argc, char **argv) {
53   glutInit(argc, argv);
54   glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);
55   glutInitWindowSize(W, H);
56   glutCreateWindow(TITLE_STRING);
57 #ifndef __APPLE__
58   glewInit();
59 #endif
60 }
61
62 void initPixelBuffer() {
63   glGenBuffers(1, &pbo);
64   glBindBuffer(GL_PIXEL_UNPACK_BUFFER, pbo);
65   glBufferData(GL_PIXEL_UNPACK_BUFFER, 4*W*H*sizeof(GLubyte), 0,
66                GL_STREAM_DRAW);
67   glGenTextures(1, &tex);
68   glBindTexture(GL_TEXTURE_2D, tex);
69   glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
70   cudaGraphicsGLRegisterBuffer(&cuda_pbo_resource, pbo,
71                                cudaGraphicsMapFlagsWriteDiscard);
72 }
73
74 void exitfunc() {
75   if (pbo) {
76     cudaGraphicsUnregisterResource(cuda_pbo_resource);
77     glDeleteBuffers(1, &pbo);
78     glDeleteTextures(1, &tex);
79   }
80 }
81
82 int main(int argc, char** argv) {
83   printInstructions();
84   initGLUT(&argc, argv);
85   gluOrtho2D(0, W, H, 0);
86   glutKeyboardFunc(keyboard);
87   glutSpecialFunc(handleSpecialKeypress);
88   glutPassiveMotionFunc(mouseMove);
89   glutMotionFunc(mouseDrag);
90   glutDisplayFunc(display);
91   initPixelBuffer();
92   glutMainLoop();
93   atexit(exitfunc);
94   return 0;
95 }
```

这里对在 main.cpp 中发生的事情作简要概述。第 1～17 行加载合适的头文件，来使你的操作系统访问必要的支持代码。其余的解释应当从底部开始讲起。第 82～95 行定义 main() 函数，main() 函数中做了以下事情：

- 第 83 行将一些用户界面指令打印到命令窗口。
- initGLUT 初始化 GLUT 库，并且设置图像窗口的规格，包括显示模式（RGBA）、缓冲区（double 型）、尺寸（W×H）和标题。
- gluOrtho2D(0, W, H, 0) 建立视觉变换（简单的投影）。

- 第 86~89 行表明键盘和鼠标的交互会用一些函数来指定，函数包括 keyborad、handleSpecialKeypress、mouseMove 和 mouseDrag（这些函数的细节在 interactions.h 中指定）。
- glutDisplayFunc(display) 指出，函数 display() 决定了在窗口中显示的内容，display() 函数有整整三行代码。第 47～49 行调用 render() 函数来计算新的像素值，调用 drawTexture() 函数来画出 OpenGL 的纹理，然后交换显示缓冲区。
- drawTexture() 函数建立一个二维 OpenGL 纹理图像，创建一个四边形图像，初始纹理坐标为（0.0f, 0.0f,），（0.0f, 1.0f），（1.0f, 1.0f）和（1.0f, 0.0f），实际是单位正方形的四个角对应的像素坐标（0, 0），（0, H），（W, H）和（W, 0）。
- 双缓冲是一种用来提高图形程序效率的常见技术。一个缓冲区提供可被读取用于显示的内存，与此同时，另一个缓冲区提供一段内存，保证下一帧的内容能够被写入。在图形序列的帧与帧之间，两个缓冲区交换它们的读 / 写角色。
- initPixelBuffer() 在第 62~72 行初始化像素缓冲区，这并不出乎意料。对于我们的目的来说，最关键的是最后一行，用 CUDA “注册” OpenGL 缓冲区。这个操作会有一些开销，但是它可以做到低开销 “映射”，如果映射成功，则将缓冲区内存的控制交给 CUDA 来进行写输出。如果没有映射成功，则返回缓冲区内存的控制给 OpenGL 用于显示。图 4.2 总结了 CUDA 和 OpenGL 间的互操作。

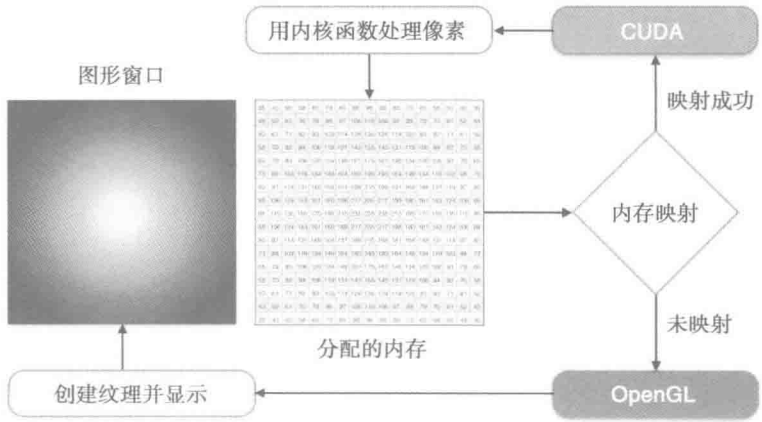


图 4.2　两种不同方式访问设备内存的图示，若映射成功，跳转到 CUDA 来存储计算结果；若未映射（即返回 OpenGL 控制）则显示结果

- glutMainLoop()，第 92 行是真正行动的开始。它反复检查输入，并且需要一种通

过调用 render 函数的 display 函数更新图像的计算，如下：

- 将像素缓冲区映射到 CUDA，并且获取一个指向缓冲区内存的 CUDA 指针，这个指针可以作为设备端输出数组。
- 调用包装函数 kernelLauncher()，这个函数启动内核来计算像素值从而更新图像。
- 映射失败而用 OpenGL 显示内容。

- 退出应用程序时，atexit(exitfunc) 执行最终清理工作，它在 main() 中返回 0（意味着函数的结束）之前取消资源的注册，并且删除 OpenGL 的像素缓冲和纹理。

在 main.cpp 的所有代码中，当你想创造属于自己的 CUDA/OpenGL 互操作应用程序时，唯一需要改变的就是 render() 函数。在这个函数中，你需要更新 kernelLauncher() 函数的参数列表。

代码清单4.6 flashlight/kernel.cu

```
#include "kernel.h"
#define TX 32
#define TY 32

__device__
unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }

__global__
void distanceKernel(uchar4 *d_out, int w, int h, int2 pos) {
  const int c = blockIdx.x*blockDim.x + threadIdx.x;
  const int r = blockIdx.y*blockDim.y + threadIdx.y;
  if ((c >= w) || (r >= h)) return; // Check if within image bounds
  const int i = c + r*w; // 1D indexing
  const int dist = sqrtf((c - pos.x)*(c - pos.x) +
                         (r - pos.y)*(r - pos.y));
  const unsigned char intensity = clip(255 - dist);
  d_out[i].x = intensity;
  d_out[i].y = intensity;
  d_out[i].z = 0;
  d_out[i].w = 255;
}

void kernelLauncher(uchar4 *d_out, int w, int h, int2 pos) {
  const dim3 blockSize(TX, TY);
  const dim3 gridSize = dim3((w + TX - 1)/TX, (h + TY - 1)/TY);
  distanceKernel<<<gridSize, blockSize>>>(d_out, w, h, pos);
}
```

代码清单 4.6 中，kernel.cu 的代码应该看起来很熟悉，不需要过多解释。主要变化就是一个包装函数 kernelLauncher()，这个函数计算网格维度，并且启动内核。注意，你将找不到任何关于主机端输出数组的内容。计算和显示都是在设备端处理的，不需要再把数据传回主机端。（这种通过 PCIe 总线进行的大量图像数据传输很耗时，而且大大抑

制了实时交互能力。）你也找不到 cudaMalloc() 来为设备端数组申请空间。main.cpp 中的 render() 函数声明了一个指针 d_out，它从 cudaGraphicsResourceGetMappedPointer() 获取值，并且提供指向为像素缓冲区分配内存的 CUDA 指针。

与内核相关的头文件在代码清单 4.7 中给出。除了包含保护函数和内核函数原型外，kernel.h 还包含 uchar4 和 int2 的预声明。这样，编译器可以在 CUDA 代码（知道它们的定义）建立或执行前，知道它们的存在。

代码清单4.7　flashlight/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

struct uchar4;
struct int2;

void kernelLauncher(uchar4 *d_out, int w, int h, int2 pos);

#endif
```

代码清单4.8　flashlight/interactions.h指定回调函数来控制flashlight（手电筒）应用程序的交互

```
#ifndef INTERACTIONS_H
#define INTERACTIONS_H
#define W 600
#define H 600
#define DELTA 5 // pixel increment for arrow keys
#define TITLE_STRING "flashlight: distance image display app"
int2 loc = {W/2, H/2};
bool dragMode = false; // mouse tracking mode

void keyboard(unsigned char key, int x, int y) {
  if (key == 'a') dragMode = !dragMode; // toggle tracking mode
  if (key == 27)  exit(0);
  glutPostRedisplay();
}

void mouseMove(int x, int y) {
  if (dragMode) return;
  loc.x = x;
  loc.y = y;
  glutPostRedisplay();
}

void mouseDrag(int x, int y) {
  if (!dragMode) return;
  loc.x = x;
  loc.y = y;
  glutPostRedisplay();
}

void handleSpecialKeypress(int key, int x, int y) {
  if (key == GLUT_KEY_LEFT)  loc.x -= DELTA;
  if (key == GLUT_KEY_RIGHT) loc.x += DELTA;
```

```
33    if (key == GLUT_KEY_UP)     loc.y -= DELTA;
34    if (key == GLUT_KEY_DOWN)   loc.y += DELTA;
35    glutPostRedisplay();
36  }
37
38  void printInstructions() {
39    printf("flashlight interactions\n");
40    printf("a: toggle mouse tracking mode\n");
41    printf("arrow keys: move ref location\n");
42    printf("esc: close graphics window\n");
43  }
44
45  #endif
```

flashlight 应用程序开发的目标是显示一个可交互式改变的图像。每个图像元素对应该位置到参考点的距离，这个参考点可以被交互式地移动。现在我们准备开始定义和实现互动。代码清单 4.8 中给出的 interactions.h 代码允许用户通过移动鼠标或按下箭头键来移动参考点（即手电筒光束的中心）。按下切换键在鼠标移动跟踪和鼠标拖曳跟踪（按下鼠标按钮）之间切换。按下 Esc 键来关闭图像窗口。这里简单描述代码所做的事情以及它是如何进行互动的：

- 第 3～6 行，设置了图像的维度、标题栏中显示的文本、每按下箭头键参考点移动的距离（以像素为单位）。
- 第 7 行，设置了初始参考位置 {W/2, H/2}，即图像的中心。
- 第 8 行，声明了一个布尔类型变量 dragMode，初始化为 false。我们使用 dragMode 在鼠标移动跟踪和“点击 – 拖曳”之间来回切换。
- 第 10～14 行，指定用键盘进行定义过的互动：
  - 按下 a 键切换 dragMode 的值来对应鼠标跟踪模式。
  - ASCII 码 27 对应 Esc 键，按下 Esc 键，关闭图像窗口。
  - glutPostRedisplay() 函数在每个回调函数的结尾被调用，根据交互输入计算出一个新的图像用于显示（调用 main.cpp 中的 display()）。
- 第 16～21 行，指定了对鼠标移动的响应。当 dragMode 切换时，return 用来确保不执行任何操作。否则，参考位置变量的成员会在计算和显示更新图像前，被设置成与鼠标位置 x、y 坐标一致（通过 glutPostRedisplay() 实现）。
- 相似地，第 23～28 行，指定了对“点击 – 拖曳”的响应。当 dragMode 为 false 时，return 语句确保不执行任何行动。否则，参考位置将被重置到鼠标拖曳的最后位置。

- 第 30～36 行，指定对具有预设动作的特殊键的响应。（注意，标准键盘互动是基于 ASCII 键码[6]处理的，所以像箭头键和功能键等特殊键，没有对应的标准 ASCII 码，它们需要被分开处理。）flashlight（手电筒）应用程序运行起来，按下箭头键，参考位置将在期望方向上移动 DELTA 个像素。
- 第 38～43 行的 printInstructions() 函数包括打印提示清单，这些清单通过控制台为用户提供交互指令说明。

尽管 flashlight 应用程序所有的代码和解释有数页之多，但不要被吓到。让我们停下来看看事情的本质。当显示总共大约 200 行的代码时，如果不考虑其可读性，那么全部的代码可以被压缩到很少的几行，所以，实际要理解和消化的代码不是非常多。可能更加重要的是，在 main.cpp 中超过一半的代码实际上根本不需要修改，就可以用来创建你自己的应用程序，当然不包括需要改变 kernelLauncher() 函数的参数列表或者定制标题栏要显示的信息。如果一开始以 flashlight 应用程序作为模板，你应该（诚挚推荐）利用 CUDA 的威力来创造你自己的图形交互应用程序，可通过替换内核函数或者在 interactions.h 中修改用户交互方式来实现。

最后，给出 Linux 下创建应用程序所需的 Makefile 文件，见代码清单 4.9。

代码清单4.9　flashlight/Makefile

```
UNAME_S := $(shell uname)

ifeq ($(UNAME_S), Darwin)
  LDFLAGS = -Xlinker -framework,OpenGL -Xlinker -framework,GLUT
else
  LDFLAGS += -L/usr/local/cuda/samples/common/lib/linux/x86_64
  LDFLAGS += -lglut -lGL -lGLU -lGLEW
endif

NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler "-Wall -Wno-deprecated-declarations"

all: main.exe

main.exe: main.o kernel.o
  $(NVCC) $^ -o $@ $(LDFLAGS)

main.o: main.cpp kernel.h interactions.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@

kernel.o: kernel.cu kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

Windows 用户需要改变一个生成的自定义属性，并且添加两个库文件 OpenGL Utility Toolkit (GLUT) 和 OpenGL Extension Wrangler (GLEW)。为了使事情变得简单，

并且确保库版本的一致性，我们发现可以很方便地将库文件复制（可以通过在 CUDA Samples 目录中搜索文件名 freeglut.dll、freeglut.lib、glew64.dll 和 glew64.lib）到工程目录中，然后通过 PROJECT ⇒ Add Existing Item 把它们加入到工程中。

可通过工程属性页改变生成的自定义属性。在解决方案资源管理器中右键单击 flashlight，然后依次选择 Properties ⇒ Configuration Properties ⇒ C/C++ ⇒ General ⇒ Additional Include Directories，并且编辑列表来添加 CUDA Samples 的 common\inc 目录。它的默认安装位置是：C:\ProgramData\NVIDIA Corporation\CUDA Samples\v7.5\common\inc。

4.3 stability 应用程序

为了讲清楚以 flashlight 应用程序作为模板创造更多有趣应用程序的想法，让我们实际动手做一下。这里，我们在 flashlight 的基础上创建一个应用程序，用来分析线性振荡器的稳定性，然后我们将这个应用程序扩展到去处理一般的单一自由度（1DoF）的系统，包括范德堡尔振荡器，一种更有趣的振荡器。

线性振荡器来源于机械学的质量 – 弹簧 – 阻尼器系统的模型（一种 RLC 电路，其性质仅与平衡点附近的所有 1DoF 系统有关）。它的数学模型包含一个二阶常微分方程（ODE），可以用其最简形式（选择合适的时间单位）表示为 $x''+2bx'+x=0$，其中 x 是到平衡点的位移，b 是阻尼常数，上撇号表示时间的导数。为了把问题化简成一个方便处理的形式，我们引入一个新的速度变量 y 将系统转化为两个一阶常微分方程，并且写出这两个给出 x、y 变化速率的一阶常微分方程：

$$x'=y$$

$$y'=-x-2by=f(x, y, t, \cdots)$$

作为伏笔，我们这里做的一切都可以推广到更多不同的 1DoF 振荡器中，只需要在 $f(x, y, t, \cdots)$ 中 y 的右边加入其他表达式。尽管我们能够写出线性振荡器的解析解，但是，在这里，我们着眼于用数值方法来使应用覆盖的范围更广，因此，我们使用有限差分法。有限差分法用离散的时间步长 dt 的倍数来计算数值（所以我们引入 $t_k=k*dt$, $x_k=x(t_k)$ 和 $y_k=y(t_k)$ 作为相关变量），并且用差分近似值代替精确的导数，即 $x' \to (x_{k+1}-x_k)/dt$, $y' \to (y_{k+1}-y_k)/dt$。这里，我们运用最简单的有限差分法，显式欧拉方法的思想是将导数替换

为有限差分表达式，并且根据一个时间步长开始的原值 x_k 和 y_k 求解时间步长结束时的新值 x_{k+1} 和 y_{k+1} 得到：

$$x_{k+1}=x_k+dt*y_k$$

$$y_{k+1}=y_k+dt*(-x_k-2by_k)$$

接下来，我们可以选择一个初始状态 $\{x_0, y_0\}$，并且计算后续连续时间步长的系统状态。

我们刚刚描述了一种源自单一初始化状态的计算解决方案（一系列状态），并且解决方案是完全串行的：这个系列中的状态计算是一个接着一个的。

然而，稳定性并不仅仅依赖于一个初始状态，而是依赖于所有初始状态的解决方案。对于一个稳定的平衡，所有附近的初始状态都会产生能够达到（或者说，至少不远离）平衡的解决方案。找到一个趋向远离平衡的解决方案，意味着它是不稳定的。与动力学和稳定性有关的更多信息，参见参考文献[7, 8]。

这种集体性质形式使稳定性测试很适合并行化：通过启动一个在平衡附近密集取样的初始化状态的计算网格，我们可以测试源于周围初始状态的解决方案。我们发现可以并行地计算成百上千个解决方案。通过 CUDA/OpenGL 的互操作，还可以实时观察结果，并进一步与结果进行互动。

特别地，我们将选择一个网格的初始状态，这些初始状态是通过定期在平衡处进行矩形中心采样得到的。我们将计算对应的解决方案，并且基于距离的微小变化，及来自平衡的 dist_r（距离比率）分配阴影值。为了展示结果，我们将为每个像素分配一个红色通道值，这些值与距离比率成正比（并且被限制为 [0, 255]），我们还会为每个像素分配一个蓝色通道值，这些值与距离比率成反比（同样被限制为 [0, 255]）。产生向平衡态收敛（意味着稳定）的解决方案的初始状态是由蓝色通道所控制的，然而产生远离平衡状态的解决方案的初始状态是由红色通道所控制的，并且收敛 / 远离的转换由蓝色与红色的值是否相等表示，即是否为紫色。

通过颜色调整来提高灰度对比度

鉴于观察到转化成灰度模式的图像时，区分红色（R）和蓝色（B）比较困难，所以这里引入公式 $G=0.3+(R-B)/2$，用绿色通道来提高对比度和亮度。用 stability（稳定性）应用程序生成的全彩图像可通过登录 www.cudaforengineers.com 了解。

在图像窗口显示的结果将由红、蓝或紫色像素组成的区域构成（在水平 x 轴和垂直 y 轴的交汇处使用绿色通道）。图 4.3 给出了同时具有收敛和远离区域的 stability 应用程序的结果预览图。

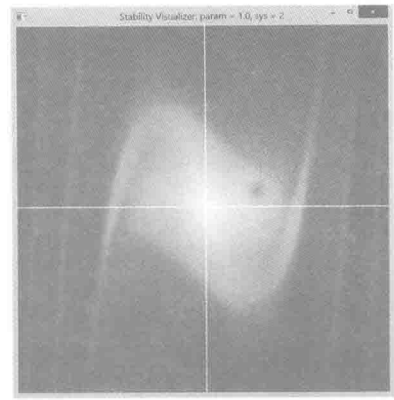


图 4.3 带阴影的 stability 图像，阴影调整为显示一个明亮的中心排斥区域和周围稍暗一些的吸引区域

我们计划产生单一系统的 stability 图像。我们同样会介绍交互，这样就可以观察到不同参数或不同系统下的稳定性的变化情况。

已经了解了关于内核和交互的想法，我们开始看代码。正如之前强调过的，与 flashlight 应用程序相比，stability 应用程序的主要变化包括如代码清单 4.10 所示的新内核函数（及一些支撑函数）以及如代码清单 4.11 所示的新交互规格说明。

代码清单4.10 stability/kernel.cu

```
#include "kernel.h"
#define TX 32
#define TY 32
#define LEN 5.f
#define TIME_STEP 0.005f
#define FINAL_TIME 10.f

// scale coordinates onto [-LEN, LEN]
__device__
float scale(int i, int w) { return 2*LEN*(((1.f*i)/w) - 0.5f); }

// function for right-hand side of y-equation
__device__
float f(float x, float y, float param, float sys) {
  if (sys == 1) return x - 2*param*y; // negative stiffness
  if (sys == 2) return -x + param*(1 - x*x)*y; //van der Pol
  else return -x - 2*param*y;
}

// explicit Euler solver
__device__
float2 euler(float x, float y, float dt, float tFinal,
             float param, float sys) {
  float dx = 0.f, dy = 0.f;
  for (float t = 0; t < tFinal; t += dt) {
    dx = dt*y;
    dy = dt*f(x, y, param, sys);
    x += dx;
    y += dy;
  }
  return make_float2(x, y);
}

__device__
unsigned char clip(float x){ return x > 255 ? 255 : (x < 0 ? 0 : x); }

// kernel function to compute decay and shading
```

```
38 __global__
39 void stabImageKernel(uchar4 *d_out, int w, int h, float p, int s) {
40   const int c = blockIdx.x*blockDim.x + threadIdx.x;
41   const int r = blockIdx.y*blockDim.y + threadIdx.y;
42   if ((c >= w) || (r >= h)) return; // Check if within image bounds
43   const int i = c + r*w; // 1D indexing
44   const float x0 = scale(c, w);
45   const float y0 = scale(r, h);
46   const float dist_0 = sqrt(x0*x0 + y0*y0);
47   const float2 pos = euler(x0, y0, TIME_STEP, FINAL_TIME, p, s);
48   const float dist_f = sqrt(pos.x*pos.x + pos.y*pos.y);
49   // assign colors based on distance from origin
50   const float dist_r = dist_f/dist_0;
51   d_out[i].x = clip(dist_r*255); // red ~ growth
52   d_out[i].y = ((c == w/2) || (r == h/2)) ? 255 : 0; // axes
53   d_out[i].z = clip((1/dist_r)*255); // blue ~ 1/growth
54   d_out[i].w = 255;
55 }
56
57 void kernelLauncher(uchar4 *d_out, int w, int h, float p, int s) {
58   const dim3 blockSize(TX, TY);
59   const dim3 gridSize = dim3((w + TX - 1)/TX, (h + TY - 1)/TY);
60   stabImageKernel<<<gridSize, blockSize>>>(d_out, w, h, p, s);
61 }
```

这里对 kernel.cu 代码作简要说明。第 1～6 行引入了 kernel.h，并且定义了一些常量（线程数、空间比例因子，用于模拟的时间步长和时间间隔）。第 8～35 行定义了新的设备端函数，它们会被内核调用：

- scale() 限定像素值的坐标范围为 [–LEN, LEN]。
- f() 给出了速度的变化率。如果你对研究其他 1DoF 振荡器感兴趣，可以编辑函数来实现你感兴趣的系统。在样例代码中，包含三个不同的版本，分别对应着三个不同的变量 sys 值。
  - 默认版本（sys=0）是上面讨论的线性阻尼振荡器。
  - 设置 sys=1 对应一个负有效刚度线性振荡器（可能一开始看起来很奇怪，但是这是一种接近于倒立摆位的情况）。
  - 设置 sys=2 对应一个个人比较喜爱的范德堡尔振荡器，它具有一个非线性阻尼项。
- euler() 对一个给定的初始状态进行模拟，并且根据轨迹的位置在模拟区间结尾返回一个 float2 型的值。（注意，float2 型允许我们将位置和速度结合为一个实体，还可以用另一种方法，就是像我们从内核函数中处理大批输出数据那样，通过传递分配来存储多个值的内存指针达到目的，然而，在这里我们并不需要这样做。）
- 第 34 行和第 35 行定义的 clip() 函数与 flashlight 应用程序的相同。新内核 stabImageKernel() 的定义从第 38 行开始。注意，参数增加了阻尼参数值 p 和

振荡器类型 s。第 40～43 行的索引计算和边界检查与 flashlight 应用程序中的 distanceKernel() 完全一致。第 44 行和第 45 行根据像素的位置并且计算以平衡点为原点的初始距离 dist_0，我们引入 {x0, y0} 作为已进行尺度放缩的 float 型的坐标值（范围是 [–LEN, LEN]）。第 47 行调用 euler() 函数，模拟了在 FINAL_TIME 时间范围内，每次按照 TIME_STEP 的时间步长进行迭代之后坐标变化的过程，并在最后返回最终的坐标值 pos。第 50 行比较了距原点的最终距离和初始距离。第 51～54 行，基于距离比较分配阴影值，其中，蓝色代表衰减到平衡（即利于稳定的投票），红色代表逐渐远离平衡（即否决稳定的其他投票）。第 52 行使用绿色通道显示在平衡点处相交的水平 x 轴和竖直 y 轴。

第 57～61 行，用正确的参数列表和待启动内核名称来定义修改后的包装函数 kernelLauncher()。

代码清单4.11 stability/interactions.h

```
#ifndef INTERACTIONS_H
#define INTERACTIONS_H
#define W 600
#define H 600
#define DELTA_P 0.1f
#define TITLE_STRING "Stability"
int sys = 0;
float param = 0.1f;
void keyboard(unsigned char key, int x, int y) {
  if (key == 27)  exit(0);
  if (key == '0') sys = 0;
  if (key == '1') sys = 1;
  if (key == '2') sys = 2;
  glutPostRedisplay();
}

void handleSpecialKeypress(int key, int x, int y) {
  if (key == GLUT_KEY_DOWN) param -= DELTA_P;
  if (key == GLUT_KEY_UP)   param += DELTA_P;
  glutPostRedisplay();
}

// no mouse interactions implemented for this app
void mouseMove(int x, int y) { return; }
void mouseDrag(int x, int y) { return; }

void printInstructions() {
  printf("Stability visualizer\n");
  printf("Use number keys to select system:\n");
  printf("\t0: linear oscillator: positive stiffness\n");
  printf("\t1: linear oscillator: negative stiffness\n");
  printf("\t2: van der Pol oscillator: nonlinear damping\n");
  printf("up/down arrow keys adjust parameter value\n\n");
  printf("Choose the van der Pol (sys=2)\n");
  printf("Keep up arrow key depressed and watch the show.\n");
```

```
36 }
37
38 #endif
```

interactions.h 如代码清单 4.11 所示，对它的修改看起来也比较简单明了。除了设置图像宽度 W 和高度 H 的 #define 声明之外，加入了 DELTA_P 来代表参数值的增量大小。第 7 行和第 8 行初始化系统编号变量 sys 和用于调节阻尼值的参数值 param。

支持一些键盘的交互：按下 Esc 退出应用程序；按下数字键 0、1 或 2 选择模拟的系统；向上箭头和向下箭头键分别增加和减小阻尼参数值，变化量为 DELTA_P。这里没有为鼠标交互提供支持，所以 mouseMove() 和 mouseDrag() 直接返回，什么事情也不做。

最后，在其他文件中有几个细节需要注意：

- kernel.h 包含 kernelLauncher() 函数的原型，因此应该复制 kernel.cu 中函数定义的第一行，替代 flashlight/kernel.cu 中原来的原型说明（加上一个分号）。
- main.cpp 中还需要一些小改变：
  - render() 中调用的 kernelLauncher() 的参数列表已经改变，并且需要改变那个调用来与修改的内核语法相匹配。
  - render() 也很适合指定欲在图像窗口标题栏里显示的信息。例如，样例代码显示应用程序名（“Stability”），下面紧跟着 param 和 sys 的值。代码清单 4.12 给出了 render() 中关于标题栏信息的改进版本和改进的内核启动调用。

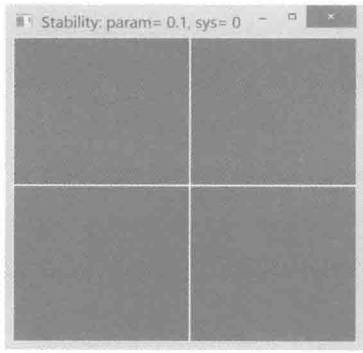
代码清单4.12　stability/main.cpp中render()函数的升级版

```
1 void render() {
2   uchar4 *d_out = 0;
3   cudaGraphicsMapResources(1, &cuda_pbo_resource, 0);
4   cudaGraphicsResourceGetMappedPointer((void **)&d_out, NULL,
5                                        cuda_pbo_resource);
6   kernelLauncher(d_out, W, H, param, sys);
7   cudaGraphicsUnmapResources(1, &cuda_pbo_resource, 0);
8   // update contents of the title bar
9   char title[64];
10  sprintf(title, "Stability: param = %.1f, sys = %d", param, sys);
11  glutSetWindowTitle(title);
12 }
```

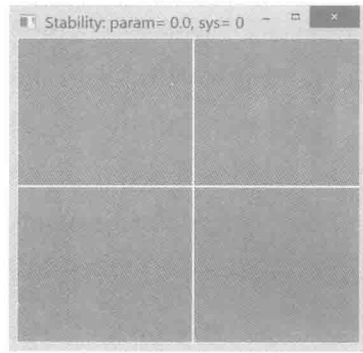
运行 stability 可视化程序

我们已经浏览了相关的代码，现在到了测试这个应用程序的时候了。在 Linux 中，创建这个工程的 Makefile 与代码清单 4.9 中提供的 flashlight 的 Makefile 相同。在 Visual Studio 中，引入的库文件和工程设置与 flashlight 中描述的一致。当你生成并运行应用程

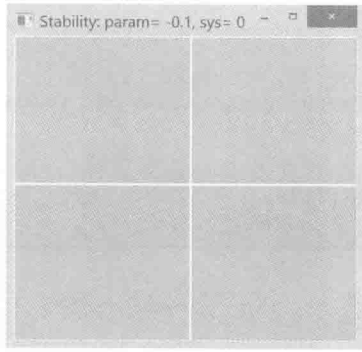
序时，应该打开两个窗口：一个是通常的命令行窗口，显示所支持的用户输入的简要汇总；另一个是图像窗口，显示稳定性结果。默认设置指定线性振荡器具有正阻尼，你可以通过显示 stability（稳定性）的标题栏来验证：param=0.1，sys=0，如图 4.4a 所示。因为非受迫性线性阻尼振荡器的所有解决方案都吸引向平衡态，所以图像窗口应该在暗区域显示坐标轴，意指处于稳定。然后，你也许会测试向下箭头键。按一下会将阻尼值从 0.1 降到 0.0（这个应该在标题栏中验证），并且你应该可以观察到区域由暗变为中度亮色，如图 4.4b 所示。阻尼为 0 的线性振荡器是中度稳定的（使用保持不断接近，但是无法到达平衡的正弦振荡）。显式欧拉微分方程求解器会产生一些小误差，导致倾向于远离原点，但是颜色体系正确地标识全部的初始状态都能得到解，该解大体上保持了到平衡态的距离。再按一下向下箭头键，将使阻尼参数值变为 -0.1，如图 4.4c 所示的亮区域表明不稳定。



a) param=0.1，暗区域表明吸引到稳定平衡态的解



b) param = 0.0，中度亮区域表明中度稳定



c) param=−0.1，亮区域表明得到的是远离稳定平衡态的解

图 4.4　不同阻尼参数的线性振荡器的 stability 可视化

现在，按下 1 键来设置 sys=1，对应于一个负有效刚度的系统，并且增加阻尼值。你现在应该看到明亮区域中带有黑色部分的轴线（和中度明亮的过度区域），如图 4.5 所示。在这种情况下，一些解移向平衡态，但是，大多数（几乎所有）初始条件都会导致远离平衡态的解，这是不稳定的。

我们将学习样例中的最后一种情况，设置阻尼 param=0.0、sys=2，即范德堡尔振荡器。因为 param=0.0，所以这个系统与无阻尼线性振荡器相同。因此，我们再一次观察中度亮区域的平衡。当你按向上箭头键来使阻尼为正时，会发生什么？平衡态被一个亮区域包围，所以附近的初始状态产生远离稳定态的解，并且平衡是不稳定的。然而，外部区域是暗区域，更外面的初始状态将产生向内吸的解。没有其他平衡点去靠近，那么这些解在哪里终止呢？原来，在阴影过渡附近有一个封闭的、吸引的回路，对应一个稳定的周期运动或“限制环”（见图 4.6）。

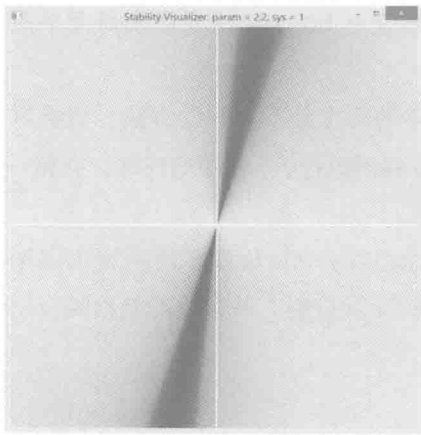


图 4.5　负刚度线性振荡器的相平面。出现一个暗区域，但是亮区域表示进入不稳定平衡态

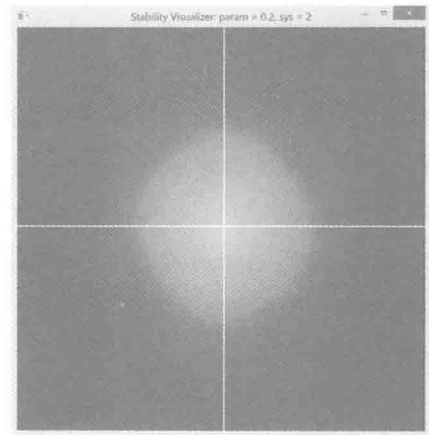


图 4.6　范德堡尔振荡器的相平面。中心亮区域表示不稳定平衡态。外部暗区域表示向内衰减的解。这些结果与在中度亮区域的周期性的稳定“限制环”轨迹的存在是一致的

注意，此处的数值稳定性分析结果应被看作是示意性的。常微分方程求解是近似的，并且我们仅测试了几百个初始状态，所以极有可能忽略一些东西。

在完成前，你可能想一直按着向上箭头键来实时观察稳定性视觉图像中成百上千的像素变化。但是不借助并行计算，你是不可能做到的。

4.4 本章小结

在本章中，我们讨论了在二维网格上定义和启动内核的要点。我们给出并解释了样例代码 flashlight 应用程序，它利用 CUDA/OpenGL 互操作，用二维计算网格的结果，来进行实时图像显示与交互。最后，我们给出了如何把 flashlight 当作一个模板，并在它的基础上进行修改来使其可以应用到动态稳定性数值求解这一实际工程问题。

4.5 推荐项目

1. 修改 flashlight 应用程序，使其变为一个“更冷 / 更热”游戏。为玩家 A 提供一个选择目标像素的接口。然后，玩家 B 根据点的颜色寻找目标像素。其中，随着点远离目标，点的颜色会变蓝（或红）。
2. 根据你的兴趣，寻找一个其他的 1DoF 系统，并且修改 stability 应用程序来研究平衡性质。
3. 显式欧拉方法也许是进行常微分方程数值求解最简单的但也最不可靠的方法。请尝试通过使用一个更加准确的常微分方程来求解方法提高 stability 应用程序的性能。龙格 - 库达（Runge-Kutta）法是扩展到多数领域的很好选择。
4. 范德堡尔限制环在 param=0.1 时近似为圆形。修改 stability 应用程序，使阴影决定于最终距离和一个新参数 rad 之间的差值。实现对 rad 的交互控制，并且运行修改的应用程序来确定限制环的大小。

参考文献

[1] Microsoft Windows Dev Center. "Direct3D," 2015, https://msdn.microsoft.com/en-us/library/windows/desktop/hh309466(v=vs.85).aspx.

[2] Mason Woo, Jackie Neider, Tom Davis, and Dave Shreiner, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 1.2, Third Edition*. (Reading, MA: Addison-Wesley, 1999).

[3] NVIDIA Corporation. "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[4] Graham Sellers, Richard S. Wright, Jr., and Nicholas Haemel, *OpenGL Superbible:*

Comprehensive Tutorial and Reference, Seventh Edition. (Boston, MA: Addison-Wesley, 2016).

[5] Randi J. Rost et al., *OpenGL Shading Language, Third Edition.* (Boston, MA: Addison-Wesley, 2010).

[6] cppreference.com, "ASCII Chart," 2015, http://en.cppreference.com/w/cpp/language/ascii.

[7] Richard H. Rand. *Lecture Notes on Nonlinear Vibrations*, Cornell University eCommons, May 2012, http://hdl.handle.net/1813/28989.

[8] Steven H. Strogatz, *Nonlinear Dynamics and Chaos, Second Edition.* (Cambridge, MA: Westview Press, 2014).

Chapter 5

第 5 章
模板与共享内存

本章将关注需要线程间交换数据这样一类应用。这类应用的计算线程间不是相互独立的，而是与计算网格中邻近的其他线程相互依赖的。我们所要实现的数学模型中将包含卷积（或者相关）运算。在该运算中，邻近线程中的数据将通过一个常系数数组进行线性组合。在计算领域，这种运算通常被称为**滤波**（filtering），而这里的常系数数组则被称为一个滤波器（filter）或者**模板**（stencil）。当多个线程竞争访问相同的数据时，线程间的相互作用会引起瓶颈，所以 CUDA 提供了一些功能来缓解这种瓶颈，提高性能。CUDA 的大多数技巧都离不开特殊类型的内存，这里我们关注的是共享内存，它使得同一个块中的线程可以高效地进行信息共享。

我们通过一个一维的例子来介绍模板计算以及共享内存的基本思想。在这个例子中，我们将通过均匀采样值来计算一个函数的二阶导数。然后我们将创建一个应用把模板和共享内存扩展到二维，这个应用使用雅可比迭代法求解拉普拉斯方程来计算和显示稳态温度分布。值得注意的是求解拉普拉斯方程在流体力学、静电学、重力以及复变函数领域也有应用。最后我们创建一个应用来锐化图片，为说明对初始实现版本改进的有效性，借助了一些性能分析手段。

5.1 线程间依赖

这一章标志着向前迈出的重要一步。我们应该花一点时间来明确陈述这一步的内容。

我们以一个 stability 应用结束了第 4 章，这个应用的功能是计算一个具有初始状态网格的振子随时间的变化过程。我们介绍了基于有限差分的导数估计，并且用它来关联每一个线程计算出来的时间历史中的连续值。这里导数的计算都处于一个单独的线程之内，并且每个线程的处理过程都和其他线程没有依赖关联。每个线程都可以访问它自己的初始状态（基于它的索引值），但是不可以访问仿真应用中其他任何线程的状态。

现在我们已经准备好运用有限差分运算来计算一个函数的导数，这个函数的每个采样值与网格中的一个点关联起来（每个点代表一个不同的线程）。具体来说，让我们考虑计算一个微分方程的数值解 $u(x)$。整个计算被一系列网格点 $x_i=ih$ 离散化了，这些点是在一条线段上具有相同步长 h 的点，其中导数由邻近点间的差值代替。并行求解程序启动一个一维计算网格，并且每一个线程都有一个变量 u 用于保存计算出来的结果 $u_i=u(x_i)$。现在有限差分公式中的系数将与邻近线程相关联的变量值组合在一起。

为什么如此关心这一点呢？因为如果你写一个简单内核程序像“计算当前线程中的一个变量的值与相邻网格中其他相同变量值的差”，CUDA 是不支持的。在 SIMT 并行程序中，每一个线程都可以访问它自己本地的每一个内核变量，但是不能访问其他线程的本地变量。那么当我们想要在线程间共享信息时该怎么避免这种限制呢？简单的方法是在设备的全局内存中申请数组，每个线程都可以在这个数组中读写它自己的 u。我们将使用这种全局数组的方法，并且说明这种方法是有效的，但是结果显示这种方法并不是非常高效。当启动一个非常大的线程网格时，可能会有数百万的线程尝试从相同的数组读写数据，这将引起同步以及存储拥塞的问题。

我们不会立马涉及所有的细节，但是你必须清楚数据存储的地方离处理器越远，处理数据所需时间越长这一基本原则。在第 3 章中，当我们第一次将数据从主机端复制到设备端时，我们已经讨论了一个与这一原则有关的应用程序。设备存储离多流处理器（SM）更近，所以相比于主机存储，它是首选的存储位置，但是使用设备存储需要通过一个相对较慢的通信总线进行数据传输。此外，当确实需要进行数据传输时，我们希望通过尽量少的传输次数来减少总的数据传输时间。

我们现在遇到了第二个关于“越近越快”原则的应用程序。在 GPU 中有不同的存储区域，并且应该优先使用离 SM 更近的存储。前面我们已经接触了全局内存（所有的设备数组存储在这里），以及寄存器存储（每个线程的本地变量存储在这里）。全局内存提供了绝大部分的设备存储容量，但它也是你在 GPU 上所能访问到的离 SM 最远的存储，所以全局内存提供了通用的但是相对较慢的存储访问。（重申一下它仍然比访问主机存储快

得多。）寄存器存储离 SM 最近，所以它提供了最快的访问速度，但是它的作用范围仅局限于单个线程内。

现在我们来介绍共享内存。它的作用是缩小存储速度与处理器访问之间的差距。共享内存和 SM 相邻，并且提供了多达 48KB 的可被同一线程块中所有线程高效访问的存储空间。可以认为共享内存就是 CUDA 为支持高效的线程间合作所提供的最主要的机制，并且在许多情况下（包括许多模板计算）使用共享内存将获得巨大的性能提升。废话不多说，让我们开始进入实例讲解吧。

5.2 一维网格上的导数计算

使用模板以及共享内存进行的计算包含一点索引计算的工作，所以为了使描述尽量简洁，我们从一维的例子开始讲解。虽然我们在一维情况下的运行时间与 CPU 上的计算相比可能得不到加速，但是当我们转移到二维情况时就能获得巨大的回报。为了便于实际讨论，我们将再次选取一个具体的例子。我们选择在第 4 章的基本有限差分方法的基础上，将其修改成具有依赖关系的变量 u 以及独立变量 x，其中的 x 由具有等长间隔 h 的离散采样点 x_i 所表示。这样一阶导数的前向差分估计变成：

$$\frac{du}{dx}(x_n)=(u_{i+1}-u_i)/h$$

其中 $u_i=u(x_i)=u(ih)$。第二个关于有限差分公式的应用程序使用中心差分来计算二阶导数：

$$\frac{d^2u}{dx^2}(x_n)=(u_{i+1}-2u_i+u_{i-1})/h^2$$

第一个例子非常简单明了：创建一个函数 sin(x) 的采样值数组，并使用中心差分公式计算出二阶导数数组。（对 sin(x) 进行两次差分仅仅会使其符号发生变化，所以很容易对结果进行验证。）

现在我们回到开始一个新应用程序时通常会遇到的问题：我们如何划分整个计算任务使得每个部分可以被一个线程所处理？为了保持简明，我们再次让每个线程计算一个导数值；也就是说，索引号为 i 的线程从设备数组 d_in 中读取必要的输入（例如 d_in[i−1]、d_in[i] 和 d_in[i+1]），然后计算本地的二阶导数，并将结果值存储到输出矩阵 d_out[i] 中。

5.2.1 实现 dd_ld_global

在代码清单 5.1 至代码清单 5.3 中，我们展示了一个使用全局内存实现的 dd_ld_

global（使用全局内存计算一维上的二阶导数）程序。

我们首先讨论代码清单 5.1 中所展示的 main.cpp 的细节。main() 函数首先计算 x 和 u 数组的值（第 14～17 行），然后执行并行导数计算（第 19 行）。随后，存储在数组 result_parallel 中的结果被写入 .csv 格式（逗号分隔的数据）的文件 result.csv 中，该文件可以使用电子制表软件（如 Excel）或者文本编辑器打开查看。

代码清单5.1　dd_ld_global/main.cpp

```
1 #include "kernel.h"
2 #include <math.h>
3 #include <stdio.h>
4
5 int main() {
6   const float PI = 3.1415927;
7   const int N = 150;
8   const float h = 2*PI/N;
9
10  float x[N] = {0.0};
11  float u[N] = {0.0};
12  float result_parallel[N] = {0.0};
13
14  for (int i = 0; i < N; ++i) {
15    x[i] = 2*PI*i/N;
16    u[i] = sinf(x[i]);
17  }
18
19  ddParallel(result_parallel, u, N, h);
20
21  FILE *outfile = fopen("results.csv", "w");
22  for (int i = 1; i < N - 1; ++i) {
23    fprintf(outfile, "%f,%f,%f,%f\n", x[i], u[i],
24            result_parallel[i], result_parallel[i] + u[i]);
25  }
26  fclose(outfile);
27 }
```

现在让我们继续看代码清单 5.2 中的内核及启动代码。希望现在你可以轻松读懂内核 ddKernel() 的代码。kernel.cu 中的另一个函数是包装函数 ddParallel()，它分配了设备端数组 d_in 和 d_out（第 14 行和第 15 行），将输入数据复制到设备端（第 16 行），在第 18 行启动了 ddKernel()，将 d_out 中的结果复制回主机端（第 20 行），最后释放设备端存储（第 21 行和第 22 行）。

代码清单5.2　dd_ld_global/kernel.cu

```
1 #include "kernel.h"
2 #define TPB 64
3
4 __global__
5 void ddKernel(float *d_out, const float *d_in, int size, float h) {
```

```
  const int i = threadIdx.x + blockDim.x*blockIdx.x;
  if (i >= size) return;
  d_out[i] = (d_in[i - 1] - 2.f*d_in[i] + d_in[i + 1])/(h*h);
}

void ddParallel(float *out, const float *in, int n, float h) {
  float *d_in = 0, *d_out = 0;

  cudaMalloc(&d_in, n*sizeof(float));
  cudaMalloc(&d_out, n*sizeof(float));
  cudaMemcpy(d_in, in, n*sizeof(float), cudaMemcpyHostToDevice);

  ddKernel<<<(n + TPB - 1)/TPB, TPB>>>(d_out, d_in, n, h);

  cudaMemcpy(out, d_out, n*sizeof(float), cudaMemcpyDeviceToHost);
  cudaFree(d_in);
  cudaFree(d_out);
}
```

最后，我们看一下代码清单 5.3 中所展示的 kernel.h 的内容。这个头文件中的主要内容是内核包装函数 ddParallel() 的原型，其余的代码是一些防止多次包含该头文件的宏定义。

代码清单5.3 dd_ld_global/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

void ddParallel(float *out, const float *in, int n, float h);

#endif
```

代码清单 5.4 是为了在 Linux 中生成该应用程序而提供的 Makefile。

代码清单5.4 dd_ld_global/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall

all: main.exe

main.exe: main.o kernel.o
  $(NVCC) $^ -o $@

main.o: main.cpp kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@

kernel.o: kernel.cu kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

生成并执行 dd_ld_global 应用程序，然后通过查看 results.csv 确保程序正确执行了。文件中的四列分别是 x、sin(x)、二阶导数的近似值以及二阶导数结果与 sin(x) 的和。最

后一列数都接近于 0，说明了有限差分逼近的正确性（以及准确性）。

一旦证明结果是正确的，我们就可以开始考虑效率问题了。虽然在本章最后介绍图片锐化应用程序时我们会使用一些性能分析方法，但是现在我们要意识到使用全局内存来实现模板计算会引入冗余的存储访问，从而导致不必要的数据拥塞。因为每个线程都会读取输入数据中和它本身以及相邻线程索引号相关联的数据，所以输入数组中的每个元素都会被请求三次。（d_in[i] 会被索引号为 i–1、i、i+1 的线程当作右邻居值、中间值，以及左邻居值分别访问。）

模板半径及冗余数据访问

模板半径是指该模板每边所覆盖的相邻元素个数。在本章一维情况下，具有二阶精度和半径 r=1 的模板的数据访问冗余量为 $2r$+1=3。而在 N 维情况下冗余量将是 $(2r+1)^N$，所以当半径与 / 或维数增加时，问题将变得更加严重。

接下来，我们使用数据分块和共享内存来提高速度，减少重复的存储传输，从而实现一个更高效的模板计算方法。

5.2.2　实现 dd_ld_shared

使用共享内存的基本思想是将大的网格分解成数据块，为一个计算线程块提供所需的输入和输出。然后我们在共享内存中创建输入和（或）输出数组，这些数组既可以被快速访问（因为它们离 SM 很近），也可以被一个线程块中的所有线程所访问。这里我们重点关注输入数据使用共享内存，因为这正是引起冗余访问问题的地方。我们的目标是对于每块输入数据仅从全局内存访问一次（为了填充共享内存中的数组），之后线程块中所有线程都可以通过共享内存快速获取所需数据。

为了便于索引，一个明智的方法是维护通常的索引 i（我们称它为全局索引，因为它确定了全局内存数组中的对应元素），同时引入一个新的局部索引 s_idx 用来记录数据在共享内存中的位置。需要注意的是仅仅将与线程相对应的数据复制到共享内存中是不够的，当处于线程块边缘时，我们还需要相邻线程块中被模板覆盖的数据。我们将在代码中使用 RAD 来命名模板半径，并且我们需要将与线程块边缘的 RAD 个相邻数组元素也包含进来。通常在处理一个半径为 RAD 的模板时，除了线程块中每个线程所对应的元素外，共享数组还必须包含分布于两端的 2*RAD 个重叠单元（halo cells），从而保证共享数组中包含所有必须的数据。线程块中的第一个线程（threadIdx.x=0）需要为它左边的

RAD 个邻居预留空间，因此它的局部索引 s_idx=RAD。局部索引和线程索引之间的一般关系是 s_idx=threadIdx.x+RAD。

既然我们有了一个系统的索引方案，那么就可以开始讲解实现细节了。幸运的是由于调用求导函数的接口没有变化，所以不需要修改 main.cpp 和 kernel.h，它们仍然与代码清单 5.1 和代码清单 5.3 中展示的一样。我们可以关注代码清单 5.5 中所展示的内核函数的共享内存版本。当我们计算出全局索引 i，并且确认它在规定范围内（第 7 行和第 8 行）后，我们就可以计算局部索引 s_idx（第 10 行），并使用 __shared__ 修饰符声明共享数组 s_in（第 11 行）。

代码清单5.5 dd_ld_shared/kernel.cu

```
 1 #include "kernel.h"
 2 #define TPB 64
 3 #define RAD 1 // radius of the stencil
 4
 5 __global__
 6 void ddKernel(float *d_out, const float *d_in, int size, float h) {
 7   const int i = threadIdx.x + blockDim.x*blockIdx.x;
 8   if (i >= size) return;
 9
10   const int s_idx = threadIdx.x + RAD;
11   extern __shared__ float s_in[];
12
13   // Regular cells
14   s_in[s_idx] = d_in[i];
15
16   // Halo cells
17   if (threadIdx.x < RAD) {
18     s_in[s_idx - RAD] = d_in[i - RAD];
19     s_in[s_idx + blockDim.x] = d_in[i + blockDim.x];
20   }
21   __syncthreads();
22   d_out[i] = (s_in[s_idx-1] - 2.f*s_in[s_idx] + s_in[s_idx+1])/(h*h);
23 }
24
25 void ddParallel(float *out, const float *in, int n, float h) {
26   float *d_in = 0, *d_out = 0;
27   cudaMalloc(&d_in, n*sizeof(float));
28   cudaMalloc(&d_out, n*sizeof(float));
29   cudaMemcpy(d_in, in, n*sizeof(float), cudaMemcpyHostToDevice);
30
31   // Set shared memory size in bytes
32   const size_t smemSize = (TPB + 2*RAD)*sizeof(float);
33
34   ddKernel<<<(n + TPB - 1)/TPB, TPB, smemSize>>>(d_out, d_in, n, h);
35
36   cudaMemcpy(out, d_out, n*sizeof(float), cudaMemcpyDeviceToHost);
37
38   cudaFree(d_in);
39   cudaFree(d_out);
40 }
```

设置共享数组大小

如果你要创建一个固定大小的共享数组，可以使用如下方法：

```
__shared__ float s_in[34];
```

并且不需要修改内核函数的调用方法。注意下面的写法会导致编译错误：

```
__shared__ float s_in[blockDim.x + 2*RAD];
```

如果你要动态申请数组，则声明时需要像下面一样加上关键字 extern：

```
extern __shared__ float s_in[];
```

并且在调用内核函数时需要在 <<<>>> 内加上第三个参数来指明所需分配的共享内存的字节大小。

```
const size_t smemSize = (TPB + 2*RAD)*sizeof(float);
ddKernel<<<(n+TPB-1)/TPB, TPB, smemSize>>>(d_out, d_in, n, h);
```

执行配置参数

还有第四个可选执行配置参数用于指定计算所在的流号（stream number）。因为本书中没有例子使用多个流，所以默认值 0 就已经够用了。如果想获取更多关于 CUDA 计算中流的相关信息，可以参考英伟达官方手册《CUDA C Programming Guide》[1] 中“CUDA C 运行时”一章的“流”部分，或者参考 Parallel Forall 栏目中的博客帖子“GPU Pro Tip: CUDA 7 Streams Simplify Concurrency”[2]。

分配好共享数组后，我们就可以将数据从全局内存转移到共享内存了。基本方案是每个线程从输入数组的全局索引位置读取元素，并将它存储到共享数组中对应该线程局部索引的位置（例如第 14 行中的 s_in[s_idx] = d_in[i]）。重叠单元也需要读取及存储，这项工作通过下面的代码片段来完成：

```
if (threadIdx.x < RAD) {
s_in[s_idx - RAD] = d_in[i - RAD];
s_in[s_idx + blockDim.x] = d_in[i + blockDim.x];
}
```

它将工作分配给 0 至 RAD-1 号线程，每一个线程读取并存储一对重叠值。对于 0 号线程有 s_idx=RAD，所以：

- s_idx-RAD 是 0，最左边的邻居 in[i-RAD] 存储在共享数组开始最左边的重叠单元里。
- s_idx+blockDim.x 是 RAD+blockDim.x，而右边第一个邻居 in[i+blockDim.x] 存储在数组末尾的最左边重叠单元中。

前 RAD 个线程不断成对的读取和存储重叠单元，直到整个共享内存都被填充。

我们已经快到达内核函数的最后一行语句了（第 22 行），在这里计算和存储所要求的二阶导的有限差分估计，但是首先我们需要注意另一重要的索引项。

回想一下内核函数启动后是异步的。为了获得大规模并行计算的能力，我们放弃了一些对执行顺序的控制，这意味着我们不能简单地假设在线程执行最后一条语句时所有的输入数据都已经被加载到共享内存中（并且可能会使用一些分配的共享数组所在存储位置原有的随机值）。为了确保所有数据都被正确存储了，我们使用了 CUDA 函数 __syncthreads()（第 21 行），该函数强制线程块中所有线程在本线程块中任意其他线程进一步执行之前完成之前的语句。同步需要一些时间，并且会降低你的加速比，所以需要合理使用，而此处需要它来保证获得正确的结果。

再次生成并执行 dd_ld_shared 应用程序，然后通过查看 results.csv 文件内容确保程序按计划工作了。如果你在使用 Linux，这个应用程序的 Makefile 和代码清单 5.4 中为 dd_ld_global 所提供的一样。

5.2.3 解决二维拉普拉斯方程：heat_2d

为了求解导数，我们已经有了一个执行模板运算的高效方法，并且离实际工程应用不远了。现在我们计算拉普拉斯方程的解，这可以用于很多领域。这个问题可以结合很多不同的环境加以说明，而我们将讨论的应用是关于指定边界温度的情况下，一个区域内的平衡温度分布 $u(x)$ 的求解。基础物理理论说明温度变化速率 $\frac{\partial u}{\partial t}$ 与每单位体积热能净流量成正比。单位体积热能净流量可以使用热流场散度来测量，而且基本扩散模型说明热流场正比于温度分布梯度。综上所述可以得出：

$$\frac{\partial u}{\partial t}=\alpha\nabla\cdot\nabla u=\alpha\nabla^2 u=\alpha\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}+\frac{\partial^2 u}{\partial z^2}\right)$$

其中 α 是导热性，∇^2 是拉普拉斯算子，最右边给出的是其在三维笛卡尔坐标下的表达式。

现在让我们专注于一个特定的问题，这个问题包含一个薄板中的平衡温度分布（不随温度变化）。这个薄板位于平面直角坐标系，并且表面是隔热的，所以在 z 轴方向没有明显的温度变化。所以右边式子中的 z 的导数等于 0 被去掉，同样的左边的 t 的导数等于 0 也被去掉，最后消去导热性 α 就得到笛卡儿坐标下的二维拉普拉斯方程：

$$\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}=0$$

这个方程需要在一定约束条件下求解。

我们希望事情简单而不平凡，并且具有一个特定的背景环境，所以我们构想一个工程场景，在该场景中我们需要分析一个作为圆形蒸汽管道的垂直支撑体的平板的稳态温度分布。这个平板是一个中间挖出圆洞（用来穿过管道），顶部两角被斜截的方形。平板的底边与地面接触，所以温度是 t_g（g 代表地面）。类似地，洞的边缘温度是 t_s（s 代表流经管道的源），而平板的其他边与周围空气相接触，温度为 t_a。我们的任务是创建一个叫作 heat_2d 的应用程序，用于在我们改变如下几项条件时计算和显示平板的稳态温度分布：

- 管道的位置、大小，以及温度。
- 斜角的大小。
- 空气以及地面的温度。

图 5.1 概括了这个问题所包含的偏微分方程及其边界条件。

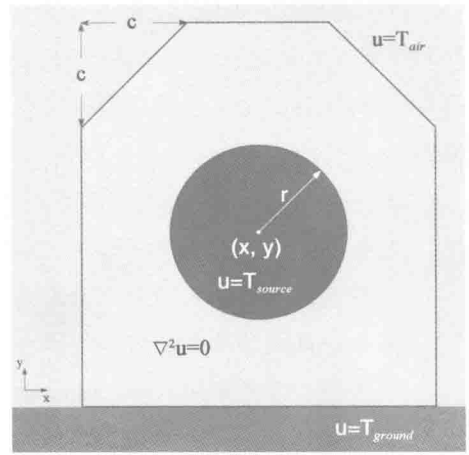


图 5.1　heat_2d 应用所需解决的方程及其边界条件图示

现在，为了进行数值计算，我们对其进行离散化，二阶导数的计算使用我们前面的有限差分公式加上第二个适合二维情况的索引来代替：

$$\frac{\partial^2 u}{\partial x^2}(x_{i,j}) \rightarrow \frac{u_{i+1,j}-2u_{i,j}+u_{i-1,j}}{h^2}$$

$$\frac{\partial^2 u}{\partial y^2}(x_{i,j}) \rightarrow \frac{u_{i,j+1}-2u_{i,j}+u_{i,j-1}}{h^2}$$

将有限差分公式加入拉普拉斯方程并求解 $u_{i,j}$ 得到：

$$u_{i,j}=(u_{i-1,j}+u_{i+1,j}+u_{i,j-1}+u_{i,j+1})/4$$

所以平衡温度分布的离散化采样值具有这样的特性，其每一个值是相邻四个值的平均数，这样我们可以得到一个新的二维的模板：

$$\left(\frac{1}{4}\right)\begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}$$

这个模板（通常被称为五点式模板，因为它包括一个中间点和四个相邻点）描述了微分方程稳态解的基本性质；那就是在平衡状态下，一个网格点的温度是周围点温度的平均值。

计算该稳态温度分布的最简单的方法可能就是重复使用上述模板对温度数组中的值求其相邻值的平均数。这种方法被称为雅可比迭代，并且这种方法的结果确实可以收敛到拉普拉斯方程的解。（提醒一下，问题规模很大时收敛过程会非常慢。要获得关于这

一点以及和它们收敛性质相关的详细讨论，请查看 Leveque 所写的《Finite Difference Methods for Ordinary and Partial Differential Equations》[3] 一书的第 4 章。）

虽然雅可比迭代可以收敛到拉普拉斯方程的一个解，但是它一般不能收敛到一个满足特定边界条件的解。为了解决这个问题，我们对内核函数进行了一点直观的修改。并不对所有的点都使用模板进行计算，而是对平板外面的点采用特定的边界条件值，对于平板内部的点才采用模板进行计算。

目前我们已经确定了问题，及其数值解方案，下面让我们继续查看代码清单 5.6 至代码清单 5.9 中所展示的具体实现。这个应用程序包括可交互的图形界面，所以它是以第 4 章中的 flashlight 应用为模板创建的，并且只有代码清单 5.6 中 main.cpp 的几行是不同的。

- render() 函数重复地调用了一个不同的内核函数，将在后面具体介绍。
- 标题栏将分别显示源、空气以及地面温度的当前边界值：t_s、t_a 以及 t_g。注意这些值将作为后面介绍的 kernel.h 中定义的边界条件数据结构体的成员。
- main() 函数启动时打印了所有用户可用的操作，申请数组用来存储二维网格的温度值，并且调用 resetTemperature() 来启动一个内核函数初始化温度数组。
- 另外还调用了一个 GLUT 的 glutIdleFunc() 函数，这是有用的，因为我们希望迭代过程在用户操作之间不断更新。

代码清单5.6 heat_2d/main.cpp

```
#include "interactions.h"
#include "kernel.h"
#include <stdio.h>
#include <stdlib.h>
#ifdef _WIN32
#define WINDOWS_LEAN_AND_MEAN
#define NOMINMAX
#include <windows.h>
#endif
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/freeglut.h>
#endif
#include <cuda_runtime.h>
#include <cuda_gl_interop.h>
#define ITERS_PER_RENDER 50

// texture and pixel objects
GLuint pbo = 0;     // OpenGL pixel buffer object
GLuint tex = 0;     // OpenGL texture object
struct cudaGraphicsResource *cuda_pbo_resource;
```

```
24
25 void render() {
26   uchar4 *d_out = 0;
27   cudaGraphicsMapResources(1, &cuda_pbo_resource, 0);
28   cudaGraphicsResourceGetMappedPointer((void **)&d_out, NULL,
29                                        cuda_pbo_resource);
30   for (int i = 0; i < ITERS_PER_RENDER; ++i) {
31     kernelLauncher(d_out, d_temp, W, H, bc);
32   }
33   cudaGraphicsUnmapResources(1, &cuda_pbo_resource, 0);
34   char title[128];
35   sprintf(title, "Temperature Visualizer - Iterations=%4d, "
36                  "T_s=%3.0f, T_a=%3.0f, T_g=%3.0f",
37                  iterationCount, bc.t_s, bc.t_a, bc.t_g);
38   glutSetWindowTitle(title);
39 }
40
41 void draw_texture() {
42   glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, W, H, 0, GL_RGBA,
43                GL_UNSIGNED_BYTE, NULL);
44   glEnable(GL_TEXTURE_2D);
45   glBegin(GL_QUADS);
46   glTexCoord2f(0.0f, 0.0f); glVertex2f(0, 0);
47   glTexCoord2f(0.0f, 1.0f); glVertex2f(0, H);
48   glTexCoord2f(1.0f, 1.0f); glVertex2f(W, H);
49   glTexCoord2f(1.0f, 0.0f); glVertex2f(W, 0);
50   glEnd();
51   glDisable(GL_TEXTURE_2D);
52 }
53
54 void display() {
55   render();
56   draw_texture();
57   glutSwapBuffers();
58 }
59
60 void initGLUT(int *argc, char **argv) {
61   glutInit(argc, argv);
62   glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);
63   glutInitWindowSize(W, H);
64   glutCreateWindow("Temp. Vis.");
65 #ifndef __APPLE__
66   glewInit();
67 #endif
68 }
69
70 void initPixelBuffer() {
71   glGenBuffers(1, &pbo);
72   glBindBuffer(GL_PIXEL_UNPACK_BUFFER, pbo);
73   glBufferData(GL_PIXEL_UNPACK_BUFFER, W*H*sizeof(GLubyte) * 4, 0,
74                GL_STREAM_DRAW);
75   glGenTextures(1, &tex);
76   glBindTexture(GL_TEXTURE_2D, tex);
77   glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
78   cudaGraphicsGLRegisterBuffer(&cuda_pbo_resource, pbo,
79                                cudaGraphicsMapFlagsWriteDiscard);
80 }
81
82 void exitfunc() {
83   if (pbo) {
```

```
    cudaGraphicsUnregisterResource(cuda_pbo_resource);
    glDeleteBuffers(1, &pbo);
    glDeleteTextures(1, &tex);
  }
  cudaFree(d_temp);
}

int main(int argc, char** argv) {
  cudaMalloc(&d_temp, W*H*sizeof(float));
  resetTemperature(d_temp, W, H, bc);
  printInstructions();
  initGLUT(&argc, argv);
  gluOrtho2D(0, W, H, 0);
  glutKeyboardFunc(keyboard);
  glutMouseFunc(mouse);
  glutIdleFunc(idle);
  glutDisplayFunc(display);
  initPixelBuffer();
  glutMainLoop();
  atexit(exitfunc);
  return 0;
}
```

说完 main.cpp 中新的内容，下面让我们看看代码清单 5.7 中所展示的头文件 kernel.h 的内容：

- 第 4 行先声明 uchar4 类型。虽然 nvcc 知道如何处理 uchar4 类型的变量，但是为了防止由 nvcc 启动的 C++ 编译器产生错误，我们还是需要这句声明。
- 第 6～11 行定义了 BC 数据结构，它使我们可以将所有边界条件信息（包括管道的坐标和半径）包含在单个变量里。
- 第 13～15 行是内核包装函数（例如调用 main.cpp 的函数）kernelLauncher() 和 resetTemperature() 的原型。

代码清单5.7 heat_2d/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

struct uchar4;
// struct BC that contains all the boundary conditions
typedef struct {
  int x, y; // x and y location of pipe center
  float rad; // radius of pipe
  int chamfer; // chamfer
  float t_s, t_a, t_g; // temperatures in pipe, air, ground
} BC;

void kernelLauncher(uchar4 *d_out, float *d_temp, int w, int h,
                    BC bc);
void resetTemperature(float *d_temp, int w, int h, BC bc);

#endif
```

既然我们已经知道了 BC 结构中的变量以及 kernel.cu 中可用的函数，我们就可以看看代码清单 5.8 中所展示的 interactions.h 的详细内容了。其中第 3～11 行包括 interactions.h 中所使用的头文件。第 13～19 行定义相关参数值（包括计算网格的宽和高以及位置和温度的变化增量），并且声明了将要存储温度数据的设备数组的指针，记录从最近初始化以来的迭代次数的计数器，以及一个用来存储边界条件的 BC 结构体变量 bc。（bc 初始化时让管道位于平板中心，半径为平板宽度的十分之一，斜面为 150 个网格大小，管道温度为 212℉⊖，空气温度为 70 ℉，地面温度为 0 ℉。）

keyboard() 回调函数允许用户重置或者终止模拟过程，并且提供了对所有边界条件的控制：

- z 键重置边界条件，并且将 iterationCount 恢复为 0。
- s 键改变管道温度。
- a 键改变空气温度。
- g 键改变地面温度。
- r 键改变管道半径。
- c 键改变平板斜面。
- Esc 键终止模拟。

mouse() 回调函数读取鼠标点击的位置，并将管道中心移到所选位置。另一个相关的回调函数是 idle()，它增加迭代次数，并且调用 glutPostRedisplay()，而 glutPostRedisplay() 又会调用 render()，render() 最后调用内核启动函数。idle() 函数使得雅可比迭代在没有用户交互时不断更新（并且向期望的解收敛）。

代码清单5.8　heat_2d/interactions.h

```
 1 #ifndef INTERACTIONS_H
 2 #define INTERACTIONS_H
 3 #include "kernel.h"
 4 #include <stdio.h>
 5 #include <stdlib.h>
 6 #ifdef __APPLE__
 7 #include <GLUT/glut.h>
 8 #else
 9 #include <GL/glew.h>
10 #include <GL/freeglut.h>
11 #endif
12 #define MAX(x, y) (((x) > (y)) ? (x) : (y))
13 #define W  640
14 #define H  640
15 #define DT 1.f // source intensity increment
```

⊖ 1℉≈-17.23℃

```
16
17 float *d_temp = 0;
18 int iterationCount = 0;
19 BC bc = {W/2, H/2, W/10.f, 150, 212.f, 70.f, 0.f}; // Boundary conds
20
21 void keyboard(unsigned char key, int x, int y) {
22   if (key == 'S')  bc.t_s += DT;
23   if (key == 's')  bc.t_s -= DT;
24   if (key == 'A')  bc.t_a += DT;
25   if (key == 'a')  bc.t_a -= DT;
26   if (key == 'G')  bc.t_g += DT;
27   if (key == 'g')  bc.t_g -= DT;
28   if (key == 'R')  bc.rad += DT;
29   if (key == 'r')  bc.rad = MAX(0.f, bc.rad-DT);
30   if (key == 'C')  ++bc.chamfer;
31   if (key == 'c')  --bc.chamfer;
32   if (key == 'z')  resetTemperature(d_temp, W, H, bc);
33   if (key == 27)   exit(0);
34   glutPostRedisplay();
35 }
36
37 void mouse(int button, int state, int x, int y) {
38   bc.x = x, bc.y = y;
39   glutPostRedisplay();
40 }
41
42 void idle(void) {
43   ++iterationCount;
44   glutPostRedisplay();
45 }
46
47 void printInstructions() {
48   printf("Temperature Visualizer:\n"
49          "Relocate source with mouse click\n"
50          "Change source temperature (-/+): s/S\n"
51          "Change air temperature    (-/+): a/A\n"
52          "Change ground temperature (-/+): g/G\n"
53          "Change pipe radius        (-/+): r/R\n"
54          "Change chamfer            (-/+): c/C\n"
55          "Reset to air temperature       : z\n"
56          "Exit                           : Esc\n");
57 }
58
59 #endif
```

现在我们开始讲解代码清单 5.9 所展示的 kernel.cu 中的功能函数、内核函数，以及包装函数的细节。从功能函数开始：

- divUp() 用来计算可以覆盖一个计算网格的特定大小线程块的数量。
- clip() 用来确保颜色值的类型是 unsigned char，并且在 [0，255] 范围内。
- idxClip() 避免超出范围的采样值。idxClip(i, N) 返回一个在 [0，N-1] 区间内的 int 类型值（例如一个长度为 N 的数组的合法索引）。

- flatten() 计算一个大小为 width×height 的二维数组中第 col 列、第 row 行元素在线性一维数组中所对应的索引。注意，flatten() 会使用 idxClip() 从而避免当模板超出网格边界时试图访问不存在的数组元素。

下面介绍内核函数：

- 首先，resetKernel() 是相当简单的。它一开始使用内建的 CUDA 索引及维度变量来计算二维几何网格每个点的索引 col 和 row。如果点在图形窗口范围内，则计算其线性索引 idx，并且为网格上的每个点内存储一个默认值（这里使用空气温度）。
- tempKernel() 首先给所有点赋初始颜色黑色（完全不透明），然后将一块（包括必要的重叠单元）已经存在的温度值加载到共享内存中。
- 对于平板范围外的点，内核函数使用指定的边界值。
- 对于考察范围内的点，内核函数通过使用模板计算得到更新后的温度值，并将结果写入全局内存数组对应的位置，从而完成一步雅可比迭代。
- 最后，更新后的温度值被截取到 [0, 255] 区间范围内，转换成 unsigned char 类型数据，并将温度编码成颜色值，编码方式按照温度低的区域蓝色分量较强，温度高的区域红色分量较强。

内核函数由包装函数或者启动函数调用：

- resetTemperature() 使用必要的网格以及线程块维度调用 resetKernel()。
- kernelLauncher() 先计算必要的网格维度，然后启动 tempKernel() 计算所有的平均值、判断边界条件以及计算更新的颜色值并通过 OpenGL 交互操作进行显示。kernelLauncher() 在 main.cpp 中被 render() 调用指定次数，所以每次屏幕刷新都要进行 ITERS_PER_RENDER 次雅可比迭代。（屏幕刷新非常快，而雅可比迭代比它还快 50 倍，并且每次迭代需要计算 40 万个值。）包括 idle() 在内的每一个回调函数都以 glutPostRedisplay() 结束，glutPostRedisplay() 会调用 display()，而 display() 又会调用 50 次内核启动函数，所以只要用户没有干预，雅可比迭代就会不断进行下去。

代码清单5.9　heat_2d/kernel.cu

```
1 #include "kernel.h"
2 #define TX 32
3 #define TY 32
4 #define RAD 1
5
```

```
int divUp(int a, int b) { return (a + b - 1) / b; }

__device__
unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }

__device__
int idxClip(int idx, int idxMax) {
  return idx > (idxMax-1) ? (idxMax-1) : (idx < 0 ? 0 : idx);
}

__device__
int flatten(int col, int row, int width, int height) {
  return idxClip(col, width) + idxClip(row, height)*width;
}

__global__
void resetKernel(float *d_temp, int w, int h, BC bc) {
  const int col = blockIdx.x*blockDim.x + threadIdx.x;
  const int row = blockIdx.y*blockDim.y + threadIdx.y;
  if ((col >= w ) || (row >= h)) return;
  d_temp[row*w+col] = bc.t_a;
}

__global__
void tempKernel(uchar4 *d_out, float *d_temp, int w, int h, BC bc) {
  extern __shared__ float s_in[];
  // global indices
  const int col = threadIdx.x + blockDim.x * blockIdx.x;
  const int row = threadIdx.y + blockDim.y * blockIdx.y;
  if ((col >= w ) || (row >= h)) return;
  const int idx = flatten(col, row, w, h);
  // local width and height
  const int s_w = blockDim.x + 2*RAD;
  const int s_h = blockDim.y + 2*RAD;
  // local indices
  const int s_col = threadIdx.x + RAD;
  const int s_row = threadIdx.y + RAD;
  const int s_idx = flatten(s_col, s_row, s_w, s_h);
  // assign default color values for d_out (black)
  d_out[idx].x = 0;
  d_out[idx].z = 0;
  d_out[idx].y = 0;
  d_out[idx].w = 255;

  // Load regular cells
  s_in[s_idx] = d_temp[idx];
  // Load halo cells
  if (threadIdx.x < RAD) {
    s_in[flatten(s_col - RAD, s_row, s_w, s_h)] =
      d_temp[flatten(col - RAD, row, w, h)];
    s_in[flatten(s_col + blockDim.x, s_row, s_w, s_h)] =
      d_temp[flatten(col + blockDim.x, row, w, h)];
  }
  if (threadIdx.y < RAD) {
    s_in[flatten(s_col, s_row - RAD, s_w, s_h)] =
      d_temp[flatten(col, row - RAD, w, h)];
    s_in[flatten(s_col, s_row + blockDim.y, s_w, s_h)] =
      d_temp[flatten(col, row + blockDim.y, w, h)];
  }

```

```
66    // Calculate squared distance from pipe center
67    float dSq = ((col - bc.x)*(col - bc.x) + (row - bc.y)*(row - bc.y));
68    // If inside pipe, set temp to t_s and return
69    if (dSq < bc.rad*bc.rad) {
70      d_temp[idx] = bc.t_s;
71      return;
72    }
73    // If outside plate, set temp to t_a and return
74    if ((col == 0) || (col == w - 1) || (row == 0) ||
75        (col + row < bc.chamfer) || (col - row > w - bc.chamfer)) {
76      d_temp[idx] = bc.t_a;
77      return;
78    }
79    // If point is below ground, set temp to t_g and return
80    if (row == h - 1) {
81      d_temp[idx] = bc.t_g;
82      return;
83    }
84    __syncthreads();
85    // For all the remaining points, find temperature and set colors.
86    float temp = 0.25f*(s_in[flatten(s_col - 1, s_row, s_w, s_h)] +
87                        s_in[flatten(s_col + 1, s_row, s_w, s_h)] +
88                        s_in[flatten(s_col, s_row - 1, s_w, s_h)] +
89                        s_in[flatten(s_col, s_row + 1, s_w, s_h)]);
90    d_temp[idx] = temp;
91    const unsigned char intensity = clip((int)temp);
92    d_out[idx].x = intensity; // higher temp -> more red
93    d_out[idx].z = 255 - intensity; // lower temp -> more blue
94  }
95
96  void kernelLauncher(uchar4 *d_out, float *d_temp, int w, int h,
97                      BC bc) {
98    const dim3 blockSize(TX, TY);
99    const dim3 gridSize(divUp(w, TX), divUp(h, TY));
100   const size_t smSz = (TX+2*RAD)*(TY+2*RAD)*sizeof(float);
101   tempKernel<<<gridSize, blockSize, smSz>>>(d_out, d_temp, w, h, bc);
102 }
103
104 void resetTemperature(float *d_temp, int w, int h, BC bc) {
105   const dim3 blockSize(TX, TY);
106   const dim3 gridSize(divUp(w, TX), divUp(h, TY));
107   resetKernel<<<gridSize, blockSize>>>(d_temp, w, h, bc);
108 }
```

完成关于实现的讨论后，可以开始生成并执行 heat_2d 应用了。代码清单 5.10 是在 Linux 下生成该应用的 Makefile。对于 Visual Studio 用户，可以像对 flashlight 所做的那样来生成和执行该应用。如果你使用默认配置来启动该应用，那么你将看到它的图形界面类似图 5.2 所示。

代码清单5.10　heat_2d/Makefile

```
1 UNAME_S := $(shell uname)
2
3 ifeq ($(UNAME_S), Darwin)
4   LDFLAGS = -Xlinker -framework,OpenGL -Xlinker -framework,GLUT
5 else
```

```
  LDFLAGS += -L/usr/local/cuda/samples/common/lib/linux/x86_64
  LDFLAGS += -lglut -lGL -lGLU -lGLEW
endif

NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -Xcompiler "-Wall -Wno-deprecated-declarations"
INC = -I/usr/local/cuda/samples/common/inc

all: main.exe

main.exe: main.o kernel.o
  $(NVCC) $^ -o $@ $(LDFLAGS)

main.o: main.cpp kernel.h interactions.h
  $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@

kernel.o: kernel.cu kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

调整颜色以提高灰度对比度

考虑到当显示图像被转成灰度图片后很难区分红色（R）和蓝色（B），所以这里的图像都根据公式 $G=0.3+(R-B)/2$ 使用绿色（G）通道来提高对比度和亮度。

heat_2d 应用产生的全彩图片可以从 www.cudaforengineers.com 获取。

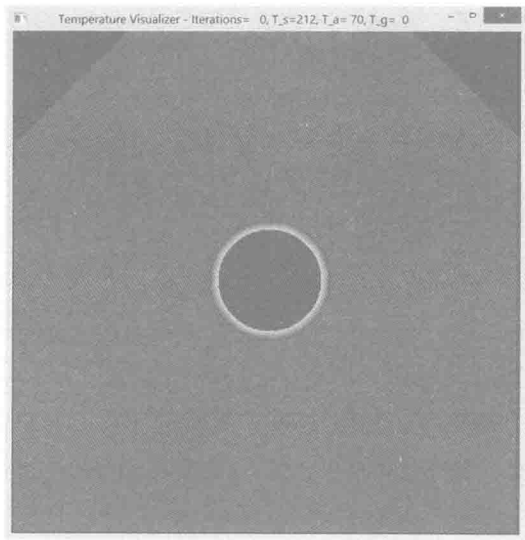


图 5.2 将颜色模式调整为灰度图像的初始配置的 heat_2d 应用

控制台窗口中打印了可用操作的列表，并且鼓励你去调节边界条件并观察温度如何变化。就像图 5.3 中所显示的，你可以生成更多有趣的温度分布。

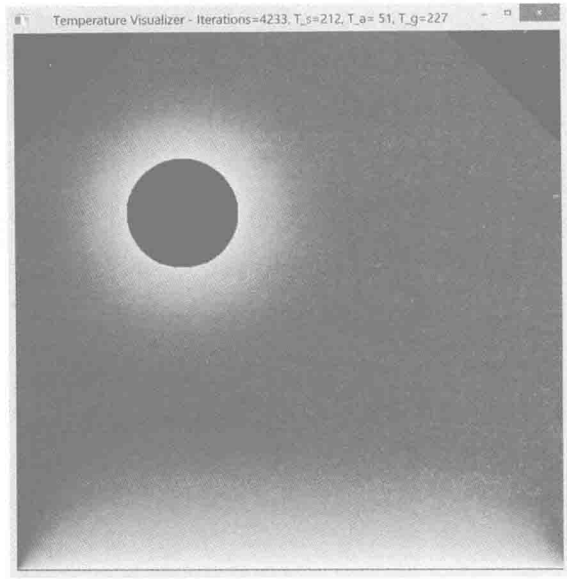


图 5.3　将颜色模式调整为灰度图像的调整配置后的 heat_2d 应用

关于 heat_2d 应用，你需要摒弃一些经验。首先，你需要注意的是，当你运行 heat_2d 时，你是在计算一个大小合适的、在一个半交互时间范围内生成一个不规则形状的二维网格（一开始为 640×640，不过你可以试着修改这个大小）上的偏微分方程的解。标题栏上显示的迭代次数可以让你直观地看到迭代速度（记住每次迭代都会调用 ITERS_PER_RENDER 次内核函数）。

注意模板加和操作已经在代码清单 5.9 的第 86～89 行中逐个明确写出了。如果你想用这个内核函数来进行更大的模板计算（例如 RAD＞1），那么你应该会想要将这个模板加和计算换成一个循环结构，所以让我们开始下一个例子，在这个例子中我们将实现一个循环结构的模板计算，并且进行一些分析和性能提升。

5.2.4　图像边缘锐化：sharpen

这里我们采用模板计算来实现一个经典的图像处理效果：边缘锐化。我们将展示这个 sharpen 应用，它对一个输入图片采用边缘锐化模板计算，并将锐化后的图片写入一个图片文件，这样你可以用喜欢的图片查看器打开查看。由于需要读写标准图片文件格式，所以我们引入了一些 CImg 的基本功能。CImg 是一个用于图像处理的、小巧的、开源的、现代 C++ 工具箱 [4]。

图像锐化也是一个模板运算，而我们所要用到的是如下所示的一个 3×3 的模板：

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

它与狄拉克 δ 内核函数（这个函数所使用的模板中间是 1，周围都是 0，不会对输入造成变化）以及平滑内核函数不同。因此锐化过程通过使用原始图像减去模糊了的版本（这种方法有时被称为非锐化掩模）来实现。注意现在这个模板是满的，所以在实现共享内存版本时，你将看到额外的代码用来存储数据块角落及边缘的重叠单元的值。

这个应用没有图形化交互功能，所以可以作为一个标准的 CUDA 7.5 应用（包含 CImg.h）来生成。完整的应用包含四个文件：代码清单 5.11 中的 sharpen/main.cpp；代码清单 5.12 中的一个头文件；代码清单 5.13 中的 Makefile（为 Linux 用户提供的）；以及一个内核文件。我们展示了三个从不同程度利用共享内存的 sharpen/kernel.cu 版本，并且提供了它们之间的性能比较：

- 一个没有使用共享内存数组的全局版本（代码清单 5.14）。
- 一个使用共享内存数组来存储输入数据的共享输入版本（代码清单 5.15）。
- 一个使用共享内存数组来加载输入以及写回输出的共享输入 / 输出版本（代码清单 5.16）。

代码清单5.11 sharpen/main.cpp

```
#include "kernel.h"
#define cimg_display 0
#include "CImg.h"
#include <cuda_runtime.h>
#include <stdlib.h>

int main() {
  cimg_library::CImg<unsigned char> image("butterfly.bmp");
  const int w = image.width();
  const int h = image.height();

  // Initialize uchar4 array for image processing
  uchar4 *arr = (uchar4*)malloc(w*h*sizeof(uchar4));

  // Copy CImg data to array
  for (int r = 0; r < h; ++r) {
    for (int c = 0; c < w; ++c) {
      arr[r*w + c].x = image(c, r, 0);
      arr[r*w + c].y = image(c, r, 1);
      arr[r*w + c].z = image(c, r, 2);
    }
  }

  sharpenParallel(arr, w, h);

```

```
26    // Copy from array to CImg data
27    for (int r = 0; r < h; ++r) {
28      for (int c = 0; c < w; ++c) {
29        image(c, r, 0) = arr[r*w + c].x;
30        image(c, r, 1) = arr[r*w + c].y;
31        image(c, r, 2) = arr[r*w + c].z;
32      }
33    }
34
35    image.save_bmp("out.bmp");
36    free(arr);
37    return 0;
38  }
```

让我们快速查看代码清单 5.11 中所示的 sharpen/main.cpp 代码的显著特点：

- 指令 #define cimg_display 0 说明将不会使用 CImg 的显示功能；我们仅仅读写图像文件，查看图像的任务由其他软件来完成。
- main() 的第一行初始化了一个 CImg 的 image 对象作为输入输出。（是的，这里涉及一些面向对象的 C++ 语言知识，但我们只会关注少量需要知道的概念，所以我们不会在这里花费太多时间。）第 8 行使用和一个图像文件对应的参数 butterfly.bmp 声明了 image。
- 第 9 行和第 10 行将变量 w 和 h 分别设置为输入图像的宽和高。
- 第 13 行以 C 语言风格声明并分配了一个 uchar4 数组用于存储图像数据。
- 第 16～22 行将 CImg 对象 image 中的图像数据复制到 in 数组中。第 18 行将 image 第 c 列、第 r 行像素第一通道（红色）值 image(c, r, 0) 存储在 in 数组对应位置的 .x 成员中：in[r*w + c].x = inImage(c, r, 0);。第 19 行和第 20 行以相同的方式存储绿色以及蓝色分量。
- 第 24 行调用内核包装函数 sharpenParallel() 来对输入数据进行锐化模板计算，并将结果存储到 out 中。
- 第 27～33 行将输出数据（存储在数组 out 中）复制到 CImg 对象 image 中。
- 第 35 行调用 .save_bmp 方法（CImg 对象提供的函数）将锐化后的结果保存到一个叫作 out.bmp 的标准可移植位图（.bmp）格式文件中。
- 第 36 行释放分配的数组。

sharpen/kernel.h 或 sharpen/Makefile 中没有什么新东西，这里略过它们直接去看内核函数的实现。

代码清单5.12 sharpen/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

struct uchar4;

void sharpenParallel(uchar4 *arr, int w, int h);
#endif
```

代码清单5.13 sharpen/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -Xcompiler -Wall

all: main.exe

main.exe: main.o kernel.o
  $(NVCC) $^ -o $@

main.o: main.cpp kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@

kernel.o: kernel.cu kernel.h
  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

现在我们来到了第一个关注焦点：代码清单 5.14 中全局内存版本的内核函数实现。我们希望内核函数代码一开始看起来是熟悉的，这样我们就可以仅仅注意一些重要的新概念：

- 第 24～27 行定义一般的行、列，以及线性索引。它们也进行边界检查。
- 第 28 行定义模板的大小（宽或高）fltSz。我们使用前缀 flt（就像 filter 中一样）来表示跟模板相关的量。
- 第 31～41 行的嵌套 for 循环用来累加模板贡献，循环的索引 cd 和 rd 分别是从掩模中心像素到某一指定像素的列偏移和行偏移。
- 注意输入数据是从全局内存数组 d_in 中读取的，并被写入到全局内存数组 d_out 中。

代码清单5.14 sharpen/kernel.cu全局内存实现

```
#include "kernel.h"
#define TX 32
#define TY 32
#define RAD 1

int divUp(int a, int b) { return (a + b - 1) / b; }

__device__
unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }

```

```
__device__
int idxClip(int idx, int idxMax) {
  return idx > (idxMax-1) ? (idxMax-1) : (idx < 0 ? 0 : idx);
}

__device__
int flatten(int col, int row, int width, int height) {
  return idxClip(col, width) + idxClip(row, height)*width;
}

__global__
void sharpenKernel(uchar4 *d_out, const uchar4 *d_in,
                   const float *d_filter, int w, int h) {
  const int c = threadIdx.x + blockDim.x * blockIdx.x;
  const int r = threadIdx.y + blockDim.y * blockIdx.y;
  if ((c >= w) || (r >= h)) return;
  const int i = flatten(c, r, w, h);
  const int fltSz = 2*RAD + 1;
  float rgb[3] = {0.f, 0.f, 0.f};

  for (int rd = -RAD; rd <= RAD; ++rd) {
    for (int cd = -RAD; cd <= RAD; ++cd) {
      int imgIdx = flatten(c + cd, r + rd, w, h);
      int fltIdx = flatten(RAD + cd, RAD + rd, fltSz, fltSz);
      uchar4 color = d_in[imgIdx];
      float weight = d_filter[fltIdx];
      rgb[0] += weight*color.x;
      rgb[1] += weight*color.y;
      rgb[2] += weight*color.z;
    }
  }
  d_out[i].x = clip(rgb[0]);
  d_out[i].y = clip(rgb[1]);
  d_out[i].z = clip(rgb[2]);
}

void sharpenParallel(uchar4 *arr, int w, int h) {
  const int fltSz = 2 * RAD + 1;
  const float filter[9] = {-1.0, -1.0, -1.0,
                           -1.0,  9.0, -1.0,
                           -1.0, -1.0, -1.0};

  uchar4 *d_in = 0, *d_out = 0;
  float *d_filter = 0;

  cudaMalloc(&d_in, w*h*sizeof(uchar4));
  cudaMemcpy(d_in, arr, w*h*sizeof(uchar4), cudaMemcpyHostToDevice);

  cudaMalloc(&d_out, w*h*sizeof(uchar4));

  cudaMalloc(&d_filter, fltSz*fltSz*sizeof(float));
  cudaMemcpy(d_filter, filter, fltSz*fltSz*sizeof(float),
             cudaMemcpyHostToDevice);

  const dim3 blockSize(TX, TY);
  const dim3 gridSize(divUp(w, blockSize.x), divUp(h, blockSize.y));

  sharpenKernel<<<gridSize, blockSize>>>(d_out, d_in, d_filter, w, h);

  cudaMemcpy(arr, d_out, w*h*sizeof(uchar4), cudaMemcpyDeviceToHost);
```

```
71   cudaFree(d_in);
72   cudaFree(d_out);
73   cudaFree(d_filter);
74 }
```

代码清单 5.15 中显示了第二种实现。重要的改变包括下面几点：

- 第 24 行的创建共享数组 s_in。
- 第 36～61 行将一块输入数据加载进共享数组中，包括第 50～60 行的边界重叠值以及第 40～49 行的角落重叠值。
- 第 102 行启动的内核函数包括第三个执行配置参数 smSz 用来指定为动态共享数组分配的存储字节数。

代码清单5.15 共享内存输入数组的sharpen/kernel.cu

```
 1 #include "kernel.h"
 2 #define TX 32
 3 #define TY 32
 4 #define RAD 1
 5
 6 int divUp(int a, int b) { return (a + b - 1) / b; }
 7
 8 __device__
 9 unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }
10
11 __device__
12 int idxClip(int idx, int idxMax) {
13   return idx > (idxMax-1) ? (idxMax-1) : (idx < 0 ? 0 : idx);
14 }
15
16 __device__
17 int flatten(int col, int row, int width, int height) {
18   return idxClip(col, width) + idxClip(row, height)*width;
19 }
20
21 __global__
22 void sharpenKernel(uchar4 *d_out, const uchar4 *d_in,
23                    const float *d_filter, int w, int h) {
24   extern __shared__ uchar4 s_in[];
25   const int c = threadIdx.x + blockDim.x * blockIdx.x;
26   const int r = threadIdx.y + blockDim.y * blockIdx.y;
27   if ((c >= w) || (r >= h)) return;
28   const int i = flatten(c, r, w, h);
29   const int s_c = threadIdx.x + RAD;
30   const int s_r = threadIdx.y + RAD;
31   const int s_w = blockDim.x + 2*RAD;
32   const int s_h = blockDim.y + 2*RAD;
33   const int s_i = flatten(s_c, s_r, s_w, s_h);
34   const int fltSz = 2*RAD + 1;
35
36   // Regular cells
37   s_in[s_i] = d_in[i];
38
39   // Halo cells
40   if (threadIdx.x < RAD && threadIdx.y < RAD) {
```

```
    s_in[flatten(s_c - RAD, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c - RAD, r - RAD, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r - RAD, w, h)];
    s_in[flatten(s_c - RAD, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c - RAD, r + blockDim.y, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r + blockDim.y, w, h)];
  }
  if (threadIdx.x < RAD) {
    s_in[flatten(s_c - RAD, s_r, s_w, s_h)] =
      d_in[flatten(c - RAD, r, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r, w, h)];
  }
  if (threadIdx.y < RAD) {
    s_in[flatten(s_c, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c, r - RAD, w, h)];
    s_in[flatten(s_c, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c, r + blockDim.y, w, h)];
  }
  __syncthreads();

  float rgb[3] = {0.f, 0.f, 0.f};
  for(int rd = -RAD; rd <= RAD; ++rd) {
    for(int cd = -RAD; cd <= RAD; ++cd) {
      const int s_imgIdx = flatten(s_c + cd, s_r + rd, s_w, s_h);
      const int fltIdx = flatten(RAD + cd, RAD + rd, fltSz, fltSz);
      const uchar4 color = s_in[s_imgIdx];
      const float weight = d_filter[fltIdx];
      rgb[0] += weight*color.x;
      rgb[1] += weight*color.y;
      rgb[2] += weight*color.z;
    }
  }
  d_out[i].x = clip(rgb[0]);
  d_out[i].y = clip(rgb[1]);
  d_out[i].z = clip(rgb[2]);
}

void sharpenParallel(uchar4 *arr, int w, int h) {
  const int fltSz = 2 * RAD + 1;
  const float filter[9] = {-1.0, -1.0, -1.0,
                           -1.0,  9.0, -1.0,
                           -1.0, -1.0, -1.0};

  uchar4 *d_in = 0, *d_out = 0;
  float *d_filter = 0;

  cudaMalloc(&d_in, w*h*sizeof(uchar4));
  cudaMemcpy(d_in, arr, w*h*sizeof(uchar4), cudaMemcpyHostToDevice);

  cudaMalloc(&d_out, w*h*sizeof(uchar4));

  cudaMalloc(&d_filter, fltSz*fltSz*sizeof(float));
  cudaMemcpy(d_filter, filter, fltSz*fltSz*sizeof(float),
             cudaMemcpyHostToDevice);

  const dim3 blockSize(TX, TY);
  const dim3 gridSize(divUp(w, TX), divUp(h, TY));
  const size_t smSz = (TX+2*RAD)*(TY+2*RAD)*sizeof(uchar4);
```

```
102   sharpenKernel<<<gridSize, blockSize, smSz>>>(d_out, d_in, d_filter,
103                                                 w, h);
104
105   cudaMemcpy(arr, d_out, w*h*sizeof(uchar4), cudaMemcpyDeviceToHost);
106   cudaFree(d_in);
107   cudaFree(d_out);
108   cudaFree(d_filter);
109 }
```

代码清单 5.16 中的第三个版本为读取输入和写回输出都创建了一个共享内存数组。因为只能创建一个共享数组，所以在第 35～37 行以及第 106 行采用了一点小技巧。

代码清单5.16　输入和输出都采用共享内存数组的sharpen/kernel.cu

```
 1 #include "kernel.h"
 2 #define TX 32
 3 #define TY 32
 4 #define RAD 1
 5
 6 int divUp(int a, int b) { return (a + b - 1) / b; }
 7
 8 __device__
 9 unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }
10
11 __device__
12 int idxClip(int idx, int idxMax) {
13   return idx > (idxMax-1) ? (idxMax-1) : (idx < 0 ? 0 : idx);
14 }
15
16 __device__
17 int flatten(int col, int row, int width, int height) {
18   return idxClip(col, width) + idxClip(row, height)*width;
19 }
20
21 __global__
22 void sharpenKernel(uchar4 *d_out, const uchar4 *d_in,
23                    const float *d_filter, int w, int h) {
24   const int c = threadIdx.x + blockDim.x * blockIdx.x;
25   const int r = threadIdx.y + blockDim.y * blockIdx.y;
26   if ((c >= w) || (r >= h)) return;
27   const int i = flatten(c, r, w, h);
28   const int s_c = threadIdx.x + RAD;
29   const int s_r = threadIdx.y + RAD;
30   const int s_w = blockDim.x + 2*RAD;
31   const int s_h = blockDim.y + 2*RAD;
32   const int s_i = flatten(s_c, s_r, s_w, s_h);
33   const int fltSz = 2*RAD + 1;
34
35   extern __shared__ uchar4 s_block[];
36   uchar4 *s_in = s_block;
37   uchar4 *s_out = &s_block[s_w*s_h];
38
39   // Regular cells
40   s_in[s_i] = d_in[i];
41
42   // Halo cells
43   if (threadIdx.x < RAD && threadIdx.y < RAD) {
```

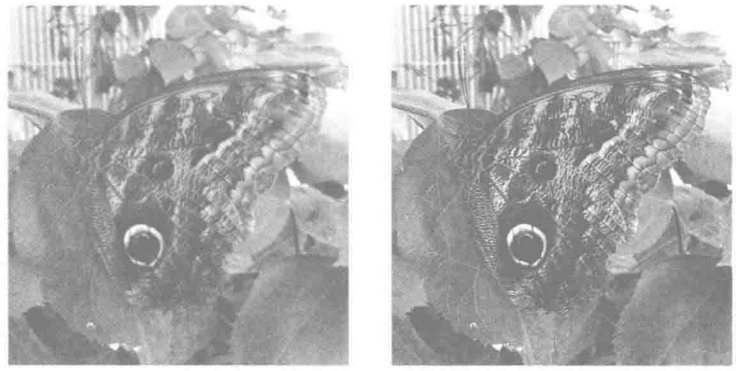
```
    s_in[flatten(s_c - RAD, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c - RAD, r - RAD, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r - RAD, w, h)];
    s_in[flatten(s_c - RAD, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c - RAD, r + blockDim.y, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r + blockDim.y, w, h)];
  }
  if (threadIdx.x < RAD) {
    s_in[flatten(s_c - RAD, s_r, s_w, s_h)] =
      d_in[flatten(c - RAD, r, w, h)];
    s_in[flatten(s_c + blockDim.x, s_r, s_w, s_h)] =
      d_in[flatten(c + blockDim.x, r, w, h)];
  }
  if (threadIdx.y < RAD) {
    s_in[flatten(s_c, s_r - RAD, s_w, s_h)] =
      d_in[flatten(c, r - RAD, w, h)];
    s_in[flatten(s_c, s_r + blockDim.y, s_w, s_h)] =
      d_in[flatten(c, r + blockDim.y, w, h)];
  }
  __syncthreads();

  float rgb[3] = {0.f, 0.f, 0.f};
  for(int rd = -RAD; rd <= RAD; ++rd) {
    for(int cd = -RAD; cd <= RAD; ++cd) {
      const int s_imgIdx = flatten(s_c + cd, s_r + rd, s_w, s_h);
      const int fltIdx = flatten(RAD + cd, RAD + rd, fltSz, fltSz);
      const uchar4 color = s_in[s_imgIdx];
      const float weight = d_filter[fltIdx];
      rgb[0] += weight*color.x;
      rgb[1] += weight*color.y;
      rgb[2] += weight*color.z;
    }
  }

  const int s_outIdx = threadIdx.y*blockDim.x + threadIdx.x;
  s_out[s_outIdx].x = clip(rgb[0]);
  s_out[s_outIdx].y = clip(rgb[1]);
  s_out[s_outIdx].z = clip(rgb[2]);

  __syncthreads();
  d_out[i] = s_out[s_outIdx];
}

void sharpenParallel(uchar4 *arr, int w, int h) {
  const int fltSz = 2 * RAD + 1;
  const float filter[9] = {-1.0, -1.0, -1.0,
                           -1.0,  9.0, -1.0,
                           -1.0, -1.0, -1.0};
  uchar4 *d_in = 0, *d_out = 0;
  float *d_filter = 0;

  cudaMalloc(&d_in, w*h*sizeof(uchar4));
  cudaMemcpy(d_in, arr, w*h*sizeof(uchar4), cudaMemcpyHostToDevice);
  cudaMalloc(&d_out, w*h*sizeof(uchar4));
  cudaMalloc(&d_filter, fltSz*fltSz*sizeof(float));
  cudaMemcpy(d_filter, filter, fltSz*fltSz*sizeof(float),
             cudaMemcpyHostToDevice);

```

```
104    const dim3 blockSize(TX, TY);
105    const dim3 gridSize(divUp(w, TX), divUp(h, TY));
106    const size_t smSz = ((TX+2*RAD)*(TY+2*RAD)+(TX*TY))*sizeof(uchar4);
107    sharpenKernel<<<gridSize, blockSize, smSz>>>(d_out, d_in, d_filter,
108                                                 w, h);
109    cudaMemcpy(arr, d_out, w*h*sizeof(uchar4), cudaMemcpyDeviceToHost);
110    cudaFree(d_in);
111    cudaFree(d_out);
112    cudaFree(d_filter);
113 }
```

第二个版本中我们创建了一个大小为 smSz 的共享数组 s_block，但是现在内核启动函数 sharpenParallel() 在第 106 行增加了分配大小。uchar4 变量的个数现在被设置为输入数据块的像素数量 ((TX+2*RAD)*(TY+2*RAD)) 和输出数据块的像素数量 (TX*TY) 的和。第 36 行和第 37 行将 s_block 分开，得到的效果就是看起来分别有对应于输入和输出的共享数组。

其中第 36 行的 uchar4 *s_in = s_block 创建了一个指向共享数组起始地址的指针，而第 37 行 uchar4 *s_out = &s_block[s_w*s_h] 在创建一个指向输入数据块存储区域下一位置的指针。这样设置后，我们写代码时就可以将 s_in 和 s_out 当作分开的数组了，后台的一些指针运算使得这样做是完全有效的。在第 81～83 行中，输出像素值被写入共享内存，并且调用 __syncthreads() 保证在将共享输出数组中的内容复制到全局内存 d_out 之前，所有输出值都已经被写入共享内存 s_out 中。内核函数完成后，结果从 d_out 再次转移到主机数组 arr 中，CImg 可以通过 arr 创建并写入输出图像文件。

浏览完代码，现在可以生成、运行这个程序并检查结果了。（确认在编译程序之前在项目目录下包含一个 butterfly.bmp 以及 CImg.h 的副本。）在图 5.4 中，我们选择了一个输入图像 butterfly.bmp 和锐化后的输出图像一起展示。图像展示了一个大猫头鹰蝶，锐化对它翅膀上的纹路结构产生了显著的增强。



a) 原始图　　b) 锐化后图

图 5.4　大猫头鹰蝶图像

测试 sharpen 程序，并且确认三个版本所产生的输出图像是相同的。确定结果一致之后，让我们来对比一下它们的性能。

我们将使用 NVIDIA Visual Profiler (NVVP) 来生成性能信息。关于 NVVP 的详细介绍可以在附录 D 中找到。启动 NVVP，打开 New Session（新建会话），填写 File（文件）入口，即你想要分析的可执行文件的完整路径。你还需要制定一个 Working Directory（工作目录）：如果可执行文件和图像所在文件夹不同，则给出输入图像所存储的文件夹。

> 性能分析提示
>
> 为了避免可能的错误，你应该在程序结束处增加代码 cudaDeviceReset(); 并在使用 NVVP 进行分析之前重新生成该应用。

图 5.5 显示了全局内存版本 sharpen 程序的分析结果，其中第一条警告（在右上的 Properties 标签和底部的 Analysis 标签中都有）是关于 Low Global MemoryLoad Efficiency 的，这提示我们若使用共享内存实现可能提高存储传输效率。

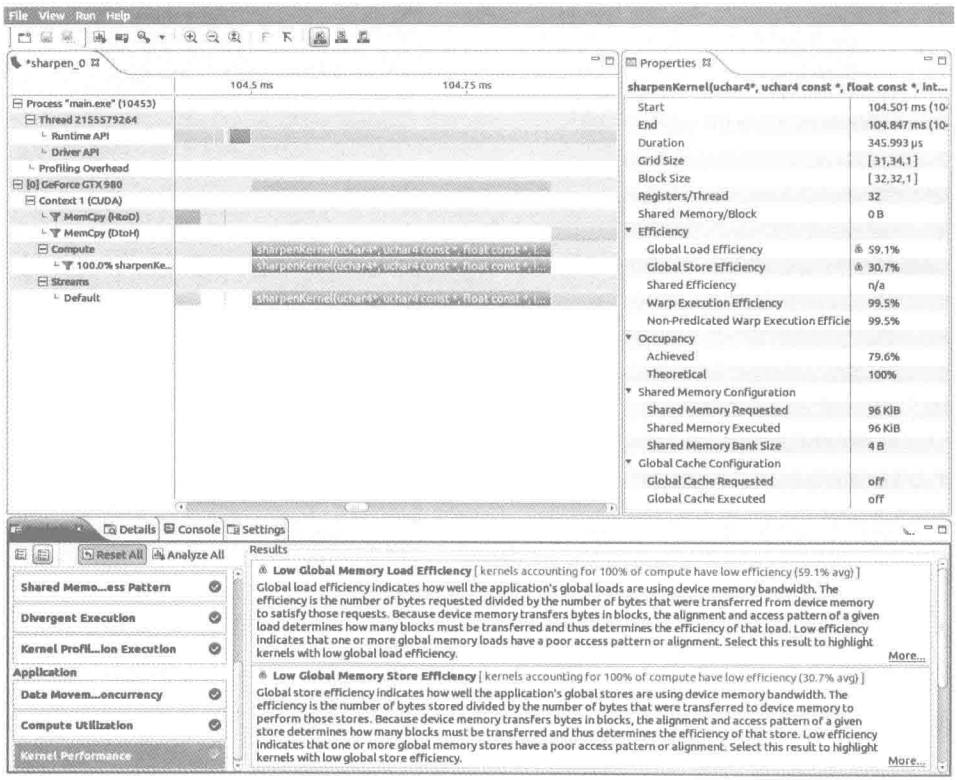


图 5.5　对应全局内存实现的 sharpen 程序的 Visual Profiler 结果

图 5.6 显示了共享对应输入数组版本 sharpen 程序的分析结果。Properties 标签显示 Global Store Efficiency 现在从 59.1% 提升到了 83.7%。内核函数持续时间（Duration）从先前的 345.993 μs 下降到 292.232μs，下降了 20%。

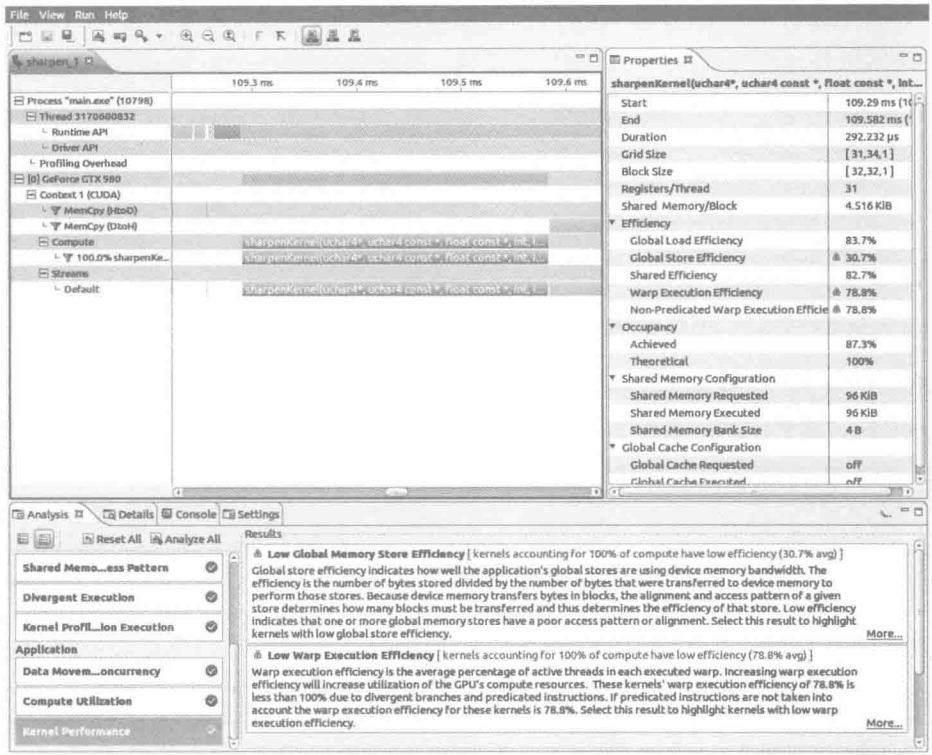


图 5.6　将输入数据加载到共享内存数组实现方式的 sharpen 程序的分析结果

结果标签中已经没有 Low Global Memory Load Efficiency 警告了，但是仍然有一个 Low Global Memory Store Efficiency 警告，并且相关的 Properties 标签中显示了 30.7% 的 Global Store Efficiency。同时有了一个新的 78.8% 的 Low Warp Execution Efficiency 警告。这个警告与写回内核函数运算结果的操作有关，而使用共享内存来存回输出数据是一个解决该警告的可能方法。

让我们看看图 5.7 中对输入、输出都使用共享内存的内核函数的分析结果，从而判断这种方法的有效性。在 Visual Profiler 窗口底部的 Analysis 标签中，选择 Kernel Performance，可以看到 Results 面板中显示了一个绿色背景的对号和一条 No Issues 信息。Properties 标签中的结果全部没有了警告。内核函数运行时间也进一步降低到了 287.336μs，但是这相较只使用共享输入数组的实现方案在时间上仅仅提高了约 2%。

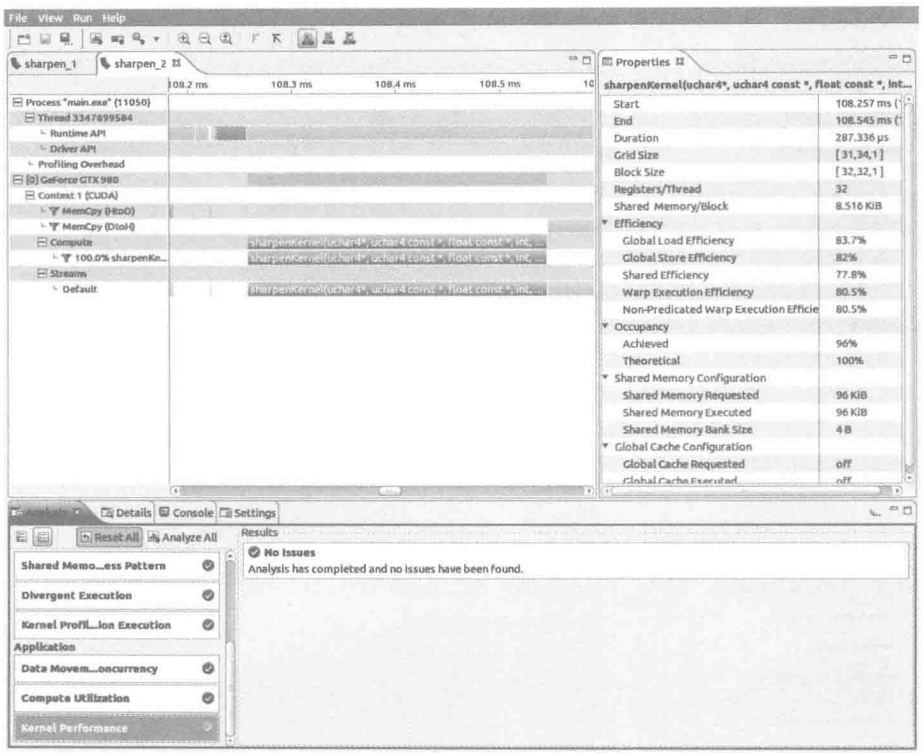


图 5.7　对输入、输出数据都使用共享内存实现方式的 sharpenKernel() 的分析结果

最终，系统可以通过将多个输出数据元素打包在单个存储事务中，来提高写回操作的效率，而不是对每一个输出元素都进行一次全局内存写操作。

另一个共享输入数组分析会话中的警告是 Low Warp Execution Efficiency (78.8%)。而共享输入 / 输出版本将线程束执行效率提高到了 80.5%，并且消除了该警告。这说明我们可以通过调整线程块规模进一步提升线程束执行效率。图 5.8 中显示了将线程块规模从 32×32 改为 128×8 后的结果。现在 Warp Execution Efficiency 高达 92.9%，并且内核函数运行时间降低到了 281.704μs（又提升了约 2%）。

总之，分析结果显示在使用共享内存以及调整线程块规模后，内核函数总体上获得了 25% 的性能提升。通过对输入数组使用共享内存，在存储传输效率以及内核函数时间测量方面都取得了显著的提升。为输出数组使用共享内存产生了显著的全局内存写回效率提升，但是对于内核函数运行时间的减少很有限。在这个例子中，我们通过增加线程块规模得到了一点回报。但是调整运行配置参数是一个凭经验的过程；对于特定的系统以及输入数据，需要通过实验来确定最好的线程块规模，而一个产生线程块规模候选值

的可靠方法是选择 32 的倍数作为 blockDim.x 的值。

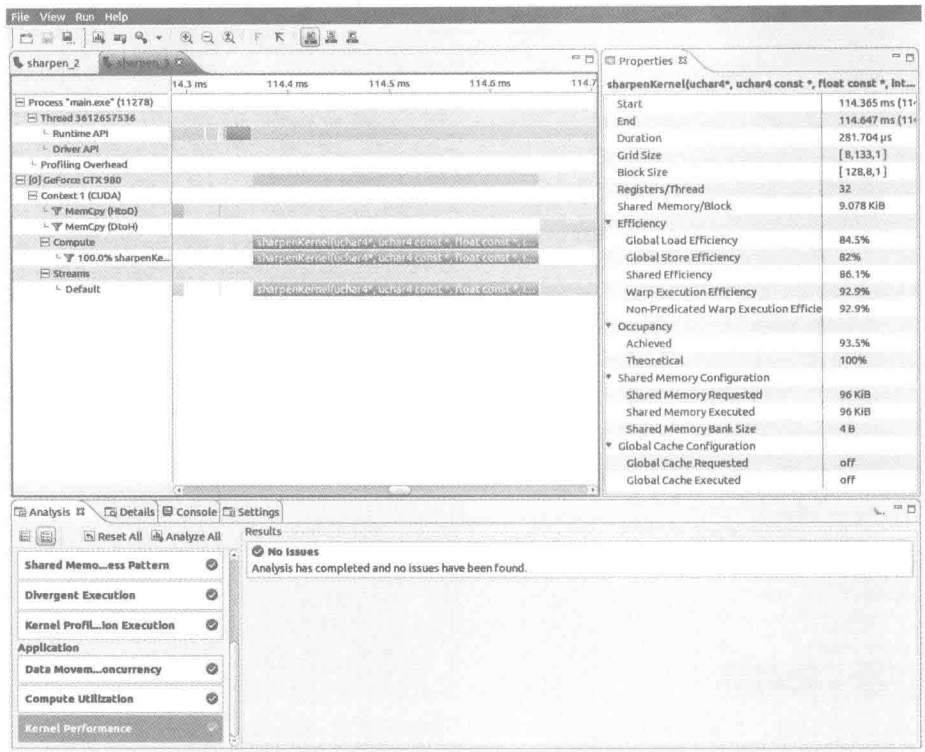


图 5.8 输入和输出数据都采用共享内存，线程块规模调整为 128×8 的 sharpenKernel() 的分析结果

5.3 本章小结

本章聚焦于一类特别的应用问题，其中的计算线程需要和网格中相邻的线程合作，而不是完全独立的。我们将与相邻线程间的一致的交互统一表述成模板计算，并且实现了三个此类的应用程序：通过离散值计算一个函数的导数（dd_1d_global 和 dd_1d_shared），通过雅可比迭代求解稳态温度分布的拉普拉斯方程（heat_2d），以及图像边缘增强（sharpen）。我们展示了仅使用存储在全局内存中的数组的初始实现，然后说明如何使用共享内存来改进实现方案。最后我们使用 NVIDIA Visual Profiler (NVVP) 来检测 sharpen 程序的三种不同版本的性能：一个仅使用全局内存数组，第二个采用了共享输入数组，最后一个版本是输入 / 输出数组都采用共享内存。我们发现对于分析时所采用的特定样本图像、执行参数，以及用于执行的 GPU，将输入数据加载进共享内存在全局加载效率以及内核函数运行时间方面都产生了显著的提升。进一步使用共享内存来存储或

者传输输出数据在全局存储效率方面产生了显著的提升，但是对于内核函数运行时间方面仅有少量降低。最后通过调整线程块规模又取得了一点提升。

这一状况恰恰反映了代码性能优化上的一个现实情况：性能提升和你在优化上所花的时间及努力可能并不成比例。在着手开始一项主要的优化工作前停下来并思考预期的代价及可能的收益是有好处的。记住，SIMT 并行模式的可扩展性是非常好的，所以有时候正确的答案只是简单地请求更多的处理器。你也应该记住阿姆达尔定律，它限制了从 1 个处理器上升到 N 个处理器并行所能获得的加速比 S：

$$S(N)=\frac{1}{(1-P)+P/N}$$

其中 P 是所有可以被并行化的部分。例如，如果一个计算任务的 90% 可以并行化，那么即使有无限多的处理器，剩下 10% 的工作仍然需要串行执行，所以使用并行可获得的最大加速比是 10 倍。

5.4　推荐项目

1. 使用以下三种方法改进 heat_2d 应用。
   a. 使用 9 点式拉普拉斯模板：

$$\left(\frac{1}{20}\right)\begin{bmatrix}1 & 4 & 1\\ 4 & -20 & 4\\ 1 & 4 & 1\end{bmatrix}$$

   b. 使用一个更大的拉普拉斯模板。
   c. 导入一张图片对其进行划分或者阈值化，从而创建一个不规则的区域，在该区域上使用边界条件。
2. heat_2d 应用提供了对输出图像的可视化以及一个迭代计数器，你认为什么可以作为收敛到稳态的可靠度量值？如何实现你的标准？
3. 围绕 sharpen 应用进行实验，看下面的改变对性能有何影响（使用 Visual Profiler 进行度量）：
   a. 选择不同尺寸的样本图像。
   b. 改变执行参数（例如线程块大小以及网格大小）。
   c. 选择不同大小的模板。

参考文献

[1] NVIDIA Corporation. "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[2] Mark Harris, "GPU Pro Tip: CUDA 7 Streams Simplify Concurrency." *Parallel Forall* (blog), 2015, http://devblogs.nvidia.com/parallelforall/gpu-pro-tip-cuda-7-streams-simplify-concurrency/.

[3] Randall J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations* (Philadelphia, PA: SIAM, 2007).

[4] David Tschumperlé, The CImg Library, 2015. http://cimg.eu.

第 6 章 Chapter 6

归约与原子操作

本章将处理一种非常有用的计算问题，其中所有线程互相作用最终只产生一个输出。很多这样的运算实际都是一种被称为归约的模式，其中包含一个输入数组，该数组的元素被组合直到产生一个输出值。包括点积（又称为内积或者数量积）、图像相似度测量、整体性质以及（稍微一般化的）直方图在内的应用都是这样的。

6.1 全局交互的线程

在第 5 章中，我们在处理计算线程之间的交互方面踏出了重要的第一步，然而模板计算仅仅包括网格中相邻线程之间的局部交互。现在我们准备处理的计算问题中所有线程都互相作用来产生一个输出。

从由模板计算所引出的邻近线程间的交互问题出发，在考虑一般的线程间交互前，需要思考并回答一些问题。

问：实际应用中的确会出现归约运算吗？

答：是的，它们经常出现。点积开始可能只是逐项的乘法，但是最后需要将所有的乘积求和；每一项都对和有用，所以该运算是归约。矩阵可以看作是向量的列表，所以矩阵 – 向量以及矩阵 – 矩阵乘法包含归约。通过对大量元素的作用进行求和得到的整体性质（例如质心以及惯性矩）包含归约运算。

问：归约和我们已经见过的应用有关系吗？

答：是的。在使用雅可比迭代求解稳态温度分布问题时，我们只是启动迭代，并且让它向稳态运行。在该章的最后，有一个训练任务询问停止的标准是什么：如何判断当前状态已经接近平衡，所以可以停止计算了？一般的方法是当进一步迭代时在温度值数组上不会再产生明显变化就停止运算。评判变化是否明显需要对温度数组的变化进行测量。无论你使用二范数（换句话说就是所有元素通过平方并求和进行组合的多维毕达哥拉斯公式）或者是无穷范数（比较数组元素并选择较大的绝对值进行组合），所有的元素都会互相作用并影响结果。上述计算中包括一个归约。

问：既然归约如此普遍，那么为什么 CUDA 7.5 Visual Studio 样板的示例代码中只有逐项操作（加法）？

答：逐项运算是易于并行化的部分。所有线程间都没有交互，这有时被称为乐不可支的并行。是的，我们可以将逐项的加法变成乘法，而且这样差不多就是要实现点乘积了；剩下的就是对所有逐项乘积求和，而这正是并行化中最具挑战性的部分。

问：既然归约这么重要，是不是应该有实现好的代码库可以供我使用？

答：是的，有可用的库实现了归约。在本章中我们提供了足够的背景知识，所以你可以实现自己的归约。我们将在第 8 章中讨论使用库来实现归约，如果你急需实现归约功能你也可以现在就阅读该章。

看完问答部分后，让我们从点积应用开始实现一些特别的归约。

6.2 实现 parallel_dot

我们先创建一个叫作 parallel_dot 的应用来并行计算一对输入向量的点积。输入向量 a 和 b 存储在两个长度都为 N 的数组中，输出是两个数组相应元素的乘积，之后求和得到的单个标量（存储在变量 res 中）：

$$res = \sum_{idx=0}^{N-1} a[idx]*b[idx]$$

因为每个元素都会对 res 产生影响，所以这是一个典型的归约问题（可能也是最常见的归约）。任何归约都需要一个变量（如 res）用于累加所有线程的贡献。在 CUDA 的 SIMT 并行模型中，由一个线程创建的私有变量（一般存储在寄存器中）不能被任何其他线程访问。而被声明为 __shared__ 的变量（存储在线程块的共享内存中）不能被线程块外的任何线程访问。为了避免这些限制，归约需要一个在内核外部声明并存储在全局内

存中的累加变量。

一个简单的方法是让每个线程从全局内存的输入数组中读取一对相应的元素，将它们相乘，并将结果与存储在全局累加变量 d_res（最终结果的设备端版本，在所有计算完成后将被复制回主机端的 res 中）中的值相加。如果你刚开始熟悉 CUDA，上述方法即使没让你恐惧或者厌恶，至少也会引起一点反感。是的，输入数据至少需要从全局内存中读取一次，那是无法避免的。然而，让每个线程都将自己的结果写入全局内存中并不是什么好主意。通过采取数据块的方法，我们将大的输入向量分割成线程块大小的片段，每个线程块只更新一次 d_res（而不是每个线程更新一次），这样可以大大减小全局内存拥塞。这种方案简述如下：

- 创建一个共享内存数组用于存储输入数组在对应分块中相应元素的乘积。
- 在继续处理之前进行同步，以保证上述共享数组已经完全填满。
- 指定一个线程（例如线程 threadIdx.x==0）将线程块共享数组（块中的所有线程都可以访问其中的数据）中所有值循环累加并存储到一个寄存器变量 blockSum 中。
- 进行一次全局内存传输（针对整个线程块）将 blockSum 加到全局累加变量 d_res 上。

代码清单 6.1 展示了使用共享内存实现的点积内核函数 dotKernel() 以及包装函数 dotLauncher()。像通常一样，我们包含头文件以及标准输入 / 输出库，然后定义包括 TBP（每线程块线程数）在内的一些常量和我们很快将介绍的 ATOMIC。在第 8～10 行，内核函数一开始像往常一样计算全局和共享索引，并且检查边界。共享数组 s_prod 在第 12 行声明。（s_ 提醒我们这个数组是共享的，而 prod 说明它将存储输入数组中对应元素的乘积。）在第 13 行，输入数组全局索引处的元素相乘，并将乘积存储在 s_prod 的局部索引处。第 14 行的 __syncthreads() 保证在任何线程继续运行之前 s_prod 数组已经被完全填满。在我们的做法中只有第一个线程（threadIdx.x==0）需要继续运行。它在第 17 行初始化寄存器变量 blockSum=0，然后在第 17～20 行中，在共享数组索引范围内将 s_prod 循环累加到 blockSum 上。我们已经在第 21 行包括了一条打印语句，这样你就可以检查 blockSum 值，并清楚地看到所有线程块并不是按指定顺序执行的。虽然线程块内的累加可以用更加并行化的风格进行，但是让我们在创建并测试完一个初始版本的 parallel_dot 之前暂且先讨论这样一个可选方案。

最后，在第 23～26 行，一个块中乘积的贡献值 blockSum 被加到全局累加变量 d_res 中。注意这里实际发生的是一个读 – 写序列操作，而且通过 #define 设置的 ATOMIC

值被编码成两种完全不同的运行方式。当 #define ATOMIC 0 时，将进行普通的加法。当 #define ATOMIC 1 时，将调用 atomicAdd(d_res, blockSum) 函数将 d_res 的值累加到 blockSum。让我们先看看这个工程所需要的其他文件，测试运行它们，看看实际工作情况。

代码清单6.1 parallel_dot/kernel.cu 包括dotKernel()以及包装函数dotLauncher()

```
#include "kernel.h"
#include <stdio.h>
#define TPB 64
#define ATOMIC 1 // 0 for non-atomic addition

__global__
void dotKernel(int *d_res, const int *d_a, const int *d_b, int n) {
  const int idx = threadIdx.x + blockDim.x * blockIdx.x;
  if (idx >= n) return;
  const int s_idx = threadIdx.x;

  __shared__ int s_prod[TPB];
  s_prod[s_idx] = d_a[idx] * d_b[idx];
  __syncthreads();

  if (s_idx == 0) {
    int blockSum = 0;
    for (int j = 0; j < blockDim.x; ++j) {
      blockSum += s_prod[j];
    }
    printf("Block_%d, blockSum = %d\n", blockIdx.x, blockSum);
    // Try each of two versions of adding to the accumulator
    if (ATOMIC) {
      atomicAdd(d_res, blockSum);
    } else {
      *d_res += blockSum;
    }
  }
}

void dotLauncher(int *res, const int *a, const int *b, int n) {
  int *d_res;
  int *d_a = 0;
  int *d_b = 0;

  cudaMalloc(&d_res, sizeof(int));
  cudaMalloc(&d_a, n*sizeof(int));
  cudaMalloc(&d_b, n*sizeof(int));

  cudaMemset(d_res, 0, sizeof(int));
  cudaMemcpy(d_a, a, n*sizeof(int), cudaMemcpyHostToDevice);
  cudaMemcpy(d_b, b, n*sizeof(int), cudaMemcpyHostToDevice);

  dotKernel<<<(n + TPB - 1)/TPB, TPB>>>(d_res, d_a, d_b, n);
  cudaMemcpy(res, d_res, sizeof(int), cudaMemcpyDeviceToHost);

  cudaFree(d_res);
  cudaFree(d_a);
  cudaFree(d_b);
}
```

代码清单 6.2 展示了头文件 kernel.h。其中包含防止重复包含的宏定义，以及使包装函数 dotLauncher() 可以被 main() 调用的原型说明。

代码清单6.2　parallel_dot/kernel.h

```
1 #ifndef KERNEL_H
2 #define KERNEL_H
3
4 void dotLauncher(int *res, const int *a, const int *b, int n);
5
6 #endif
```

parallel_dot 应用的 main.cpp 如代码清单 6.3 所示。一开始它声明了 cpu_res（用于存储 CPU 版本的结果）和 gpu_res（用于存储 GPU 版本的结果）。第 9 行和第 10 行声明指针型输入数组 a 和 b。第 9～16 行分配并初始化了输入数组的存储空间（为了保持简单所有元素都为 1）。第 18～21 行计算并向控制台打印了串行参考结果。第 23 行调用了内核包装函数 dotLauncher()，并且在第 24 行向控制台打印了并行 GPU 计算结果。最后我们释放为输入数组分配的内存。

代码清单6.3　parallel_dot/main.cpp

```
 1 #include "kernel.h"
 2 #include <stdio.h>
 3 #include <stdlib.h>
 4 #define N 1024
 5
 6 int main() {
 7   int cpu_res = 0;
 8   int gpu_res = 0;
 9   int *a = (int*)malloc(N*sizeof(int));
10   int *b = (int*)malloc(N*sizeof(int));
11
12   //Initialize input arrays
13   for (int i = 0; i < N; ++i) {
14     a[i] = 1;
15     b[i] = 1;
16   }
17
18   for (int i = 0; i < N; ++i) {
19     cpu_res += a[i] * b[i];
20   }
21   printf("cpu result = %d\n", cpu_res);
22
23   dotLauncher(&gpu_res, a, b, N);
24   printf("gpu result = %d\n", gpu_res);
25
26   free(a);
27   free(b);
28   return 0;
29 }
```

代码清单 6.4 中给出了用于在 Linux 下编译该应用的 Makefile。

代码清单6.4 parallel_dot/Makefile

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3
4 all: main.exe
5
6 main.exe: main.o kernel.o
7   $(NVCC) $^ -o $@
8
9 main.o: main.cpp kernel.h
10  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
11
12 kernel.o: kernel.cu kernel.h
13  $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

我们已经看完了所有代码，现在可以生成并执行该应用了。

代码中的参数值指定输入数组含有 1024 个元素（每一个都是 1），并且每个线程块有 64 个线程，所以当你执行这个应用时，应该可以获得可控制的几个输出：一行是作为参照的 CPU 的结果，一行是每个线程块的索引号和块内的和，一行是 GPU 的结果。除此之外，输出结果应该是可明确的：每个线程块的和应该是 64，而最终的结果应该是 1024。

现在让我们将 parallel_dot/kernel.cu 的第 4 行改为 #define ATOMIC 0。重新生成并运行该应用看看会发生什么。你的结果应该和图 6.1 中显示的 parallel_dot 的样例输出相似，并且有几个值得注意的特点。在第 1 行我们可以看到 CPU 正确地计算出分别具有 1024 个元素为 1 的向量的点积结果为 1024。在接下来的第 16 行，我们看到每个长度为 64 的线程块计算出正确的点积贡献值，并且我们再次发现线程块的执行顺序是不受控制的。在倒数第 2 行，我们发现虽然每个线程块都计算出了正确的贡献值，但是全局累加器最终却没有得到预期的 1024。在这个例子中最终结果是 640，也就是 10×64，而不是 16×64，从结果来看有 6 个线程块的贡献值丢失了。（注意简单地重新执行该应用会产生不同的线程块执行顺序以及不同的 gpu_res 值。试着重复运行你的应用，并观察返回值的变化。）

到底什么地方出错了呢？ 16 个线程块中每块的线程 0 从全局内存中读取 d_res 值，加上它的 blockSum，并将结果存回 d_res 存储的内存位置。问题是这些操作的结果依赖于它们执行的顺序，并且我们无法控制该顺序。

具体来说，让我们考虑线程块 11（blockIdx.x==11）和线程块 10（blockIdx.x==10），根据图 6.1 的输出，它们恰好是最先开始执行的两个线程块。线程块 11 恰好第一个执行，

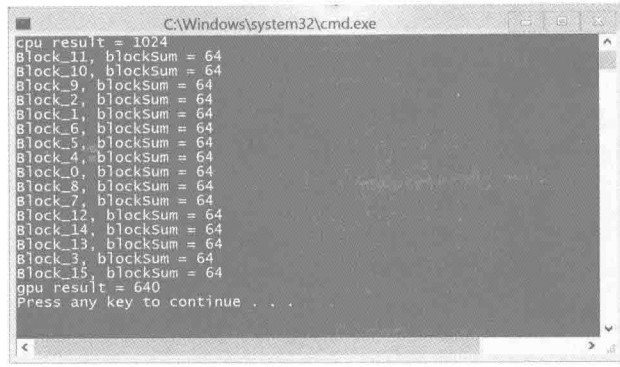


图 6.1　parallel_dot 应用的初始输出

并且它的 0 号线程从 d_res 中读到 0。同样线程块 11 中的 0 号线程会加上它的 blockSum 值 64，并将 0+64 存储回 d_res 所在的内存位置。线程块 10 的 0 号线程也会遵循同样的指令序列，但是和线程块 11 的执行顺序无法保证。如果线程块 10 恰好在线程块 11 更新完 d_res 值之前读取了它的值，那么线程块 10 也会读到 0 并写回 64。如果线程块 10 恰好在线程块 11 更新完 d_res 值之后读取其值，那么线程块 10 将读到 64 并写回 128。

这种情况被称为竞争条件，其计算结果依赖于操作顺序，并且顺序无法控制，而竞争条件会造成未定义的行为。为了解决这个问题，我们需要对执行顺序进行一定控制——在这个例子中，需要确保当一个特定线程占用了全局累加变量后（例如为了执行读 – 修改 – 写操作序列），任何其他线程在该线程完成它的操作序列之前都不可以占用这个全局累加变量。

CUDA 中用于处理竞争条件的机制是一组被称为原子操作的函数。单词原子（atom）源自古希腊语，意思是不可分割。一个原子函数通过一种类似于图书馆图书借阅模式的机制将读 – 修改 – 写操作序列当作一个不可分割的单元进行执行。当一个原子操作读取一个值后，该变量即被“检出”，并且不可被其他操作读取。当操作序列的修改 – 写部分完成后，该变量被“归还”到栈中从而可以再次被其他访问者读取。

好消息是原子操作的确可以解决我们的竞争问题。坏消息是这需要一定的代价：原子操作强制一些串行化，并会造成一定的减速。这很好地提醒了我们，即使速度很快，但如果结果不对，那也算不上计算加速。为了获得正确的结果，我们需要使用原子操作，而除此之外都应该避免原子操作。

现在我们已经提出使用原子操作来解决竞争条件问题了，让我们将 parallel_dot/kernel.cu 的第 4 行恢复成 #define ATOMIC 1 来加以验证。重新生成你的应用并运行它，

你将会看到一个和图 6.2 相似的结果，其中显示了正确的结果 1024。你应该多次运行该应用来确信所产生的正确结果是可靠的。

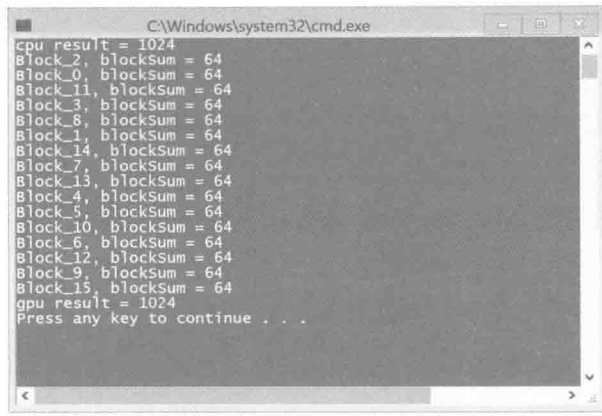


图 6.2 使用 atomicAdd() 的 parallel_dot 应用的正确输出

除了在这个例子中能满足我们需求的 atomicAdd()，CUDA 还提供了另外 10 个原子函数：atomicSub()、atomicExch()、atomicMin()、atomicMax()、atomicInc()、atomicDec()、atomicCAS()（CAS 表示比较并交换）以及三个按位操作函数 atomicAnd()、atomicOr() 和 atomicXor()。详细信息请参考《CUDA C Programming Guide》[1]。

一个浮点类型的警告

注意标准的归约定义包含一个满足结合律的 2- 输入、1- 输出操作，这很好地描述了我们通常所认为的加法运算。然而，浮点加法只是近似满足结合律；对一个浮点数组求和所得的准确结果依赖于运算顺序，并且当你的求和元素中存在量级上的巨大差异时，改变求和顺序将会产生很大的影响。这不是一本关于数值分析的书，所以我们不会讨论所有的细节（更多相关内容参见 David Goldberg 的“What Every Computer Scientist Should Know About Floating-Point Arithmetic”[2]），但是这的确说明了为什么我们在第一个例子中使用 int 作为输入数组类型。

6.3 计算整体属性：centroid_2d

在本节中我们使用原子操作在二维上进行归约。特别地，我们将使用一个位于白色背景上的物体图像作为输入。（我们将选择一个州的地图作为物体，但是你可以选择自己的图像，背景区域将作为物体的外部。）形心（centroid）就是简单地求物体内所有像素的

平均位置。但是我们也可以认为形心是整个图像所有像素位置的加权平均值，权重因子是如下函数：

$$\chi_{col,\,row}=\begin{cases}0, & \{col,\,row\}\in exterior\\ 1, & \{col,\,row\}\in interior\end{cases}$$

形心的列坐标 $\bar{c}$ 是通过将所有像素的列索引乘上权重值之后进行加和，并除以所有权重的和得到的，行坐标 $\bar{r}$ 也是相似的。（因为我们使用 $\bar{c}$ 和 $\bar{r}$ 作为形心坐标，所以称列和行索引为 col 和 row。）形心坐标如下所示：

$$\bar{c}=\frac{\sum_{row}\sum_{col}\chi_{col,\,row}*col}{\sum_{row}\sum_{col}\chi_{col,\,row}},\quad \bar{r}=\frac{\sum_{row}\sum_{col}\chi_{col,\,row}*row}{\sum_{row}\sum_{col}\chi_{col,\,row}}$$

其中求和是针对所有像素的，也就是说 col 的取值范围是从 0 到 width-1，row 的取值范围是从 0 到 height-1。

如代码清单 6.5 至代码清单 6.7 所示，centroid_2d 应用的实现包含通常的内核、头文件，以及 main 文件。由于该应用需要读写图像，所以也包含了第 5 章中介绍的用于图像处理包的 CImg.h 头文件。

让我们先看看代码清单 6.5，其中包含了 centroidKernel() 以及包装函数 centroidParallel() 的代码。

代码清单6.5　centroid_2d/kernel.cu

```
#include "kernel.h"
#include <stdio.h>
#include <helper_math.h>
#define TPB 512

__global__
void centroidKernel(const uchar4 *d_img, int *d_centroidCol,
                    int *d_centroidRow, int *d_pixelCount,
                    int width, int height) {
  __shared__ uint4 s_img[TPB];

  const int idx = threadIdx.x + blockDim.x * blockIdx.x;
  const int s_idx = threadIdx.x;
  const int row = idx/width;
  const int col = idx - row*width;

  if ((d_img[idx].x < 255 || d_img[idx].y < 255 ||
       d_img[idx].z < 255) && (idx < width*height)) {
    s_img[s_idx].x = col;
    s_img[s_idx].y = row;
    s_img[s_idx].z = 1;
  }
  else {
    s_img[s_idx].x = 0;
    s_img[s_idx].y = 0;
    s_img[s_idx].z = 0;
  }
```

```
    __syncthreads();

    // for (int s = 1; s < blockDim.x; s *= 2) {
    //     int index = 2*s*s_idx;
    //     if (index < blockDim.x) {
    //         s_img[index] += s_img[index+s];
    //     }
    //     __syncthreads();
    // }

    for (int s = blockDim.x/2; s > 0; s >>= 1) {
      if (s_idx < s) {
        s_img[s_idx] += s_img[s_idx + s];
      }
      __syncthreads();
    }

    if (s_idx == 0) {
      atomicAdd(d_centroidCol, s_img[0].x);
      atomicAdd(d_centroidRow, s_img[0].y);
      atomicAdd(d_pixelCount, s_img[0].z);
    }
}

void centroidParallel(uchar4 *img, int width, int height) {
    uchar4 *d_img = 0;
    int *d_centroidRow = 0, *d_centroidCol = 0, *d_pixelCount = 0;
    int centroidRow = 0, centroidCol = 0, pixelCount = 0;

    // Allocate memory for device array and copy from host
    cudaMalloc(&d_img, width*height*sizeof(uchar4));
    cudaMemcpy(d_img, img, width*height*sizeof(uchar4),
               cudaMemcpyHostToDevice);

    // Allocate and set memory for three integers on the device
    cudaMalloc(&d_centroidRow, sizeof(int));
    cudaMalloc(&d_centroidCol, sizeof(int));
    cudaMalloc(&d_pixelCount, sizeof(int));
    cudaMemset(d_centroidRow, 0, sizeof(int));
    cudaMemset(d_centroidCol, 0, sizeof(int));
    cudaMemset(d_pixelCount, 0, sizeof(int));

    centroidKernel<<<(width*height + TPB - 1)/TPB, TPB>>>(d_img,
        d_centroidCol, d_centroidRow, d_pixelCount, width, height);

    // Copy results from device to host.
    cudaMemcpy(&centroidRow, d_centroidRow, sizeof(int),
               cudaMemcpyDeviceToHost);
    cudaMemcpy(&centroidCol, d_centroidCol, sizeof(int),
               cudaMemcpyDeviceToHost);
    cudaMemcpy(&pixelCount, d_pixelCount, sizeof(int),
               cudaMemcpyDeviceToHost);

    centroidCol /= pixelCount;
    centroidRow /= pixelCount;

    printf("Centroid: {col = %d, row = %d} based on %d pixels\n",
           centroidCol, centroidRow, pixelCount);

    // Mark the centroid with red lines
```

```
88    for (int col = 0; col < width; ++col) {
89      img[centroidRow*width + col].x = 255;
90      img[centroidRow*width + col].y = 0;
91      img[centroidRow*width + col].z = 0;
92    }
93    for (int row = 0; row < height; ++row) {
94      img[row*width + centroidCol].x = 255;
95      img[row*width + centroidCol].y = 0;
96      img[row*width + centroidCol].z = 0;
97    }
98
99    // Free the memory allocated
100   cudaFree(d_img);
101   cudaFree(d_centroidRow);
102   cudaFree(d_centroidCol);
103   cudaFree(d_pixelCount);
104 }
```

内核函数一开始声明了一个共享数组 s_img 以及通常的一维索引：全局内存数组的 idx 以及共享内存数组的 s_idx。row 和 col 由一维索引导出（而不是从二维内核启动参数导出）并且只在第 19 行和第 20 行的赋值中使用。事实上启动一个一维内核就足够了（并且很方便）。使用二维所造成的改变是我们需要累加两个坐标上的贡献值，而不仅仅是一个坐标。在计算每一个坐标时我们也需要计算权重的和（在本例中就是内部像素的个数，非白色像素），所以我们共计算三个和（行和的贡献值、列和的贡献值以及像素个数贡献值）。为了保存这三个和，我们可以使用三个独立的数组，但是这里我们选择使用一个 unit4 数组 s_img。第一个分量 s_img[].x 存储列贡献值，第二个分量 s_img[].y 存储行贡献值，第三个分量 s_img[].z 存放像素个数贡献值。（我们只需要存放三个值，所以应该使用 uint3，但我们选择 uint4 是因为它可以直接泛化到三维形心计算，并且两个 uint4 变量使用 32 字节内存，这对高效的内存事务是一个很合适的大小。）

第 17 行和第 18 行测试了像素索引以及颜色值。如果像素索引 idx 超出了输入数组，或者该索引处图像像素是白色，它就被当作一个外部像素（并且 0 被存储到 x、y 和 z 通道上）。否则，该像素的贡献值被加到共享内存块中。对 __syncthreads() 的调用确保在任何线程进行求和之前，该线程块所需用来求和的值都已经被存储到共享内存数组中了。

剩下的就是将共享数组中的贡献值求和，并使用原子操作增加全局累加器来避免资源竞争。在点积例子中，我们采用了非常简单且完全串行的方法，选择一个特定的线程来计算线程块的和。我们提到有更加细粒度的并行（也更加高效的）方法，而且事实上有很多。在 2007 年的一次著名演讲中，Mark Harris 演示了 7 个越来越高效的（而且颇具技巧性的）改进 [3]。如果你有一个计算能力 3.0 或以上的 GPU（基于 Kepler 架构），那么还有一系列其他高效归约的技巧，在 Parallel Forall 博客 [4] 中有详细说明。此处并不假设

有 Kepler 架构，并限制我们只使用 Kepler 架构之前 GPU 所能使用的两个有效的技术。第 30～36 行（被注释了，但是你可以拿来实验）实现了第一种在线程块内并行求和的方法。基本的思想是每隔一个线程（相应的跨度为 2）用它本身索引所在值加上相邻索引的值。对所有线程进行同步之后，将跨度加倍到 4，并使每第四个线程将它本身索引所在值加上比它高两个位置处的值。只要跨度小于线程块大小，我们就不断同步、加倍跨度，并求和。根据循环终止条件。线程块的和最终保存在索引 0 处，而这个值可以使用 atomicAdd() 加到全局累加器中。这种方法取得了显著的进步，将线程块内求和所需的串行步数从 O(blockDim.x) 降低到 O(log(blockDim.x))，但是在内存访问效率上存在不足。（关于大 O 标记的详细信息推荐参看 [5]。）

第 38～43 行展示了一个在内存访问方面更加高效的实现。它也进行类似的数对求和，但它是将数组相距较远的一对元素加起来并将结果存放到相邻的位置上，而不是将一对相邻元素加起来并将结果存放到数组中相距一段距离的部分。请参看 [4] 获得进一步的细节。（注意这里的求和针对的是 uint4，并且包含了 helper_math.h 来提供向量运算的定义。注意在程序生成期间花些心思，确保这里的代码恰当地进行了集成。）

第 52～104 行给出了包装函数 centroidParallel() 的实现，该函数准备并调用内核启动函数。希望这里看起来不那么令人意外，而且这里只有少量需要特别注意的要点。第 53 行和第 54 行声明了指向图像数据以及三个全局设备内存中累加器变量的指针。第 55 行声明的主机端变量将保存来自显存的值。紧接着是一系列内存操作，用于为输入数据以及全局累加器变量分配和初始化必要的全局内存。第 70 行和第 71 行启动了内核，就像上面提到的，值得注意的是这里只需要一个一维内核。第 74～85 行将结果复制回主机，并且向控制台打印一个摘要。第 88～97 行使用红色通道创建垂直和水平轴，使用图像中心作为原点，最后内核释放所分配的内存。

代码清单 6.6 中的头文件 centroid_2d/kernel.h 包含 CUDA 内建向量类型的前置声明，以及将在 main.cpp 中被调用的函数 centroidParallel() 的原型。main.cpp 在代码清单 6.7 中展示。

代码清单6.6 centroid_2d/kernel.h

```
1 #ifndef KERNEL_H
2 #define KERNEL_H
3
4 struct uchar4;
5
6 void centroidParallel(uchar4 *img, int width, int height);
7
8 #endif
```

代码清单 6.7 中展示的文件 centroid_2d/main.cpp 开始是通常使用的 #include 和 #define 指令，并使用 CImg 库导入和导出图像。centroidParallel() 函数是包装函数，它调用内核函数 centroidKernel()，并且一旦内核调用完成，它将保存结果图像。

代码清单6.7　centroid_2d/main.cpp

```
#include "kernel.h"
#define cimg_display 0
#include "CImg.h"
#include <cuda_runtime.h>

int main()
{
  // Initializing input and output images
  cimg_library::CImg<unsigned char> inImage("wa_state.bmp");
  cimg_library::CImg<unsigned char> outImage(inImage, "xyzc", 0);
  int width = inImage.width();
  int height = inImage.height();

  // Initializing uchar array for image processing
  uchar4 *imgArray = (uchar4*)malloc(width*height*sizeof(uchar4));

  // Copying CImg data to image array
  for (int row = 0; row < height; ++row) {
    for (int col = 0; col < width; ++col) {
      imgArray[row*width + col].x = inImage(col, row, 0);
      imgArray[row*width + col].y = inImage(col, row, 1);
      imgArray[row*width + col].z = inImage(col, row, 2);
    }
  }

  centroidParallel(imgArray, width, height);

  // Copying image array to CImg
  for (int row = 0; row < height; ++row) {
    for (int col = 0; col < width; ++col) {
      outImage(col, row, 0) = imgArray[row*width + col].x;
      outImage(col, row, 1) = imgArray[row*width + col].y;
      outImage(col, row, 2) = imgArray[row*width + col].z;
    }
  }

  outImage.save("wa_state_out.bmp");
  free(imgArray);
}
```

除了指定一个华盛顿州的地图作为输入文件外，main() 函数没有太多新特点。你可以选择一个不同的输入文件，但是无论你选择什么，都要将其复制到项目文件夹中。代码清单 6.8 中用于在 Linux 中编译该应用的 Makefile，通过设置以确保包含 CUDA 样例程序中的 common/inc 目录，来提供对 helper_math.h 的访问。

为了成功地在 Visual Studio 中生成并运行该应用，需要使用 PROJECT ⇒ Add Existing Item 将输入图像文件添加到工程中，并在工程属性页中做一个修改。在解决方

案浏览器（Solution Explorer）中点击 centroid_2d 工程，并依次选择 PROJECT ⇒ Properties ⇒ Configuration Properties ⇒ C/C++ ⇒ General ⇒ Additional Include Directories，使编辑列表包含 CUDA 样例的 common\inc 目录。它的默认安装路径是 C:\ProgramData\NVIDIA Corporation\CUDA Samples\v7.5\common\inc。

现在生成并运行这个应用。

代码清单6.8 centroid_2d/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall
INC = -I/usr/local/cuda/samples/common/inc

all: main.exe

main.exe: main.o kernel.o
  $(NVCC) $^ -o $@

main.o: main.cpp kernel.h
  $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@

kernel.o: kernel.cu kernel.h
  $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@
```

图 6.3 中显示了华盛顿州流域边界地图的样例输入图像。打印到命令行窗口的结果显示基于 504 244 个像素贡献值的形心位于 530 列、317 行。

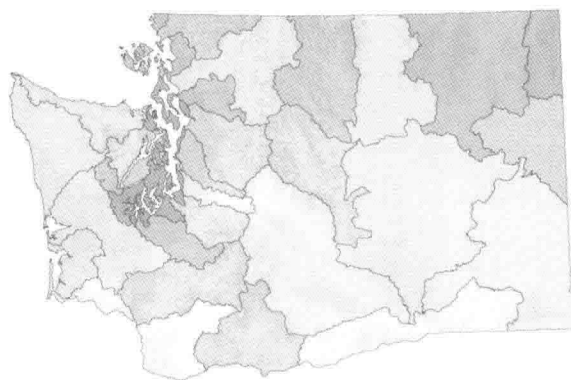


图 6.3 输入图像显示华盛顿州流域（地图来自美国地质调查局，http://wa.water.usgs.gov/data/realtime/adr/interactive/index2.html）

最后，在图 6.4 中，输出图像显示了交叉于图形中心的坐标轴。计算出来的该州形心位于奇兰县南边，韦纳奇以西几英里[⊖]，这和大家普遍接受的位置相符。

⊖ 1 英里≈1.609 千米。——编辑注

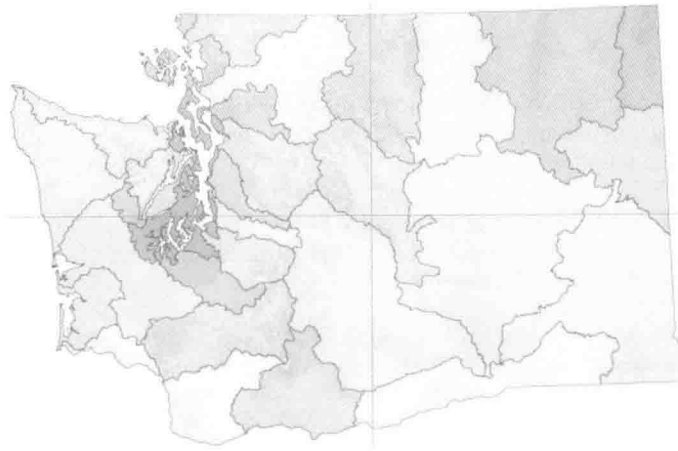


图 6.4　输出图像，坐标轴位于形心处

6.4　本章小结

在本章中，我们研究了计算归约的问题，一个计算网格中的所有线程都会对结果产生影响。我们从实现点积开始，这也许是很多归约应用的最终目的。我们使用共享内存计算一个线程块的贡献值，从而引入资源竞争，并展示了如何使用原子操作来解决这一问题。我们也将实现的归约扩展到计算图像形心问题上，并且看到虽然该问题是二维的，但是一个一维计算网格就足以处理这个问题了。在每个例子中，我们亲手实现归约，NVIDIA 的 Thrust 库中也提供了标准的归约实现代码库。在第 8 章中将提供另外一种使用 Thrust 库实现的 centroid_2d。

6.5　推荐项目

1. 常见的归约包括查找一个列表中的最大值和最小值。修改 parallel_dot 应用来创建一个 L_inf 应用用于计算一个数组中最大的绝对值。
2. 探索一个归约的执行时间如何随着内核启动时选择的线程块大小的变化而变化。
3. 组合一组不同大小的图像，并且将形心计算时间随图片大小的变化绘制成图。
4. 为第 5 章中的 heat_2d 应用实现一个基于归约的迭代停止条件。

参考文献

[1] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[2] David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic." *Computing Surveys*, March 1991, http://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html.

[3] Mark Harris, *Optimizing Parallel Reduction in CUDA*, NVIDIA. https://docs.nvidia.com/cuda/samples/6_Advanced/reduction/doc/reduction.pdf.

[4] Justin Luitjens, "Faster Parallel Reductions on Kepler," *Parallel Forall* (blog), 2014. http://devblogs.nvidia.com/parallelforall/faster-parallel-reductions-kepler/.

[5] Paul E. Black. "Big-O Notation." National Institute of Standards and Technology (NIST), 2012, http://www.nist.gov/dads/HTML/bigOnotation.html.

第 7 章 Chapter 7

三维数据交互

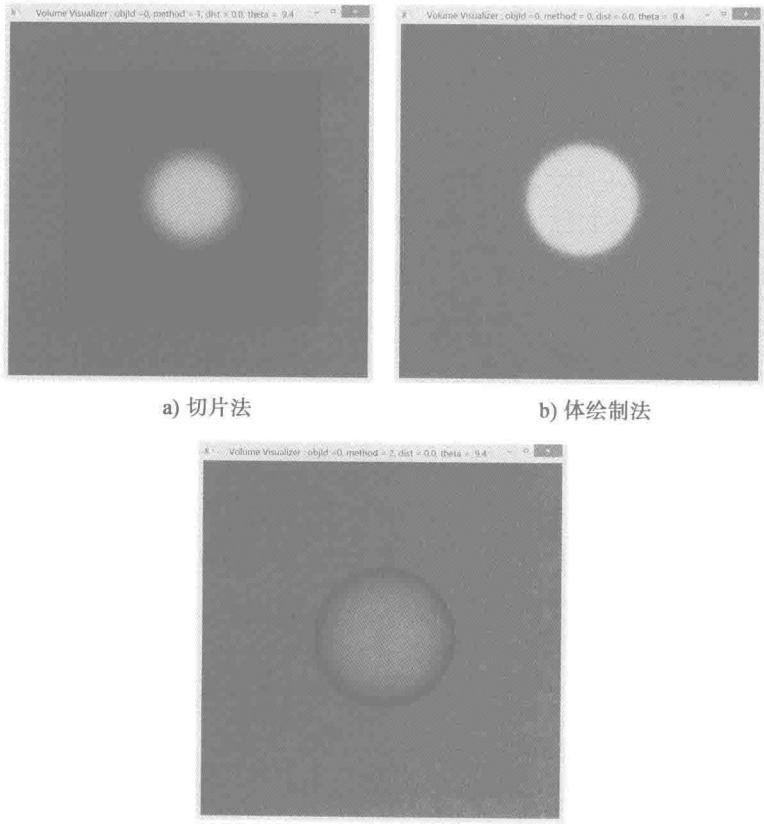
本章编写的一些应用程序涉及三维数据的生成、可视化和交互。对三维函数进行可视化的一般方法，第一步是在其连续数据网格上进行采样，因此几乎所有三维绘图程序都会在程序执行过程中生成一个三维网格数据。另外，三维网格数据可以通过多种设备（物理上）或应用程序（数学上）获得，下面是一些例子：

- 诸如计算机断层扫描（CT）和正电子发射断层扫描（PET）系统的医学体扫描仪。CT 扫描技术也被用于机械零部件检验。
- 流体动力学中的直接数值模拟（DNS），其中包括空气动力学和水动力学。
- 激光多普勒测速仪（LDV）中流体速度场的实验数据。
- 地质与地球物理中的地震勘探方法。
- 在考古和土木工程中应用的探地雷达系统。
- 利用数字光处理（DLP）型 3D 打印机从一堆图像中生成物理实体。
- 视频处理（将时间视为第三个维度）。

我们将首先调用并行程序对函数在三维网格上进行采样，这个程序相对于二维的情况（第 4 章）改动很小。当我们有了三维网格数据，接着将考虑可视化和交互的问题。考虑到我们的屏幕（与视觉系统）是二维的这个基本条件，我们将提供方法（切片法、体绘制法和光线投射法）来获得能够观察和交互的二维图像，我们使用 flashlight 中的代码作为 CUDA 和 OpenGL 间交互的模板，修改 flashlight 包括三步：

- 在 kernel.cu 中，编写内核函数用于计算将被显示的图像数据。
- 在 interactions.h 中，编写键盘与鼠标的回调函数（用于响应用户的操作），并提供操作指南。
- 在 main.cpp 中，修改 render() 函数中内核函数的名字和参数，并在窗口标题栏显示一些有用的信息。

本章与前几章的不同之处在于我们调用三维内核函数来生成将要被显示的数据集。在这一章的最后，你会生成类似图 7.1 的显示图像，也会看到一些更加有趣的数据集。



a) 切片法　　b) 体绘制法

c) 光线投射法

图 7.1　使用由 vis_3d 提供的可视化方法生成的三维距离场（在一个包围盒中）的样例图片

记住上述计划，让我们直接进入正题。先从三维内核代码的编写开始。

7.1　计算三维网格数据：dist_3d

三维网格是二维网格的扩展，因此我们将使用相似的上下文和语言。在第 4 章中，我们用了图像处理中的术语，用 c 和 r 分别代表图像中像素的列序号和行序号。从二维到三维，构成单张图像的二维像素（pixel）扩展成为构成图像序列的三维体素（voxel）。

除了行和列索引，我们需要一个新的整数类型来代表图像层的序号，我们将使用变量 s（可以认为是 stratum 或 stack），范围从 0 到 D-1，其中 D 表示图像层的深度（depth）。我们像之前计算行与列的索引一样（利用 CUDA 内置变量 blockDim、blockIdx 和 threadIdx）来计算“层”的索引：

```
int s = blockIdx.z * blockDim.z + threadIdx.z;
```

由于每个图像每行有 w 个体素，每层有 w×h 个体素，因此体素的一维数组索引就是：

```
int i = c + r*w + s*w*h;
```

我们还需要设定内核启动参数，因此除了 TX、TY（分别代表线程块的宽度和高度），还引入了 TZ 来代表线程块的深度。

现在，dim3 变量的所有分量都被用来指定线程块的大小了：

```
dim3 blockSize(TX, TY, TZ);
```

而线程网格的三个维度由 gridSize 所指定：

```
dim3 gridSize(divUp(W, TX), divUp(H, TY), divUp(D, TZ));
```

函数 divUp() 是一个常用技巧，用于确保线程块的数量能够覆盖所有的线程。

代码清单 7.1 展示了计算三维网格上的数据点到一个定点的距离的程序片段。我们再次从一个简单的例子开始，计算网格中的点相对一个参考点的距离值。但是我们将进一步在一个三维可视化应用中几乎完全不变地使用这个内核函数，只是将 distance() 函数换成一个计算着色值的函数。

代码清单7.1　dist_3d/kernel.cu, 这段代码能够并行地计算三维网格上的数据点到一个定点的距离

```
1 #define W 32
2 #define H 32
3 #define D 32
4 #define TX 8 // number of threads per block along x-axis
5 #define TY 8 // number of threads per block along y-axis
6 #define TZ 8 // number of threads per block along z-axis
7
8 int divUp(int a, int b) { return (a + b - 1)/b; }
```

```
 9
10 __device__
11 float distance(int c, int r, int s, float3 pos) {
12   return sqrtf((c - pos.x)*(c - pos.x) + (r - pos.y)*(r - pos.y) +
13                (s - pos.z)*(s - pos.z));
14 }
15
16 __global__
17 void distanceKernel(float *d_out, int w, int h, int d, float3 pos) {
18   const int c = blockIdx.x * blockDim.x + threadIdx.x; // column
19   const int r = blockIdx.y * blockDim.y + threadIdx.y; // row
20   const int s = blockIdx.z * blockDim.z + threadIdx.z; // stack
21   const int i = c + r*w + s*w*h;
22   if ((c >= w) || (r >= h) || (s >= d)) return;
23   d_out[i] = distance(c, r, s, pos); // compute and store result
24 }
25
26 int main() {
27   float *out = (float*)calloc(W*H*D, sizeof(float));
28   float *d_out = 0;
29   cudaMalloc(&d_out, W*H*D*sizeof(float));
30   const float3 pos = {0.0f, 0.0f, 0.0f}; // set reference position
31   const dim3 blockSize(TX, TY, TZ);
32   const dim3 gridSize(divUp(W, TX), divUp(H, TY), divUp(D, TZ));
33   distanceKernel<<<gridSize, blockSize>>>(d_out, W, H, D, pos);
34   cudaMemcpy(out, d_out, W*H*D*sizeof(float), cudaMemcpyDeviceToHost);
35   cudaFree(d_out);
36   free(out);
37   return 0;
38 }
```

我们希望你发现这段代码十分眼熟并且有较好可读性，但有一部分代码被改动并值得指出（由于问题上升到了三维）。

- 我们使用 calloc 来分配数组 out 而不是用静态数组，因为三维网格的规模往往会导致栈溢出错误。
- 注意到线程块三个维度 TX、TX 和 TZ 是较小值。那是因为对于一个线程块来说，它最多能够拥有 1024 个线程（对于计算能力 2.0 到 5.2 的 GPU 来说），因此 8×8×8 是一个可行的大小（512），而如果设成 16×16×16 则需要 4096 个线程，这将不能成功启动内核。通常，并没有要求线程块必须是方的，而且你会发现，将 blockDim.x 指定为 32 的倍数会具有更好的性能。例如，dim3 gridSize(32, 4, 4) 是可行的而且更具有性能优势。
- 现在，参考点 pos 是一个 float3 类型。三维勾股定理中出现了 pos.x、pos.y、pos.z 以及体素的行、列、层索引。
- 每个维度都有边界检测，超出网格范围的线程会被简单地直接返回：

```
if ((c >= w) || (r >= h) || (s >= d)) return;
```

上述部分是三维内核的基础，请自行构建并执行 dist_3d 应用并使用调试工具验证输出结果是否正确。(阅读本章末尾的建议项目 1 来获取更多建议。) 注意，在 Linux 环境下，构建项目需要的 Makefile 文件和构建 dist_2d 时所用的一样。

7.2　查看三维数据并与之交互：vis_3d

本章开头说过，当前的显示技术是二维的，所以我们将重点讨论如何从三维数据中获取有意义的二维图像。二维图像的显示和交互使用 flashlight 作为与 OpenGL 交互的模板。所以，尽管使用三维内核来生成了三维数据，显示与交互仍然需要二维内核去计算二维图像。

> 须知
>
> 你可能会问，可视化方面有哪些知识是必备的呢？诚然，本节中只出现了少量与 CUDA 相关的新知识。但是我们这里的主要目标是向广大工程师展示 CUDA 的适用性以及有效性，并且本节可能是最好的时机。如果你曾经使用过你最喜欢的软件来绘制三维等值线，并且对其显示出结果所需的漫长等待时间（或者当你改变视角方向时更新显示的延迟）很介意，那么这正是一个很好的机会来体验在图形互操作性方面基于 GPU 的并行计算的强大威力。

为了说明如何从三维数据获得二维图像，我们将尝试使用一个简单的模型——X 光或荧光成像：一个点光源发射光线穿过三维数据集并打到一个二维平板上。我们采用放射科学中的语言，将三维体素的值当作**密度**，将二维像素的值当作（光）**强度**。每个像素的强度值由从光源到平面之间的众多密度值来决定。不同可视化方法的区别就在于从密度数据得到强度值的方法不同。

让我们来看看三种不同的方法：

- 切片法：我们用一块平面穿过数据集，在光线命中平面的位置计算强度值（未命中的部分将显示一个默认的背景值）。
- 体绘制法：我们通过对光线路径上的密度值进行积分来获得二维图像。
- 光线投射法：我们认为密度函数中暗含了三维物体的结构，并选择一个阈值作为这个物体的表面。找出光线最初击中物体表面的位置（如密度到达阈值的地方）并根据这个位置的特性来得到二维图像。我们将使用兰伯特反射模型（model of

Lambertian reflectance），根据物体表面的法向量和视角来计算强度值。（法向量与密度的导数有关并由有限差分近似方法获得。）

现在我们遇到了并行程序设计的基本问题：如何将任务划分到单个线程？这里，我们继续使用 flashlight 中的模型，假定一个线程计算一个像素的强度值。（使用这种方法是为了描述方便。你可以通过使用一个线程来计算多个像素的强度值来提高性能。）

上述的每一种方法都包括一个判断光线与盒子（含有三维数据）是否相交的计算，所以让我们看看光线 - 盒子相交检测的实现。在这里，我们不重复造轮子，而是使用一段现有的代码来完成任务，这段代码能够在 CUDA Samples 的 2_Graphics 目录下的 volumeRender 应用内找到，实际上一部分是参考了 SIGGRAPH 的笔记[1]。我们所需要的只是稍微修改一下 intersectBox() 函数并且用一个数据结构 Ray 来代表一条有向线段，如代码清单 7.2 所示。Ray 数据结构由两个 float3 类型分量组成：o 分量表示光线的起始位置，d 分量代表光线的方向。

代码清单7.2 Ray数据结构、线性插值函数paramRay()以及光线-盒子相交检测函数intersectBox()

```
typedef struct {
  float3 o, d; // origin and direction
} Ray;

__device__ float3 paramRay(Ray r, float t) { return r.o + t*(r.d); }

// intersect ray with a box from volumeRender SDK sample
__device__ bool intersectBox(Ray r, float3 boxmin, float3 boxmax,
  float *tnear, float *tfar) {
  // compute intersection of ray with all six bbox planes
  const float3 invR = make_float3(1.0f)/r.d;
  const float3 tbot = invR*(boxmin - r.o), ttop = invR*(boxmax - r.o);
  // re-order intersections to find smallest and largest on each axis
  const float3 tmin = fminf(ttop, tbot), tmax = fmaxf(ttop, tbot);
  // find the largest tmin and the smallest tmax
  *tnear = fmaxf(fmaxf(tmin.x, tmin.y), fmaxf(tmin.x, tmin.z));
  *tfar = fminf(fminf(tmax.x, tmax.y), fminf(tmax.x, tmax.z));
  return *tfar > *tnear;
}
```

一条从 p_0 到 p_1 的线段可以用公式 $p(t)=p_0+t*(p_0-p_1)$ 表达，其中 t 的变化范围是 [0, 1]。在 Ray 结构中（称其为 pixRay），p_0 存储在 pixRay.o 中，(p_1-p_0) 存储在 pixRay.d 中，我们可以用下面一段代码来创建并初始化光线，该光线通过光源（名为 source 的 float3 变量）和像素位置（名为 pix 的 float3 变量）：

```
Ray pixRay;
pixRay.o = source;
pixRay.d = pix - source;
```

盒子中存有数据，更确切地说是一个 AABB 包围盒（axis-aligned bounding box），它被设定了两个顶点 boxmin 和 boxmax（分别位于盒子的下 - 左 - 前和上 - 右 - 后）。intersectBox() 函数的参数包含对光线的描述 pixRay，一个 AABB 包围盒（由 boxmin 和 boxmax 描述）和两个用于存储坐标信息的 float 指针 tnear 与 tfar（这两个参数将用于估计光线是否进入了盒子）。注意 intersectBox() 函数的返回值是布尔类型。false 表示相交检测失败（光线没有击中盒子）。如果光线击中了盒子，将会返回 true，并将光线进入和离开盒子的信息储存在 tnear 和 tfar 中。然后我们就可以用线性插值函数 paramRay() 通过 tnear 和 tfar 来获得光线进入和离开盒子的坐标 near 与 far。

计算一个像素的强度值的步骤如下：

- 计算像素的坐标 pix，声明 / 初始化 pixRay。
- 调用光线 - 盒子相交检测函数：
  - 如果光线没有命中盒子，使用默认背景值作为像素的强度值。
  - 如果光线命中了盒子，创建一个被裁剪的光线（称为 boxRay），它从 near 到 far，然后调用一个特定的着色函数来计算强度值，着色函数的参数包括 boxRay、图像序列的密度数据和其他所需参数。

现在，我们开始关注着色函数，我们将展现每个函数的代码片段，然后基于 flashlight 将它们整合起来，产生完整的 vis_3d 的程序代码。

7.2.1　切片法

我们从切片法开始，因为它的计算过程最为简单。一个切片能够显示密度数据的一个截面。如果光线 boxRay 与切片相交，强度值就由交点附近的密度值决定。如果光线没有与切片相交，就返回一个默认的背景值。我们选择的背景值与在光线完全未击中盒子时所显示的背景值不同，这样就能使盒子的轮廓清晰可见。

注意到交点不总是落在整数点上，所以就要用插值的方法通过交点附近的密度值来估计交点处的密度值。

为了简化相应的一些坐标变换，选择三维数据集作为参考系，将坐标原点设定在众多体素的中心，坐标轴和体素各边平行。z 轴分量代表图像不同的层。在这个坐标系下，坐标原点 $\{x, y, z\}=\{0, 0, 0\}$ 就固定在众多体素的中心 $\{c_0, r_0, s_0\}=\{\lfloor W/2 \rfloor, \lfloor H/2 \rfloor, \lfloor D/2 \rfloor\}$。（注意到这些值必须是整数，所以使用向下取整来使 $\{c_0, r_0, s_0\}$ 是整数。）这样，我们就有了从空间坐标系到图像坐标系的简单变换：

$$c=x+c_0$$

$$r=y+r_0$$

$$s=z+s_0$$

接下来，我们就需要确定光线 - 切片交点 pos 处的密度值。如果那个点的坐标都是整数值，那么密度值就能直接从数组 dens 中获得。如果那个点的坐标不是正好落在整数格点上（体素顶点），而是落在体素内，我们就需要使用三线性插值的方法得到密度值。交点坐标的整数部分表示体素的下 - 左 - 前顶点，小数部分用于进行插值运算。

用来确定密度值的实际代码由代码清单 7.3 给出。各个方向上的单位增量分别为 dx、dy 和 dz，它们用于确定体素 8 个顶点的坐标，这些坐标通过三位二进制数来区分（例如，dens000 表示下 - 左 - 前顶点，dens111 表示上 - 右 - 后顶点）。代码最后的公式用于简单地从各个顶点通过插值得到交点处的密度值，这可能有些长，但我们还会在别的可视化方法中再次用到它。

代码清单7.3 从三维网格数据计算密度值，先裁剪了超出范围的部分，再进行三线性插值

```
__device__ int clipWithBounds(int n, int n_min, int n_max) {
  return n > n_max ? n_max : (n < n_min ? n_min : n);
}

__device__ float density(float *d_vol, int3 volSize, float3 pos) {
  int3 index = posToVolIndex(pos, volSize);
  int i = index.x, j = index.y, k = index.z;
  const int w = volSize.x, h = volSize.y, d = volSize.z;
  const float3 rem = fracf(pos);
  index = make_int3(clipWithBounds(i, 0, w - 2),
    clipWithBounds(j, 0, h - 2), clipWithBounds(k, 0, d - 2));
  // directed increments for computing the gradient
  const int3 dx = {1, 0, 0}, dy = {0, 1, 0}, dz = {0, 0, 1};
  // values sampled at surrounding grid points
  const float dens000 = d_vol[flatten(index, volSize)];
  const float dens100 = d_vol[flatten(index + dx, volSize)];
  const float dens010 = d_vol[flatten(index + dy, volSize)];
  const float dens001 = d_vol[flatten(index + dz, volSize)];
  const float dens110 = d_vol[flatten(index + dx + dy, volSize)];
  const float dens101 = d_vol[flatten(index + dx + dz, volSize)];
  const float dens011 = d_vol[flatten(index + dy + dz, volSize)];
  const float dens111 = d_vol[flatten(index + dx + dy + dz, volSize)];
  // trilinear interpolation
  return (1 - rem.x)*(1 - rem.y)*(1 - rem.z)*dens000 +
    (rem.x)*(1 - rem.y)*(1 - rem.z)*dens100 +
    (1 - rem.x)*(rem.y)*(1 - rem.z)*dens010 +
    (1 - rem.x)*(1 - rem.y)*(rem.z)*dens001 +
    (rem.x)*(rem.y)*(1 - rem.z)*dens110 +
    (rem.x)*(1 - rem.y)*(rem.z)*dens101 +
    (1 - rem.x)*(rem.y)*(rem.z)*dens011 +
    (rem.x)*(rem.y)*(rem.z)*dens111;
}
```

通过裁剪和插值操作后，sliceShader()函数显得简单明了。它的代码在代码清单 7.4 中，包括一些辅助函数：

- planeSDF() 返回一点到平面的距离，平面由法向量（float3 norm）和到原点的距离（float dist）确定。
- rayPlaneIntersect() 执行光线与平面的相交检测。如果 planeSDF() 返回了两个符号相反的距离值，光线就与平面相交，rayPlaneIntersect() 将返回 true，交点处的参数将被存入 t。如果光线与平面未相交，rayPlaneIntersect() 返回 false。

代码清单7.4　sliceShader()函数和它的辅助函数planeSDF()和rayPlaneIntersect()

```
__device__ float planeSDF(float3 pos, float3 norm, float d) {
  return dot(pos, normalize(norm)) - d;
}

__device__
bool rayPlaneIntersect(Ray myRay, float3 n, float dist, float *t) {
  const float f0 = planeSDF(paramRay(myRay, 0.f), n, dist);
  const float f1 = planeSDF(paramRay(myRay, 1.f), n, dist);
  bool result = (f0*f1 < 0);
  if (result) *t = (0.f - f0) / (f1 - f0);
  return result;
}

__device__ uchar4  sliceShader(float *d_vol, int3 volSize, Ray boxRay,
  float gain, float dist, float3 norm) {
  float t;
  uchar4 shade = make_uchar4(96, 0, 192, 0); // background value
  if (rayPlaneIntersect(boxRay, norm, dist, &t)) {
    float sliceDens = density(d_vol, volSize, paramRay(boxRay, t));
    shade = make_uchar4(48, clip(-10.f * (1.0f + gain) * sliceDens),
                        96, 255);
  }
  return shade;
}
```

sliceShader() 函数有以下输入参数：

- 一个指向密度值的指针 float *d_vol，它是一个指向设备数组的指针。
- 图像堆栈的维度（int3 volSize）。
- 光线，boxRay。
- 用于调节强度大小的 float 类型参数 gain。
- 用于指定平面的参数 float dist 与 float3 norm。

sliceShader() 函数先执行光线 – 平面相交检测，然后依据不同情况要么返回默认的背景值（如果没有相交），要么在计算出光线与平面交点处的密度值的前提下，再计算该处的绿色分量，绿色分量通过下式进行计算：

```
shade.y = clip(-10.f * (1.0f + gain) * sliceDens);
```

其中 sliceDens 是 slicePoint（交点）处的密度值，gain 是用于调节强度的因子，clip() 将结果转换到 uchar 的有效范围 0 到 255。

7.2.2 体绘制法

体绘制法的基本思想是，像素的值是光线（boxRay）路径上的密度值的积分。代码清单 7.5 是其具体实现。这次依然只使用绿色通道，所以为红色与蓝色通道设定初始值。我们使用有限和来近似计算积分，有限和的项数为 numSteps，微分因子为 dt，积分路径长度为 len。首先初始化变量（accum=0.f;），然后沿着 boxRay 对 accum 进行累加，直到密度值超过阈值 threshold。for 循环的最后两行用于获得更新的当前位置和密度值。当我们走完了 boxRay 路径，accum 被 clip 函数转换成为 uchar 类型，作为该像素的绿色分量。

代码清单7.5 volumeRenderShader()

```
__device__ uchar4 volumeRenderShader(float *d_vol, int3 volSize,
  Ray boxRay, float threshold, int numSteps) {
  uchar4 shade = make_uchar4(96, 0, 192, 0); // background value
  const float dt = 1.f/numSteps;
  const float len = length(boxRay.d)/numSteps;
  float accum = 0.f;
  float3 pos = boxRay.o;
  float val = density(d_vol, volSize, pos);
  for (float t = 0.f; t<1.f; t += dt) {
    if (val - threshold < 0.f) accum += (fabsf(val - threshold))*len;
    pos = paramRay(boxRay, t);
    val = density(d_vol, volSize, pos);
  }
  if (clip(accum) > 0.f) shade.y = clip(accum);
  return shade;
}
```

7.2.3 光线投射法

为了确定采用光线投射法的像素值，我们沿着光线搜索目标密度值（这个密度值勾勒出物体的一个表面），然后根据该处局部的表面法向量来计算像素值。代码清单 7.6 是 raycastShader() 的实现。rayCastShader() 包括以下输入参数：

- 一个指向密度值数组的指针（float *d_vol）。
- 图像数据的大小（int3 volSize）。
- 由包围盒裁剪的光线（Ray boxRay）。
- 物体表面的距离阈值（float dist）。

代码清单7.6　raycastShader()

```
__device__ uchar4 rayCastShader(float *d_vol, int3 volSize,
  Ray boxRay, float dist) {
  uchar4 shade = make_uchar4(96, 0, 192, 0);
  float3 pos = boxRay.o;
  float len = length(boxRay.d);
  float t = 0.0f;
  float f = density(d_vol, volSize, pos);
  while (f > dist + EPS && t < 1.0f) {
    f = density(d_vol, volSize, pos);
    t += (f - dist) / len;
    pos = paramRay(boxRay, t);
    f = density(d_vol, volSize, pos);
  }
  if (t < 1.f) {
    const float3 ux = make_float3(1, 0, 0), uy = make_float3(0, 1, 0),
                 uz = make_float3(0, 0, 1);
    float3 grad = {(density(d_vol, volSize, pos + EPS*ux) -
                    density(d_vol, volSize, pos))/EPS,
                   (density(d_vol, volSize, pos + EPS*uy) -
                    density(d_vol, volSize, pos))/EPS,
                   (density(d_vol, volSize, pos + EPS*uz) -
                    density(d_vol, volSize, pos))/EPS};
    float intensity = -dot(normalize(boxRay.d), normalize(grad));
    shade = make_uchar4(255*intensity, 0, 0, 255);
  }
  return shade;
}
```

raycastShader() 函数返回一个代表颜色的 uchar4 类型结构体。计算从 while 循环开始，沿着光线 boxRay 一直到包围盒的边界，或者直到与阈值 dist 接近到一个足够小的误差（EPS）。

求根、步长大小和符号距离函数

确定某一函数中给定函数值在何处取到的过程被称为“求根”。这是一个重要研究课题，至少可以用一整章进行讨论[2]，并且还被广泛认为是未解决的问题。对于没有限定的一般函数，要可靠地计算它的根的确是个大问题。如果函数值能够在任意小的间隔内有很大的变化，那将没有一个可以确定搜索步长的可靠方法。

这里，我们不会遇到这种棘手问题。我们的数据集有一个最小的间隔（一个体素的间隔），选择一个相对于体素来说足够小的步长就能确保不会有细节被错过。

在有符号距离函数的情况下，我们能够做得更好。计算符号距离函数（并减去距离阈值）就能够得到该点到表面的最短距离，所以我们能够安全地用这个距离作为步长。如果向正确的方向搜索，就能找到待求的根。如果向其他方向搜索，将会到达一个距离表面较近的点，我们能够重复这个步骤，而且知道不会有根被遗漏。综上所述，一定能够在很少的步数内找到足够精确的光线 – 表面的交点。

如果 while 循环由于光线离开包围盒而终止，将返回一个默认的背景值。否则，如果 while 循环由于遇到了密度阈值（也就是说，光线打中了表面），我们就计算该点处的梯度（即表面法向量），并根据梯度向量与光线向量的点积计算像素值（在这里我们将结果存储在绿色通道中）。

梯度 grad 的分量是密度函数沿坐标轴的导数。float3 类型变量 ux、uy 和 uz 是坐标轴方向的单位向量，用于计算 grad.x、grad.y 和 grad.z 的公式，在第 4 章已经使用过有限差分方法。

boxRay.d 和 grad 的单位方向向量的点乘就是光线与法向量之间的 cos 值，根据 Lambert 辐射模型（Lambertian radiance model），可以近似计算漫反射的光强，以此计算像素值。

着色模型（Shading Models）、光线跟踪（Ray Tracing）和计算机图形学（Computer Graphics）

这是另一个大课题。有一部关于表面和光照场景模型的完整著作。关于光线追踪（这说明光线可以反射和折射）、辐射模型（将每个对象同时作为辐射的发射器和吸收器）和许多其他计算机图形学方面细节的讨论，请参考经典著作[3]。

7.2.4 创建 vis_3d 应用

现在已经讨论完所有的着色函数，就让我们把代码整合起来，看看 vis_3d 应用的完整代码。我们再一次基于第 4 章的 flashlight 应用，用相同的文件和命令，不过要做如下的修改：

- vis_3d/main.cpp（见代码清单 7.7），相对于 flashlight/main.cpp 只有一些小改动：
  - 第 29～30 行，修改了 kernelLauncher() 的参数。
  - 第 33～35 行，修改了图形窗口标题栏的信息。
  - 第 91 行，加入 volumeKernelLauncher() 函数，用于生成将要被可视化的三维数据。
  - 第 94 行，加入新的回调函数 createMenu()，用于创建菜单，选择可视化所用的物体 / 函数。
- vis_3d/interactions.h（见代码清单 7.8），设定几何相关的常数和窗口尺寸，打印用户指南并定义包括弹出式菜单内容和相应动作在内的用户交互新方式。
- vis_3d/kernel.h（见代码清单 7.9），定义了新的数据类型、volumeKernelLauncher()

的原型（启动三维内核函数）和 kernelLauncher() 的原型（启动二维网格负责渲染）。

- vis_3d/kernel.cu（见代码清单 7.10），定义了内核启动函数和内核函数。

在这个案例中，内核函数调用了一系列的设备函数，包括一些辅助函数。其中有 paramRay()、scrIdxToPos() 和 intersectBox()，它们用来与 func() 一起创建三维数据网格；yRotate() 用来改变观察视角；另外还有三个着色函数。为使 vis_3d/kernel.cu 不至于变得太长，我们创建了两个分开的文件，其中 vis_3d/device_funcs.cu 用于保存定义，而 vis_3d/device_funcs.cuh 用于保存原型声明。

代码清单7.7　vis_3d/main.cpp

```
#include "interactions.h"
#include "kernel.h"
#include <stdio.h>
#include <stdlib.h>
#ifdef _WIN32
#define WINDOWS_LEAN_AND_MEAN
#define NOMINMAX
#include <windows.h>
#endif
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/freeglut.h>
#endif
#include <cuda_runtime.h>
#include <cuda_gl_interop.h>

// texture and pixel objects
GLuint pbo = 0;     // OpenGL pixel buffer object
GLuint tex = 0;     // OpenGL texture object
struct cudaGraphicsResource *cuda_pbo_resource;

void render() {
  uchar4 *d_out = 0;
  cudaGraphicsMapResources(1, &cuda_pbo_resource, 0);
  cudaGraphicsResourceGetMappedPointer((void **)&d_out, NULL,
                                       cuda_pbo_resource);
  kernelLauncher(d_out, d_vol, W, H, volumeSize, method, zs, theta,
                 threshold, dist);
  cudaGraphicsUnmapResources(1, &cuda_pbo_resource, 0);
  char title[128];
  sprintf(title, "Volume Visualizer : objId =%d, method = %d,"
                 " dist = %.1f, theta =  %.1f", id, method, dist,
                 theta);
  glutSetWindowTitle(title);
}

void draw_texture() {
  glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, W, H, 0, GL_RGBA,
         GL_UNSIGNED_BYTE, NULL);
```

```
  glEnable(GL_TEXTURE_2D);
  glBegin(GL_QUADS);
  glTexCoord2f(0.0f, 0.0f); glVertex2f(0, 0);
  glTexCoord2f(0.0f, 1.0f); glVertex2f(0, H);
  glTexCoord2f(1.0f, 1.0f); glVertex2f(W, H);
  glTexCoord2f(1.0f, 0.0f); glVertex2f(W, 0);
  glEnd();
  glDisable(GL_TEXTURE_2D);
}

void display() {
  render();
  draw_texture();
  glutSwapBuffers();
}

void initGLUT(int *argc, char **argv) {
  glutInit(argc, argv);
  glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);
  glutInitWindowSize(W, H);
  glutCreateWindow("Volume Visualizer");
#ifndef __APPLE__
  glewInit();
#endif
}

void initPixelBuffer() {
  glGenBuffers(1, &pbo);
  glBindBuffer(GL_PIXEL_UNPACK_BUFFER, pbo);
  glBufferData(GL_PIXEL_UNPACK_BUFFER, W*H*sizeof(GLubyte) * 4, 0,
               GL_STREAM_DRAW);
  glGenTextures(1, &tex);
  glBindTexture(GL_TEXTURE_2D, tex);
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
  cudaGraphicsGLRegisterBuffer(&cuda_pbo_resource, pbo,
                               cudaGraphicsMapFlagsWriteDiscard);
}

void exitfunc() {
  if (pbo) {
    cudaGraphicsUnregisterResource(cuda_pbo_resource);
    glDeleteBuffers(1, &pbo);
    glDeleteTextures(1, &tex);
  }
  cudaFree(d_vol);
}

int main(int argc, char** argv) {
  cudaMalloc(&d_vol, NX*NY*NZ*sizeof(float)); // 3D volume data
  volumeKernelLauncher(d_vol, volumeSize, id, params);
  printInstructions();
  initGLUT(&argc, argv);
  createMenu();
  gluOrtho2D(0, W, H, 0);
  glutKeyboardFunc(keyboard);
  glutSpecialFunc(handleSpecialKeypress);
  glutDisplayFunc(display);
  initPixelBuffer();
  glutMainLoop();
  atexit(exitfunc);
```

```
102     return 0;
103 }
```

接着，开始修改用户交互功能。编辑 interactions.h 中的回调函数，见代码清单 7.8。文件从定义常量（包括图像大小、图像堆栈大小和线程块大小）和变量（包括物体大小的参数、观察距离、观察角度、着色方法、阈值，以及切片的法向量和距离）开始。

新的回调函数 mymenu() 使用一个 switch() 语句来指定图像编号 id 的整数值（菜单通过鼠标右键单击创建，然后在菜单项上释放鼠标来选择）。mymenu() 的最后一行将 id 作为参数传递给 volumeKernelLauncher()。三维网格或者图像堆栈的数据在每次菜单选择后被实时计算（基于下面会介绍的 device_funcs.cu 内的函数）。

下一个回调函数 keyboard() 根据键盘输入的不同 unsigned char 类型的值执行不同的动作。+ 键和 – 键控制缩放（本质上是移动 source）。d/D 键移动切面，而 z 键还原默认。可视化方法也通过键盘选择：v（体绘制），r（光线投射），f（切片法）（只是因为在 QWERTY 键盘布局中，f 在 v 和 r 之间）。

最后，handleSpecialFunctionKeypress() 函数根据特殊功能按键（不产生 unsigned char 类型的值）对应的 int 类型的值执行不同的动作。左右方向键改变 theta 的值，它将被传递到 kernel.cu，改变观察角度（通过改变 source 和 pix 的值）。上下方向键改变 threshold 的值，它将被传递到着色函数中，改变表面阈值或者切片的位置。

代码清单7.8　vis_3d/interactions.h

```
 1 #ifndef INTERACTIONS_H
 2 #define INTERACTIONS_H
 3 #include "kernel.h"
 4 #include <stdio.h>
 5 #include <stdlib.h>
 6 #ifdef __APPLE__
 7 #include <GLUT/glut.h>
 8 #else
 9 #include <GL/glew.h>
10 #include <GL/freeglut.h>
11 #endif
12 #include <vector_types.h>
13 #define W  600
14 #define H  600
15 #define DELTA  5 // pixel increment for arrow keys
16 #define NX 128
17 #define NY 128
18 #define NZ 128
19
20 int id = 1; // 0 = sphere, 1 = torus, 2 = block
21 int method = 2; // 0 = volumeRender, 1 = slice, 2 = raycast
22 const int3 volumeSize = {NX, NY, NZ}; // size of volumetric data grid
23 const float4 params = {NX/4.f, NY/6.f, NZ/16.f, 1.f};
24 float *d_vol; // pointer to device array for storing volume data
```

```
25 float zs = NZ; // distance from origin to source
26 float dist = 0.f, theta = 0.f, threshold = 0.f;
27
28 void mymenu(int value) {
29   switch (value) {
30   case 0: return;
31   case 1: id = 0; break; // sphere
32   case 2: id = 1; break; // torus
33   case 3: id = 2; break; // block
34   }
35   volumeKernelLauncher(d_vol, volumeSize, id, params);
36   glutPostRedisplay();
37 }
38
39 void createMenu() {
40   glutCreateMenu(mymenu);          // Object selection menu
41   glutAddMenuEntry("Object Selector", 0); // menu title
42   glutAddMenuEntry("Sphere", 1);       // id = 1 -> sphere
43   glutAddMenuEntry("Torus", 2);      // id = 2 -> torus
44   glutAddMenuEntry("Block", 3);      // id = 3 -> block
45   glutAttachMenu(GLUT_RIGHT_BUTTON);     // right-click for menu
46 }
47
48 void keyboard(unsigned char key, int x, int y) {
49   if (key == '+') zs -= DELTA; // move source closer (zoom in)
50   if (key == '-') zs += DELTA; // move source farther (zoom out)
51   if (key == 'd') --dist; // decrease slice distance
52   if (key == 'D') ++dist; // increase slice distance
53   if (key == 'z') zs = NZ, theta = 0.f, dist = 0.f; // reset values
54   if (key == 'v') method = 0; // volume render
55   if (key == 'f') method = 1; // slice
56   if (key == 'r') method = 2; // raycast
57   if (key == 27) exit(0);
58   glutPostRedisplay();
59 }
60
61 void handleSpecialKeypress(int key, int x, int y)  {
62   if (key == GLUT_KEY_LEFT) theta -= 0.1f; // rotate left
63   if (key == GLUT_KEY_RIGHT) theta += 0.1f; // rotate right
64   if (key == GLUT_KEY_UP) threshold += 0.1f; // inc threshold (thick)
65   if (key == GLUT_KEY_DOWN) threshold -= 0.1f; // dec threshold (thin)
66   glutPostRedisplay();
67 }
68
69 void printInstructions() {
70   printf("3D Volume Visualizer\n"
71          "Controls:\n"
72          "Volume render mode                              : v\n"
73          "Slice render mode                               : f\n"
74          "Raycast mode                                    : r\n"
75          "Zoom out/in                                     : -/+\n"
76          "Rotate view                                     : left/right\n"
77          "Decr./Incr. Offset (intensity in slice mode): down/up\n"
78          "Decr./Incr. distance (only in slice mode)    : d/D\n"
79          "Reset parameters                                : z\n"
80          "Right-click for object selection menu\n");
81 }
82
83 #endif
```

拥有了这些交互功能（同时也鼓励你自己尝试着写一些你认为有用的交互功能），我们可以开始看 kernel.cu 中的代码。“包装”函数 volumeKernelLauncher() 和 kernelLauncher() 的原型在 kernel.h 中（见代码清单 7.9），具体实现在 kernel.cu 中（见代码清单 7.10）。volumeKernelLauncher() 调用 volumeKernel() 以启动一个三维网格去计算描述物体的三维数据。kernelLauncher() 调用 renderKernel() 以启动一个二维网格去计算各个像素的值。

代码清单7.9　vis_3d/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

struct uchar4;
struct int3;
struct float4;

void kernelLauncher(uchar4 *d_out, float *d_vol, int w, int h,
  int3 volSize, int method, int zs, float theta, float threshold,
  float dist);
void volumeKernelLauncher(float *d_vol, int3 volSize, int id,
                          float4 params);
#endif
```

代码清单7.10　vis_3d/kernel.cu

```
#include "kernel.h"
#include "device_funcs.cuh"
#include <helper_math.h>
#define TX_2D 32
#define TY_2D 32
#define TX 8
#define TY 8
#define TZ 8
#define NUMSTEPS 20

__global__
void renderKernel(uchar4 *d_out, float *d_vol, int w, int h,
  int3 volSize, int method, float zs, float theta, float threshold,
  float dist) {
  const int c = blockIdx.x*blockDim.x + threadIdx.x;
  const int r = blockIdx.y*blockDim.y + threadIdx.y;
  const int i = c + r * w;
  if ((c >= w) || (r >= h)) return; // Check if within image bounds
  const uchar4 background = {64, 0, 128, 0};
  float3 source = {0.f, 0.f, -zs};
  float3 pix = scrIdxToPos(c, r, w, h, 2*volSize.z-zs);
  // apply viewing transformation: here rotate about y-axis
  source = yRotate(source, theta);
  pix = yRotate(pix, theta);
  // prepare inputs for ray-box intersection
  float t0, t1;
  const Ray pixRay = {source, pix - source};
  float3 center = {volSize.x/2.f, volSize.y/2.f, volSize.z/2.f};
```

```
  const float3 boxmin = -center;
  const float3 boxmax = {volSize.x - center.x, volSize.y - center.y,
                         volSize.z - center.z};
  // perform ray-box intersection test
  const bool hitBox = intersectBox(pixRay, boxmin, boxmax, &t0, &t1);
  uchar4 shade;
  if (!hitBox) shade = background; // miss box => background color
  else {
    if (t0 < 0.0f) t0 = 0.f; // clamp to 0 to avoid looking backward
    // bounded by points where the ray enters and leaves the box
    const Ray boxRay = {paramRay(pixRay, t0),
                        paramRay(pixRay, t1) - paramRay(pixRay, t0)};
    if (method == 1) shade =
      sliceShader(d_vol, volSize, boxRay, threshold, dist, source);
    else if (method == 2) shade =
      rayCastShader(d_vol, volSize, boxRay, threshold);
    else shade =
      volumeRenderShader(d_vol, volSize, boxRay, threshold, NUMSTEPS);
  }
  d_out[i] = shade;
}

__global__
void volumeKernel(float *d_vol, int3 volSize, int id, float4 params) {
  const int w = volSize.x, h = volSize.y, d = volSize.z;
  const int c = blockIdx.x * blockDim.x + threadIdx.x; // column
  const int r = blockIdx.y * blockDim.y + threadIdx.y; // row
  const int s = blockIdx.z * blockDim.z + threadIdx.z; // stack
  const int i = c + r * w + s * w * h;
  if ((c >= w) || (r >= h) || (s >= d)) return;
  d_vol[i] = func(c, r, s, id, volSize, params); // compute and store
}

void kernelLauncher(uchar4 *d_out, float *d_vol, int w, int h,
  int3 volSize, int method, int zs, float theta, float threshold,
  float dist) {
  dim3 blockSize(TX_2D, TY_2D);
  dim3 gridSize(divUp(w, TX_2D), divUp(h, TY_2D));
  renderKernel<<<gridSize, blockSize>>>(d_out, d_vol, w, h, volSize,
    method, zs, theta, threshold, dist);
}

void volumeKernelLauncher(float *d_vol, int3 volSize, int id,
  float4 params) {
  dim3 blockSize(TX, TY, TZ);
  dim3 gridSize(divUp(volSize.x, TX), divUp(volSize.y, TY),
    divUp(volSize.z, TZ));
  volumeKernel<<<gridSize, blockSize>>>(d_vol, volSize, id, params);
}
```

volumeKernelLauncher() 函数在初始化时被调用（volumeKernel() 执行），而在菜单选择了新的物体后将被再次调用。kernelLauncher() 函数在每一次屏幕刷新时被调用（renderKernel() 执行）。volumeKernel() 与 dist_3d 应用中的 distanceKernel() 是一样的（在这里我们用 func() 代替了勾股定理公式），所以我们把重点放到其他内核函数。

renderKernel() 由一些熟悉的索引计算和边界检测开始。光源和像素的位置由如下方

式计算得到。光源被定在 {0.f, 0.f, -zs}，z 轴穿过屏幕中心并与屏幕垂直（光源和屏幕中心之间的距离为图像堆栈尺寸的两倍）。观察系统（光源和屏幕）允许沿着 z 轴移动（通过改变 zs），并且能够绕着 y 轴转动（通过改变 theta），见第 20～24 行。坐标系统和图像堆栈轴向一致，原点定在图像堆栈的中心（float3 center）。第 29 行和第 30 行构造的包围盒用于第 33 行的光线 - 盒子相交检测。如果光线未击中盒子，返回一个默认的背景值。如果光线击中了盒子，就如第 39 行构造一个 boxRay 用于描述这个包围盒中的光线，然后将 boxRay 根据 method 取值传递给不同着色函数。

启动函数和着色器函数需要一系列的设备函数作为辅助，它们的代码在 vis_3d/device_funcs.cu（和它的头文件 vis_3d/device_funcs.cuh）中，见代码清单 7.11 和代码清单 7.12。

代码清单7.11　vis_3d/device_funcs.cuh

```
 1 #ifndef DEVICEFUNCS_CUH
 2 #define DEVICEFUNCS_CUH
 3
 4 typedef struct {
 5   float3 o, d; // origin and direction
 6 } Ray;
 7
 8 __host__ int divUp(int a, int b);
 9 __device__ float3 yRotate(float3 pos, float theta);
10 __device__ float func(int c, int r, int s, int id, int3 volSize,
11   float4 params);
12 __device__ float3 paramRay(Ray r, float t);
13 __device__ float3 scrIdxToPos(int c, int r, int w, int h, float zs);
14 __device__ bool intersectBox(Ray r, float3 boxmin, float3 boxmax,
15   float *tnear, float *tfar);
16 __device__ uchar4 sliceShader(float *d_vol, int3 volSize, Ray boxRay,
17   float threshold, float dist, float3 norm);
18 __device__ uchar4 volumeRenderShader(float *d_vol, int3 volSize,
19   Ray boxRay, float dist, int numSteps);
20 __device__ uchar4  rayCastShader(float *d_vol, int3 volSize,
21   Ray boxRay, float dist);
22
23 #endif
```

代码清单7.12　vis_3d/device_funcs.cu

```
 1 #include "device_funcs.cuh"
 2 #include <helper_math.h>
 3 #define EPS 0.01f
 4
 5 __host__ int divUp(int a, int b) { return (a + b - 1) / b; }
 6
 7 __device__
 8 unsigned char clip(int n) { return n > 255 ? 255 : (n < 0 ? 0 : n); }
 9
10 __device__ int clipWithBounds(int n, int n_min, int n_max) {
```

```
11   return n > n_max ? n_max : (n < n_min ? n_min : n);
12 }
13
14 __device__ float3 yRotate(float3 pos, float theta) {
15   const float c = cosf(theta), s = sinf(theta);
16   return make_float3(c*pos.x + s*pos.z, pos.y, -s*pos.x + c*pos.z);
17 }
18
19 __device__ float func(int c, int r, int s, int id, int3 volSize,
20                       float4 params) {
21   const int3 pos0 = {volSize.x/2, volSize.y/2, volSize.z/2};
22   const float dx = c - pos0.x, dy = r - pos0.y, dz = s - pos0.z;
23   // sphere
24   if (id == 0) { return sqrtf(dx*dx + dy*dy + dz*dz) - params.x; }
25   else if (id == 1) { // torus
26     const float r = sqrtf(dx*dx + dy*dy);
27     return sqrtf((r - params.x)*(r - params.x) + dz*dz) - params.y;
28   }
29   else { // block
30     float x = fabsf(dx) - params.x, y = fabsf(dy) - params.y,
31           z = fabsf(dz) - params.z;
32     if (x <= 0 && x <= 0 && z <= 0) return fmaxf(x, fmaxf(y, z));
33     else {
34       x = fmaxf(x, 0), y = fmaxf(y, 0), z = fmaxf(z, 0);
35       return sqrtf(x*x + y*y + z*z);
36     }
37   }
38 }
39
40 __device__ float3 scrIdxToPos(int c, int r, int w, int h, float zs) {
41   return make_float3(c - w/2, r - h/2, zs);
42 }
43
44 __device__ float3 paramRay(Ray r, float t) { return r.o + t*(r.d); }
45
46 __device__ float planeSDF(float3 pos, float3 norm, float d) {
47   return dot(pos, normalize(norm)) - d;
48 }
49
50 __device__
51 bool rayPlaneIntersect(Ray myRay, float3 n, float dist, float *t) {
52   const float f0 = planeSDF(paramRay(myRay, 0.f), n, dist);
53   const float f1 = planeSDF(paramRay(myRay, 1.f), n, dist);
54   bool result = (f0*f1 < 0);
55   if (result) *t = (0.f - f0) / (f1 - f0);
56   return result;
57 }
58
59 // Intersect ray with a box from volumeRender SDK sample.
60 __device__ bool intersectBox(Ray r, float3 boxmin, float3 boxmax,
61   float *tnear, float *tfar) {
62   // Compute intersection of ray with all six bbox planes.
63   const float3 invR = make_float3(1.0f)/r.d;
64   const float3 tbot = invR*(boxmin - r.o), ttop = invR*(boxmax - r.o);
65   // Re-order intersections to find smallest and largest on each axis.
66   const float3 tmin = fminf(ttop, tbot), tmax = fmaxf(ttop, tbot);
67   // Find the largest tmin and the smallest tmax.
68   *tnear = fmaxf(fmaxf(tmin.x, tmin.y), fmaxf(tmin.x, tmin.z));
69   *tfar = fminf(fminf(tmax.x, tmax.y), fminf(tmax.x, tmax.z));
70   return *tfar > *tnear;
```

```
}

__device__ int3 posToVolIndex(float3 pos, int3 volSize) {
  return make_int3(pos.x + volSize.x/2, pos.y + volSize.y/2,
                   pos.z +  volSize.z/2);
}

__device__ int flatten(int3 index, int3 volSize) {
  return index.x + index.y*volSize.x + index.z*volSize.x*volSize.y;
}

__device__ float density(float *d_vol, int3 volSize, float3 pos) {
  int3 index = posToVolIndex(pos, volSize);
  int i = index.x, j = index.y, k = index.z;
  const int w = volSize.x, h = volSize.y, d = volSize.z;
  const float3 rem = fracf(pos);
  index = make_int3(clipWithBounds(i, 0, w - 2),
    clipWithBounds(j, 0, h - 2), clipWithBounds(k, 0, d - 2));
  // directed increments for computing the gradient
  const int3 dx = {1, 0, 0}, dy = {0, 1, 0}, dz = {0, 0, 1};
  // values sampled at surrounding grid points
  const float dens000 = d_vol[flatten(index, volSize)];
  const float dens100 = d_vol[flatten(index + dx, volSize)];
  const float dens010 = d_vol[flatten(index + dy, volSize)];
  const float dens001 = d_vol[flatten(index + dz, volSize)];
  const float dens110 = d_vol[flatten(index + dx + dy, volSize)];
  const float dens101 = d_vol[flatten(index + dx + dz, volSize)];
  const float dens011 = d_vol[flatten(index + dy + dz, volSize)];
  const float dens111 = d_vol[flatten(index + dx + dy + dz, volSize)];
  // trilinear interpolation
  return (1 - rem.x)*(1 - rem.y)*(1 - rem.z)*dens000 +
    (rem.x)*(1 - rem.y)*(1 - rem.z)*dens100 +
    (1 - rem.x)*(rem.y)*(1 - rem.z)*dens010 +
    (1 - rem.x)*(1 - rem.y)*(rem.z)*dens001 +
    (rem.x)*(rem.y)*(1 - rem.z)*dens110 +
    (rem.x)*(1 - rem.y)*(rem.z)*dens101 +
    (1 - rem.x)*(rem.y)*(rem.z)*dens011 +
    (rem.x)*(rem.y)*(rem.z)*dens111;
}

__device__ uchar4  sliceShader(float *d_vol, int3 volSize, Ray boxRay,
  float gain, float dist, float3 norm) {
  float t;
  uchar4 shade = make_uchar4(96, 0, 192, 0); // background value
  if (rayPlaneIntersect(boxRay, norm, dist, &t)) {
    float sliceDens = density(d_vol, volSize, paramRay(boxRay, t));
    shade = make_uchar4(48, clip(-10.f * (1.0f + gain) * sliceDens),
                        96, 255);
  }
  return shade;
}

__device__ uchar4 volumeRenderShader(float *d_vol, int3 volSize,
  Ray boxRay, float threshold, int numSteps) {
  uchar4 shade = make_uchar4(96, 0, 192, 0); // background value
  const float dt = 1.f/numSteps;
  const float len = length(boxRay.d)/numSteps;
  float accum = 0.f;
  float3 pos = boxRay.o;
  float val = density(d_vol, volSize, pos);
```

```
131   for (float t = 0.f; t<1.f; t += dt) {
132     if (val - threshold < 0.f) accum += (fabsf(val - threshold))*len;
133     pos = paramRay(boxRay, t);
134     val = density(d_vol, volSize, pos);
135   }
136   if (clip(accum) > 0.f) shade.y = clip(accum);
137   return shade;
138 }
139
140 __device__ uchar4 rayCastShader(float *d_vol, int3 volSize,
141   Ray boxRay, float dist) {
142   uchar4 shade = make_uchar4(96, 0, 192, 0);
143   float3 pos = boxRay.o;
144   float len = length(boxRay.d);
145   float t = 0.0f;
146   float f = density(d_vol, volSize, pos);
147   while (f > dist + EPS && t < 1.0f) {
148     f = density(d_vol, volSize, pos);
149     t += (f - dist) / len;
150     pos = paramRay(boxRay, t);
151     f = density(d_vol, volSize, pos);
152   }
153   if (t < 1.f) {
154     const float3 ux = make_float3(1, 0, 0), uy = make_float3(0, 1, 0),
155                  uz = make_float3(0, 0, 1);
156     float3 grad = {(density(d_vol, volSize, pos + EPS*ux) -
157                     density(d_vol, volSize, pos))/EPS,
158                    (density(d_vol, volSize, pos + EPS*uy) -
159                     density(d_vol, volSize, pos))/EPS,
160                    (density(d_vol, volSize, pos + EPS*uz) -
161                     density(d_vol, volSize, pos))/EPS};
162     float intensity = -dot(normalize(boxRay.d), normalize(grad));
163     shade = make_uchar4(255*intensity, 0, 0, 255);
164   }
165   return shade;
166 }
```

将内核代码拆分成多个文件被称为“生成可重定位设备代码”

我们选择将 vis_3d 应用的 CUDA 代码拆分成多个文件（使它们的长度可以控制）。这个决定需要**分别编译**（separate compilation）和生成**可重定位设备代码**（relocatable device code）。CUDA 5.0 之前的版本是不支持这个功能的（所有的内核代码为了编译不得不在一个文件中）。

现在，“分别编译”功能是受支持的，但是你需要设置 nvcc 的参数标识 --relocatable-device-code=true 或者简写为 -rdc=true。

为了在 Visual Studio 环境下设定 rdc 标识，需要先打开工程属性页（鼠标右击属性管理器内的工程或者从菜单中选择属性页），然后依次选择 Configuration Properties ⇒ CUDA C/C++ ⇒ Common ⇒ Generate Relocatable Device Code ⇒ Yes (-rdc=true)，见图 7.2。

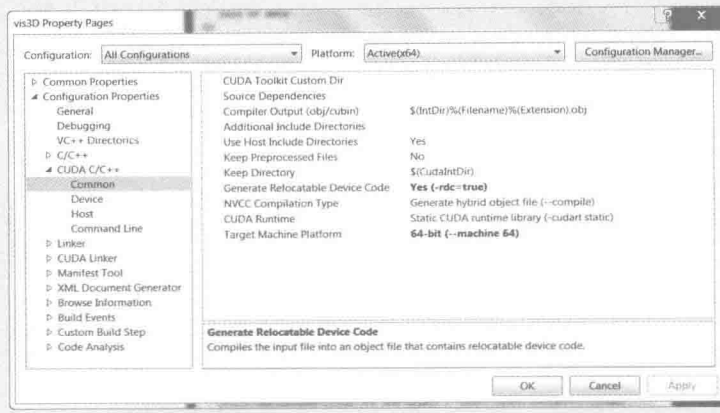


图 7.2　Visual Studio 属性页，生成可重定位设备代码设定为 Yes(-rdc=true) 以使内核代码能在多个文件内

Linux 用户将在文件 vis_3d/Makefile 的第 11 行找到合适的编译器标识，见代码清单 7.13。

代码清单7.13　vis_3d/Makefile

```
1 UNAME_S := $(shell uname)
2
3 ifeq ($(UNAME_S), Darwin)
4   LDFLAGS = -Xlinker -framework,OpenGL -Xlinker -framework,GLUT
5 else
6   LDFLAGS += -L/usr/local/cuda/samples/common/lib/linux/x86_64
7   LDFLAGS += -lglut -lGL -lGLU -lGLEW
8 endif
9
10 NVCC = /usr/local/cuda/bin/nvcc
11 NVCC_FLAGS=-Xcompiler "-Wall -Wno-deprecated-declarations" -rdc=true
12 INC = -I/usr/local/cuda/samples/common/inc
13
14 all: main.exe
15
16 main.exe: main.o kernel.o device_funcs.o
17   $(NVCC) $^ -o $@ $(LDFLAGS)
18
19 main.o: main.cpp kernel.h interactions.h
20   $(NVCC) $(NVCC_FLAGS) -c $< -o $@
21
22 kernel.o: kernel.cu kernel.h device_funcs.cuh
23   $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@
24
25 device_funcs.o: device_funcs.cu device_funcs.cuh
26   $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@
```

做完最后的这些配置，是时候编译并运行 vis_3d 应用了。你需要选择合适的数据规模才能做到实时交互。一个搭载 GeForce GT 640M 的笔记本能够处理 600×600 图像大

小和 512×512×512 图像堆栈大小的问题（更大的三维数据集将有可能超过可用设备内存的大小，所以你需要估计你的 GPU 能够分配的最大设备内存）。鼠标右键单击图像窗口能够打开一个弹出菜单并能选择用于可视化的物体。请尝试以不同的着色方法、观察角度和其他可调参数来观察物体。

使用 vis_3d 可视化的“甜甜圈”三维数据的截图，请见图 7.3、图 7.4 和图 7.5。

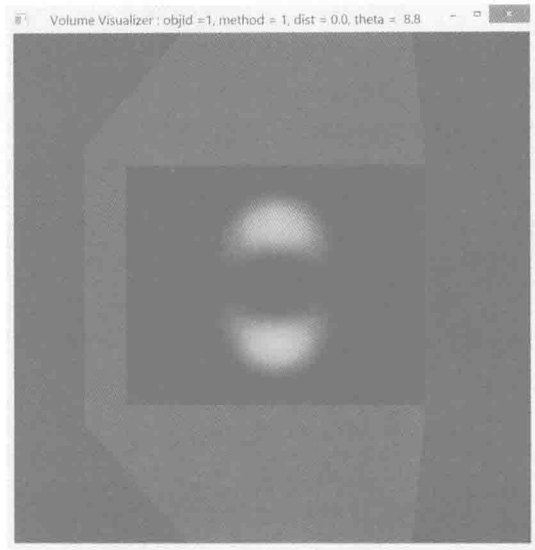


图 7.3 使用切片法观察“甜甜圈”的屏幕截图

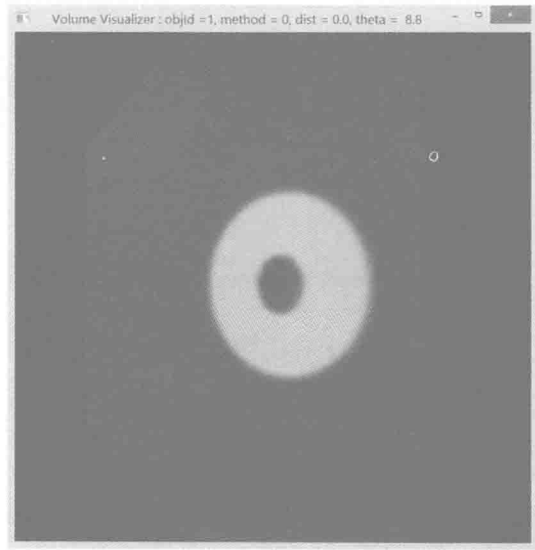


图 7.4 使用体绘制法观察“甜甜圈”的屏幕截图

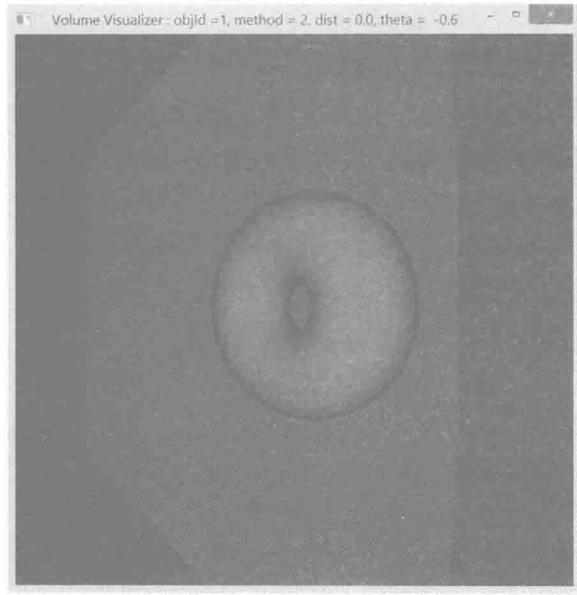


图 7.5　使用光线投射法观察“甜甜圈”的屏幕截图

7.3　本章小结

在本章我们学习了如何启动三维内核函数和如何与三维数据进行实时交互，并使用了三种不同的可视化技术：切片法，体绘制法和光线投射法。我们希望你在完成本章的学习后，能够有信心去编写自己的 CUDA 三维交互项目。这里有一些督促你继续前进的建议。

7.4　推荐项目

1. 修改 dist_3d 应用，使用 CPU 计算其相应的结果。比较 CPU 和 GPU 的结果，证明它们是一致的。为 CPU 和 GPU 的计算计时，并在不同的规模下比较两者的运算时间。
2. 通过使用更好的求根技术来替换现有的技术，扩展 raycastShader() 函数使得它能够处理更一般的数据集。
3. 使用更高级的积分程序例如梯形积分方法（以两步的平均值而不是直接以一端的值来计算面积）来增强 volumeRenderShader() 函数。
4. 想想如何在 vis_3d 应用中输入数据，观察它并与之交互。下一次你拍 CT 或核磁时，

记得拷贝一份数据，并对它进行可视化。

5. 实现全方位的三维交互（而不是只能绕一个轴旋转）。
6. 实现一个程函方程（eikonal equation）求解器，能够从图像堆栈创建有向距离网格（signed distance grid）。（更多细节，请参阅[4]。）

参考文献

[1] G. Scott Owen, "Ray - Box Intersection." (1998) http://www.siggraph.org/education/materials/HyperGraph/raytrace/rtinter3.htm.

[2] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery, *Numerical Recipes: The Art of Scientific Computing*. (New York, NY: Cambridge University Press, 1992).

[3] John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley, *Computer Graphics: Principles and Practice, Third Edition*. (Boston, MA: Addison-Wesley, 2014).

[4] J. A. Sethian, *Level Set Methods and Fast Marching Methods: Evolving Interfaces in Computational Geometry, Fluid Mechanics, Computer Vision, and Materials Science*. (New York, NY: Cambridge University Press, 1999).

第 8 章 Chapter 8

CUDA 函数库实践

本章会介绍一些软件函数库。这些函数库可以让你利用 CUDA 的能力，而无须亲自编写全部代码。在讲解这些 CUDA 函数库的过程中，我们将明确指出一些并行程序设计的模式，并通过具体的例子来说明这些模式是如何为你所用的。同时，本章还涉及一些转而采用基于函数库来实现的程序，包括图像处理以及距离、质心的计算，另外也加入了一些新的示例，涉及对圆周率 π 进行的蒙特卡罗模拟、对图像空间中距离的计算，以及对线性方程组的求解等。

8.1 自定义的与现有的

到目前为止，我们已经投入了大部分精力来构建自定义的应用程序，主要通过使用 CUDA API 编程工具以及对 C 语言进行扩展延伸来完成构建。然而，这里还是有其他的情况，更适合考虑使用一个已经存在的、不需要自己构建的代码库：

- 你需要某种特定的系统功能，它们需要你弄清一系列细节，否则无法轻易地实现相应的功能。我们曾经两次遇到这样的情况：
  - 交互式图形：编写代码与显示器上所示的内容进行交互，实际上涉及处理你的系统和显示设备的规格。这时可以使用一个已存在的图形库（例如我们在前几章中用到的 OpenGL）来提供必要的抽象层，它们可以使交互式图形处理的执行独立于特定的硬件细节。

- 图像文件：相比于处理多种图像文件格式的细节，在涉及图像文件输入 / 输出的项目中，我们使用 CImg 库来从图像文件中传入和传出数据。

- 你应该对代码的可靠性和可维护性十分关注，并且希望使用那些由专家们编写并进行了充分测试的程序；另外也希望在未来能对代码的重要功能加以改进，以确保其具有一个持久可用的生命期。

我们现在可以确认代码库可以发挥重要作用的场合，但也要承认使用它们是需要一些开销的——即便是那些免费的代码库。这些开销指的就是你每次学习使用一个新的代码库时需要投入的时间和精力，但是你也不应因此望而却步。随着经验的积累，学习使用新的代码库会变得更加容易。

我们将快速浏览一些与 CUDA 相关的代码库，这样你可以了解如何使用它们。当谈及权衡开销与收益时，这里有几件事情值得铭记：

- 并行模式：毫无疑问，成本 - 效益的分析一定程度上受我们使用特定的编码工具的频率影响，在很多应用场合提到的并行程序设计构造方式都统一称为“并行模式”。这样的模式也为我们创造和交流我们所设计产品的结构提供了一个很方便的环境。当代码库允许使用常用并行模式后，就会变得特别实用，所以当我们浏览代码库时，会明确地给出模式的类型。McCool、Robison 以及 Reinders[1] 已经定义了下面这一组并行模式：包括超标量序列任务图、选择、映像、集合、模板、划分、归约、扫描以及包装。在本章，我们将会遇到其中的一部分模式。

- CUDA 基础知识：既然 CUDA 库如此实用，为什么之前我们要投入那么多精力去使用 CUDA C 语言来编写应用程序呢？之前的努力都付之东流了吗？我确信这样的担心是完全没有必要的，即使你最终成为了极度依赖 CUDA 库的用户（或者通过其他的编程语言来调用 CUDA 的能力），你使用 CUDA C 语言开发应用程序的经验对你来说依然很有用。你会对使用的代码库的“后台”如何运行深入理解，这会让你更好地去辨别 CUDA 库的局限性并学习去利用那些库和编程语言的长处。

设定好了本章的知识背景后，让我们开始学习一些 CUDA 库。鉴于已经存在太多的 CUDA 库，所以全面地浏览所有的库是不现实的。这里我们的目标是只提及部分 CUDA 库，对它们的使用提供说明性的例子，并指出它们所使用的并行模式。

8.2　Thrust 库

CUDA 工具包中的 Thrust 库文档 [2] 提供了一个完备而简明的描述：

Thrust 是一个供 CUDA 使用的 C++ 模板库，它以标准模板库（Standard Template Library，STL）为基础。通过一个可用 CUDA C 语言进行完全操控的高级接口，Thrust 可以让你进行最小限度的编程工作，来实现高性能的并行应用程序的开发。

Thrust 提供了丰富的数据并行原语集合，例如扫描、排序和归约，这些可以与简明、易读的源代码组合在一起去实现复杂的算法。通过依照这些高层次的抽象来描绘你的计算，你就可以留给 Thrust 库自动选择最有效的实现方式的余地。因此，Thrust 库可以被用于 CUDA 应用程序的快速原型设计中，适用于对程序员工作效率比较在意的场合，也同样适用于健壮性和绝对性能至关重要的生产过程。

有关 Thrust 库的更多信息可参考 Thrust 项目网站 [3]。

既然 Thrust 是作为一个 C++ 模板库供 CUDA 使用的，我们应当花一些时间去介绍它的基础，也就是标准模板库。在本书中，我们常用的是 C 语言风格，其中的函数定义具有特异性（type-specific），因此一个函数的定义只适用于单一的参数类型列表。C++ 则提供了更多的灵活性，表现为允许函数按照通用的类型使用 C++ 中的模板和重载两大特性（即允许一个函数名在不同的数据输入类型下表示不同的操作），C++ 支持变量模板、类模板和函数模板（这将与这部分讨论的内容密切相关）。由于 C++ 语言源于 C 语言，并且我们之前已经使用了 C++ 编译器来编译 C 语言，因此我们的开发工具是完全支持使用这样的 C++ 特性的。

C++ 标准库包含多种容器（container）供处理给定类型的变量集合使用，其中最常用的容器为向量，这是一种具有自动调整大小能力的序列容器。

因为标准库模板不是唯一用 vector 作为命名方式的软件库，因此要使用命名空间（反映本质的前缀标识符）来解决名称冲突。特别地，C++ 标准库的一切内容的名称前都会加上 std::，因此一个向量的定义形式为 std::vector（或者你可以在代码文件中加入声明 using namespace std; 来引入整个 std 名字空间，虽然这样做确实会提供一些便利，不过这也会牺牲代码的可读性，因此在多数情况下，我们会使用包含命名空间前缀的完整表述）。

Thrust 是 CUDA 的一个模板库，使用命名空间 thrust。不同于容器 std::vector，Thrust 需要使用 thrust::host_vector 和 thrust::device_vector。Thrust 向量的单个元素的访

问可以通过使用 C 语言风格的方括号或 C++ 语言风格下的迭代器来实现。

Thrust 使用基于向量和迭代器的语法，并且为分配和释放内存，创建、获取、复制向量内容以及对向量进行操作提供内置的功能（不需要显式调用 cudaMalloc()、cudaMemcpy() 或 cudaFree() 方法）。Thrust 还提供了执行多种并行模式的指令，包括选择、映射、归约、扫描和划分，还有其他常见的操作，如排序和向量比较。虽然我们集中于使用 CUDA 在 GPU 上执行 Thrust 库提供的代码。Thrust 也包含了供 OpenMP 执行的“后台”和线程构建块（threading building blocks，TBB），因此 Thrust 库的代码在 GPU 与多核 CPU 系统之间是可移植的。[5, 6, 7]

Thrust 库提供了一个抽象层，这免除了我们编写自定义内核代码的必要性，但是这个额外的抽象层也会使调试出现一些麻烦，同时也需要一些特定的 C++ 知识。

8.2.1 使用 inner_product() 计算向量的模

在了解了一些相关的背景知识后，让我们看一些使用 Thrust 库的例子。首先考虑一个仅用到 Thrust 库内置函数的简单例子，并在此基础上添加一些个性化的内容。

代码清单 8.1 展现了有关并行地计算向量的长度（即向量的模）的 Thrust 库版本的代码，即其值等于向量自身的点积，然后进行平方根运算后的数量（也被称作 N 维毕达哥拉斯公式），对应于如下公式。

$$\|v\| = \sqrt{\sum_{i=0}^{N-1} (v_i)^2} = \sqrt{v \cdot v}$$

点积（或称作数量积）的计算涉及将每一项与自身相乘并加和得到最终的结果，所以在计算过程中出现了映射和归约两种模式。

代码清单8.1 norm/kernel.cu，应用Thrust库计算向量的模

```
#include <thrust/device_vector.h>
#include <thrust/inner_product.h>
#include <math.h>
#include <stdio.h>
#define N (1024*1024)

int main() {
  thrust::device_vector<float> dvec_x(N, 1.f);
  float norm = sqrt(thrust::inner_product(dvec_x.begin(),
    dvec_x.end(), dvec_x.begin(), 0.0f));
  printf("norm = %.0f\n", norm);
  return 0;
}

```

第 1～4 行包含了在这段代码中用到的 Thrust 库特性的头文件（以及数学函数库和输入 / 输出库），在第 5 行定义了向量的长度 N。main() 函数首先声明了名为 dvec_x 的 device_vector 类型的对象，用来容纳 N 个初值均置为 1.0 的 float 型变量。第 9 行和第 10 行计算了 dvec_x 与自身相乘得到的数量积，并通过平方根运算得到模数。thrust::inner_product 的参数包括第一个向量的起始和终止迭代器，以及为第二个输入向量设定的起始迭代器（在本例中，通过 device_vector 的成员函数 .begin() 方法和 .end() 方法获得）和之后元素归约乘法的初始值。在输出模的值后，main() 函数返回 0 并结束。之后你就可以编译并运行代码，来核实所得到的结果是否正确；由于每一项的值均为 1，进行平方后的值为 1，之后进行数量积运算后的结果是 N，所以模的值为$\sqrt{N}$，在本例中即为 1024。本程序的生成文件和建立过程如代码清单 8.2 所示。

> Thrust 库代码一定要放在 CUDA 文件中
>
> 注意一定要将你的 Thrust 库代码放入一个 .cu 文件来使其参与编译！

代码清单8.2　norm/Makefile

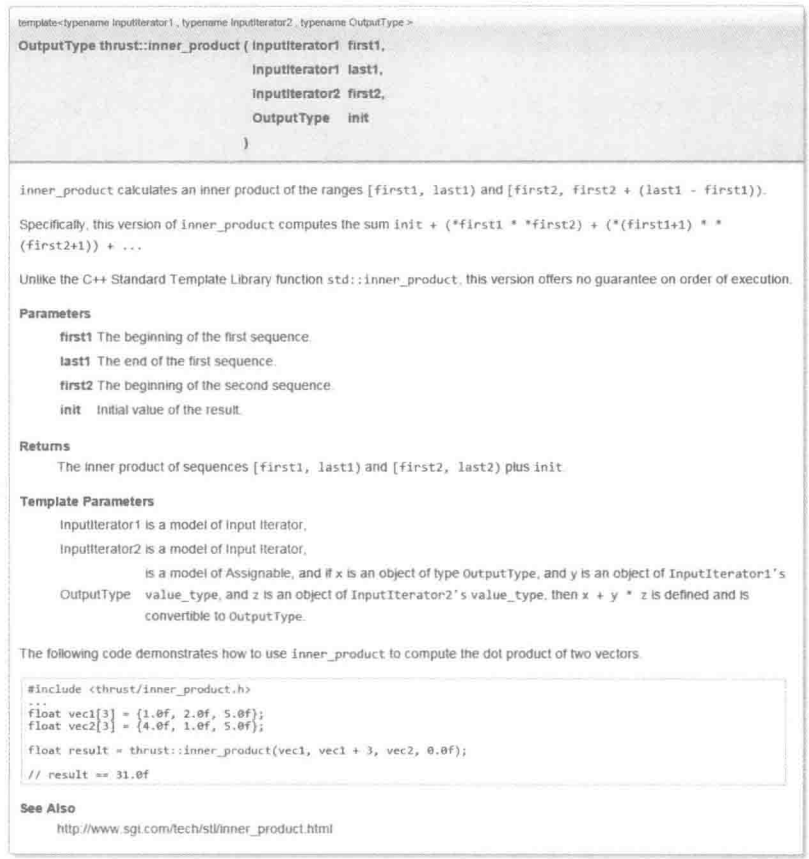
```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3
4 main.exe: kernel.cu
5   $(NVCC) $(NVCC_FLAGS) $^ -o $@
```

现在我们来看一个重要的问题：你应该怎样去了解 inner_product 以及它的语法呢？参考一些优良的样例代码是一个好的开始，但是当你摆脱样例代码的限制后，你就需要了解 Thrust 库的全部文件。样例程序和全部的 API 文件都可以在 Thrust 工程网站 [3] 在线查看。

网站上的文件目录中也包含了一个 Thrust 库的维基百科链接，那里提供了很多其他有用的资料，包括快速入门指南（Quick Start Guide）、常见问题与解答（FAQ）以及有关错误处理和调试的文档。

我推荐你去浏览 Thrust 库的网页，但是现在先回到我们的目标上来，即寻找有关于 inner_product 以及它的语法的信息。进入网站 http://thrust.github.io/，选择 Documentation ⇒ Thrust API (Doxygen) 来建立一个列出以迭代器和算法作为初始的 Thrust 模块。我们在寻找一个可以计算数量积的算法，并且（再次多亏了我们的 CUDA 背景知识）我们知道这涉及一个归约过程。可能经历了一番四处寻找后，我们最终选择 Algorithms ⇒ Reductions ⇒ Transformed Reductions，并找到文件，如图 8.1 所示。这样的文件向你提

供一个囊括丰富信息的环境，但经过一些练习之后，你就会熟练地找到你需要的东西。



template<typename InputIterator1 , typename InputIterator2 , typename OutputType >

OutputType thrust::inner_product (InputIterator1 first1,
InputIterator1 last1,
InputIterator2 first2,
OutputType init
)

inner_product calculates an inner product of the ranges [first1, last1) and [first2, first2 + (last1 - first1)).

Specifically, this version of inner_product computes the sum init + (*first1 * *first2) + (*(first1+1) * *(first2+1)) + ...

Unlike the C++ Standard Template Library function std::inner_product, this version offers no guarantee on order of execution.

Parameters

first1 The beginning of the first sequence.
last1 The end of the first sequence.
first2 The beginning of the second sequence.
init Initial value of the result.

Returns

The inner product of sequences [first1, last1) and [first2, last2) plus init.

Template Parameters

InputIterator1 is a model of Input Iterator.
InputIterator2 is a model of Input Iterator.
OutputType is a model of Assignable, and if x is an object of type OutputType, and y is an object of InputIterator1's value_type, and z is an object of InputIterator2's value_type, then x + y * z is defined and is convertible to OutputType.

The following code demonstrates how to use inner_product to compute the dot product of two vectors.

```
#include <thrust/inner_product.h>
...
float vec1[3] = {1.0f, 2.0f, 5.0f};
float vec2[3] = {4.0f, 1.0f, 5.0f};

float result = thrust::inner_product(vec1, vec1 + 3, vec2, 0.0f);

// result == 31.0f
```

See Also

http://www.sgi.com/tech/stl/inner_product.html

图 8.1 有关 thrust::inner_product 的文档

在本文档中，Returns 的部分（连同阴影标题下的两行文字）证实了这个算法可以完成我们要找的数量积运算。Parameters 部分描述了输入变量的列表，底部的文本框中的代码片段阐明了它的用法。

使用包含数组的 Thrust 库（代替向量）

不管怎样，此文件中的样例碰巧使用了常规的 C 语言数组作为输入代替使用 Thrust 向量，所以不能明显地看到迭代器，此外还使用了一些指针运算。因为数组的名称就是指向它第一个元素的指针，所以第一个数组的名称 vec1 就是指定第一个数组起始位置的迭代子，即 first1。（同样的，第三个参数 vec2 就是迭代子 first2 的值。）第二个参数 last1 是一个指示 vec1 末尾的迭代器。在样例代码中，这个值被用于指针运

算：vec1 指向长度为 3 的数组的起始元素，因此 vec1+3 指向 3 个元素数组的末尾。如果你使用向量，你可以在返回迭代器的处理过程中使用 begin() 和 end() 成员函数，并且不一定要直接参与指针运算。

需要注意的是我们刚刚为之前的一个问题确定了一个高效的解决方法。在第 5 章的 heat_2d 程序中，我们执行雅可比迭代来逼近稳态温度分布。在建立了临界条件后，我们开始进行迭代并让其持续运行。在那一章的结尾，我们对建立一个收敛性判定准则的可行性提出讨论，这一准则可为确定迭代停止的时间提供依据。在第 6 章中，我们指出一个合理的终止准则可以基于迭代器间气温数组变化的范数之上，并且通过使用 thrust::inner_product 方法，我们可以仅用一行代码来计算这样的范数。

8.2.2　使用 transform() 计算距离

现在我们来看一下程序 dist_1d_thrust 的 Thrust 实现，这与第 3 章的距离计算程序类似。这一例子再次计算了从一个参考点到一条线段上均匀分布的点序列的距离。相关的代码展示在了代码清单 8.3 中，我们将会逐行分析它。

代码清单8.3　dist_1d_thrust/kernel.cu

```
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/sequence.h>
#include <thrust/transform.h>
#include <math.h>
#include <stdio.h>
#define N 64

using namespace thrust::placeholders;

// Define transformation SqrtOf()(x) -> sqrt(x)
struct SqrtOf {
  __host__ __device__
  float operator()(float x) {
    return sqrt(x);
  }
};

int main() {
  const float ref = 0.5;
  thrust::device_vector<float> dvec_x(N);
  thrust::device_vector<float> dvec_dist(N);
  thrust::sequence(dvec_x.begin(), dvec_x.end());
  thrust::transform(dvec_x.begin(), dvec_x.end(),
                    dvec_x.begin(), _1 / (N - 1));
  thrust::transform(dvec_x.begin(), dvec_x.end(),
                    dvec_dist.begin(), (_1 - ref)*(_1 - ref));
  thrust::transform(dvec_dist.begin(), dvec_dist.end(),
```

```
29                      dvec_dist.begin(), SqrtOf());
30   thrust::host_vector<float> h_x = dvec_x;
31   thrust::host_vector<float> h_dist = dvec_dist;
32   for (int i = 0; i < N; ++i) {
33     printf("x=%3.3f, dist=%3.3f\n", h_x[i], h_dist[i]);
34   }
35   return 0;
36 }
```

第 1～4 行指定了我们将用到的 Thrust 特性所包含的文件。其余包含的头文件是 math.h 和 stdio.h。

这里，先介绍 main() 函数的内容，之后再回过头来讨论中间的代码。第 20 行将参考位置 ref 置为 0.5，这与我们起初的例子一致；之后我们直接进入到 Thrust 的使用。（注意这里选择的实现方式主要是为了表达清楚，并不是最优的操作。最优的操作方式更可能是将多个步骤合并或整合成一个单独的更复杂的表达方式，但这可能与我们当前的目的相反，所以我们将有关于代码整合的讨论留在这一章后面去讲解。）

第 21 行和第 22 行创建 Thrust 设备向量来保存输入的值和输出的距离。第 23～25 行用区间 [0, 1] 上的等间隔的值来填充输入向量 dvec_x，这一操作分两个步骤完成：sequence(d_x.begin(), d_x.end()) 中用值为 0.0, 1.0, ..., N–1 的序列来填充向量。为了产生期望得到的 [0,1] 区间上的值，我们需要用 N-1 来除每一个值。这里我们使用了 thrust::transform 进行除法运算（映射模式的另一种表现形式），它的参数包括了输入向量的第一个和最后一个迭代器以及它们的转换。dvec_x 既是输入向量也是输出向量，前三个参数分别为 dvec_x.begin()、dvec_x.end() 和 dvec_x.begin()。真正棘手的部分是最后一个指定转换的参数。一个函数不能像一个参数那样直接传递，因此需要一些更加复杂的东西。

在本例中，转换部分只涉及一个标准算术运算，并且我们可以使用 Thrust 库的位置标志符特性来创建一个可行的转换。位置标志符的格式使用一个下划线后接数字的形式，其中数字指的是对应它在参数列表中的位置的参数，标准算术运算按正常的形式进行。这样我们就可以解释 thrust::transform 的第 4 个参数。表达式 _1/(N–1) 表示输出的转换应当包含第一个输入（第一个输入向量中的元素）除以（N–1）。（注意输入向量是以 float 的形式进行声明的，所以要进行浮点数除法运算。）位置标志符有特定的命名空间 thrust::placeholders，这也是我们要在第 9 行中使用指令 using namespace 的原因。

在第 26 行和第 27 行中，我们再一次使用了位置标识符表示法。(dvec_x[i]-ref)* (dvec_x[i]-ref) 表示从参考点到第 i 个格点的距离的平方，因此 thrust::transform(dvec_

x.begin(), dvec_x.end(), dvec_dist.begin(), (_1-ref)*(_1-ref)) 将输出设备向量用平方后的距离进行填充。现在我们需要计算每一项的平方根，但是 sqrt() 不是一个标准算术运算，它的定义属于标准数学库，我们在这里使用的转换已经超过了位置标志符的功能，所以需要一个函数对象（function object）或函子（functor）。

定义一个函子比定义一个函数复杂多少呢？其实并不是很复杂，让我们详细看一下。我们可以创建自己的自定义函数（让我们将其命名为 SqrtOf()）并让它可以同时由 CPU 和 GPU 进行编译来调用包含在数学库中的 sqrt() 函数，代码如下：

```
__host__ __device__
float SqrtOf(float x)
{
  return sqrt(x);
}
```

创建函子的版本中只需要以下变化：

1. 不同于定义名为 SqrtOf 的函数，可以通过 struct SqrtOf{ }; 将函数 SqrtOf 的定义包裹起来，创建一个同名的 struct（在 C++ 专业术语中属于公共类）。

2. 在函数定义中，用 operator() 代替 SqrtOf。

通过对函数定义进行上述的这些改动，你应该可以写出在第 12～17 行中所对应的函子版本的代码。

对函子的思考

如果可行，你可以按以下方式对函子进行思考。我们真正做的是创建了一个名为 SqrtOf 的 struct 结构，它包含函数调用运算符（写成圆括号）的替代（重载）定义。因为在其他语境下，括号用来给参数进行分组或显示优先级顺序，所以我们创建了一个 SqrtOf() 的替代性定义（调用 sqrt()），即括号接在 struct 名称 SqrtOf 后面。

以上涵盖了我们之前跳过部分的代码，也包括 main() 函数中剩余的内容，涉及将结果传回主存并将它们显示在用于监测的命令窗口上。注意在创建时用对应的设备向量内容的值分别填充每个主机向量（第 30 行和第 31 行），所有的向量内存会自动释放（不需要显式调用 free() 或 cudaFree() 方法），并且在命令窗口上的结果应当与使用我们早先计算距离的方法得到的结果一致。最后，建立过程的文档与 norms 中的相同，已经展示在了代码清单 8.2 中。

拉姆达表达式（Lambda expressions，也叫拉姆达函数）作为 C++ 的一个便捷的函数，将函数之间的传递变得更加简洁、更具可读性。拉姆达表达式可以用来代替函子和

其他匿名函数。近期 CUDA 7.5 将设备拉姆达表达式的一个试验性的特性进行推介。因此，现在（尽管是试验性的）我们可以在我们的 Thrust 算法中使用拉姆达表达式。代码清单 8.4 展示了使用拉姆达表达式的 dist_1d_thrust 程序。

代码清单8.4 使用拉姆达表达式的dist_1d_thrust/kernel.cu

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/sequence.h>
#include <thrust/transform.h>
#include <math.h>
#include <stdio.h>
#define N 64

int main() {
  const float ref = 0.5;
  thrust::device_vector<float> dvec_x(N);
  thrust::device_vector<float> dvec_dist(N);
  thrust::sequence(dvec_x.begin(), dvec_x.end());
  thrust::transform(dvec_x.begin(), dvec_x.end(), dvec_x.begin(),
    [] __device__ (float x) { return x / (N-1); });
  thrust::transform(dvec_x.begin(), dvec_x.end(), dvec_dist.begin(),
    [=] __device__ (float x) { return (x-ref)*(x-ref); });
  thrust::transform(dvec_dist.begin(), dvec_dist.end(),
    dvec_dist.begin(), [] __device__ (float x) { return sqrt(x); });
  thrust::host_vector<float> h_x = dvec_x;
  thrust::host_vector<float> h_dist = dvec_dist;
  for (int i = 0; i < N; ++i) {
    printf("x=%.3f, dist=%.3f\n", h_x[i], h_dist[i]);
  }
  return 0;
}
```

相关的文件内容在代码清单 8.5 中给出。Visual Studio 用户需要在 Configuration Properties ⇒ CUDA C/C++ ⇒ Command Line 下的附加选项框中进入编译器标志 --expt-extended-lambda。

代码清单8.5 dist_1d_thrust/Makefile，兼容试验性的拉姆达特征

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall --std=c++11 --expt-extended-lambda

main.exe: kernel.cu
  $(NVCC) $(NVCC_FLAGS) $^ -o $@
```

代码也可以通过整合（fusion）变得更加简短（并且更高效），也就是将并行代码中的多个步骤的功能进行合并。关于一维距离的函子实现的整合版本，即 dist_1d_fused，在代码清单 8.6 中进行了展示。

代码清单8.6　dist_1d_fused/kernel.cu, 整合后的对于dist_1d的Thrust实现

```
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/iterator/counting_iterator.h>
#include <thrust/transform.h>
#include <math.h>
#include <stdio.h>
#define N 64

// DistanceFrom(ref,n)(x)->sqrt((x/(n-1)-ref)*(x/(n-1)-ref))
struct DistanceFrom {
  DistanceFrom(float ref, int n) : mRef(ref), mN(n) {}

  __host__ __device__
  float operator()(const float &x) {
    float scaledX = x / (mN - 1);
    return std::sqrt((scaledX - mRef)*(scaledX - mRef));
  }
  float mRef;
  int mN;
};

int main() {
  const float ref = 0.5;
  thrust::device_vector<float> dvec_dist(N);
  thrust::transform(thrust::counting_iterator<float>(0),
    thrust::counting_iterator<float>(N), dvec_dist.begin(),
    DistanceFrom(ref, N));

  thrust::host_vector<float> hvec_dist = dvec_dist;
  float *ptr = thrust::raw_pointer_cast(&hvec_dist[0]); // debugging
  for (int i = 0; i < N; ++i) {
    printf("x[%d]=%.3f, dist=%.3f\n", i, 1.f*i/(N - 1), hvec_dist[i]);
  }
  return 0;
}
```

注意全部的计算是通过在第 25～27 行中对 thrust::transform 的单独调用完成的。前两个参数是用作虚拟输入序列生成器 counting_iterator 的起始值和终止值。第三个参数给出了输出向量的起始迭代器，最后一个参数指定了函子 DistanceFrom() 的调用。计数迭代器完成在输入数组中存储一个常规序列的值的等效操作，函子完成剩下的工作：它带有两个指定了参考点和数组长度的参数，计算 [0, 1] 区间上对应每个迭代器的坐标值，从 ref 中计算距离并将结果存储在 dvec_dist 中。（第 30 行中有些独立的指针声明是一种用调试工具使输出数据可视化的方法，在这部分的末尾将对其进行具体的解释。）

8.2.3　使用 generate()、transform() 以及 reduce() 对 π 的值进行估计

另一个可以说明 Thrust 库的便捷之处的著名的例子是使用蒙特卡罗法估计圆周率 π。它的计算方案很简单：生成较大数目的 N 个位于 [0, 1]×[0, 1] 的单位正方形中的坐标点，

并统计出那些位于单位圆第一象限部分中的点的数目 N_1，如图 8.2 所示。操作的前提是二维空间中的随机点均匀地分布在区域中，因此这 N 个点落在不同区域的数目之比应当接近于它们所在区域面积之比。半径 $r=1$ 的圆的第一象限的面积为 $\frac{1}{4}\pi r^2=\frac{\pi}{4}$，单位正方形的面积为 $1\times1=1$，因此面积之比为 $\frac{\pi}{4}$。最终结果为 $\frac{N_1}{N}\approx\frac{\pi}{4}$，所以我们可以通过计算 $4\frac{N_1}{N}$ 得出对于 π 的蒙特卡罗估计。

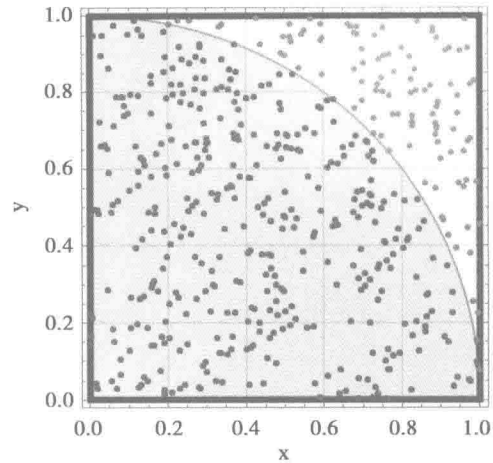


图 8.2 对于随机分布在单位正方形的大量点，位于单位圆第一象限中的点的比例近似于其面积的比例为 $\frac{\pi}{4}$

代码清单 8.7 中展示了完整的 Thrust 实现过程，以通常的 #include 声明作为起始，包含我们要用到的 Thrust 函数、数学库以及输入 / 输出库。

代码清单8.7 thrustpi/kernel.cu, 对于π的蒙特卡罗估计的简单Thrust实现

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/reduce.h>
#include <thrust/generate.h>
#include <thrust/transform.h>
#include <math.h>
#include <stdio.h>
#define N (1 << 20)

using namespace thrust::placeholders;

int main(void)
{
  thrust::host_vector<float> hvec_x(N);
  thrust::host_vector<float> hvec_y(N);
  thrust::generate(hvec_x.begin(), hvec_x.end(), rand);
  thrust::generate(hvec_y.begin(), hvec_y.end(), rand);
  thrust::device_vector<float> dvec_x = hvec_x;
  thrust::device_vector<float> dvec_y = hvec_y;
  thrust::transform(dvec_x.begin(), dvec_x.end(), dvec_x.begin(),
    _1 / RAND_MAX);
  thrust::transform(dvec_y.begin(), dvec_y.end(), dvec_y.begin(),
    _1 / RAND_MAX);
  thrust::device_vector<float>dvec_inCircle(N);
  thrust::transform(dvec_x.begin(), dvec_x.end(), dvec_y.begin(),
    dvec_inCircle.begin(), (_1*_1 + _2*_2)<1);
  float pi =
    thrust::reduce(dvec_inCircle.begin(), dvec_inCircle.end())*4.f/N;
  printf("pi = %f\n", pi);
```

```
30    return 0;
31 }
```

第 8 行中指定了 N 的值。第 10 行中指定了 thrust:: placeholders 的命名空间以免去在位置标志符表达式的下划线前添加前缀的必要。

位移位操作

注意 << 不是指“远小于”的意思。这里它是一个移位操作符，所以 1<<20 表示取整数 1 的二进制表达式并左移 20 位（换句话说，即为 $2^{20}=1,048,576$）。

main() 函数中的方法要尽可能简单明确：

1. 创建主存向量 hvec_x 和 hvec_y 来保存 N 个点的 x 和 y 坐标（见第 14 行和第 15 行）。

2. 使用 thrust::generate 将随机数填入 hvec_x 和 hvec_y（见第 16 行和第 17 行）。（注意这要用到主机上的整型随机数生成器。）

3. 创建设备向量 dvec_x 和 dvec_y 并将在主机上生成的随机数的值复制到其中（见第 18 行和第 19 行）。

4. 用 RAND_MAX（随机数生成器可以生成的最大整数）去除随机整数，使用 thrust::transform（带有位置标志符转换）来产生位于 [0, 1] 上的随机浮点数（见第 20～23 行）。

5. 创建一个设备向量 dvec_inCircle 来将单位圆以外的点记为 0，或将单位圆以内的点记为 1（即如果满足 $x^2+y^2<1$，或者用位置标志符方面的术语来表达，即 (_1*_1+_2*_2)<1)（见第 24～26 行）。

6. 统计圆内点的数量（对 dvec_inCircle 应用 thrust::reduce），乘以 4.f/N 来得到对于 π 的估计，并将结果显示在命令窗口中（见第 28 行和第 29 行）。

有关此程序建立的文档和 norm 对应的文档相同，已展示在代码清单 8.2 中。你可以编译并运行程序以查证它生成了一个合理的数值。希望通过这个例子，可以让你在不需要做任何显式编程的前提下，快速地把一些基础的部分拼在一起来完成并行计算。代码清单 8.8 中展示了使用拉姆达表达式估计 π 值的修改版本的代码，此方法不仅可以完成整合转换并减少步骤，也可以免去使用设备向量 dvec_inCircle 的必要。

代码清单8.8　thrustpi/kernel.cu，蒙特卡罗法估计π的整合后的Thrust实现

```
1 #include <thrust/host_vector.h>
2 #include <thrust/device_vector.h>
3 #include <thrust/generate.h>
4 #include <thrust/count.h>
```

```
#include <stdio.h>
#define N (1 << 20)

int main() {
  thrust::host_vector<float> hvec_x(N), hvec_y(N);
  thrust::generate(hvec_x.begin(), hvec_x.end(), rand);
  thrust::generate(hvec_y.begin(), hvec_y.end(), rand);
  thrust::device_vector<float> dvec_x = hvec_x;
  thrust::device_vector<float> dvec_y = hvec_y;
  int insideCount =
    thrust::count_if(thrust::make_zip_iterator(thrust::make_tuple(
      dvec_x.begin(), dvec_y.begin())), thrust::make_zip_iterator(
      thrust::make_tuple(dvec_x.end(), dvec_y.end())),
      [] __device__ (const thrust::tuple<float, float> &el) {
        return (pow(thrust::get<0>(el)/RAND_MAX, 2) +
                pow(thrust::get<1>(el)/RAND_MAX, 2)) < 1.f; });
  printf("pi = %f\n", insideCount*4.f/N);
  return 0;
}
```

所有的并行计算代码都在第 14～20 行中整合后的声明中出现。在第 18～20 行中组合使用 thrust::make_tuple 和 thrust::make_zip_iterator 将拉姆达表达式中参与运算的 x 和 y 坐标进行配对。拉姆达表达式从 dvec_x 和 dvec_y（分别使用 thrust::get<0> 和 thrust::get<1>）中获得值并检验从原点开始的距离是否小于参数值，即 1.0f。这里不需要任何向量来保存结果；它们直接通过 thrust::count_if 进行加和。注意这个程序要与代码清单 8.5 中所示的文件进行联合。

作为对代码整合的进一步示范，我们呈现了基于 Thrust 库的实现来替换在第 6 章中程序 centroid_2d 中的内核 centroid_2d/kernel.cu。Thrust 的实现过程如代码清单 8.9 所示。值得注意的是，这次的代码比之前的版本简洁许多。

代码清单8.9　centroid_2d/kernel.cu使用Thrust库

```
#include "kernel.h"
#include <helper_math.h>
#include <thrust/device_vector.h>
#include <thrust/iterator/counting_iterator.h>
#include <thrust/iterator/zip_iterator.h>
#include <thrust/transform_reduce.h>
#include <thrust/tuple.h>
#include <stdio.h>

struct PixelFunctor {
  PixelFunctor(int width) : mWidth(width) {}
  template <typename T>
  __host__ __device__ int3 operator()(const T &el) {
    const int idx = thrust::get<0>(el);
    const uchar4 pixel = thrust::get<1>(el);
    const int r = idx / mWidth;
    const int c = idx - r*mWidth;
    int pixVal = (pixel.x < 255 || pixel.y < 255 || pixel.z < 255);
    return make_int3(pixVal*c, pixVal*r, pixVal);
```

```
20    }
21    int mWidth;
22  };
23
24  void centroidParallel(uchar4 *img, int width, int height) {
25    thrust::device_vector<uchar4> invec(img, img + width*height);
26    thrust::counting_iterator<int> first(0), last(invec.size());
27    int3 res = thrust::transform_reduce(thrust::make_zip_iterator(
28      thrust::make_tuple(first, invec.begin())),
29      thrust::make_zip_iterator(thrust::make_tuple(last, invec.end())),
30      PixelFunctor(width), make_int3(0, 0, 0), thrust::plus<int3>());
31
32    int centroidCol = res.x / res.z;
33    int centroidRow = res.y / res.z;
34    printf("Centroid: {col = %d, row = %d} based on %d pixels\n",
35           centroidCol, centroidRow, res.z);
36
37    for (int c = 0; c < width; ++c) {
38      img[centroidRow*width + c].x = 255;
39      img[centroidRow*width + c].y = 0;
40      img[centroidRow*width + c].z = 0;
41    }
42
43    for (int r = 0; r < height; ++r) {
44      img[r*width + centroidCol].x = 255;
45      img[r*width + centroidCol].y = 0;
46      img[r*width + centroidCol].z = 0;
47    }
48  }
```

这里还有很多有关 Thrust 的知识要去学习，但是在继续学习之前，让我们再考虑一个需要掌握的问题：怎样把 Thrust 设备向量中的数据传到 CUDA 内核中去呢？相反，如果数据在设备数组中，怎样才能用 Thrust 函数对其进行操作呢？事实证明以下两种策略都可以实现：

- **将 CUDA 设备数组作为参数传到 Thrust 函数中。**像往常一样通过声明指针和分配设备内存的方式创建一个设备数组。

  ```
  float *ptr;
  cudaMalloc(&ptr, N * sizeof(float));
  ```

  Thrust 提供 thrust::device_pointer 作为一种创建指针的方式，让它们可以被传到 Thrust 函数中，如下所述：

  ```
  thrust::device_ptr<float>dev_ptr(ptr);
  ```

  之后设备指针可以被传到 Thrust 算法：

  ```
  thrust::fill(dev_ptr, dev_ptr+N, 1.0f);
  ```

  记得去包含必要的头文件，以及在操作完成后释放设备内存（cudaFree(ptr);）。

- **将 Thrust 设备向量作为参数传到 CUDA API 函数或内核。**一个 CUDA 内核函

数允许指向数组的指针作为参数，但是如何处理 Thrust 设备向量还不明了。这一问题可以由 thrust::raw_pointer_cast 来解决，这一方法的名称很有描述性。（记得要包含头文件！）一旦你创建了一个设备向量（例如，thrust::device_vector<float>dvec_x(N);），你就可以按如下方法创建一个常规指针：

```
float* ptrToX = thrust::raw_pointer_cast(&dvec_x[0]);
```

并且原始指针适合于作为内核参数使用。

相关 Windows 调试的说明

因为 Thrust 向量在调试器中无法直接显示，所以如果你强制转换一个原始指针为 Thrust 的 host_vector，你可以在监视（watch）窗口中输入指针名称和要考虑的元素个数，并用逗号隔开，进而监测数值。例如，在代码清单 8.6 的第 30 行中创建的指针提供了一种在 Visual Studio 中观察结果的方法。在调试模式下编译程序，并在第 35 行后面添加一断点。开始调试时，打开监视窗口，输入 ptr,64 来查看存储在 hvec_dist 中的距离值。

我们希望这些介绍能为你深入学习 Thrust 库提供足够的引导，下面让我们去了解其他实用的库。

8.3 cuRAND 库

你也许已经注意到，当我们使用 Thrust 库来估算 π 的值时，我们在主机端生成了随机点坐标（并将随机坐标储存于主机向量中）。如果你想将该部分计算并行化，就需要了解有关 cuRAND 的知识，即 CUDA 随机数生成库 [9]。

cuRAND 库的一般描述解释了其“灵活的使用模式”，意思是指存在一个用于产生大量随机数的主机 API（函数在主机端调用但在 GPU 上并行执行）及一个可在内核中调用的内联实现。我们估计 π 的例子中遵循随机数大量生成模式，我们生成了一系列随机数，之后将它们用于紧随其后的计算过程，因此我们在生成随机点坐标时，采用 cuRAND 主机 API 来代替之前的方法。改进后的使用 cuRAND 来估算 π 的代码在代码清单 8.10 中给出，添加了 cuRAND 头文件 curand.h 来包含文件。

代码清单8.10 thrustpi/kernel.cu是基于cuRAND库产生随机点坐标

```
1 #include <curand.h>
2 #include <thrust/device_vector.h>
```

```
 3 #include <thrust/count.h>
 4 #include <math.h>
 5 #include <stdio.h>
 6 #define N (1 << 20)
 7
 8 int main() {
 9   curandGenerator_t gen;
10   curandCreateGenerator(&gen, CURAND_RNG_PSEUDO_DEFAULT);
11   curandSetPseudoRandomGeneratorSeed(gen, 42ULL);
12   thrust::device_vector<float>dvec_x(N);
13   thrust::device_vector<float>dvec_y(N);
14   float *ptr_x = thrust::raw_pointer_cast(&dvec_x[0]);
15   float *ptr_y = thrust::raw_pointer_cast(&dvec_y[0]);
16   curandGenerateUniform(gen, ptr_x, N);
17   curandGenerateUniform(gen, ptr_y, N);
18   curandDestroyGenerator(gen);
19   int insideCount =
20     thrust::count_if(thrust::make_zip_iterator(thrust::make_tuple(
21       dvec_x.begin(), dvec_y.begin())), thrust::make_zip_iterator(
22       thrust::make_tuple(dvec_x.end(), dvec_y.end())),
23       [] __device__ (const thrust::tuple<float, float> &el) {
24         return (pow(thrust::get<0>(el), 2) +
25                 pow(thrust::get<1>(el), 2)) < 1.f; });
26   printf("pi = %f\n", insideCount*4.f/N);
27   return 0;
28 }
```

第 9～11 行包括了使用 cuRAND 进行的设定：

- 在第 9 行声明一个名为 gen 的 curandGenerator_t 对象。（你可以将 curandGenerator_t 当作一种由 cuRAND 创建的适用于随机数生成器的类型。）
- 建立一个带有参数的特定的随机数生成器，参数对应 gen 地址和特定生成器的选择。CURAND_RNG_PSEUDO_DEFAULT 选取默认的伪随机数生成器（由于它不需要状态输入，所以显得稍微简单一些）（见第 10 行）。
- 第 11 行为随机数生成器设定种子。curandSetPseudoRandomGeneratorSeed() 的参数包括生成器和种子的值，42ULL 表示值为 42 的 unsigned long long int 类型。

接下来看一下第 12 行和第 13 行中设备向量 dvec_x 和 dvec_y 的创建。注意 cuRAND 允许我们在 GPU 上生成随机坐标值，因此现在不需要使用我们之前使用的主机向量 hvec_x 和 hvec_y。

现在我们获得了使用 Thrust 的指针强制转换能力的首次机会。我们将使用 cuRAND 函数 curandGenerateUniform() 来生成一个均匀分布的随机数集合，curandGenerateUniform() 预设的参数包括一个 curandGenerator_t 对象（来生成随机值），一个指向设备数组的指针（生成值被储存的地方）以及将要生成的值的数量。为了将随机值存入设备数组，我们在第 14 行和第 15 行中使用 thrust::raw_pointer_cast() 创建两个指针 ptr_x 和

ptr_y，并指向每个设备向量的起始元素。随机数通过 curandGenerate-Uniform() 进行计算并在第 16 行和第 17 行通过来自 dvec_x 和 dvec_y 强制转换的指针 ptr_x 和 ptr_y，存储到设备向量 dvec_x 和 dvec_y 中。完成随机值的生成后，在第 18 行中通过 curandDestroyGenerator() 释放掉 cuRAND 使用的资源。剩余的代码和之前的相同，建立程序的文件在代码清单 8.11 中给出。在 Visual Studio 中，通过打开项目属性页，依次选择 Linker ⇒ Input ⇒ Additional Dependencies ⇒ Edit，并添加 curand.lib 的方式来链接 cuRAND 库。

代码清单8.11 thrustpi/Makefile用于包含cuRAND能力

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall --std=c++11 --expt-extended-lambda
3 LIBS = -lcurand
4
5 main.exe: kernel.cu
6   $(NVCC) $(NVCC_FLAGS) $^ -o $@ $(LIBS)
```

生成并运行代码来了解它的执行过程。再次重申，这个例子为演示目的，所以比较简单。另外有一个与 Thrust/cuRAND 更紧密耦合和更加高效的示例，见参考文献 [9]。

8.4 NPP 库

英伟达性能原语库（The NVIDIA Performance Primitives library，NPP）是一个可被用于图像、视频和信号处理的并行函数的集合。从版本 7.5 开始，NPP 声称要实现与 CPU 执行相比 5～10 倍的性能提升，并且它包含大约 1900 种对于图像的原语处理和大约 600 种对于信号的原语处理，为加速代码的开发提供了便利。这里我们使用 NPP 来创建一个对第 5 章应用程序 sharpen 的基于库的实现，但在正式了解代码的细节之前，我们需要简单介绍一下库文件。

链接库文件

使用代码库有时需要包含头文件或链接对象文件。附录 D 中给出了有关如何在 Visual Studio 中实现这些操作的综合介绍。这里我们给出关于 NPP 的细节。Windows 用户需要告知系统将 nppc.lib 与 nppi.lib 和 npps.lib 进行链接，它们中分别有图像处理函数和信号处理函数。在 Visual Studio 中，你可以通过打开项目属性页，选择 Linker ⇒ Input 来开始此操作，之后可以单击 Additional Dependencies，选择 Edit，将需要的库添加到列表中，这样它们就会在列表中显示，如图 8.3 所示。

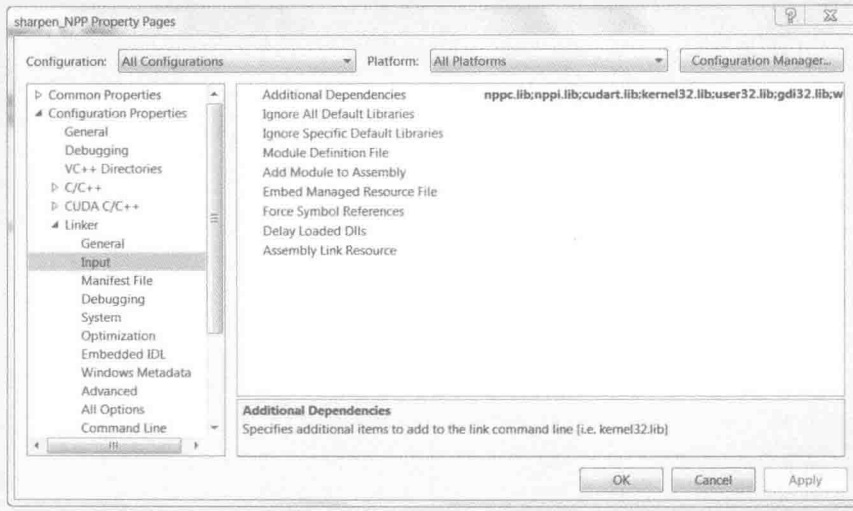


图 8.3　Linker ⇒ Input ⇒ Additional Dependencies 中包括 nppc.lib 和 nppi.lib

Linux 用户使用静态链接选项，并且如果你进行了静态链接，你也需要链接 cuLIBOS 库。在我们的文档中，提供了一种静态链接的方法，代码清单 8.12 给出了一个用于链接 cuLIBOS 和 NPP 库的示例文档。

代码清单8.12　sharpen_npp/Makefile

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3 LIBS += -lnppi_static -lnppc_static -lculibos
4
5 all: main.exe
6
7 main.exe: main.cpp
8   $(NVCC) $(NVCC_FLAGS) $^ -o $@ $(LIBS)
```

如果你想知道应该怎样去获取与此相关的信息，这里有一些资源供你使用。在一个搜索引擎中输入“CUDA NPP 附加依赖项”有助于成功地找到关于解决此问题的有用信息。然而这可能会更突显出 CUDA 样例的价值。如果你运行的 CUDA 样例需要使用相关的库，你可以打开它的属性页来检测包括附加依赖项在内的各项设置。

8.4.1　sharpen_npp

代码清单 8.13 展示了我们使用 NPP 对 sharpen 程序的再实现过程。注意在这个版本中没有 kernel.cu，所有操作都在一个 main.cpp 文件中执行，我们将在列出代码内容之后

讨论如何去找到正确的库函数并理解调用它们的语法。

代码清单8.13 sharpen_npp/main.cpp使用NPP库版本的sharpen程序

```
#define cimg_display 0
#include "CImg.h"
#include <cuda_runtime.h>
#include <npp.h>
#include <stdlib.h>
#define kNumCh 3

void sharpenNPP(Npp8u *arr, int w, int h) {
  Npp8u *d_in = 0, *d_out = 0;
  Npp32f *d_filter = 0;
  const Npp32f filter[9] = {-1.0, -1.0, -1.0,
                            -1.0,  9.0, -1.0,
                            -1.0, -1.0, -1.0};

  cudaMalloc(&d_out, kNumCh*w*h*sizeof(Npp8u));
  cudaMalloc(&d_in, kNumCh*w*h*sizeof(Npp8u));
  cudaMalloc(&d_filter, 9*sizeof(Npp32f));
  cudaMemcpy(d_in, arr, kNumCh*w*h*sizeof(Npp8u),
             cudaMemcpyHostToDevice);
  cudaMemcpy(d_filter, filter, 9*sizeof(Npp32f),
             cudaMemcpyHostToDevice);
  const NppiSize oKernelSize = {3, 3};
  const NppiPoint oAnchor = {1, 1};
  const NppiSize oSrcSize = {w, h};
  const NppiPoint oSrcOffset = {0, 0};
  const NppiSize oSizeROI = {w, h};

  nppiFilterBorder32f_8u_C3R(d_in, kNumCh*w*sizeof(Npp8u), oSrcSize,
    oSrcOffset, d_out, kNumCh*w*sizeof(Npp8u), oSizeROI, d_filter,
    oKernelSize, oAnchor, NPP_BORDER_REPLICATE);

  cudaMemcpy(arr, d_out, kNumCh*w*h*sizeof(Npp8u),
             cudaMemcpyDeviceToHost);
  cudaFree(d_in);
  cudaFree(d_out);
  cudaFree(d_filter);
}

int main() {
  cimg_library::CImg<unsigned char> image("butterfly.bmp");
  const int w = image.width();
  const int h = image.height();
  Npp8u *arr = (Npp8u*)malloc(kNumCh*w*h*sizeof(Npp8u));

  for (int r = 0; r < h; ++r) {
    for (int c = 0; c < w; ++c) {
      for (int ch = 0; ch < kNumCh; ++ch) {
        arr[kNumCh*(r*w + c) + ch] = image(c, r, ch);
      }
    }
  }

  sharpenNPP(arr, w, h);

  for (int r = 0; r < h; ++r) {
    for (int c = 0; c < w; ++c) {
```

```
57          for (int ch = 0; ch < kNumCh; ++ch) {
58            image(c, r, ch) = arr[kNumCh*(r*w + c) + ch];
59          }
60        }
61      }
62      image.save_bmp("out.bmp");
63      free(arr);
64      return 0;
65  }
```

代码与往常一样以一些 #include 指令作为开始，包括 CUDA 运行时、CImg 库（被反复用于图像输入和输出）以及 NPP 头文件，此外还有两个 #define 指令：我们不会使用 CImg 进行图像显示，因此 cimg_display 被设置为 0，另外 kNumCh 被设置为 3 来指定图像数据中色彩通道的数量。

在第 8～37 行通过使用 sharpenNPP() 来实现整体的模板计算，其中应用了 NPP 库函数。函数以熟悉的方式开始：声明两个 Npp8u（unsigned char）类型的指向设备数组 d_in 和 d_out 的指针，并定义一个 Npp32f（float）类型的指向数组 d_filter 的指针来保存第 11～13 行中定义的浮点型模板系数。第 15～21 行为设备数组分配内存并将输入的图像数据传到设备上。第 22～26 行设置了应用如下模板的 NPP 算法中的参数：

- oKernelSize 将模板的大小设定为 3x3。
- oAnchor 将对齐点设为指数 {1,1}，因此模板中心的 9 可以与当前像素对齐。
- oSrcSize 设定待处理图像的维数。
- oSrcOffset 设定了可以开始应用模板的像素索引。由 {0,0} 开始代表在第一个像素上处理。
- oSizeROI 设定了感兴趣矩形区域（region of interest，ROI）的像素范围。指定的值将 ROI 匹配到整个图像的一个区域。

本程序的主要工作出现在第 28～30 行，执行对 NPP 算法的调用：

```
nppiFilterBorder32f_8u_C3R(d_in, kNumCh*w*sizeof(Npp8u), oSrcSize,
  oSrcOffset, d_out, kNumCh*w*sizeof(Npp8u), oSizeROI, d_filter,
  oKernelSize, oAnchor, NPP_BORDER_REPLICATE);
```

函数名称 nppiFilterBorder32f_8u_C3R() 的含义可按如下方式进行分解：

- nppi 表示一个 NPP 图像处理算法。
- Filter 名副其实：它应用了一个滤波器（也可称作模板）。
- Border 表明这一算法处理模板在图像边缘以外时的方式。最后一个参数 NPP_BORDER_REPLICATE 表明“拐角处重叠区域”的值从最邻近的像素值进行复制。

- 32f 表明模板包含 32 位 float。
- 8u 表明输入数据类型是 8 位 unsigned char 类型。
- C3 表明图像的数据有三个色彩通道。
- R 表明在矩形 ROI 上进行操作。

函数使用结束时将经过滤波的图像数据传回主存并释放为设备数组分配的内存。

main.cpp 的剩余部分几乎与第 5 章中的 sharpen 程序相同。例外的是数据的数组变成了 unsigned chars 类型，Npp8u 是对应 unsigned char 型的 NPP 库类型名。

关于代码的解释介绍到这里，接下来要生成和运行程序，代码清单 8.12 展示了在 Linux 上建立该程序所需要的文档。你需要确认 sharpen_npp 程序再次生成了与 sharpen 程序相同的结果，即图 5.4 中所示的样例图像。

NPP 文档导览

让我们花一些时间来明确 NPP 代码的信息源。你怎样找到一个满足要求的 NPP 算法？你怎样决定传递何种参数？NPP 提供了成千上万种算法，因此找到恰当的算法显得很困难，但这也并不是不可能。所有的 NPP 算法都在一个全面的文档 [10] 中进行了描述，这一大部头电子书（一般有大约 3000 页）最初看来是令人生畏的，所以让我们将它分解为多个易于理解的部分。

NPP 文档的第 1～4 章（“英伟达原语性能”(NVIDIA Performance Primitives)、“通用 API 约定”(General API Conventions)、“信号处理特定 API 约定”(Signal-Processing Specific API Conventions) 以及“图像处理特定 API 约定”(Image-Processing Specific API Conventions)）共有 20 页关于使用 NPP 的很有阅读价值的通用背景方面的材料。

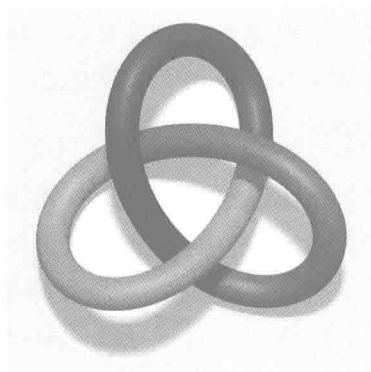
一旦你阅读了那份材料，你可以集中精力于第 5 章“模块索引”(Module Index)，它提供了一个 NPP 模块的完整列表。(图像处理操作包括算术逻辑操作，色彩取样和转换、压缩、标记及分割，数据交换，滤波，几何和线性变换，形态学操作，统计量操作，内存管理，以及阈值 / 比较操作。) 每个模块的函数都与具体的文档相链接。

对于图像锐化的例子来说，我们使用一个二维卷积与自定义的滤波器相结合，因此只需要通过检测几个候选项来证实卷积的实用性并单击相关的链接（版本 7.5 中的第 1221 页）。那里可以找到几页具有各种“特点”的相关函数，但是，在阅读了 NPP 文档前四章的内容后，找到具有正确特点和适当规格的输入和输出数据类型就不再困难了。我们基于上述原因选择 nppiFilterBorder32f_8u_C3R() (也就是一个用于处理 8 位 unsigned char 类型图像，使用带有三种色彩通道、一个边界和一个矩形的 ROI 的 32 位

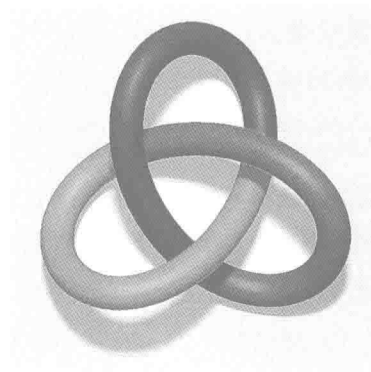
float 型滤波器）。在确定选择一种函数后，单击它的名称链接到参数说明（也在上文中提到过）。注意一些更复杂的参数提供了对于更具体的解释的链接。

8.4.2　更多使用 NPP 进行的图像处理操作

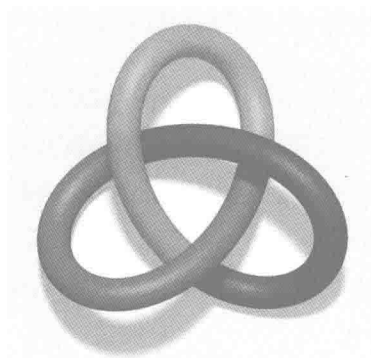
让我们增强程序，使其不仅可以锐化图像，也可以生成一张转换（或交换）了色彩通道的图像，或与另一张图像的色彩进行着色值的叠加，并计算出一种判断结果图像与原图像不同程度的量度。你可以参考 NPP 文档来寻找可以生成像图 8.4 中所示的那些图像的函数。



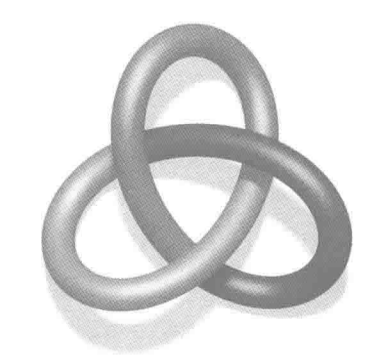
a) 原始的红绿蓝三色



b) 锐化后的版本



c) 将颜色由红绿蓝（RGB）变为蓝绿红（BGR）



d) 对于初始的和色彩变更后的图像进行像素加和

图 8.4　三色的三叶图示的渲染（这些图像的彩色版本位于 www.cudaforengineers.com。）

（改编自 Wikimedia Commons，Jim.belk 的原始图像，网址为 http://commons.wikimedia.org/wiki/File:Tricoloring.png。）

对于特定的操作，我们要明确 NPP 函数的调用：

- **变更色彩通道。**

```
nppiSwapChannels_8u_C3R(d_in, kNumCh*w*sizeof(Npp8u), d_out,
kNumCh*w*sizeof(Npp8u), oSizeROI, aDstOrder);
```

函数名称可以解释为这是一个 NPP 图像处理算法，用于交换 8 位、3 通道的矩形图像的通道。唯一的新参数是最后一个，它指定了通道的排列顺序，在我们的例子中，我们通过以下规格将 RGB 排列为 GBR。

```
const int aDstOrder[3] = {1, 2, 0};
```

（注意 aDstOrder[3] = {0, 1, 2} 会使图像维持原样。）这两行代码（aDstOrder 的定义和 nppiSwapChannels_8u_C3R() 的调用）可以替换代码清单 8.13 中第 28～30 行对 nppiFilterBorder32f_8u_C3R() 的调用，以执行色彩交换而不是锐化操作。

- **像素级叠加。**nppiAdd_8u_C3RSfs() 执行像素的加和。函数要求将输出数据 d_out 与输入图像数据 d_in 相加，并将结果重写入 d_out 中，具体如下：

```
nppiAdd_8u_C3RSfs(d_in, kNumCh*w*sizeof(Npp8u), d_out,
kNumCh*w*sizeof(Npp8u), d_out, kNumCh*w*sizeof(Npp8u), oSizeROI, 1);
```

后缀 Sfs 表明函数包含缩放技术来处理输出范围可能超过 8 位无符号整型范围的情况。因为我们要完成最大值为 255+255 的加和（并且所有更大的值要舍到 255，产生舍入），我们指定一个指数为 1 的缩放，因此所得的和值被 $2^1=2$ 除来确保结果在 [0, 255] 区间上，并且不需要对更大的值进行舍入。注意得到的图像会暗一点，因此我们通过为每个像素值加上一个常数的方法将它变亮，具体如下：

```
const Npp8u aConstants[3] = {32, 32, 32};
nppiAddC_8u_C3IRSfs (aConstants, d_out, kNumCh*w*sizeof(Npp8u),
oSizeROI, 0);
```

- **计算图像距离。**nppiNormDiff_L2_8u_C3R() 计算了图像之间的欧几里得距离（或图像差分的 L^2- 范数）。注意进行差分范数算法需要为中间结果分配内存，因此需要几行额外的代码。代码清单 8.14 中展示了建立和调用 nppiNormDiff_L2_8u_C3R() 的代码。距离的计算在第 7 行和第 8 行中执行并在第 11～13 行中显示到命令窗口。这一代码段可以插入到修改图像时调用的 NPP 库的代码后面，用来计算输入图像和修改后的输出图像之间的距离。

代码清单8.14　使用nppiNormDiff_L2_8u_C3R()计算图像间“距离”的代码段

```
1   Npp64f *aNorm = 0;
2   cudaMalloc(&aNorm, 3 * sizeof(Npp64f));
3   Npp8u *pDeviceBuffer = 0;
4   int bufferSize;
5   nppiNormDiffL2GetBufferHostSize_8u_C3R(oSizeROI, &bufferSize);
6   cudaMalloc(&pDeviceBuffer, bufferSize);
```

```
  nppiNormDiff_L2_8u_C3R(d_in, kNumCh*w*sizeof(Npp8u), d_out,
    kNumCh*w*sizeof(Npp8u), oSizeROI, aNorm, pDeviceBuffer);
  Npp64f res[3];
  cudaMemcpy(res, aNorm, 3*sizeof(Npp64f), cudaMemcpyDeviceToHost);
  for (int i = 0; i < 3; ++i) {
    printf("%f\n", res[i]/(w*h));
  }
```

样例结果如图 8.4 所示。a 为输入图像；b 为锐化后的图像；c 为色彩交换后的图像；d 则是原始图像与色彩交换后的图像的叠加结果图。虽然我们可能不容易通过肉眼观察区分出原始图像与锐化后的图像，但是我们可以基于图像距离范数来分辨它们。注意图像距离是以分量形式计算的；也就是说，将红色、绿色和蓝色通道的值分开。如果你想得到一个单独的距离标量值，可以先利用代码清单 8.15 中的代码将其转换为灰度。

下面是对于计算与输入图像距离所得结果的汇总：

- **输入图像**。三个色彩通道得出差分为 0.0 的范数（所有对象和它本身的距离均应为 0，所以结论显然正确）。
- **锐化后的图像**。红、绿、蓝三个色彩通道的差分范数分别为 0.01122、0.01131 和 0.01129。
- **色彩交换后的图像**。红、绿、蓝三个色彩通道的差分范数分别为 0.08716、0.08813 和 0.08521，转化为灰度之后的距离范数为 0.07697。正如所料，灰度距离的测量对色彩交换的敏感度比特定颜色的测量低。
- **原始图像与色彩交换后的图像的和**。红、绿、蓝三色的差分范数分别为 0.05028、0.04999 和 0.04877，转换为灰度值之后的距离为 0.04375。注意得到的值呈现为原图像和色彩交换后图像的平均值，并且得到的图像距离值远小于进行色彩交换后的图像，但仍然明显大于锐化后的图像。

代码清单8.15　转化为灰度值后计算距离

```
// Allocate memory for a double.
Npp64f *pNormDiff = NULL;
cudaMalloc(&pNormDiff, sizeof(Npp64f));

// Allocate memory for scratch buffer.
Npp8u *pDeviceBufferGray = NULL;
int bufferSizeGray;
nppiNormDiffL2GetBufferHostSize_8u_C1R(oSizeROI, &bufferSizeGray);
cudaMalloc(&pDeviceBufferGray, bufferSizeGray);

// Convert to grayscale.
nppiRGBToGray_8u_C3C1R(d_in, kNumCh*w*sizeof(Npp8u), d_temp_gray1,
  w*sizeof(Npp8u), oSizeROI);
nppiRGBToGray_8u_C3C1R(d_temp, kNumCh*w*sizeof(Npp8u), d_temp_gray2,
  w*sizeof(Npp8u), oSizeROI);
```

```
16
17 // Grayscale norm diff.
18 nppiNormDiff_L2_8u_C1R(d_in, kNumCh*w*sizeof(Npp8u), d_temp,
19   kNumCh*w*sizeof(Npp8u), oSizeROI, pNormDiff, pDeviceBufferGray);
20
21 // Get and print result.
22 Npp64f res = 0;
23 cudaMemcpy(&res, pNormDiff, sizeof(Npp64f), cudaMemcpyDeviceToHost);
24 printf("%f\n", res/(w*h));
```

虽说阅读这个大部头文档需要浏览技巧，NPP 提供了大量用于图像和信号处理的算法，它们由专家级程序员开发、测试和维护并供你使用。

8.5 线性代数中的 cuSOLVER 和 cuBLAS 实践

现在我们来学习另一个在代码库发展和库使用方面都有重要意义的领域：线性方程组求解。线性代数库的研究先于 GPU 计算，其中最有意义的、被广泛使用的基于 CPU 的线性代数库包括基本线性代数库（Basic Linear Algebra Subprograms，BLAS）和更高级别的线性代数子程序包（Linear Algebra Package，LAPACK）[11]。cuBLAS 是英伟达对 BLAS 库 [12] 的 GPU 加速实现。在版本 7.0 的 CUDA 工具包中介绍的 cuSOLVER 是类似于 LAPACK[13] 的 GPU 加速版本。cuSOLVER 包含三个独立的库：

- cuSolverDN 提供因式分解以及稠密矩阵的解决方法。
- cuSolverSP 基于稀疏矩阵的 QR 分解解决稀疏矩阵问题和最小二乘法问题。
- cuSolverRF 加速实现了共享同一稀疏模式的多元矩阵再分解方法。

虽然 cuBLAS 和 cuSOLVER 提供了大量的函数，但这里我们将讨论解决其中的三个并将它们进行组合来实现一个实用的功能：解决多元线性回归或最小二乘问题。

我们将直接跳转到旨在计算线性模型的具体例子，该模型如下：发电厂每小时的产能效率与四个输入变量有关分别为：温度（T）、排气真空度（V）、环境压力（P）和相对湿度（H）。其运算模型的输出可被描述为输入的线性函数：

$$c_0 + c_T * T_i + c_V * V_i + c_H * H_i + c_P * P_i = b_i$$

如果我们有五组测量值来决定这五个未知的系数 c_0, c_T, c_V, c_H 和 c_P，之后这就变成了一个很好解决的 5×5 矩阵问题，可进行手动求解。不管怎样，多亏了加州大学欧文分校（UCI）机器学习资源库 [14, 15] 对于联合循环发电场相关数据的贡献，它使得我们有超过 9500 组可用的测量值（注意，为便于说明，我们对变量的名称稍做改动）。因此我们要处理一个庞大的严重超定（overdetermined）的问题（即方程数大于变量数）而不是小型的

唯一确定的问题（五个方程式包括五个变量）。我们将这个问题在完整的数据集上的情况，留给你来探索，本书构建一个基于前九组测量值的例子。（进行更大量的计算并不成问题，但是输入全部的数据组并构造矩阵涉及读入一个 .csv 文件，这超过了本书的范畴。）

每一个测量值都对应了一个方程，它们的测量变量分别与 5 个未知的系数相乘并最终加和得到测量的结果值 b。之后我们可以将方程组表示成一个单独的矩阵方程式 $Ac=b$，其中 A 为 9×5 的矩阵，它的列元素值由每个变量的测量值组成（例如，A 的第一列为测出的温度值的向量），b 是测出的产能效率的向量（或 9×1 的矩阵），最终我们要解得一个系数向量（或 5×1 矩阵）$c=[c_0, c_T, c_V, c_H, c_P]^T$，这样的代数系统不容易直接求解，但可以使用 QR 分解方法 [16, 17] 将其转化为易处理的形式。基本思想是任何矩阵 A 都可以依据以下特性，由矩阵 Q 和 R 来表示。

- $QR=A$
- Q 是标准正交的矩阵（它的逆矩阵与转置矩阵相等），所以 $Q^TQ=I$，其中 I 为单位矩阵。
- R 是上三角矩阵。

应用分解 $A=QR$ 并在矩阵方程两边同时乘 Q^T，我们得到 $Q^TQRc=Q^Tb$。左边可进行化简（因为 $Q^TQ=$I，$IR=R$），最终得到 $Rc=Q^Tb$。此方程容易处理，因为 R 是上三角矩阵。所以最后一个方程只涉及 c 的最后一项，因此我们可以直接处理这一项，之后用代入的方式依次解决方程中的未知项。此问题具体的数据如表 8.1 所示。

表 8.1　联合循环发电厂数据集里的样例数据

T	V	P	H	B
8.34	40.77	1010.84	90.01	480.48
23.64	58.49	1011.40	74.20	445.75
29.74	56.90	1007.15	41.91	438.76
19.07	49.69	1007.22	76.79	453.09
11.80	40.66	1017.13	97.20	464.43
13.97	39.16	1016.05	84.60	470.96
22.10	71.29	1008.20	75.38	442.35
14.47	41.76	1021.98	78.41	464.00
31.25	69.51	1010.25	36.83	428.77
6.77	38.18	1017.80	81.13	484.31

问题描述完成之后，我们可以按以下三个主要的步骤来研究通过库来实现该问题：

1. 稠密矩阵的QR分解由函数cusolverDnSgeqrf()给出。函数名结合了稠密矩阵库的名称（cusolverDn）、浮点数据类型的指示符（S）以及进行QR分解的简短指令（geqrf）。

2. 变形后等式的右侧Q^Tb由cusolverDnSormqr()计算，该名称结合了稠密矩阵库、数据类型指示符，以及“使用QR分解通过乘以Q重写矩阵（overwrite matrix by multiplying with Q from QR decomposition）”这一方法名称的缩写（ormqr）。

3. 反过来通过解$Rc=Q^Tb$求c的值用到函数cublasStrsm()。这里的名称包含了相关的库（cublas）、数据类型指示符（S表示浮点型）以及“解三角矩阵”（triangular solve matrix）方法名称的缩写（trsm）。

代码清单8.16中展示了基于库实现多元线性回归例子的代码。

代码清单8.16 linreg/main.cpp，使用cuSOLVER库和cuBLAS库解决多元线性回归问题

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <cusolverDn.h>
#include <cublas_v2.h>

#define MIN(X, Y)  ((X) < (Y) ? (X) : (Y))

int main() {
  // Create A (m by n) and b (m by 1) on host and device.
  const int m = 9, n = 5;
  const int lda = m, ldb = m;

  float A[m*n] = {
    1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
    8.34, 23.64, 29.74, 19.07, 11.8, 13.97, 22.1, 14.47, 31.25,
    40.77, 58.49,56.9,49.69,40.66,39.16,71.29,41.76, 69.51,
    1010.84, 1011.4, 1007.15, 1007.22, 1017.13, 1016.05, 1008.2,
    1021.98, 1010.25,
    90.01,74.2,41.91,76.79,97.2,84.6,75.38,78.41, 36.83};
  float b[m] = {
    480.48, 445.75,438.76,453.09,464.43, 470.96,442.35,464, 428.77};

  float *d_A = 0, *d_b = 0;
  cudaMalloc(&d_A, m*n*sizeof(float));
  cudaMemcpy(d_A, A, m*n*sizeof(float), cudaMemcpyHostToDevice);
  cudaMalloc(&d_b, m*sizeof(float));
  cudaMemcpy(d_b, b, m*sizeof(float), cudaMemcpyHostToDevice);

  // Initialize the CUSOLVER and CUBLAS context.
  cusolverDnHandle_t cusolverDnH = 0;
  cublasHandle_t cublasH = 0;
  cusolverDnCreate(&cusolverDnH);
  cublasCreate(&cublasH);

  // Initialize solver parameters.
  float *tau = 0, *work = 0;
  int *devInfo = 0, Lwork = 0;
```

```
38   cudaMalloc(&tau, MIN(m,n)*sizeof(float));
39   cudaMalloc(&devInfo, sizeof(int));
40   const float alpha = 1;
41
42   // Calculate the size of work buffer needed.
43   cusolverDnSgeqrf_bufferSize(cusolverDnH, m, n, d_A, lda, &Lwork);
44   cudaMalloc(&work, Lwork*sizeof(float));
45
46   // A = QR with CUSOLVER
47   cusolverDnSgeqrf(cusolverDnH, m, n, d_A, lda, tau, work, Lwork,
48                    devInfo);
49   cudaDeviceSynchronize();
50
51   // z = (Q^T)b with CUSOLVER, z is m x 1
52   cusolverDnSormqr(cusolverDnH, CUBLAS_SIDE_LEFT, CUBLAS_OP_T, m, 1,
53                    MIN(m, n), d_A, lda, tau, d_b, ldb, work, Lwork,
54                    devInfo);
55   cudaDeviceSynchronize();
56
57   // Solve Rx = z for x with CUBLAS, x is n x 1.
58   cublasStrsm(cublasH, CUBLAS_SIDE_LEFT, CUBLAS_FILL_MODE_UPPER,
59               CUBLAS_OP_N, CUBLAS_DIAG_NON_UNIT, n, 1, &alpha, d_A,
60               lda, d_b, ldb);
61   // Copy the result and print.
62   float x[n] = {0.0};
63   cudaMemcpy(x, d_b, n*sizeof(float), cudaMemcpyDeviceToHost);
64   for (int i = 0; i < n; ++i) printf("x[%d] = %f\n", i, x[i]);
65
66   cublasDestroy(cublasH);
67   cusolverDnDestroy(cusolverDnH);
68   cudaFree(d_A);
69   cudaFree(d_b);
70   cudaFree(tau);
71   cudaFree(devInfo);
72   cudaFree(work);
73   return 0;
74 }
```

代码以包含必要的库的指令作为开始，并定义了一个有用的宏 MIN。main() 函数以相关的容量的声明开始，包括在增广矩阵中的行数 m 和列数 n（即，A 矩阵的第 1 列初始化为 1，然后与 c_0 相乘）以及等式两边的矩阵维数（本例中均为 m）。第 13～21 行采用了对输入数据的硬编码，你可能想将它们替换为可以从一个文件中读取的数据以灵活处理更大的问题。第 23～27 行分配设备内存并传送 A 与 b 中的数据。第 29～44 行做些准备工作，包括在第 43 行中调用一个库函数来决定计算所需的工作缓存的容量大小。之后的第 47、52 和 58 行中调用了三个之前讨论过的主要的库函数以计算 QR 分解，并构造变形后的等式的左侧部分，再反过来求解未知系数向量，在第 63 行和第 64 行中将它们传回主存并显示最终的结果。剩余的代码通过销毁之前创建的上下文以及释放分配的内存，来完成清理工作。

在 Linux 系统上运行此代码的文档如代码清单 8.17 所示。在 Windows 下的 Visual

Studio 中，一定要向附加链接输入依赖项中添加 cublas.lib 和 cusolver.lib，可参考在本章中稍早时介绍的“链接库文件”，见图 8.3。

代码清单8.17 linreg/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall
LIBS += -lcublas_static -lcusolver_static -lculibos

all: main.exe

main.exe: main.cpp
  $(NVCC) $(NVCC_FLAGS) $< -o $@ $(LIBS)
```

基于数据集里前九个示例数据运行代码，得到如下系数向量：

$$c=[834.18, -2.149, -0.378, -0.300, -0.209]^T$$

线性回归关系为：

$$b=843.18-2.149*T-0.378*V-0.300*P-0.209*H$$

并且我们可以通过将数据集里其他样例数据输入程序来进行测试（这就是为什么我们要额外留出不参与计算的样例数据）。将表 8.1 中的最后一行数据值代入回归公式得到 $b=483.21$，与表中的实际值相差 0.2%。

最后是关于错误处理的注解：附录 D 中给出的错误处理的方法对于使用 cuBLAS 和 cuSOLVER API 是完全有效的，因此 helper_cuda.h 中的函数 checkCudaErrors() 可以用来处理 cuBLAS 和 cuSOLVER 错误。尽管这里为了清晰起见省略了错误处理过程，我们还是建议你在开发 cuBLAS 和 cuSOLVER 应用时，将所有的 CUDA 调用，包括 cuBLAS 和 cuSOLVER 封装起来，放入 checkCudaErrors() 下进行错误处理。

8.6 cuDNN 库

机器学习在 GPU 计算中已经成为最热门的主题之一，所以你需要了解与此相关的英伟达 CUDA 深度神经网络库 cuDNN，以及深度学习训练系统 NVIDIA DIGITS 。cuDNN 可以用于广泛使用的深度学习框架的执行，包括 Caffe、Theano 以及 Torch。具体的内容，请参考 cuDNN 网站 https://developer.nvidia.com/cudnn 和 DIGITS 网站 https://developer.nvidia.com/digits。

8.7　ArrayFire 库

最后，我们简单介绍一下 ArrayFire 库，它的定义如下 [18]：

ArrayFire 是一个提供易于使用 API 的用于并行计算的高性能软件库。其基于数组的函数集使并行编程更容易理解。

对于那些拥有 Matlab 背景的读者，ArrayFire 可能会更具诱惑，因为相较于 C 语言，它的编程风格与 Matlab 的更接近。ArrayFire 也是开源的和跨平台的，因此它可以利用基于 GPU 的 SIMT 并行的优点，但同时也可以在其他的硬件上执行（对于执行性能的预期也要做适当调整），基于 ArrayFire 构建的应用可以非常简明，如果你感兴趣，应该去检验它们的实例以得到对于其特性的更具体的认识。

8.8　本章小结

我们已经示范了一些可供你使用的 CUDA 库。我们希望这些样例代码可以为你发掘更多的库抛砖引玉，它们会使你的编码更加简洁、高效。

8.9　推荐项目

1. 在估算 π 值的 Thrust 版本例子中，产生区间 [0, 1] 上的随机坐标的方法为：生成随机整数的向量，之后除以随机整数生成器产生的最大值 RAND_MAX，这两个运算均在主机上执行。这一特定的随机数生成器仅仅是主机函数，因此计算需要在主机端完成。然而，这对除法运算而言，没有利用设备端的并行性，并不明智。修改代码使随机数在主机上生成，但在 GPU 中并行地进行除法运算，可以使用 thrust::transform 和一个设备向量。
2. 借助于代码清单 8.10 中展示的 cuRAND 版本的 π 估计，考虑以下问题。
   a. 如果你多次运行程序，你会始终得到相同的结果吗？这对于蒙特卡罗算法来说是好事还是坏事？
   b. 进行试验改变第 11 行中的种子值，这是否生成了新的结果？如果保持新的种子值不变，再次运行程序是否再次得到相同的结果？
   c. 探索 cuRAND 来学习状态和偏移。修改代码使其每次运行带有不同随机数集并（正确地）产生不同的结果。

3. 使用 Thrust 库，修改 dist1D_fused 来创建 dist2D_fused，使其成为可以与第 4 章中的 dist2D 函数具有相同功能的程序。建议使用一个计数迭代器来提供一个虚拟的线性索引和一个可计算 row 和 col 索引的函数，将它们转化为 x、y 坐标，并计算其与参考点的距离。
4. 使用 NPP 库来定量地验证 sharpen 和 sharpenNPP 对于给出的输入图像得到了相同的结果。
5. 创建一个交互式图像处理程序来展示 NPP 库提供的一些能力。（基于 flashlight 应用的 OpenGL 交互操作特性或基于 CImg 的图像显示特性或其他可供你选择的库。）
6. 使用代码清单 8.9 中阐述的 thrust::tuple 和 thrust::zip_iterator 实现对 π 的估计。
7. 合并一个实现读入标准文件输入格式的代码，例如用逗号隔开的值（comma-separated-value，CSV），并尝试把线性回归程序应用到更大的数据集。

参考文献

[1] Michael D. McCool, Arch D. Robison, and James Reinders, *Structured Parallel Programming: Patterns for Efficient Computation* (New York, NY: Elsevier, 2012).

[2] NVIDIA Corporation, "Thrust," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/thrust.

[3] Jared Hoberock and Nathan Bell, Thrust Project website, 2015, http://thrust.github.io.

[4] "C++ Reference," 2015, http://en.cppreference.com.

[5] Mark Harris, "Expressive Algorithmic Programming with Thrust," *Parallel Forall* (blog), 2012, http://devblogs.nvidia.com/parallelforall/expressive-algorithmic-programming-thrust/.

[6] OpenMP Architecture Review Board, OpenMP, 2015, http://openmp.org.

[7] Intel, "Threading Building Blocks." (2015) https://www.threadingbuildingblocks.org.

[8] Jared Hoberock and Nathan Bell, Thrust wiki page, 2015, https://github.com/thrust/thrust/wiki.

[9] NVIDIA Corporation, "Thrust and cuRAND Example," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/curand/device-api-overview.html#thrust-and-curand-example.

[10] NVIDIA Corporation. *NVIDIA Performance Primitives (NPP) Version 7.0* [NPP Library Documentation], 2015, http://docs.nvidia.com/cuda/pdf/NPP_Library.pdf.

[11] Edward Anderson et al., *LAPACK Users' Guide* (Philadelphia, PA: SIAM, 1999).

[12] NVIDIA Corporation, "cuBLAS," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cublas.

[13] NVIDIA Corporation, "cuSOLVER" NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cusolver.

[14] M. Lichman, "Welcome to the UC Irvine Machine Learning Repository!" UCI Machine Learning Repository, 2013, http://archive.ics.uci.edu/ml.

[15] Pinar Tufekci, "Prediction of Full Load Electrical Power Output of a Base Load Operated Combined Cycle Power Plant Using Machine Learning Methods," *International Journal of Electrical Power & Energy Systems* 60 (2014): 126–40. http://dx.doi.org/10.1016/j.ijepes.2014.02.027.

[16] Gene H. Golub and Charles F. Van Loan, *Matrix Computations, Fourth Edition* (Baltimore, MD: Johns Hopkins University Press, 2012).

[17] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery, *Numerical Recipes: The Art of Scientific Computing* (New York, NY: Cambridge University Press, 1992).

[18] P. Yalamanchili et al., ArrayFire, 2015, http://www.arrayfire.com/docs/index.htm.

Chapter 9 第 9 章

探索 CUDA 生态系统

与 CUDA 有关的资源合集（包括书、网站、博客、软件、文档）经常被称为“CUDA 生态系统”。作为本书的最后一章，提供一系列材料的索引，它们能帮助你探索 CUDA 生态系统。这些材料的简短描述可以帮助你找到 CUDA 某一方面或某种应用的资料。

9.1 主要资源的权威列表

让我们从一个简短的资源列表开始，列表包含了你应该了解的内容。

9.1.1 CUDA 空间

可以利用的基本资源是英伟达的 CUDA 空间网站：https://developer.nvidia.com/cuda-zone，你可能在下载 CUDA 工具包或者注册为 CUDA 开发者的时候已经访问过该网站了。现在该页面提供了以下六个类别的链接：

- 关于 CUDA（About CUDA）对 CUDA 在各行各业的应用实例提供了概览和链接，这些领域包括生物信息学、计算流体和结构力学、设计自动化、影像等。
- 入门（Getting Started）提供了编程语言（我们现在使用 C/C++，但是 CUDA 也可以用 Fortran、Python、Matlab、Mathematica 等语言来访问）、编译指令（可以让编译器根据你已有的代码进行并行计算）、优化库（你可以使用由他人写好、测

试并维护的代码）的链接。这些链接也有助于你识别兼容 CUDA 的硬件以及引导你注册 CUDA 并行编程的在线课程。

- **工具和生态系统**（Tools & Ecosystem）包含了一些应用、库、分析工具以及管理一个 GPU 集群的工具。
- **学术合作**（Academic Collaboration）提供了一些可用的教育和培训材料。
- **CUDA 下载**（CUDA Downloads）在这里你可以下载最新版本 CUDA、入门指南和发行说明。
- **资源**（Resources）提供了一大堆按类别分类的链接。有些应该被单独列出来：
  - **文档和引用**（Docs and References）链接到了 CUDA 在线文档，包括发行说明、入门指南和编程指南。CUDA 工具箱的文档值得你花一些时间去研究。你应该定期访问这里。
  - **教育和培训**（Education & Training）给出了 GTC 展示和 GTC 研讨会的链接。GTC 全称是 GPU 技术大会（GPU Tech Conference），是一个每年春天在加利福尼亚的圣何塞举行的 GPU 专家和爱好者的集会。每年都会呈现几百场关于 CUDA 的讲座，大多数都会被录像并且可以通过网络观看。GTC 研讨会是 GTC 的在线扩展，持续一整年。它提供附加话题的讲座。这里有许多非常好的 GTC 讲座，你应该选择你感兴趣的领域进行关注。

9.1.2　其他的主要网络资源

CUDA 开发的确是一项需要靠网络提供帮助的活动。当你碰到隐秘的错误时，把错误信息复制下来并用你最喜爱的搜索引擎搜索是最有用的获得帮助的方法之一。当然，这里也有许多其他的网络资源你应该了解，下面列举了其中的部分资源：

- Parallel Forall（http://devblogs.nvidia.com/parallelforall/，被喜欢数学符号的人缩写为“||∀”）。该博客由专家（如 Mark Harris，英伟达 GPU 计算软件的首席科学家）定期发布有用的和富有启发性的内容。
- 当你在网上搜索某个话题或者错误信息的时候，如果你找到了一个近期的 StackOverflow 的链接，那么打开它看一看。StackOverflow 是这么描述自己的：“StackOverflow 是一个针对专业程序员和编程爱好者的问答网站，它完全免费，不需要注册。”它满足我们目标的原因是这里有许多知识渊博的专家会回答贴出的问题。在 StackOverflow 的搜索框里输入 CUDA 会返回有关 CUDA 的结果：

https://stackoverflow.com/questions/tagged/cuda。

- **维基百科上 CUDA 页面**，这是一个便利的地方，可以查看某些 GPU 计算能力的细节，也可以查看基于计算能力的技术特性和技术说明。

9.1.3 在线课程

这里有三个可以学习的在线课程，它们包含了有关 CUDA 的重要内容：

- Udacity CS344: Intro to Parallel Programming（**并行编程入门**）使用 CUDA 来介绍并行计算的基本概念。该课程的描述如下："该课程专为学习 GPU 编程、算法和优化技术的开发人员、科学家、工程师、研究者和学生量身打造。"讲授者是加州大学戴维斯分校的 John Owens 教授和英伟达研究中心的资深总监 David Luebke 博士。学生可以按照自己的时间安排来免费学习该课程。在简明和精心设计的课程中会穿插练习题和编程作业。由于这是一个计算机科学课程，所以假定你有 C/C++ 的编程经验。即使你没有期望的背景，课程的前部分讲解也值得你花费时间（包括用校车和跑车来类比 CPU 和 GPU 计算这样的形象比喻）。
- Heterogeneous Parallel Programming（**异构并行编程**）是基于伊利诺伊大学的一个课程，该课程的目标是完成一个包含 CPU 和 GPU 的并行系统。课程开始部分是基于 CUDA 的，后面部分使用其他的并行平台（OpenCL、OpenACC 和 C++AMP）。讲座是由胡文美（Wen-mei Hwu）教授讲授（他的名字会在 9.1.4 节中再次出现），内容已经较 2012 年的初始版本有所更新。近年来该课程持续 9 周（1 月至 4 月）。现可以免费从 Coursera 访问该课程。
- Programming Massively Parallel Processors with CUDA（**CUDA 大规模并行处理器程序设计**）是一系列斯坦福 2010 年春季课程的讲座。讲座由 Jared Hoberock 和 Nathan Bell 博士（他们都参与了 Thrust 的创造）以及一些特邀嘉宾讲授。该课程假定你有丰富的编程背景。此外有些材料有一点陈旧，但是课程开始部分的讲解以及随后讨论的应用都很有价值。视频可以免费从 iTunes 上下载。

9.1.4 CUDA 书籍

你应该了解一些有关 CUDA 的书籍。下面简短描述每一本书：

1.《CUDA by Example: An Introduction to General-Purpose GPU Programming》，作者：Jason Sanders、Edward Kandrot，由 Addison-Wesley 于 2011 年出版，ISBN-13：

9780131387683。

这本书使许多人，包括本书作者 Duane Storti 进入 CUDA 的世界。这是一本引人入胜且编排紧凑的书，值得每一个学习 CUDA 的人阅读。这种基于实例的方法很有效，能使读者积极地参与其中。当然，注意这本书的目标读者是有经验的 C 开发者，而非一般从事技术的读者。该书的很大一部分内容都涉及处理多个计算流，这些内容与你想使用多个 GPU 或者通过一个流传输数据而另一个流进行计算的方式来隐藏延迟的时候有关。（我们认为多流编程是一个高级主题，CUDA 初学者不需要了解它，因此直到现在我们也没有涉及这个主题。）此外需要注意的是，这本书未更新的时间达到了从 CUDA 发布到现在时长的 3/4。当然它的陈旧也是可以理解的，考虑到它要维持与 OpenGL 的互通性以及它对公用功能（在 cutil.h 中）的依赖——这些功能分布在 CUDA 的早期样例文件中但已不被官方支持。（在新版本的 CUDA 中，这些公用功能已经被替换成了 helper 文件。）总之，这是一本不错的书。如果该书的第 2 版出版了，我们马上就会买来阅读。

2.《*Programming Massively Parallel Processors*: *A Hands-On Approach*》，作者：David B. Kirk、Wen-mei Hwu，由 Morgan Kaufmann 于 2010 年出版，ISBN-13: 9780123814722。（译者注：原书第 2 版已由机械工业出版社出版。）

这是另一本早期进入 CUDA 市场的书。这本书针对更广泛的技术人员，并且假设读者有一些（但不多）C 编程背景。该书的目标是有效地用 CUDA 武装读者，以使他们能开发自己的 CUDA 项目。它提供了 CUDA 介绍性的背景材料（也包括了 OpenCL 的章节，这是一个在异构系统上进行并行计算的候选平台）和一些实例，包括矩阵乘法、MRI 重构以及分子可视化。

3.《*CUDA Application Design and Development*》，作者：Rob Farber，由 Morgan Kaufmann 于 2011 年出版，ISBN-13：9780123884268。

该书假设读者有较强的编程背景，但是该书是从零开始介绍 CUDA 的。该书内容足够高阶，可以用来对一些应用做深层的处理，包括机器学习和视频流的实时处理。该书的作者在《*Dr. Dobb's Journal*》负责一系列有关 CUDA 编程的文章，并且有能力以迷人和易于理解的方式展现关于 CUDA 的主题。（令人惋惜的是，《*Dr. Dobb's Journal*》已经停止运营了，因此不会再出现新的文章了。）

4.《*CUDA Programming*: *A Developer's Guide to Parallel Computing with GPUs*》，作者：Shane Cook，由 Morgan Kaufmann 于 2012 年出版，ISBN-13：9780124159334。（译者注：该书已由机械工业出版社引进出版。）

这是另一本专业开发 CUDA 应用的书籍。(用作者的话说，“在真实的场景由真正的开发者来使用 CUDA。”) 该书假设读者有 C/C++ 的知识，它足够新，可以用在开普勒硬件上 (例如计算能力为 3.X 的 GPU)。另外该书强调编写高性能的代码。

5.《*CUDA Handbook: A Comprehensive Guide to GPU Programming*》，作者：Nicholas Wilt，由 Addison-Wesley 于 2013 年出版，ISBN-13：9780321809469。(译者注：已由机械工业出版社引进出版。)

书如其名，这本书是一本关于 CUDA 方方面面的综合手册，由 CUDA 的发起人之一编写。它不是为初学者编写的，但是随着你深入重要 CUDA 项目的细节，你会在这本书中发现许多“是什么”“怎么样”以及“为什么”等问题的答案。在 2013 年出版的书籍中，这本书是比较与时俱进的，覆盖了 CUDA 5.0 版本以及包含开普勒架构的硬件。

6.《*Professional CUDA C Programming*》，作者：John Cheng、Max Grossman 和 Ty McKercher，由 Wiley 于 2014 年出版，ISBN-13：9781118739327。

如书名所示，这本书的目标人群是 CUDA 专业开发人员。组织的目标是将 CUDA 的方方面面展示给有经验的程序员，而不是用例子来吸引 CUDA 新手。正如该书作者在序言中所说的，“这是一本由程序员书写、针对程序员，并且专注于程序员开发 CUDA 时所需内容的书。”当你创建出有意义的项目并且你准备优化它们的结构或性能时，这本书值得一看。如果你是能聘请雇员去创建项目的少数幸运儿，你的雇员很可能想要获得这本书。这本书是 2014 年出版的，内容与时俱进。如果你使用的 Linux 系统配有 CUDA 6.0 或者更高版本、开普勒或更高版本的硬件架构 (也就是说计算能力≥3.0)，可以轻松运行书中的样例代码。

7.《*GPU Computing Gems: Emerald Edition and Jade Edition*》，由 Wen-mei Hwu 编著，由 Morgan Kaufmann 于 2011 年出版，ISBN-13：9780123849885 (Emerald 版本)，9780123859631 (Jade 版本)。

这两卷书是众所周知的 GPU 精粹系列，但是特别专注于使用 GPU 来进行计算。两卷书提供了相关领域顶级开发者编写的大量 GPU 计算的实例 (大约 1400 页)。当你需要代码完成某个具体目标时，如果你能在 GPU 精粹中找到它，那么你就可以直接使用这些高效的、调试好的、可依赖的代码。

8.《*CUDA Fortran for Scientists and Engineers: Best Practices for Efficient CUDA Fortran Programming*》，作者：Gregory Ruetsch、Massimiliano Fatica，由 Morgan Kaufmann 于 2013 年出版，ISBN-13：9780124169708。

如果你是 Fortran 用户（或者有想并行化的 Fortran 代码），这本书可能会适合你。

9.2　更多资源

先前列举的资源在我们写书的时候已经存在并且得到了广泛应用。然而，CUDA 生态系统正在加速发展，相关的资源列表将持续增长。这里列举了一些你可能想尝试的资源。

9.2.1　CUDA 样例

下载的 CUDA 中包含一系列示范代码叫作 CUDA 样例（在一些早期版本叫作 CUDA SDK 样例），如果不提它们真是一种疏忽。当你需要去看某些 CUDA 函数或特征的实现时，这些样例就有用了。当你想开始一个新项目的时候，如果你能找到相关的样例代码，这些代码就能给你提供一些建议（和 / 或问题），它们有助于你在正确的方向上建立你的项目。采用专门的库或者特性的样例也能给你提供一些信息，这些信息可能涉及一些定制的生成参数，例如包含额外的头文件、链接到额外的库文件等。

9.2.2　CUDA 语言和库

在 CUDA 空间中，选择工具和生态系统，然后就能看到不断增长的各种各样兼容 CUDA 的语言和 API（应用程序编程接口），以及一堆库（超出在第 8 章中已经介绍的 Thrust、NPP、cuRAND、cuSOLVER、cuBLAS 和 ArrayFire 等库之外）。如果想看最新的条目，可以访问 https://developer.nvidia.com/gpu-accelerated-libraries。如果要开始使用新的语言或者库，可能会有一个复杂的学习曲线，但是你可以通过使用那些经过彻底测试、优化和维护的代码来很好地弥补时间和精力上的不足。

9.2.3　更多的 CUDA 书籍

CUDA 的加速发展已经被不断涌现的相关书籍所证明。除了之前列举的权威书单外，这里也列举了一些跟领域相关的研究型书籍。

1.《Generation of Radiographs from 3D Anatomical Models Using the GPU: Parallel DRR Algorithms Using CUDA GPUs》，作者：Andre Cardoso，由 Lambert Academic 于 2012 年出版，ISBN-13：9783838378756。

2.《CPU-Based Application Transformation to CUDA: Transformation of CPU-Based

Applications to Leverage on Graphics Processors Using CUDA》，作者：Anas Mohd Nazlee、Fawnizu Azmadi Hussin，由 Lambert Academic 于 2012 年出版，ISBN-13：9783659171215。

3.《Accelerating MATLAB with GPU Computing: A Primer with Examples》，作者：Jung W. Suh、Youngmin Kim，由 Morgan Kaufmann 于 2013 年出版，ISBN-13：9780124080805。

4.《Designing Scientific Applications on GPUs》，作者：Raphael Couturier，由 Chapman and Hall/CRC 于 2013 年出版，ISBN-13：9781466571624。

5.《GPU Power for Medical Imaging: A Practical Approach to General-Purpose Computing with CUDA Machine Learning》，作者：Francisco Xavier，由 Lambert Academic 于 2014 年出版，ISBN-13：9783659251894。

6.《Parallel Computing for Data Science: With Examples in R, C++ and CUDA》，作者：Norman Matloff，由 Chapman and Hall/CRC 于 2015 年出版，ISBN-13：9781466587014。

7.《Multicore and GPU Programming: An Integrated Approach》，作者：Gerassimos Barlas，由 Morgan Kaufmann 于 2015 年出版，ISBN-13：9780124171374。

正如 CUDA 的其他方面一样，相关书籍的列表也在迅速改变。随时关注“通往”（go to）权威资源，以了解新的发展动态。

9.3 本章小结

我们现在已经来到本书的结尾了。祝贺你坚持到这里，并且我们也衷心希望你的收获能配得上你投入的时间与精力。在你学习本书的过程中，如果有哪一个样例促使你想到了有趣的项目，那么我们的目的便达到了，同时也预示你步入了正轨：使用 CUDA 支持的大规模并行计算的能力去完成新颖和令人惊喜的事情。

在许多人都谈论着 3D 打印（另一种发生在我们身边的尖端技术）如何代表了制造业的大众化的主要力量时，你正在亲身参与并行计算的大众化。我们希望你能好好利用这些新的资源以便增长你的兴趣，甚至可能增长人类的知识。再次感谢你与我们一同分享 CUDA 之旅，当你继续在那激动人心的 CUDA 世界中旅行时，祝你好运。

9.4 推荐项目

此时，你应该很有资格去探索一些更具挑战的 CUDA 项目了，兹列于下：

1. 确定一个 CUDA 的库、语言，或 API（它有看上去有用的特征或函数），并且将它们用

在你个人感兴趣的问题上。

2. 确定一个有趣的 CUDA 样例，弄明白它要做什么以及它是如何达到目标的。写下你的描述并且将它与你身边了解 CUDA 的朋友分享。
3. 观看一个有趣的 GTC 讲座。（如果你没有明确的个人兴趣，就从一个主题讲座开始。主题讲座都包含了重要而有趣的高质量内容。）写下一个快速简介，并将它分享给对 CUDA 感兴趣的朋友。
4. 访问 http://stackoverflow.com，并且在搜索框中输入“ CUDA”，单击票数标签，并且阅读排名前 10 的问题和回答。
5. 审视在线的 CUDA 课程中的部分内容。
6. 查找另一本 CUDA 书籍，看能否找到一些与你的目标有关的内容。
7. 找一本新发行的 CUDA 书籍。审视书的内容和呈现风格。写一篇评论和你的朋友分享。
8. 学习最新、最棒地利用 CUDA 大规模并行计算能力的应用。

Appendix A

附录 A

硬件设置

如果要让一台计算机支持 CUDA 并行计算平台，必须保证其硬件和软件均支持 CUDA。硬件的支持主要为计算机是否有支持 CUDA 的 GPU 来决定。本章将介绍如何查看计算机是否有支持 CUDA 的 GPU，不同的操作系统查看方式不同，本章分别对 Windows、OS X 以及 Linux 操作系统的查看方式进行了介绍，读者可根据自己的操作系统有针对性地阅读（如果你的计算机生产商已经将 NVIDIA 显卡类型的绿色小标签贴到了计算机显眼的位置上，那么你可以直接阅读 A.4 节）。此外，我们还讨论了如何获取以及如何安装支持 CUDA 的 GPU，不同的硬件平台，操作不尽相同。再次强调，根据自己的实际情况选择性地阅读本章。

A.1 在 Windows 下查看 NVIDIA GPU 型号

在 Windows 桌面单击鼠标右键，若弹出菜单中没有 NIVIDA 控制面板（NVIDIA Control Panel ）选项，则请阅读 A.5 节；如果有该选项，则单击打开 NIVIDA 控制面板，单击“Home”图标，将会出现如图 A.1 所示的界面，界面中显示了一张写着 NVIDIA 控制面板的图片，图片最下方写着该机器的 GPU 型号（此处为 GeForce 840M）。确定 GPU 型号之后则可直接阅读 A.4 节。

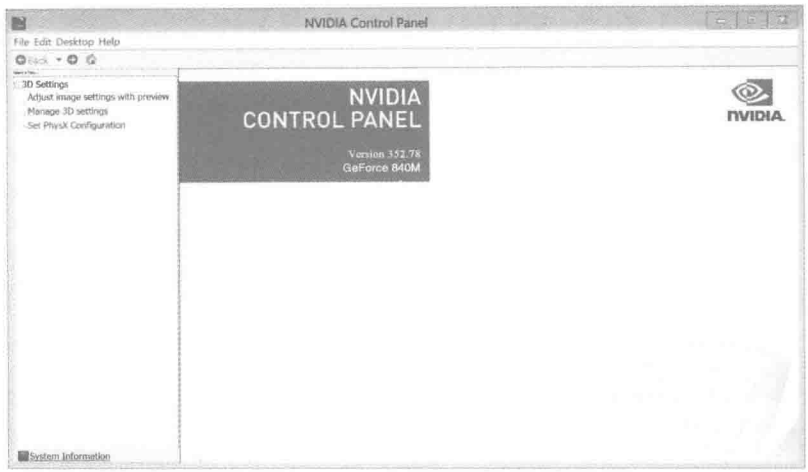


图 A.1 NVIDIA 控制面板 Home 页所显示的 NVIDIA GPU 信息

A.2 在 OS X 下查看 NVIDIA GPU 型号

单击苹果菜单，选择关于本机（About This Mac）选项，单击显示器（Displays）选项卡可以看到本机的显示器信息和 GPU 型号，如图 A.2 所示，该机器的 GPU 型号为 GeForce GT 650M。确定 GPU 型号之后则可直接阅读 A.4 节。若显示结果中没有 NVIDIA GPU，则请阅读 A.5 节。

图 A.2 关于本机中显示器选项卡所显示的 NVIDIA GPU 信息

A.3 在 Linux 下查看 NVIDIA GPU 型号

从命令行中输入以下命令（在 Ubuntu 中可以通过 Ctrl+Alt+T 组合键打开命令行）：

```
lspci | grep -i nvidia <Enter>
```

这条命令会列出该计算机上的所有外围设备（ls 是 list 的缩写，pci 表示显卡之类的外围设备与 CPU 连接的通信总线）。grep 命令是一个模式匹配工具，列出的所有外设结果将输送给 grep 命令，该命令会找出所有包含 nvidia 单词的外设并显示输出，-i 表示 grep 命令搜索时忽略关键字的大小写。我们试着在一台 Linux 机器上输入该命令，命令行返回的结果是：01:00.0 VGA compatible controller: NVIDIA Corporation GF108[GeForce GT 620] (rev a1)，这表示当前机器的显卡为 GeForce GT 620。

如果计算机没有 NVIDIA 显卡，则请阅读 A.5 节。如果发现有 NVIDIA 显卡，查看显卡型号后请阅读 A.4 节。

A.4 确认计算能力

NVIDIA CUDA 的官方网址提供了所有 CUDA 显卡与其计算能力的对应关系表。访问 https://developer.nvidia.com/cuda-gpus 可以查询自己机器显卡的计算能力。

目前，支持 CUDA 的 GPU 主要有 Tesla、Quadro、NVS、GeForce 以及 TEGRA/Jetson 这几个系列。本附录示例中三个操作系统所涉及的 GPU 都是 GeForce 类型的 GPU，因此我们需要单击“CUDA-Enabled GeForce Products”链接，然后会显示两类产品的显卡计算能力结果，分别是 GeForce Desktop Products 与 GeForce Notebook Products，从这两列结果中查找显卡对应的计算能力。本附录 Windows 系统示例中的显卡是一款搭载在笔记本电脑上的 GeForce 840M 显卡，计算能力为 5.0。Mac 系统的显卡为 GeForce GT 650M，计算能力为 3.0。Linux 系统的显卡是 GeForce GT 620，计算能力为 2.1。

读者可根据自己的显卡型号到指定列去查找显卡的计算能力。如果你的机器装有 GPU 且计算能力在 2.0 以上，那么你可以开始阅读附录 B，安装相应的软件。

为什么更新计算能力？

- 如果显卡的计算能力低于 2.0，则需要升级显卡的计算能力，否则无法运行本书中讨论到的代码。
- 如果你需要使用托管内存（Managed Memory）（例如第 3 章中涉及的显存使用），

则显卡的计算能力必须在3.0以上。

- 如果你需要用到动态并行（本书中未曾涉及），则显卡的计算能力必须在3.5以上。更多关于计算能力的细节请查看NVIDIA CUDA Zone的官方文档或维基百科上的CUDA主页：https://en.wikipedia.org/wiki/CUDA。
- 如果用户需要进行大规模的双精度计算，则可以着重考虑使用Tesla卡（该系列显卡的价格较高且功率也较大）。

硬件术语

通常，我们说的GPU一般指的是像GeForce 840M这样指定的显卡型号，而不是GPU芯片。该显卡搭载着一颗Maxwell架构的GM108芯片。因此，当我们谈及CUDA硬件时，主要涉及4个概念：显卡的类型名/编号、GPU芯片的名称/编号、计算能力和架构类型。本书主要关注的是显卡的类型名/编号（因为在我们查看系统硬件或购买新显卡时显卡类型名/编号是最简单、最常用的表达方式）以及计算能力（许多应用程序对计算能力有指定的要求）。然而，读者仍需对GPU的架构有所了解，因为不同架构的GPU处理单元以及显存的大小特性都有所不同。历代NVIDIA CUDA架构术语如下：

- Tesla：最早的CUDA卡（起源于2007年），计算能力在1.0到1.3之间。
- Fermi：第二代CUDA卡（起源于2010年），计算能力在2.0到2.1之间。Fermi架构显卡几乎能运行本书涉及的所有代码。
- Kepler：第三代CUDA卡（起源于2012年），计算能力在3.0到3.7之间。Kepler架构显卡能运行本书涉及的所有代码。
- Maxwell：第四代CUDA卡（起源于2014年），计算能力5.x。
- Pascal：第五代CUDA卡，2016年发布。

A.5 升级计算能力

升级计算能力有一定的前提条件，对于像Mac以及笔记本这类电脑，需要看生产商是否设置了一定的硬件配置，而对于安装了Windows或Linux的台式机，则需要看其是否允许更新。

A.5.1 Mac 或笔记本电脑所配置的 GPU 支持 CUDA 计算

对于 Mac 或笔记本电脑而言，安装新的硬件组件十分困难。若读者使用的是此类电脑，升级计算能力则意味着需要重新购买一台内置支持 CUDA 的 GPU 的电脑。

苹果公司一直使用着不同生产商提供的不同类型的 GPU，因此，如果想在 OS X 下运行 CUDA，首先需要在苹果专卖店购买一台配置有 NVIDIA 显卡的 Mac。

在笔记本电脑市场中，配置有支持 CUDA 的 GPU 的笔记本电脑并不多，但它们的价格普遍十分合理。大多数的笔记本电脑都配置着一块 Intel 集成显卡和一块支持 CUDA 的 NVIDIA 独立显卡（有时候也用 NVIDIA Optimus 表示），集成显卡负责显示（如显示屏的图形显示），NVIDIA GPU 用来做计算。一般配置有支持 CUDA 的 GPU 的笔记本电脑都被称作游戏本。读者可以访问 NVIDIA 的 Notebook 主页（http://www.nvidia.com/object/notebooks.html），根据自己青睐的 GPU 型号，找到对应厂商提供的配置有该 GPU 的笔记本电脑。

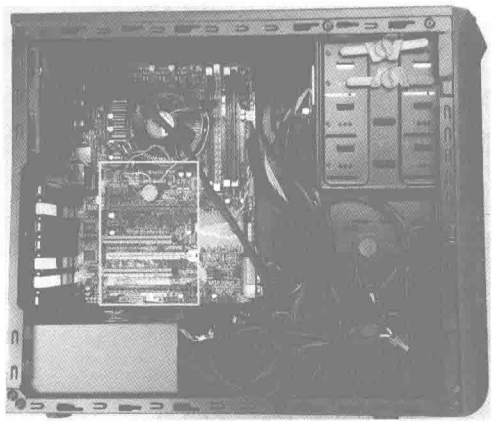
在结束对笔记本电脑的讨论之前，我们先来了解一下功耗。目前许多强大的游戏本中都配置着高端移动 GPU，例如 GeForce GTX 900M 系列。这些游戏本在计算性能和散热能力上表现都非常出色。这种高端显卡的功率大概在 100W，需要很好的风扇和排风管道帮助散热。读者可以根据自己的实际需求选择一款最适合的笔记本电脑，但也有一些性价比高的选择，例如配置有 GeForce 840M 的笔记本电脑。GeForce 840M 拥有 384 颗计算核心，计算能力 5.0，拥有 2GB 显存，功率仅 30W。这类笔记本电脑只需采用被动散热，在承担很多计算任务时，也能兼具轻量级和静音特点。

A.5.2 台式机

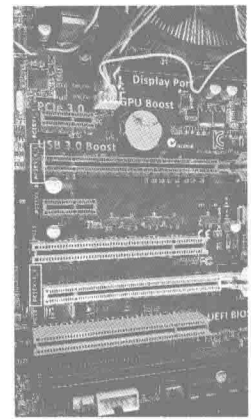
如果读者使用的是台式机，那么就可以自己扩展安装一个 GPU，但前提是你得有勇气打开你的机箱（电源关闭的情况下）。机箱打开后，有两个关键硬件需要注意：

- 主板上的 PCIe-3.0 插槽
- 连接 PCIe 与电源之间的供电线

图 A.3a 是机箱打开之后的显示图，里面包含主板和电源。图 A.3b 为主板上外设插槽局部放大图。左侧的印刷体字表示连接头的类型。这台机器中有两个 PCIe 插槽，分别是上方标记为 PCIEX16_1 的蓝色插槽和下方标记为 PCIEX16_2 的白色插槽，方框里为两个插槽的名称。



a) 台式机机箱内部硬件展示图，内含主板和外设



b) 主板外设插槽局部放大图，主要为两个PCIe x16插槽，方框内为插槽的编号名称

图 A.3 打开机箱

如果你的机器有 PCIe x16 插槽，那么就可以安装支持 CUDA 计算的显卡。有些支持 CUDA 的显卡可以直接通过 PCIe x16 插槽获得供电，而不需要额外的电源线，这种方式比较简单。类似的显卡有 GeForce GTX 750 Ti，其拥有 640 个核，计算能力为 5.0，最大额定功率为 60W（一个 300W 功率的电源即可满足）。如果你的机箱过小，可以考虑 GeForce GT 620（96 个核，计算能力 2.1）。GeForce GT 620 的计算能力并不强悍，但它只有一般显卡的二分之一宽，较扁平，适合许多较小的机箱。

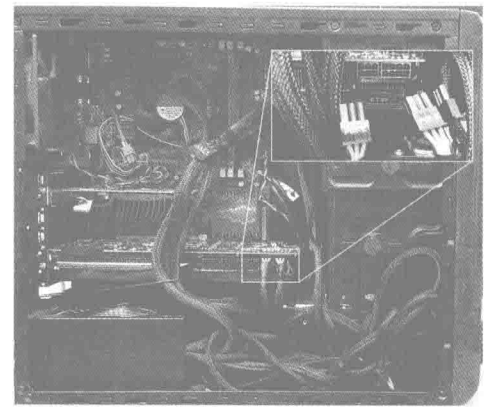


图 A.4 两个 PCIe 插槽分别安装有 GPU 的台式机效果图。PCIEX16_1 安装的是 GeForce GT 610 显卡，PCIEX16_2 安装的是 GeForce GTX 980 显卡。小框内为与 GeForce GTX 980 连接的额外电源接头，而放大框内为其与电源接头未连接时针口外露的状态

图 A.3 中的台式机主板有两个 PCIe x16 插槽和两个 PCIe 电源接头，可以插两块支持 CUDA 的显卡，其中一个可以是需要额外电源的高端显卡。图 A.4 显示了一台在 PCIEX16_1 上安装 GeForce GT 610，在 PCIEX16_2 上安装 GeForce GTX 980 的台式机机箱。在图片右侧是局部放大图，为清晰起见，PCIe 6 针口和 6+2 针口电源接口并未连接电源接头。如果读者有这两块显卡，则可以用低端卡负责显示，用高端卡负责计算任务，这是使用 CUDA

时常见的一种硬件配置方式。

检查完主板上 PCIe x16 插槽和 PCIe 电源接头之后，读者即可根据自己的需要选择相应类型的显卡（电源的功率是一个重要信息，读者需学会辨认电源功率）。你可以访问 CUDA GPU 主页（https://developer.nvidia.com/cuda-gpus）选择自己感兴趣的显卡，然后单击规格（Specifications）查看电源接头和电源供电能力的详细信息。

确定感兴趣的 GPU 之后，将其型号输入到搜索引擎中查看其价格以及是否能在市场上买到。新 GPU 通常会定期地投放到市场，价格也会随着时间变化，因此我们可以对一些新的改良后的 GPU 有所期待。对于购买娱乐性的显卡，只要确定 GPU 型号之后，向你最中意的供应商发送订单即可。

当 GPU 卡送到之后，其安装非常简单。在确认关掉电源之后，打开机箱（若检查 PCIe 插槽和电源接头之后机箱封上了），拆开 GPU 卡包装，然后将其插入 PCIe x16 插槽即可。注意 GPU 只能朝向一面插入（带金属片的那头应紧靠机箱外侧使其能够利用螺丝与机箱固定）。将 GPU 牢固地插到插槽中，若有必要需连接额外电源，然后封上机箱。

之后启动计算机，接入因特网，进入操作系统后，系统应能够识别到新安装的硬件设备，之后只需按照要求下载指定驱动。关于如何检查系统识别的硬件结果，请仔细阅读 A.1 节至 A.3 节，查看机器上安装的显卡型号。至此，硬件安装部分全部结束，接下来你可以继续按照附录 B 进行软件配置。

附录 B Appendix B

软件设置

有了配备 GPU 的计算机之后，接下来就是安装 CUDA 的相关软件了。本章我们将对 CUDA 相关软件和它们的安装过程进行介绍。注意，不同的操作系统下软件安装步骤不同，我们将针对 Windows、OS X 以及 Linux 三种操作系统的软件安装分别进行介绍。

B.1 在 Windows 下安装 CUDA 相关软件

只要有一台合适的 PC（能够运行 Windows 7 及以上版本系统）就可以开始安装相关软件了。主要步骤如下：

- 创建系统还原点。
- 安装 Microsoft Visual Studio 集成开发环境（IDE）。
- 一键安装 CUDA 软件，包含 CUDA Toolkit、CUDA Sample、Visual Studio 版 Nsight（IDE 插件）以及 GPU 的 CUDA 驱动程序。

注意，此处讨论的软件版本都是针对当前的最新版，即 Microsoft Visual Studio 2013 社区版、Visual Studio Nsight 4.7 版以及 CUDA 7.5 版。

B.1.1 创建还原点

在安装软件之前，可以先创建一个系统还原点，以防止意外发生。从 Windows 的开始菜单中打开控制面板，在其搜索框（右上角处）中输入 Create a restore point（创建还原

点），单击搜索将打开系统属性窗口的系统保护页，单击窗口下方的 Create（创建）按钮创建系统还原点。

B.1.2 安装 IDE

此处需要下载的软件主要有 CUDA Toolkit 和 Microsoft Visual Studio IDE。由于 CUDA Toolkit 包含 Nsight，这款软件主要是用来对 CUDA 代码进行调试和分析，是以插件的方式安装在 Visual Studio 中，因此，在安装 CUDA Toolkit 之前应先安装 Microsoft Visual Studio。这些软件目前仍在不断研发中，因此不断会有新的版本定期更新。此处，我们将安装目前兼容性较好的版本组合：

- Microsoft Visual Studio 2013 社区版
- CUDA 7.5，内附 Nsight 4.7

读者可以访问 www.visualstudio.com（或使用你最喜欢的搜索引擎输入“Visual Studio 2013 Community Edition”进行搜索），按照具体提示下载安装 Visual Studio。注意，Visual Studio 的安装可能需要一点时间，在此期间你可以品尝一杯或者两杯咖啡。

B.1.3 安装 CUDA Toolkit

安装好 Visual Studio 之后，接着就要进入 CUDA Zone 了。之所以叫“CUDA Zone”，是因为这个名字是 NVIDIA CUDA 网址 https://developer.nvidia.com/cuda-zone 的组成部分。在该网页中，可以找到所有关于 CUDA 的信息，包括相关工具和资源、入门材料、培训和课程课件以及相关（大多数目前我们都能用到）的附属下载（如图 B.1 所示）。

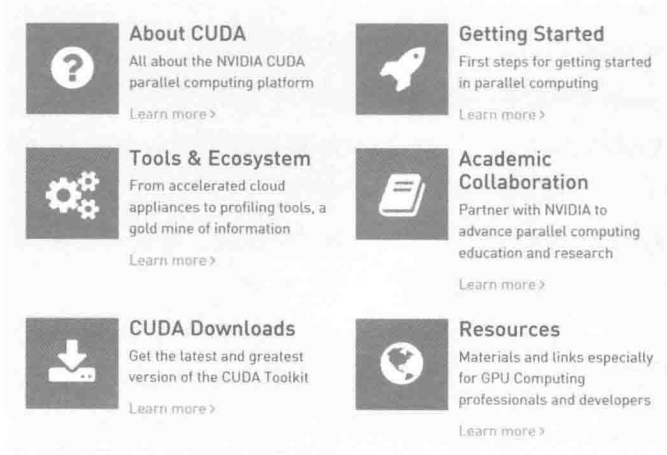


图 B.1 CUDA Zone 提供的链接

初次访问 CUDA Zone 网页时，直接将页面滑动到底部，点击“Developer Program”链接（标题“GET INVOLVED”下方[⊖]），然后以 CUDA 开发者身份进行注册，这样才能访问所有 CUDA 相关资源[⊜]。以 CUDA 开发者身份注册账号之后，回到 CUDA Zone 页面，点击“CUDA DOWNLOAD”图标进入当前 CUDA 下载页，如图 B.2 所示。

图 B.2 CUDA Zone 中 CUDA 7.0 版的下载页面

注意，图 B.2 中显示的是 CUDA 7.0 版的下载页面，因为在写作本书时，CUDA 7.5 版还只是一个候选版本，官方并未正式发布（读者阅读的时候 CUDA7.5 版可能已经正式发布了）。如果当前有候选发布版本，只需在 CUDA Zone 中搜索，就能够找到候选版本的下载页。CUDA 7.5 候选版本的下载页面，如图 B.3 所示。注意，文档链接对 CUDA 的学习非常有帮助，如果你下载的版本有快速入门指南，推荐你仔细阅读一下。

图 B.3 CUDA 7.5 候选版本的下载页面

⊖ 译者在翻译时该网页已移除“Developer Program”链接。——译者注

⊜ 截至本书中文版出版时，CUDA Toolkit 的下载已开放，无需注册。——译者注

无论是下载 CUDA 的正式发布版本还是候选版本，都需根据操作系统类型下载指定版本。下载时主要有两个可执行文件（EXE）供选择，即 Network Installer 和 Local Installer，根据你的实际需要选择其中一个即可。Network Installer 的文件大小较小，但在运行该可执行程序进行软件安装过程中需连接到互联网下载必要文件。Local Installer 的文件较大，下载较耗时，但该可执行程序在安装时无需连接到互联网。

下载完 CUDA Toolkit 安装程序之后打开可执行文件并单击 OK，将文件解压到临时文件夹下并保存。解压完文件之后 CUDA 安装程序会自动启动，然后会出现以下几个需要操作的步骤：

1. 系统兼容性检测以及协议许可浏览，单击 AGREE AND CONTINUE（同意并继续）按钮。

2. 选择快速或自定义安装方式。除非你有特殊原因需要自定义安装，一般情况下选择快速安装，单击 NEXT（下一步）按钮开始安装。

3. 当安装程序准备安装 CUDA 对应的 GPU 驱动时，Windows 安装窗口会弹出询问“Would you like to install this device software”（“愿意安装这个设备软件吗？”），单击 Install（安装）按钮，开始安装“NVIDIA 显示适配器”的驱动软件。

> 闪烁警告！
>
> 注意，在安装驱动的过程中显示屏可能出现闪烁，甚至出现几秒钟完全黑屏的现象。不用惊慌，这是安装过程中的正常现象。（此外，在安装软件之前我们创建的系统还原点也是为了防止意外发生，所以更不用惊慌。）

安装过程需要花费几分钟，安装程序直接将四个软件合并安装了，而之前则需要分别安装以下四个程序：

- CUDA Toolkit：CUDA 基础软件。
- CUDA GPU 设备驱动：该软件负责告诉支持 CUDA 计算的 GPU 在 CUDA 环境中如何工作。
- Nsight Visual Studio 版：该软件集成在 IDE 中，负责调试和分析运行在 GPU 上的 CUDA 代码。
- CUDA Samples：官方收集的 CUDA 示例程序。

当安装结束之后，将看到如图 B.4 所示的窗口，显示实际安装（或未安装）的程序。

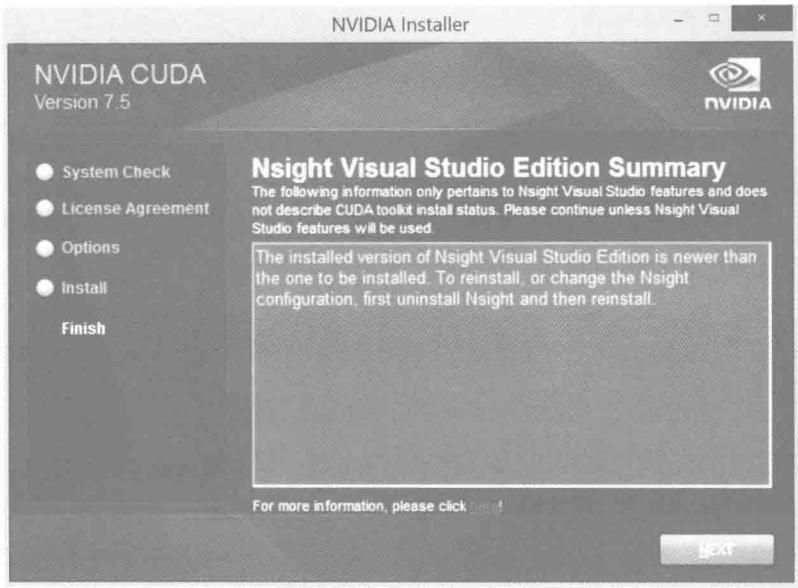


图 B.4 CUDA 安装结束窗口

单击 NEXT（下一步），勾选“Launch Documentation”与“Launch Samples”选项，单击关闭退出安装程序（如图 B.5 所示）。

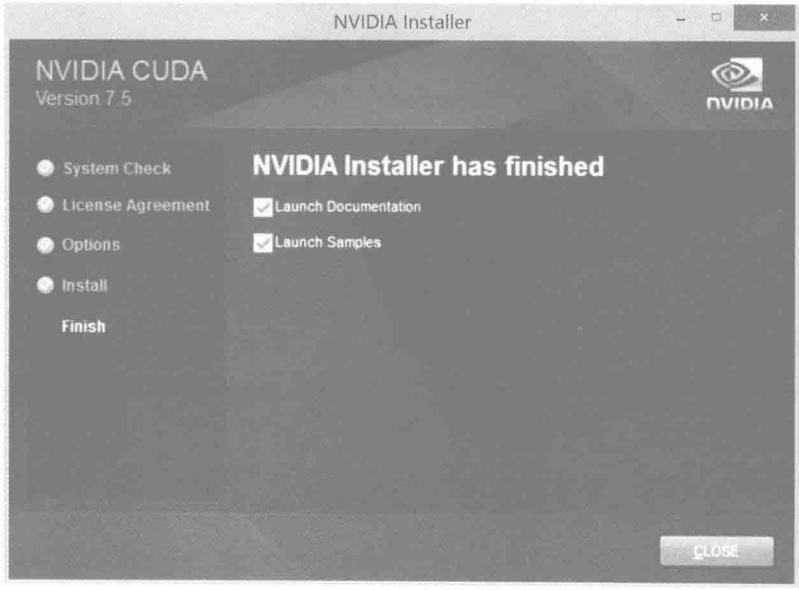


图 B.5 包含启动文档和示例程序的 CUDA 安装程序结束界面

B.1.4 运行初始测试程序

安装完成之后，可以运行两个样例程序进行测试，一个用来测试 CUDA 能否正常运行，另一个用来检测系统的性能。CUDA Samples 中包含了许多 CUDA 相关的应用程序，运行其中的程序就可以进行测试。

在 CUDA 安装结束时，如果勾选了“Launch Samples”选项，那么 Windows 的资源管理器会自动打开 CUDA Samples 所在的文件夹，如图 B.6 所示。

图 B.6 CUDA 7.5 对应的示例程序，高亮的文件为所有示例程序的 Visual Studio 2013 解决方案文件

如果在安装过程中没有选择“Launch Samples”选项，可以通过以下方式访问 CUDA Samples。在 Windows 7 中，打开 Windows 开始菜单，依次选择 All Programs ⇒ NVIDIA Corporation ⇒ CUDA Samples ⇒ V7.5 ⇒ Browse CUDA Samples。在 Windows 8 中，在开始页面底部单击向下的箭头进入应用页面，滑动窗口找到“NVIDIA Corporation”项，单击“Browse CUDA Samples”即可打开如图 B.6 所示的文件夹。这个文件夹下包含了 NVIDIA 收集的所有与 CUDA 相关的样例程序代码，需要将它们编译成可执行文件才能运行。附录 C 会介绍使用 Visual Studio 编译可执行程序的一些必要细节，但此处，我们只需双击一下鼠标然后按住一个功能键就能将 CUDA Samples 中的所有样例都编译

成可执行程序。将鼠标悬停在 Samples_vs2013 文件上，然后会出现一个小信息框，确认里面写的类型是 Microsoft Visual Studio Solution。双击它，Visual Studio 会打开这个文件。

Visual Studio 打开 Samples_vs2013 文件之后，界面显示如图 B.7 所示。现在只需按下 F7 功能键即可开始编译（编译这些样例程序会花费一点时间，这会儿你可以品尝一杯咖啡）。开始编译之后，Visual Studio 窗口底部会出现一个绿色的进度条，编译结束之后会消失。

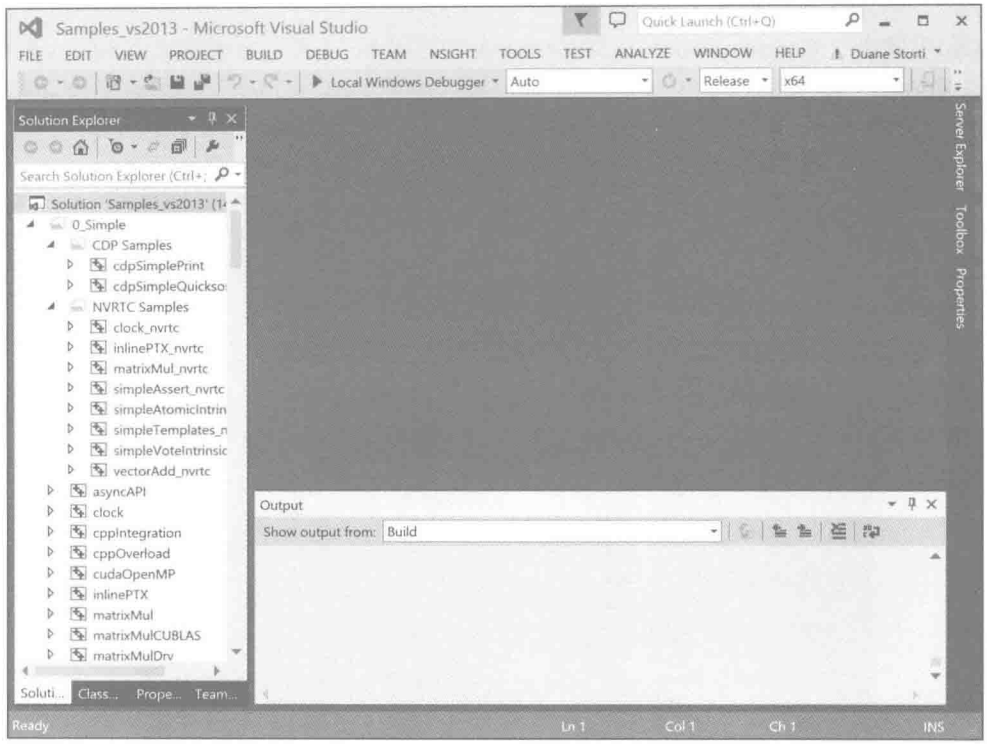


图 B.7 Visual Studio 打开 Samples_vs2013 解决方案文件的显示界面

编译结束之后，所有程序的可执行文件在图 B.6 中的 bin 子文件夹下。双击 bin ⇒ win64 ⇒ Release，进入该文件夹下可以看到所有可执行程序（查看时可以让资源管理器按照类型对文件排序，所有可执行程序会按照字母顺序排列在一起）。只需双击即可运行指定的 CUDA 样例程序。目前为止，若想验证之前所有步骤都正常运行，只需运行 deviceQuery 程序，该程序会将当前机器的 CUDA 计算能力相关信息输出到控制台。如果我们双击运行这个程序，其会将信息结果输出到控制台中，但控制台窗口同时也会一闪而过，根本无法阅读到任何信息，该程序需要以另外一种方式运行：

1. 在 Visual Studio 中，选择 TOOLS ⇒ Visual Studio Command Prompt 打开控制台窗口。

2. 在文件资源管理器中右键单击 Release 文件夹（bin/win64 文件夹下），在弹出的菜单中选择 Copy address as text。

3. 在命令行中输入“cd”命令（改变当前工作目录），然后单击右键粘贴之前复制的 CUDA Samples 可执行程序的路径，按回车进入该文件夹。

4. 在命令行中输入 deviceQuery，并按下回车键，deviceQuery 就能在命令行中运行了。

deviceQuery 程序的运行结果如图 B.8 所示。结果中显示了可用的 CUDA 设备，识别了 GPU 型号（GeForce 840M）、当前 CUDA 运行版本（7.5）、计算能力（5.0）、显存大小（2048MB，即 2GB）、设备上 CUDA 计算核心数目（384）以及一些其他的属性信息。

图 B.8　deviceQuery 程序在命令行中的运行结果

如果你也能得到类似的结果，那么恭喜你，你的计算机已经成功安装了 CUDA 程序。deviceQuery 只提供了一些 CUDA 计算能力等相关的信息，还有许多有趣的示例程序等待你去运行。

到目前为止，我们已经验证了 CUDA 软件是否正常运行（连同支持 CUDA 计算的显卡是否正常工作也进行了验证）。现在读者可以返回阅读第 1 章，也可以继续阅读附录 C，来了解更多关于 C 语言编程的必备知识。

B.2 在 OS X 下安装 CUDA 相关软件

在一些其他的书中，通常我们认为 Mac 运行的 OS X 系统是 Linux 操作系统的一种（因为这两种平台的编译方式相同），但它们安装 CUDA 的方式还是有明显的区别。本节将介绍如何在一台支持 CUDA 计算的 Mac 机器上运行 CUDA。

下载安装 CUDA Toolkit

当验证了计算机配备了支持 CUDA 计算的 GPU 之后，就可以安装相关必要软件了，其中包括 Xcode 和 CUDA Toolkit。

Xcode 是一套 OS X 的官方软件开发工具，从苹果应用商店即可免费下载使用该软件。下载 Xcode 之后，在命令行终端窗口中输入“xcode-select--install”安装命令行工具。

Xcode 命令行工具安装之后，就可以在 CUDA Downloads 页面下载 CUDA Toolkit 安装程序了。访问 https://developer.nvidia.com/cuda-downloads，选择 Mac OSX 选项（如图 B.9 所示），然后选择是联网安装程序或者本地安装程序（联网安装程序需保证安装过程中网络连接畅通）。下载之后双击安装程序，根据提示进行 CUDA Toolkit 的安装。

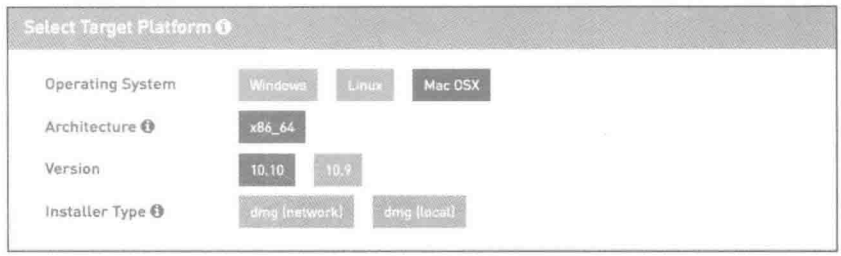


图 B.9 CUDA 7.5 OSX 版下载页面

安装结束之后，使用 TextEdit 打开 shell 配置文件（对于 bash Shell，使用命令 open-e ~/.bash_profile），加入以下两行代码：

```
export PATH=/usr/local/cuda/bin:$PATH
export DYLD_LIBRARY_PATH=/usr/local/cuda/lib:$DYLD_LIBRARY_PATH
```

这样下次在命令行中使用 NVIDIA C 编译器 nvcc 时，系统路径中能直接找到该程序。使用命令 nvcc –version 可以查看当前安装的 nvcc 版本。

至此，OS X 系统上的安装就告一段落，接下来读者可以根据下文继续下一步设置。

B.3 在 Linux 下安装 CUDA 相关软件

CUDA Toolkit 支持几种不同的 Linux 发行版。https://developer.nvidia.com/cuda-downloads 页面中 Linux 选项下罗列了兼容 CUDA 7.5 的几种 Linux 发行版，如图 B.10 所示。本节将以 Ubuntu 14.04（LTS）版为例介绍安装步骤。对于其他发行版，安装步骤大致相似，一些额外的步骤在 https://docs.nvidia.com/cuda/pdf/CUDA_Getting_Started_Linux.pdf 中也进行了详细介绍。

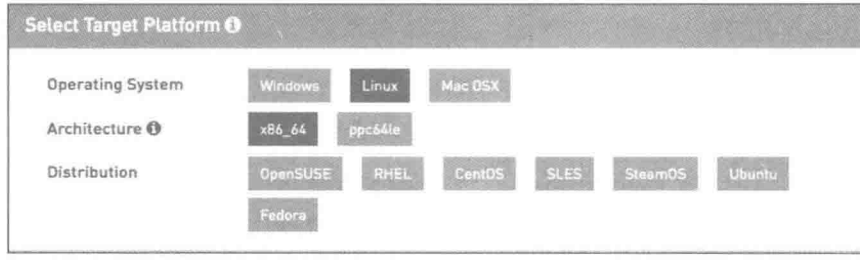


图 B.10 下载页面中兼容 CUDA 7.5 的 Linux 发行版

B.3.1 准备 CUDA 安装所需的相关系统软件

安装 CUDA 之前，需在命令行（使用 Ctrl+Alt+t 组合键可在 Ubuntu 中快速打开命令行窗口）中输入 gcc--version，确认 GNU Compiler Collection（GCC）是否已安装。如果输入之后返回的结果不是 GCC 版本编号，而是 command not found 或者 currently not installed，则需输入 sudo apt-get install gcc，来安装 gcc。

B.3.2 下载并安装 CUDA Toolkit

访问 CUDA Downloads 主页 https://developer.nvidia.com/cuda-downloads，根据系统版本选择相应 Linux 选项，然后选择联网安装程序或者本地安装程序（联网安装程序需保证安装过程中网络连接畅通）。单击链接将下载一个软件库包。右键单击该软件包图标，选择“Open with the Ubuntu Software Center”，单击 Install 按钮。当软件包安装结束之

后，使用命令行执行以下步骤：

- 输入 sudo apt-get update 命令更新系统版本库信息。
- 输入 sudo apt-get install cuda 命令安装 CUDA Toolkit。

CUDA 安装结束之后建议重启计算机。但是此处我们将推迟重启，先更新系统路径，使系统能定位到 CUDA Toolkit 相关文件。以下是一些必要的步骤：

- 使用文本编辑器打开 profile 配置文件（例如 gedit ~/.profile）。
- 在文件的末尾加入一行代码：export PATH=/usr/local/cuda/bin:$PATH，然后保存文件。
- 使用文本编辑器打开 UNIX shell 启动文件（例如 gedit ~/.bashrc），在文件末尾加入代码：export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH，然后保存文件。

至此，退出文本编辑器，重启计算机（使修改的配置生效）。重启之后，使用 printenv | grep cuda 命令查看这些改动是否成功。如果系统能够成功定位 CUDA 文件，则会返回类似如下输出结果：

```
LD_LIBRARY_PATH=/usr/local/cuda/lib64:
PATH=/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:...
```

此外，也可以使用 nvcc--version 命令来确认 nvcc（NVIDIA C 编译器，用来编译 CUDA 代码）已正确安装且已包含在系统路径中。

B.3.3 在用户目录下安装示例

CUDA Toolkit 中附带的示例程序一般安装在系统目录下，由于用户对系统目录下的文件没有写和编辑的权限，如果用户想修改这些示例程序，可以将所有示例程序复制一份到用户具有写权限的文件夹下。CUDA Toolkit 中有一个脚本就可以做这件事，执行 cuda-install-samples-7.5.sh ~ 命令将示例程序复制到 ~/NVIDIA_CUDA-7.5_Samples 文件夹下。使用 cd ~/NVIDIA_CUDA-7.5_Samples 命令可以进入这份示例程序所在的新文件夹进行查看，使用 ls 命令查看文件夹下的所有文件。

B.3.4 运行初始测试程序

我们鼓励读者去尝试每一个示例程序，进入 CUDA 示例程序的文件夹下，输入 make 命令编译这些程序（注意，编译可能花费一些时间）。编译完成之后，运行 deviceQuery 程序测试 CUDA 计算能力。deviceQuery 程序的代码在 1_Utilities 文件夹

下，输入 cd 1_Utilities/deviceQuery 切换到这个文件夹下，然后输入 ./deviceQuery 运行该程序。deviceQuery 程序的输出结果将在终端窗口中显示，对系统测试的结果如图 B.11 所示，我们可以看到计算机的 GPU 型号为 GeForce GT 650M，计算能力为 3.0，显存为 512MB 显存，并且拥有 384 个 CUDA 核心。

图 B.11 deviceQuery 示例程序的输出结果

通过运行一个 CUDA 程序验证了 CUDA 软件能够正常运行（同时也验证了显卡支持的 CUDA 计算能力）。现在你可以返回阅读第 1 章，也可以继续阅读附录 C，来了解更多关于 C 语言编程的必备知识。

附录 C Appendix C

C 语言编程须知

本附录覆盖了 C 语言编程的基础知识，它们是进行 CUDA 编程的基础。我们从一般的 C 语言描述开始，然后介绍创建、编译，以及运行 C 语言程序的过程。在这篇附录的最后，我们将详细讨论 dist_v1 和 dist_v2，这两个计算距离的程序将作为我们使用 CUDA 进行并行化的初始示例。

C.1 C 语言特性

下面简要说明你需要知道的关于 C 语言编程的三个基本特性：

- C 是一个编译语言，因此我们将按 C 语法编写人类可读的代码（例如代码中包含我们熟识的单词以及标点符号），并且我们使用一个叫作编译器的软件（可以通过命令行或者集成开发环境（IDE）启动）将人类可读的代码转换成计算机的处理单元可以“理解”并执行的机器码。

上面关于编译的描述是真实情况的简化，但是关于链接器、PTX 码等内容目前还不是我们必须知道的。（如果你想了解这些细节，Mike McGrath 所著的《C Programming in Easy Steps》一书提供了简明的描述[1]，而与 CUDA 相关的特定细节在“CUDA C Programming Guide”中介绍了[2]。）

编译是把典型的双刃剑。它的好处是编译过的代码非常高效，并且允许你创建的程序充分利用计算机的处理能力。坏处是在代码开发过程中我们必须付出一些额外的努力。

每次编辑过代码后，在运行新版本的程序之前我们都得停下来重新编译并检查错误。稍后我们将亲自看看这些步骤。（相对于编译语言，另一种典型的选择是解释语言，它没有明显的编译步骤，但是通常付出的代价是运行前没有足够的语法检查，并且运行时效率较低。）

- C 是一个类型化语言，因此我们每定义一个变量都需要一个声明语句告诉系统这个变量是什么类型的。因为系统知道每种数据类型需要多少存储空间，所以类型声明和花费一点努力使得计算机在运行你的程序时可以高效利用资源的主题是一致的。
- C 函数的参数是值传递的（与引用传递相对）。对我们而言，这在讨论处理数组数据时变得特别重要。当你写了一个使用数组作为输入或者输出的函数时，系统是否应该复制一份数组（这也许很方便，但是将大量占有有限的内存）然后执行操作？ C 语言的作者们非常重视内存的高效利用，所以选择不进行复制。相反，他们决定使用一个指针，这是一小段数据用来指明可以在系统内存的什么位置找到已有的数组。（任何标准的 C 参考书[3, 4, 5]都包含关于指针和指针运算的扩展讨论。我们将避免陷入指针运算中。）在这篇附录末尾展示的关于 dist_v2 应用中的指针，是为了提供开始学习 CUDA 所需要了解的指针的知识。

有了这些简介后，让我们开始探究必要的细节。

C.2 C 语言基础

一个 C 程序包含一系列符合标准语法，并且一般以分号结尾的语句。这种我们最常使用的语句包括变量声明、赋值、函数定义、函数调用以及运行控制。每条语句的标准语法包括一个或多个保留的关键字、标点符号以及一些你必须保证完全正确的括号。（记住除了它的处理能力，你的计算机是一个非常没有理解能力的机器，它只知道你实际写的代码，而不知道你为什么写这些代码。不好的消息是有时你得将大量精力花在代码细节上；而好消息是你的开发环境应该包含语法检查工具从而大大减轻你作为程序员的负担。）

说到语法细节，让我们通过说明一些一般规则来介绍标点、括号以及其他事项：

- 圆括号 () 被用于以下几个目的：
  - 函数以及控制语句的一组参数。

- 指明一个数学公式中操作符的优先级，所以（1+2）*3 计算结果为 9，而 1+（2*3）计算结果为 7。其实 1+2*3 的计算结果也是 7，因为乘法比加法优先级高，但是往往明确的处理这类情况会更安全，而不是假设系统会正确地判断你的意图。

- 花括号 {} 用于将语句分组，以及为数组初始化提供值。
- 方括号 [] 用于索引数组中的条目。在声明数组时，方括号中的一个整数指明了数组中的元素个数。
- 井号 # 指明预处理指令。这些代码行不是 C 语言的真正组成部分，并且不以分号结尾。
  - 我们使用 #include 从其他文件中引入代码。例如，#include<stdio.h> 引入了标准输入输出头文件，而 #include"aux_functions.h" 从位于相同目录下的 aux_functions.h 文件中引入代码。
  - 我们使用 #define 来简化对代码中多次出现的常量（并且有时也包括函数）的管理。
- 基本的数学运算加、减、乘、除使用符号（+，-，*，/）来表示，并且其中两个还有其他用处：
  - 星号 * 也和指针一起使用（我们将在后面介绍）。
  - 斜杠 / 也出现在文件路径中，可能更重要的是用于注释。双斜杠 // 表示该行的剩余部分是注释（不会被编译器转换成机器语言的代码部分），并且可以用 /* 开头，以 */ 结尾来写跨越多行的注释。

C.3　数据类型、声明以及赋值

C 语言包括几个内建的或原生的数据类型，包括 int、float、double 以及 char，分别表示整数、实数以及字符。需要注意的是我们讨论的是数字计算，所以每种数据类型都存储在固定大小内存中，并只能表示有限的范围。计算机的存储是二进制的，并且基本的存储单元是单个 0 或者 1 即一位。通常将 8 位作为一组称为字节是很方便的。一个 char 可以被转换成一个字符码，通常占用 1 个字节的存储空间，而一个 int 或者 float 通常占用 4 个字节（或者 32 位）存储空间。这些大小是依赖于具体系统的（并且像 long、short 以及 unsigned 等修饰符也会有影响），但是 C 语言有一个运算符 sizeof() 来帮我们避

免关注这些细节。

每次引入一个新变量时，我们都需要指定它的数据类型，使系统可以知道它基本的属性（尤其是需要分配多少内存来存储这个变量）。这种说明是通过具有以下语法的声明语句完成的：

```
typeName variableName;
```

其中 typeName 是一个已知的数据类型（如 int、float 或者 char），而 variableName 是任意的你为变量起的名字。选择变量名时有一些需要遵守的规则：

- 以字母开头，而不是数字或者特殊字符。
- 避免使用保留的关键字。（C 语言有几十个关键字，完整的列表可以在网上获取[6]。）
- 不要包含数学运算符标点符号，或者空格。
- 下划线是允许的，并且在 CUDA 中经常出现。（我们将看到大量像 h_out 和 d_out 的名字。）
- 可以混合使用大小写来使名字更易读。C 语言中的名字是大小写敏感的，所以 myname 和 myName 都是合法的并且是不同的。将首字母大写的单词组合起来是一种有用的命名习惯，由于看起来像驼峰所以被称为驼峰命名法（camel Case）。
- my name——中间有空格，不是合法的名字。
- my_name 是一个合法的替代命名方式。用下划线连接单词通常被称为 snake_case 或者 under_score。

一旦我们声明了一个变量，我们便可以使用具有以下语法的赋值语句将一个值与该变量关联：

```
myName = myValue;
```

其中 myName 是我们所定义的变量，而 myValue 是一个具有合适类型的值（所以你可以认为一个单个的等号就是赋值操作符）。

例如，我们可以使用如下语句声明一个叫作 myInt 的整数：

```
int myInt;
```

并且使用如下语句将值 8 赋给它：

```
myInt = 8;
```

C 语言不仅支持所有四则数学运算（加法、减法、乘法以及除法），还提供了方便可选的将数学运算和赋值组合在一条语句中的方法，该方法被称为复合赋值或者增量赋值，

例如，i+=j 和 i=(i+j) 是相同的。

C.4　定义函数

C 语言支持函数作为使用单个名字来速记一组语句的方式。函数提供了一种基本的机制将代码组织成可测试（提高可靠性以及可维护性）并且可重用（避免重复编码）的块。函数也为作用域提供了上下文，这样我们可以创建只有函数内可见的变量。我们将看到创建范围限制在特定函数内的变量的能力将有助于重用变量名，相比于创建新的名字这的确减少了混乱。

下面是定义函数的基本语法：

```
returnType functionName(type0 arg0, type1 arg1, …, typeLast argLast)
{
    statement0;
     …;
    statementLast;
}
```

我们可以创建一个具有单个返回值的函数（返回值可以被赋给一个变量），并且 returnType 指定了返回值的数据类型。函数名后面跟着用圆括号括起来的、逗号分隔的、具有 0 个或者多个条目的列表，每一个条目都是一个由数据类型（type0 等）接着一个空格以及参数名（arg0 等）组成的有类型的参数。

注意这里一般语法的定义比较冗长，所以建立一些简化的表达方式：

- args 是逗号分隔的参数列表的简化（也就是用它替代 arg0, arg1, … , argLast）。
- typedArgs 是逗号分隔的有类型的参数列表的简化（也就是用它替代 type0 arg0, type1 arg1, … ,typeLast argLast）。
- statements 是分号分隔的语句的简化（也就是用它替代 statement0；statement1；… ；statementLast）。

这样定义一个函数的语法就被简化成更容易阅读和记忆的形式：

```
returnType functionName(typedArgs)
{
    statements
}
```

后面我们将定义和执行许多函数，但是为了创建我们的第一个程序，先来关注一个叫 main() 的函数。因为所有 C 程序都有一个 main() 函数用来指定从何处开始运行，以及程序执行过程中包含什么。

C.5 生成应用：创建、编译、运行和调试

下面让我们编写第一个 C 程序，它非常简单。这个程序由一个 main() 函数构成，函数中包含声明变量并给变量赋值的代码，如代码清单 C.1 所示。

代码清单C.1 declare_and_assign/main.cpp

```
1 int main()
2 {
3   int i;
4   float x;
5   i = 2;
6   x = 1.3f;
7
8   return 0;
9 }
```

注意我们的 main() 函数没有参数，所以用于包含参数的圆括号是空的。我们声明了两个变量，一个整数 i 以及一个单精浮点数 x，然后我们分别给它们赋值为 2 和 1.3f。（在一个数字后面加一个 f 说明这是一个浮点数。）在 main() 结尾处，我们遵循惯例，通过返回 0（与 main() 函数的定义返回一个整数一致）来表明运行成功。

在继续更复杂（也更有趣）的程序之前，我们先对该程序进行完整的创建、编译、执行，以及调试。完成这些过程的具体步骤将取决于你的平台以及开发软件，所以我们分别就 Windows 和 Linux 系统提供说明。

C.5.1 在 Windows 中生成应用程序

现在你需要使用开发环境进行工作了。打开 Visual Studio 并创建一个新工程（或者在 Visual Studio 的起始页选择 File ⇒ New）。在已有的模板中选择 NVIDIA ⇒ CUDA 7.5 Runtime（如果它没有被默认选择的话），并输入应用程序的名称。当你准备好单击 OK 来创建一个名叫 declare_and_assign 的新工程时，Visual Studio 的弹出窗口如图 C.1 所示，这正是这个简单的应用程序要完成的。（注意这个例子里面只含有 C 语言代码，与 CUDA 无关，但是我们将提前开始进行一些创建 CUDA 工程的练习。）

图 C.2 为打开新工程时 Visual Studio 的窗口。左边的 Solution Explorer 面板显示了和该工程相关的文件，包括一个叫 kernel.cu 的 CUDA 文件，它被打开，其代码显示在右上角的面板中。

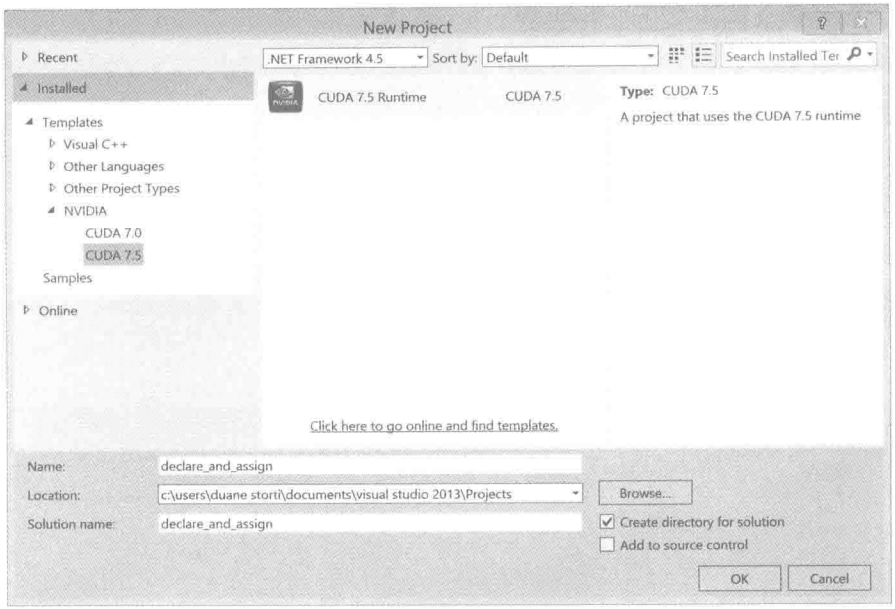


图 C.1　在 Visual Studio 中创建一个新的 CUDA 工程

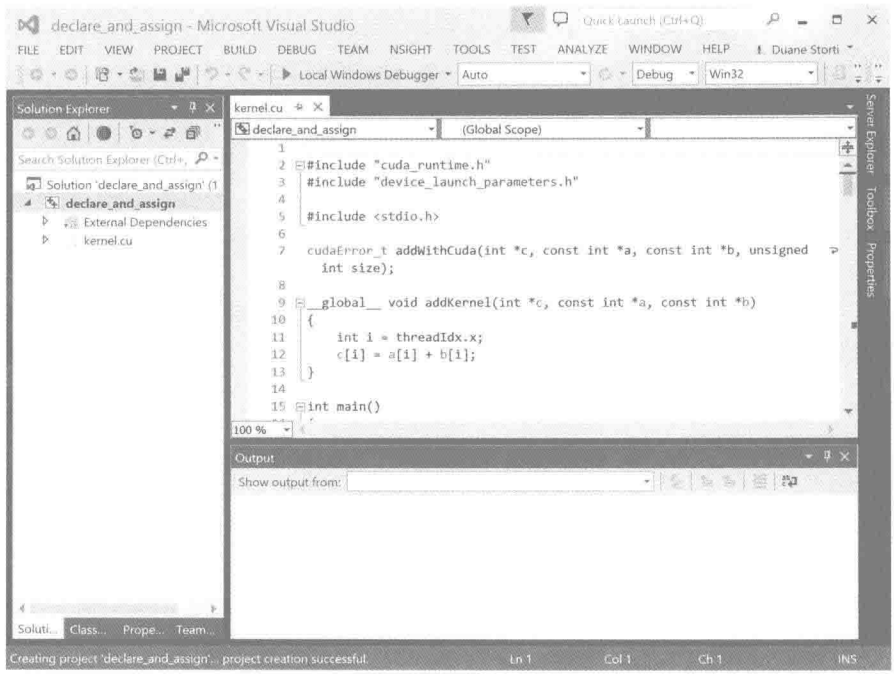


图 C.2　创建了新工程的 Visual Studio 窗口

一个新工程会默认打开一个包含将两个数组相加的简单 CUDA 应用的代码。由于我们的应用中只有 C 语言代码而不包含 CUDA 代码，所以需按如下步骤向工程中添加 main.cpp，并删除 kernel.cu：

1. 选择 PROJECT ⇒ Add New Item。

2. 选择 C++file，并像图 C.3 中一样输入名称 name.cpp。

3. 将代码清单 C.1 中的代码输入 main.cpp。

4. 通过右键单击 Solution Explorer 面板中的 kernel.cu 并选择 Remove ⇒ Delete 将其从工程中移除。

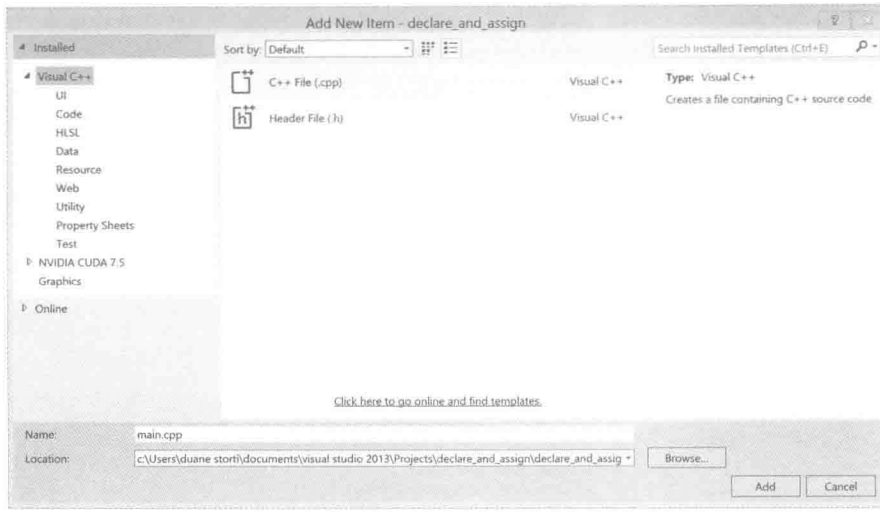


图 C.3　在 Visual Studio 中使用 Add New Item 窗口添加 main.cpp

输入代码后，下一步就是编译和运行了。可以通过选择 BUILD ⇒ Build solution 或者使用快捷键 F7，编译 / 生成代码。你应该养成一个好习惯，认真看一下 Output 面板中出现的信息（因为在我们进行更认真地调试时这些信息将变得非常关键），但是现在我们真正关心的是能否看到如下信息：

```
=== Build: 1 succeeded, 0 failed, 0 skipped ===
```

它出现在输出信息的结尾，告诉我们可执行文件已经成功生成了。

然后我们可以通过选择 DEBUG ⇒ Start without debugging 或者使用快捷键 Ctrl+F5 来运行该可执行文件。运行这个应用，发生了什么？你应该会看到一个显示着 Press any key to continue…信息的控制台窗口，并且按下任意键后会关闭这个窗口。

这个过程没有让你惊喜的，由于我们没有包含任何提供输出的代码，希望你也不要为没有任何输出而感到奇怪。为什么我们没有加上老生常谈的向控制台输出“Hello World”的语句呢？首先，这需要用到稍后才讨论的输入 / 输出格式问题。另一方面，也许是更重要的，通过向控制台打印感兴趣的东西来跟踪程序实际做了什么是一种不明智的方式。相反，让我们立马开始习惯于使用开发工具。

具体来说，假如我们想确认变量是否被赋予了我们期望的值。最好的方法是在调试模式下运行该程序，这要求代码在调试模式下编译。所以重新设置你的 Solution Configurations，就像图 C.4 中显示的一样，它位于 HELP 菜单下方，将 Release 改成 Debug，并将 Solution Platforms 改为 x64（而不是 win32）。

当在 Debug 模式下执行时，我们可以使用 DUBUG ⇒ Step Into 或者快捷键 F11 按代码行单步运行，但是对于执行完更长的代码来说，每次单步运行是很不方便的。

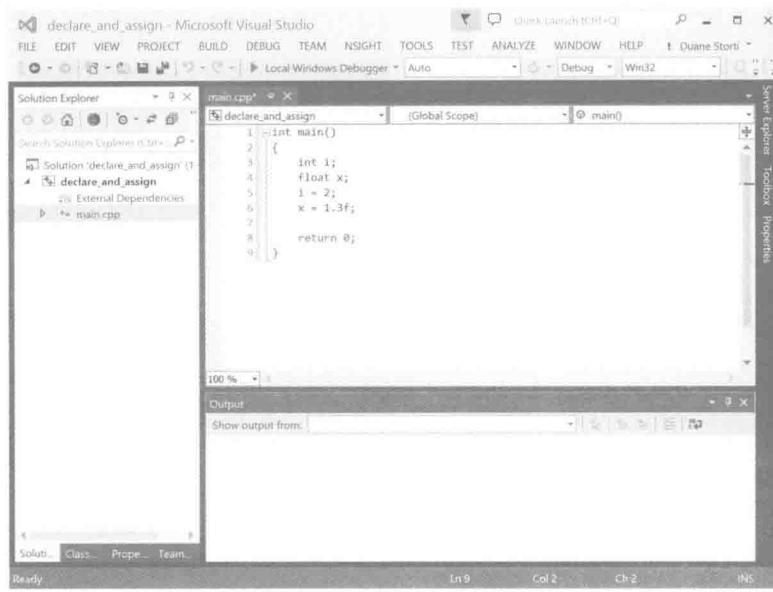


图 C.4　删除 kernel.cu 后的 Visual Studio 窗口。右上角显示了 Solution Configurations 和 Solution Platforms

更好的选择是在要做状态检查的位置设置断点。例如，让我们在给 i 赋值的第 5 行设置断点。一种方法是单击代码面板的第 5 行，然后选择 DEBUG ⇒ Toggle break point，或者使用快捷键 F9。面板左边缘出现一个大红点表明断点的位置，而且你可以点击那个位置来打开或关闭这个断点。现在如果我们使用 F5 开始调试（并且打开断点），程序将会运行到断点处，并且像图 C.5 中一样，你可以在 Locals 面板中观察变量的当前值。在这

点你将看到 i 和 x 的值，这些值没有特定意义，因为赋值操作还没有执行。

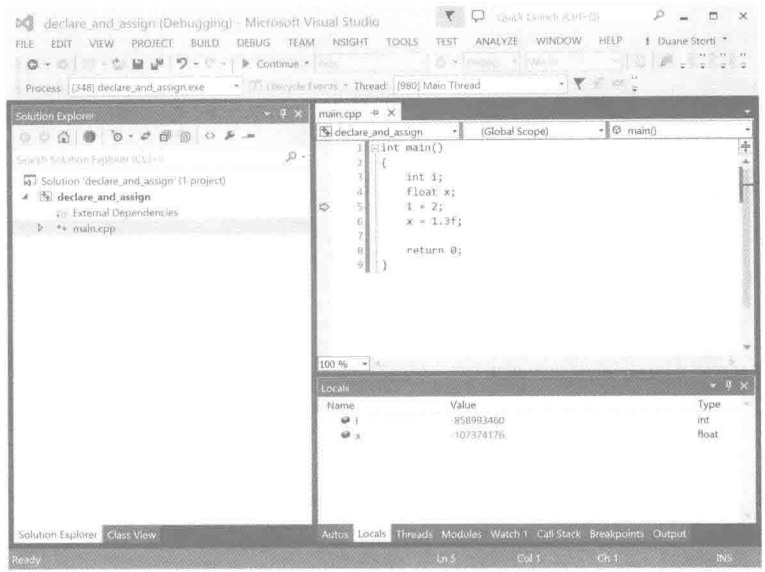


图 C.5　以调试模式运行 declare_and_assign 到第 5 行

单步执行（通过按下 F11 键）一次后箭头将移到第 6 行，这时 Locals 面板应该像图 C.6 一样，其中 i 具有一个可识读的值，并以红色展示在 Locals 面板中。

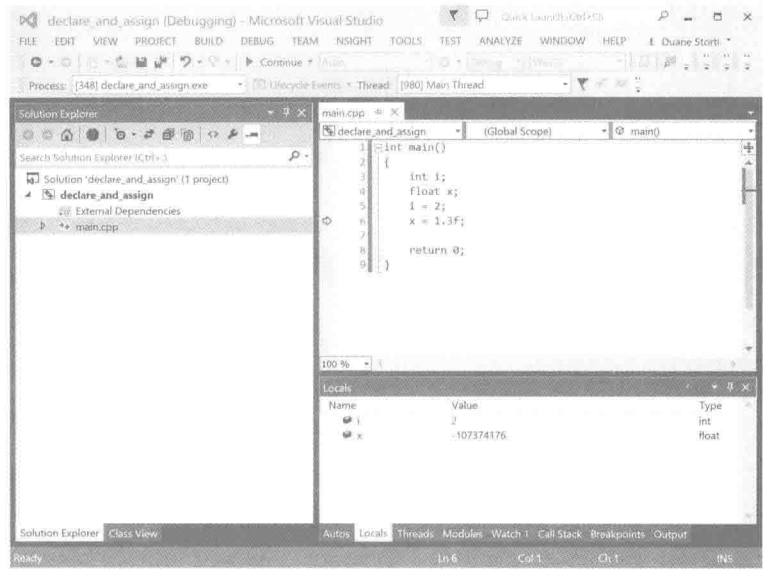


图 C.6　在调试模式下运行完赋值语句 i=2 到达第 6 行

再次单步执行（按下 F11 键），如图 C.7 所示，Locals 窗口中的内容发生了变化。i 的值现在显示为黑色（只有最近语句的赋值显示为红色），并且变量 x 被显示为红色的可识读的值。注意这个值并不是精确的 1.3，而是在给定精度下二进制系统所能表示的最接近的值。

你可以单步越过最后一行，或者选择 DEBUG ⇒ Stop Debuging 来结束运行。现在我们完成了第一个 C 程序的运行，而且如果你使用 Windows 系统，而不是 Linux/OSX 系统，那么你就可以直接跳到 C.6 节了。

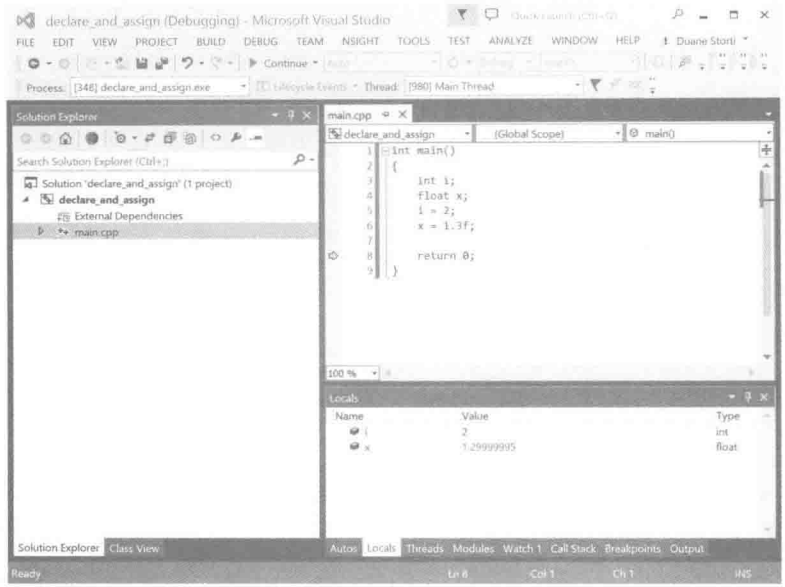


图 C.7　在调试模式下执行完两个赋值后到达第 8 行

C.5.2　在 Linux 中生成应用程序

这里我们使用下面的软件来看看在 Linux/OS X 系统中创建一个应用的过程：

- 使用一个文本编辑器：为了简明起见，我们在本书中的展示使用 Ubuntu 默认编辑器 gedit，以及 OS X 默认编辑器 TextEdit。但是我们推荐你使用带有 CUDA C++ 语言包的 Sublime Text 3，它在所有平台都是可用的。
- gmake（GNU Make）用于使编译 / 链接过程自动化。
- nvcc（NVIDIA CUDA 编译器）用于编译源代码。
- cuda-gdb（NVIDIA CUDA 调试器）用于对可执行文件进行调试。

在整本书中，当必要时，都提供了用于自动化编译 / 链接的 Makefile 文件。在 GNU

的网站[7]上可以获得完整的 GNU Make 文档。

为了在 Linux/OS X 系统中创建 declare_and_assign 应用，我们需要编译代码清单 C.1 中的源代码，并且链接必要的库。在这个只有一个 main.cpp 包含源代码的简单例子中，我们可以通过下面的命令生成可执行文件：

```
nvcc -g -G -Xcompiler -Wall main.cpp -o main.exe
```

其具体解释如下：

- nvcc 启动 NVIDIA CUDA 编译器。
- -g（--debug 的缩写）告诉编译器要为主机代码生成调试信息。
- -G（--device-debug 的缩写）告诉编译器要为设备代码生成调试信息。
- main.cpp-o main.exe 告诉 nvcc 以 main.cpp 为输入，并将输出存储为 main.exe。

另外我们也可以创建一个 Makefile 文件来控制从源代码生成可执行文件。Makefile 提供了很多好处，并且当工程中含有多个源文件时变得尤其重要。我们在此处引入 Makefile，并且将为我们的应用提供 Makefile。代码清单 C.2 中给出了 declare_and_assign 的 Makefile 文件。

代码清单C.2 declare_and_assign/Makefile

```
main.exe: main.cpp
    nvcc -g -G -Xcompiler -Wall main.cpp -o main.exe
```

Makefile 语法

Makefile 中包含一些变量赋值和规则。例如 NVCC=/usr/local/cuda/bin/nvcc，变量赋值看起来和 C 语言赋值相似（没有分号结尾），并且允许你在 Makefile 的剩余部分使用 NVCC 来作为完整路径的缩写。一个规则包含三个部分：目标、依赖以及命令。一个规则的第一行包含目标（紧接着一个冒号）和依赖。规则的其余行是一系列命令，每行开头都是以 TAB 缩进的。（为了避免出现问题，必须以 TAB 缩进，而不是空格。）

代码清单 C.3 给出了一个等价的缩写版本的 Makefile。

代码清单C.3 缩写版的declare_and_assign /Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall

main.exe: main.cpp
    $(NVCC) $(NVCC_FLAGS) $< -o $@
```

生成过程中的文件

从源代码生成可执行应用过程的详细讨论对于我们学习CUDA来说并不是非常重要，但是这里为感兴趣的人提供了一个简明的总结。对于我们所写的高级语言源文件，其中.cpp后缀文件中为C/C++代码，.cu后缀文件中含有CUDA代码，.h或者.cuh是头文件。预处理程序处理完预处理命令后生成另一个版本的C/C++/CUDA代码。编译器生成汇编代码，之后汇编器将汇编代码转换成机器码，并存入以.o为后缀的目标文件中。最后链接器将所有目标文件链接到一起生成可执行文件。

有了代码和Makefile后，我们就可以创建可执行应用了。通过如下步骤创建declare_and_assign应用：

1. 创建一个projects目录来存放所有的工程。在终端上使用cd命令进入projects目录。

2. 使用mk declare_and_assign命令为该工程创建一个目录，然后使用cd declare_and_assign进入该目录。

3. 使用gedit main.cpp命令创建main.cpp文件（对于OS X，首先使用命令touch main.cpp，然后使用open-e main.cpp）。

4. 将代码清单C.1中的内容输入main.cpp。

5. 通过gedit Makefile创建Makefile（同样的在OS X中，首先touch Makefile，然后open-e Makefile）。

6. 将代码清单C.2中的内容输入Makefile。

7. 通过敲入命令make来编译源文件。

现在可执行文件已经准备好了，如图C.8所示，通过如下步骤使用调试器运行应用并检查结果：

1. 通过cuda-gdb main.exe命令将可执行文件加载到调试器中。

2. 使用start命令开始调试，并使用info locals命令显示局部变量。第一步运行到第5行（第一条执行语句），第一个info locals显示i和x，它们还没有被初始化成期望的值。

3. 使用next运行到下一条指令，并且再次检查局部变量。我们现在在第6行，info locals告诉我们i已经被设置成2，而x仍然未定义。

4. 重复上述步骤运行到第8行，我们观察到info locals现在同时显示了i和x所赋的值。注意这里x并不恰好是1.3，而是在给定精确度的二进制系统中最可能的近似值。

5. 最后使用continue运行该应用程序，并通过quit退出调试器。

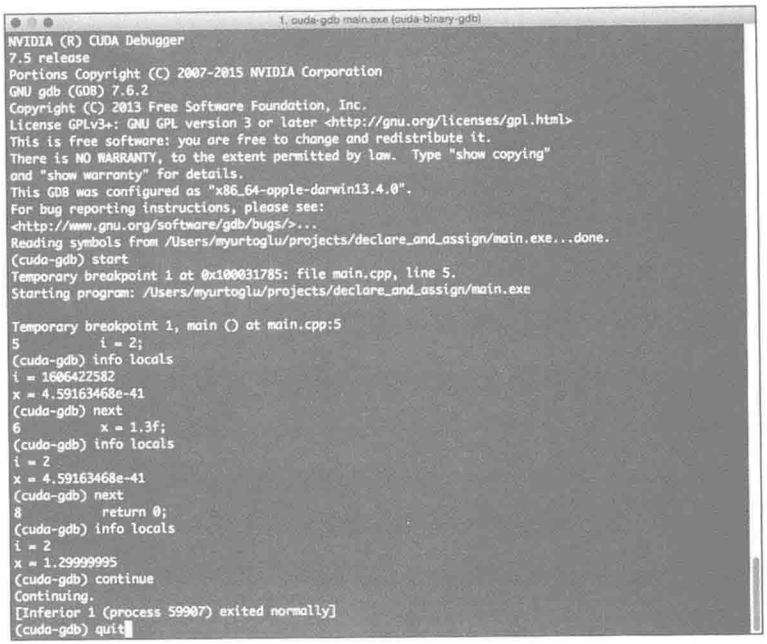


图 C.8 调试 declare_and_assign 过程中的 cuda-gdb 命令以及结果

我们已经完成了第一个 C 应用的程序运行，接下来将继续讨论其他重要的 C 语言概念。

C.6 数组、内存分配以及指针

到现在为止我们已经创建、编译、执行并调试了一个简单的 C 语言程序，所以是时候逐渐增加一些我们所需的其他 C 语言元素了。首先讨论的是数组以及包括 for 循环在内的控制语句。

你可以使用数组来存储多个特定类型的元素，它是一个给定类型的变量的索引序列。在本附录末尾的 dist_v1 和 dist_v2 应用程序提供了这样的例子，其中我们将输入值和 / 或输出值存在数组中。

具体来讲，考虑一组整数输入值以及一组浮点输出值。与普通变量相同，一个数组的名字是通过声明语句来建立的，这和变量声明很像，唯一的区别是需要在名字后面加上方括号来表明这是一个数组。在声明中，方括号内的常数表明数组的长度。在其他语句中，方括号内的整数表明一个元素在数组中的索引。

这里让我们创建一个名为 in 的输入数组。我们可以声明一个数组 in 来存储 3 个整数，以及一个数组 out 来存储 3 个浮点数：

```
int in[3];
float out[3];
```

关于数组声明有几个很重要的事项需要注意：

- 编译时方括号中的值必须是常量，因为只能为固定数量的元素分配内存。在 C 语言中是不支持动态改变数组大小的。通常采用宏定义指令来处理这一情况，如下示例代码所示（其中还包含单行及多行注释的示例）：

```
#define  N 3 //Compiler directive has no semicolon.
/* The lines below declare 2 arrays.
   Each array has length N. */
int     in[N];
float   out[N];
```

- 回想 C 语言中的索引策略是从 0 开始计数，所以这里数组中的元素是 in[0]、in[1] 以及 in[2]。
- 与简单变量一样，你可以在声明的同时赋初值。例如，int in[] = {1, 2, 3}；声明了具有 3 个元素的数组（这里方括号里面可以是空的，因为数组长度可以从初始值列表推出）并赋初值，等价于 in[0] = 1; in[1] = 2; in[2] = 3; 。
- 到目前为止我们对数组的描述都有点大而化之。我们还没有提到使用 int[] = {1, 2, 3}；进行声明与创建 3 个整型变量并分别按顺序赋上花括号内的值有什么区别。事实上，我们声明的数组名是一个指向一定大小（等于元素数量乘以每个元素的字节数；这里是 N*sizeof(int)）的连续内存块中第一个元素的指针。元素编号（包含在方括号中）告诉系统要找到数组中的指定元素需要在内存空间中跨过多少步固定的大小（例如存储每个指定类型的元素需要的字节数）。实际上我们是故意不关注这一点，因为这一特性目前还不重要。我们将在后面使用数组作为输入输出的函数示例代码中明确处理指针，并查看必要的细节，并且本书第 2 章中会讨论在设备（GPU）端创建数组时包含显式的内存分配。
- 不要尝试一下子给整个数组赋值。的确，你可以创建另一个大小为 3 的整型数组 int inCopy[3]；但是 inCopy = in 不会将 in 中所有相应的元素赋值给新数组 inCopy，而是会产生一条错误信息，因为 C 语言不能这样工作。这样的赋值只能一次一个元素的完成，这完美地将我们引入对控制语句的简要讨论。

C.7 控制语句：for、if

控制语句允许你指定某部分的代码将会（或者不会）执行的情形。在 C 语言中，你需要知道如何使用控制语句，但是现在我们仅仅关注 for 循环和 if 语句。

C.7.1 for 循环

for 循环是典型的串行程序结构，因为它包含一个明确定义的循环或者循环索引变量。循环内的代码根据循环索引值一次接一次地重复执行直到满足退出条件。注意 for 循环在我们的走向 CUDA 并行计算的旅行路线上扮演着关键作用，因为我们最开始的并行化就是对串行 for 循环的转换。

for 循环的语法形式如下所示：

```
for (start; test; increment)
{
    statements
}
```

其中 start 通常是一个声明循环索引变量并给它赋初值的语句，test 是一个指定退出条件的语句，而 increment 是一个说明每执行一次循环体后如何改变循环索引值的语句。花括号内包含了每次循环体内需要执行的语句序列。

作为例子，让我们解决前面提到的将一个数组（称为 in）中的元素复制到另一个数组（称为 inCopy）中的问题。示例代码如下：

```
#define N 3
int main()
{
     int in[] = {1,2,3};
     int inCopy[N];

     for (int i=0; i < N; ++i)
     {
          inCopy[i] = in[i];
     }

     return 0;
}
```

一开始我们使用一条编译指令 #define 将符号 N 替换成数字 3（作为数组长度常量），然后是必不可少的 main() 函数。像前面一样，我们声明 in 并为它赋初值 1 至 3。声明了一个长度为 3 的整型数组 inCopy，紧接着是按上述语法所描述的一个 for 循环。循环索引是一个名为 i 的初始值为 0 的 int 型变量。（注意 i 的作用域限制在这个循环内。这个

循环外的相同符号表示一个不同的变量，所以不需要为每一个循环的循环索引起一个新名字。）循环继续执行直到 i 不再小于 N（N 被替换成 3），而 ++i 在每执行一次循环体后将 i 的值加 1。由于每次循环都将 in 中的第 i 个元素复制到 inCopy 中相应的位置，并且 i 初始化为 0，所以第一次循环将 in[0] 中存储的值 1 赋给 inCopy[0]。i 的值从 0 增加到 1，然后回到循环顶部。因为此时 i 的值为 1（N 已经被替换成 3），循环条件 i＜N 满足，所以进行第二次循环，将 in[1] 中的值 2 赋给 inCopy[1]。这个过程继续（为元素赋值直到并包括 inCopy[2]）直到 i 增加到 3，此时继续循环测试失败，退出循环。继续执行到 main() 的最后一行，返回整数 0 表明执行完成。

如前所述，C 语言数组的索引是从 0 开始的，这是一个普遍的编程常识。为了执行一些需要数组中每一个元素（作为输入或者输出）的代码，需要使用一个索引初始为 0，并且在每次迭代中增加并测试小于元素数目的循环（因为一个长度为 N 的数组的索引为 0, 1, …, N-1）。注意一些语言使用其他的索引策略，但是对于 C 语言代码，我们应习惯从 0 开始计数。

最后需要注意的是我们目前所描述的数组还是很形象的，而且并没有真正涉及指针。我们将在后面的示例程序中扩展这一点，在那里我们将构造一个输入和输出都存储在数组中的函数。

C.7.2　if 语句

关键字 if 允许你指定一段只有在测试条件满足时才会执行的代码。测试条件包括相等（==）、不等（!=）、大于（>）、小于（<）、大于等于（>=）以及小于等于（<=）。可以通过逻辑运算符“与”（记作 &&）、“或”（记作 ||）以及取反又称为“非”（记作 !）来组合多个条件。C 语言用 0 表示 FALSE，所以 (a>b) 实际是 ((a>b)!=0) 的缩写，表示判断 a>b 不是 FALSE 是否成立。

标准的语法如下：

```
if (condition)
{
    statements
}
```

当条件不满足时，你也可以编写 else 语句提供其他可选的语句来执行。标准的语法如下：

```
if (condition)
{
    statements
```

```
}
else if (condition)
{
    statements
}
else
{
    statements
}
```

这为在不同条件下执行不同语句的计算提供了可能。作为一个实例，假设我们声明了一个整数 n，并给 n 赋了一个值，并且我们想给 y 赋一个调整后的 x 值（例如，如果 x>0 则 y=x，如果 x<=0 则 y=0）。下面使用 if-else 句来完成这一工作：

```
if (x > 0)
{
   y = x;
}
else
{
   y = 0;
}
```

与 if 语句相关的其他知识点：

- 花括号内的语句用分号结束，但是花括号后面不需要分号。
- 有多种表达方式可以达到同样效果，并且你可以看到这种结构的其他备选方案也是非常方便的（而且仍然具有可读性），特别是当花括号中只有一条或者两条语句时。给 y 赋上调整后的 x 的值的代码可以写得更紧凑：

```
if (x > 0) y = x; else y = 0;
```

或者使用三元运算符，例如：

```
y = (x > 0) ? x : 0;
```

但是我们发现使用花括号以及缩进可以使代码更加具有可读性。（稍后我们将看到用一开始所示的格式使用花括号以及缩进来编写代码对调试工具来说是最好的。）找到适合你的风格并坚持下去。

C.7.3 其他控制语句

结合了一个条件测试并重复（或循环）评估的控制语句由关键字 while 和 do 完成。你可以通过 continue 跳过一个循环索引值的执行，或者通过 break 或 goto 完全离开循环。通过 switch 语句加上多个 case 值可以指定一个根据一个表达式的值决定的多条不同的执行路径，其中还可以包含一个 default 路径。请查看标准 C 语言教材来获取详细信息[5]。

C.8　示例 C 程序

现在我们可以在我们通往 CUDA 旅程上构建第一个应用程序了。这个应用程序计算一个参照点与 N 个 0 至 1 区间内等距分布的点的距离。这里我们构建两个版本，分别为 dist_v1 和 dist_v2，这也在第 1 章中出现，并作为我们后继转换成并行 CUDA 应用的初始示例。

这两个距离应用程序会产生相同的输出，但是它们的结构是不同的：

1. dist_v1 使用一个 for 循环来调用一个计算单个输入的距离的函数。这个版本为初始并行化提供了最简单的素材。

2. dist_v2 有更多的结构。输入数据存储在一个数组中，并且 for 循环被移到了一个叫作 distanceArray() 的函数中，它被调用一次计算并存储所有输出距离值。对 dist_v2 进行并行化工作量更大，但是它为使用 CUDA 进行并行化提供了更好的结构。

C.8.1　dist_v1

dist_v1 的代码显示在代码清单 C.4 中。

代码清单C.4　dist_v1/main.cpp

```
#include <math.h>  //Include standard math library containing sqrt.
#define N 64     // Specify a constant value for array length.

// A scaling function to convert integers 0,1,...,N-1
// to evenly spaced floats ranging from 0 to 1.
float scale(int i, int n)
{
  return ((float)i) / (n - 1);
}

// Compute the distance between 2 points on a line.
float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

int main()
{
  // Create an array of N floats (initialized to 0.0).
  // We will overwrite these values to store our results.
  float out[N] = {0.0f};

  // Choose a reference value from which distances are measured.
  const float ref = 0.5f;

  /* for loop to scale the index to obtain coordinate value,
   * compute the distance from the reference point,
   * and store the result in the corresponding entry in out. */
```

```
29    for (int i = 0; i < N; ++i)
30    {
31      float x = scale(i, N);
32      out[i] = distance(x, ref);
33    }
34
35    // It is conventional for main() to return zero
36    // to indicate that the code ran to completion.
37    return 0;
38  }
```

上面的代码用于计算一个参照点 ref 到一系列 x 的距离值的输出数组（称为 out，长度 N＝64），x 是通过缩放循环索引 i 在 0.0 到 1.0 范围内得到的均匀分布的点。我们首先包含标准数学库头文件（这样我们就可以使用开方函数 sqrt()），然后定义一个常量 N（设置为 64）作为点的数量以及输出数组的长度。第 6～9 行定义的第一个函数 scale() 将 0,…,N-1 范围内的整数转换为 0.0 至 1.0 区间内的一个浮点型坐标，而定义在第 12～15 行的函数 distance() 计算两点之间的直线距离。

main() 函数首先在第 21 行创建了一个长度为 N 的浮点型数组，并初始化为 0.0（将会被重写来存储我们的输出），并且在第 24 行设置用于计算距离的参照点 ref＝0.5f。从第 29 行开始是 for 循环，第 31 行对每一个循环索引（从 0 到 N-1）都计算一个对应的缩放位置并存在 x 中，然后在第 32 行计算与参照点之间的距离并存储在 out[i] 中。当 i 增加到 N，计算退出 for 循环，并且 main() 返回 0 表示代码已经执行完成。

用如下方法将代码清单 C.4 中的代码转换成可执行应用：

❑ 在 Windows 中：

1. 打开 Visual Studio 并创建一个叫作 dist_v1 的新工程。

2. 添加 main.cpp 到工程中：选择 PROJECT ⇒ Add New Item，然后选择 C++ 文件，输入名称 main.cpp，单击 Add。后缀 .cpp 表示 C++ 并表示它的内容要作为 C/C++ 代码来编译。而 .cu 文件包含的则是 CUDA 代码，并且需要使用 nvcc 来编译。

3. 将代码清单 C.4 中的代码录入 main.cpp。

4. 在 Solution Explorer 面板中右键单击 .cu 内核文件图标并选择 Remove 将其从工程中移除。

5. 设置编译选项为 Debug 和 x64，然后选择 BUILD ⇒ Build solution 或者按 F7 键。

6. 在第 37 行设置一个断点，调试并检查结果。

❑ 在 Linux 中：

1. 创建一个 dist_v1 目录。

2. 创建一个新文件 main.cpp。将代码清单 C.4 中的代码录入 main.cpp 并保存这个文件。

3. 创建 Makefile 并录入代码清单 C.2 中的内容。

4. 通过在控制台中输入 make 来编译源文件。

5. 使用 cuda-gdb main.exe 命令将可执行文件加载进调试器中。

6. 使用命令 break main.cpp:37 在第 37 行设置一个断点。

7. 通过 run 开始调试并检查结果。

使用调试工具检查结果，过程和你在 declare_and_assign 中做得差不多，但是你需要检查存储在数组 out 中的值。在 Linux 中这没什么变化，info locals 或者 print out 会将数组 out 中的所有元素发送到终端上。在 Visual Studio 的 Locals 面板中，out 所在行的左侧将会有一个小三角形，如图 C.9 所示。点击这个小三角形将打开一个可滚动的列表显示所有存储在 out 中的元素。

不管怎样，当你查看结果，将看到值从 0.5 开始（缩放值 0 到参照点 0.5 的距离），逐渐下降，数组中间接近 0，然后又增加回 0.5（从 0.5 到缩放值 1.0 的距离）。

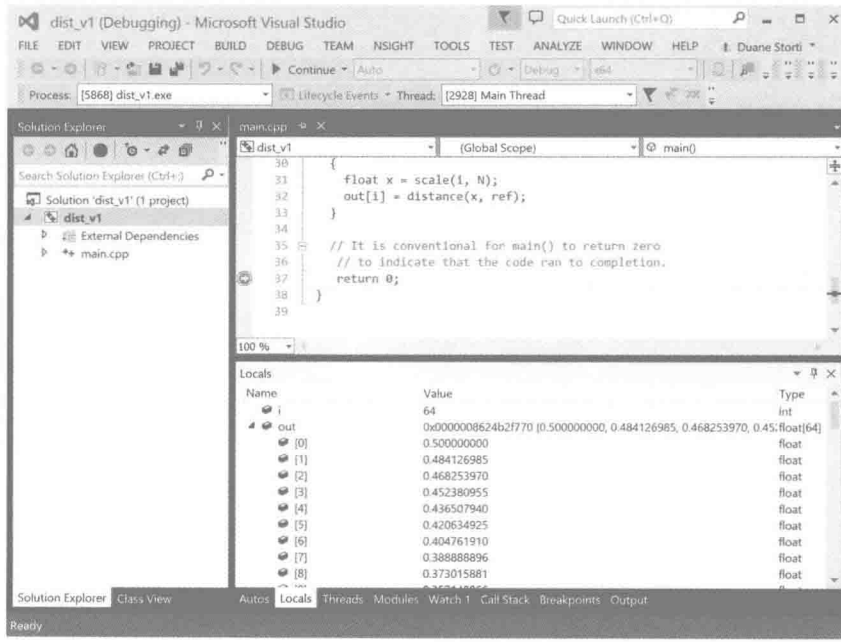


图 C.9　调试器执行到第 37 行时的 Visual Studio 窗口，Locals 面板中 out 旁的三角形已经被点开来显示数组中的元素列表

C.8.2 dist_v2

在第二个版本的距离应用中，我们创建了一个数组存储所有的输入坐标值以及一个函数计算整个输入数组的所有距离值。第一个版本 dist_v1 不是很具有代表性，因为没有实际输入数据（除了参照点）。现在我们来做一些更典型的例子并处理一组输入数据从而得到一组输出数据。（这里输入数据通过一个初始 for 循环来创建，但是该基本流程将同样适用于从一个文件或者输入设备读取输入数据。）我们将所有函数（除了 main()）放到一个单独的文件 aux_functions.cpp 中从而增加了一点结构性，并且创建了一个对应的头文件 aux_functions.h 用于包含必要原型声明。

下面开始介绍 dist_v2 的代码。main.cpp、aux_functions.h 以及 aux_functions.cpp 的代码，如代码清单 C.5 至 C.7 所示。

代码清单C.5 dist_v2/main.cpp

```
#include "aux_functions.h"
#define N 64    // Specify a constant value for array length.

int main()
{
  // Create 2 arrays of N floats (initialized to 0.0).
  // We will overwrite these values to store inputs and outputs.
  float in[N] = {0.0f};
  float out[N] = {0.0f};

  // Choose a reference value from which distances are measured.
  const float ref = 0.5f;

  // Iteration loop computes array of scaled input values.
  for (int i = 0; i < N; ++i)
  {
    in[i] = scale(i, N);
  }

  // Single function call to compute entire distance array.
  distanceArray(out, in, ref, N);

  return 0;
}
```

main() 函数中的代码与代码清单 C.4 中显示的 dist_v1/main.cpp 相比只有少量修改。除了用于存储输出距离的数组 out 外，还有一个数组 in 用于存储缩放的输入值，并且 for 循环（原来计算缩放输入值并计算 / 存储距离值）的工作被分成两部分。现在 for 循环仅仅计算并存储缩放输入值。计算并存储距离值的任务被归入了 distanceArray()。

注意 main() 函数中调用了 scale() 以及 distanceArray()，但是它们的定义都没有出现。我们已经说过在函数调用之前编译器至少得知道函数声明或者原型，这一依赖性通过头

文件得到满足。在这个例子中，我们将辅助函数 scale() 和 distanceArray() 定义在一个名叫 aux_functions.cpp 的单独文件里。然后我们将 main() 可调用的每个辅助函数的原型添加到 aux_functions.h 文件中，并且在 main.cpp 代码顶部包含 aux_functions.h，这样编译器就可以获得所有必要的信息了。

代码清单 C.6 显示了完整的头文件。头文件主要包含函数原型以及解释性的注释。除了函数原型，还有三条编译器指令构成包含保护（或者头文件保护），这用于防止试图多次包含原型而引起的编译错误。

代码清单C.6　dist_v2/aux_functions.h

```
#ifndef AUX_FUNCTIONS_H
#define AUX_FUNCTIONS_H

// Function to scale input on interval [0,1]
float scale(int i, int n);
// Compute the distance between 2 points on a line.
float distance(float x1, float x2);
// Compute scaled distance for an array of input values.
void distanceArray(float *out, float *in, float ref, int n);

#endif
```

aux_functions.cpp 的代码显示在代码清单 C.7 中。scale() 和 distance() 函数和 dist_v1 中的一样。有趣的是增加了 distanceArray()，这个函数获取一个输入数组（同时还有一个参照点和数组长度）并产生一个输出数组。这是我们第一次遇到参数中带数组的函数，这是一个很好的机会让我们回顾 C.1 节中所说的关于 C 语言是如何处理参数为数组的函数的：C 语言将指向数组存储开始处的内存地址指针值传递进去，而不是传递整个数组。C 语言在声明时使用星号表明一个指针类型变量，所以 distanceArray 的前两个参数 float *out 和 float *in 表示指向浮点型数组的指针。

代码清单C.7　dist_v2/aux_functions.cpp

```
#include "aux_functions.h"
#include <math.h>

float scale(int i, int n)
{
  return ((float)i) / (n - 1);
}

float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

void distanceArray(float *out, float *in, float ref, int n)
```

```
15 {
16   for (int i = 0; i < n; ++i)
17   {
18     out[i] = distance(in[i], ref);
19   }
20 }
```

distanceArray() 函数包含一个 for 循环，循环体内读取输入数组的第 i 个元素 in[i]，计算其到 ref 的距离，并存储到输出数组对应的位置 out[i] 中。注意函数原型告诉了编译器 in 和 out 是指向浮点数的指针，所以在函数体内数组名字前就不需要函数原型中的类型和星号了。

指针运算

除了用于声明，星号还用于解引用运算符，返回指针变量所指内存处的值。虽然有些情况下显式的解引用是合适的，但是对于数组操作来说是不必要的。如果数组的名字是一个指向该数组起始位置的指针。例如，in 指向输入数组的起始位置——那么为什么我们不使用型号来访问数组中的值呢？答案是我们不仅仅使用名称 in。我们将 in 当作一个数组，所以我们的代码中包含的不是 in，而是 in[i]。编译器将 in[i] 分解为 *(in+i)，这翻译成自然语言就是“将 in 增加 i 并返回所在位置存储的值”。因为 in 指向输入数组的起始位置，并且编译器知道按存储一个浮点数所需的内存大小来增加指针，所以 in[i] 返回了第 i 个元素。如果你真的包含了一个星号，可能会发生意外，因为存储在数组中的二进制值将被无意地当成内存地址。

注意 distanceArray() 返回类型为 void，这一开始看起来可能有点奇怪，但实际在 C 语言中是很正常的。由于 C 语言参数是按值传递的，所以一个函数无法直接修改它的任何参数。如果你想在一个 C 语言函数中修改一个实体（一个变量或者数组元素），传进去的参数应该是该实体的指针，而不是实体本身。指针（实体的内存地址）保持不变，但是函数可以在这个实体的位置存储一个新值；而且这一切都不需要函数返回任何值。（在第 3 章中，我们将看到有几类非常重要的函数的返回值类型只能是 void，不允许返回任何值。）

是时候生成并测试这个应用了。如果你在运行 Linux，那么过程和 dist_v1 只有一点点的不同。除了创建一个包含代码清单 C.5 至 C.7 中的代码文件 main.cpp、aux_functions.h 以及 aux_functions.cpp 的目录外，你还需要在 dist_v2 目录中创建一个新的 Makefile。代码清单 C.8 中给出了 dist_v2/Makefile 的代码。生成这个应用，用 cuda-gdb 在 main() 末尾的返回语句上设置一个断点，并验证结果和 dist_v1 中的一致。

代码清单C.8　dist_v2/Makefile

```
NVCC = /usr/local/cuda/bin/nvcc
NVCC_FLAGS = -g -G -Xcompiler -Wall

all: main.exe

main.exe: main.o aux_functions.o
    $(NVCC) $^ -o $@

main.o: main.cpp aux_functions.h
    $(NVCC) $(NVCC_FLAGS) -c $< -o $@

aux_functions.o: aux_functions.cpp aux_functions.h
    $(NVCC) $(NVCC_FLAGS) -c $< -o $@
```

在 Visual Studio 中，创建一个新的 dist_v2 工程，将 main.cpp 添加到工程中，并像你在 dist_v1 中所做的那样删除 kernel.cu。然后向工程中添加新文件 aux_functions.cpp 以及 aux_functions.h。（注意当你选择 Project ⇒ Add New Item ⇒ C++ 时，你可以选择添加一个 .h 文件，或者一个 .cpp 文件。）将代码清单 C.5 至 C.7 中的代码录入对应的文件中，并在 Debug 模式下编译。在 main() 底部的返回语句处设置一个断点，开始调试，并在 Locals 面板中查看结果。验证 in 数组中的元素从 0 均匀增长到 1，并且 out 数组中的距离值和 dist_v1 计算的值一致。

阅读消息和警告

如果你遵循上述指示，你的编译应该是成功的。但是有时候事与愿违，而我们希望为这种情况做好准备，因此让我们制造一个错误。将 aux_functions.cpp 中的 #include<math.h> 注释掉，并重新编译这个应用。编译器会产生一条错误信息，这里提醒你注意编译时产生的错误和警告。图 C.10 显示的错误信息出现在 Visual Studio 的 Output 面板中。用于指示整体成功或者失败的“末尾带线文字”非常重要，但是要习惯于查看其上部。从其上两行我们可以知道 aux_functions.cpp 的第 11 行发生了一个特定的错误（错误 C3861）。你现在拥有几个有用的工具来解决这个问题。你可以在搜索引擎中输入 error C3861 或 identifier not found 来找到额外的信息，例如“即使使用参数相关的查找，编译器也无法找到一个标识符的引用。”你甚至可以在 Output 面板中单击这个错误使光标移动到代码面板中对应的位置。

这些工具对于解决当前的问题可能有点大材小用，sqrt 无法识别仅仅是因为我们忘记了包含数学库头文件，但是我们强烈建议你在编译时养成仔细查看详细信息的习惯——而不仅仅是查看末尾带线文字。

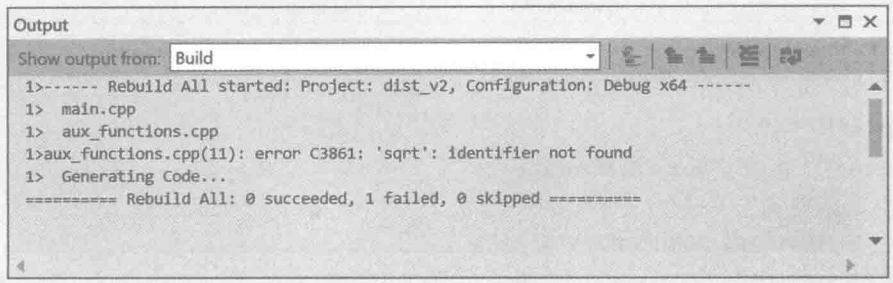


图 C.10 Visual Studio 的 Output 面板显示在代码编译时产生的错误信息

C.8.3 含有动态内存的 dist_v2

至此已经讨论了我们所需要的大部分 C 语言的关键概念，但是还有一个主题需要注意。我们目前所创建的数组都具有较小的固定长度，所以这些数组可以放入栈内存中。但是栈内存是有限的，而且当你将 dist_v2/main.cpp 中的 #define N 64 换成 #define N 20000000 时将引发段错误。对于更大的数组（而且大数组在 CUDA 中是非常重要的）或者在编译时不知其大小的数组，我们需要采取一个不同的方法。代码清单 C.9 中显示了一个使用动态内存管理的改进版 dist_v2 来处理大数组的方法。

代码清单C.9 带有动态内存管理的dist_v2/main.cpp

```
#include "aux_functions.h"
#include <stdlib.h> // supports dynamic memory management

#define N 20000000 // A large array size

int main()
{
  float *in = (float*)calloc(N, sizeof(float));
  float *out = (float*)calloc(N, sizeof(float));
  const float ref = 0.5f;

  for (int i = 0; i < N; ++i)
  {
    in[i] = scale(i, N);
  }

  distanceArray(out, in, ref, N);

  // Release the heap memory after we are done using it.
  free(in);
  free(out);
  return 0;
}
```

修改后的代码只包含少量的变化：

- 包含 stdlib.h 头文件来支持对动态内存管理函数的调用。
- 在第 8 行和第 9 行我们通过显式的声明指针 *in 和 *out，并用 (float*)calloc(N, sizeof(float)) 的返回值给它们赋值，来创建数组，这样会分配（并初始化为 0）足够的连续内存来存储 N 个浮点数。调用 calloc 会返回所分配内存的起始位置，并且 (float*) 会将返回值转换成“指向浮点类型的指针”类型。最终结果是数组的名字又是一个指向数组在内存中起始位置的指针；但是现在它是堆内存而不是栈内存，所以数组大小可以很大，而且可以在运行时决定。
- 在第 20 行和第 21 行，我们释放了分配用于存储数组的内存。

将 dist_v2/main.cpp 中的代码替换成代码清单 C.9 中的修改版本，在调试模式下编译，在第 20 行（在计算之后，内存释放之前）设置一个断点，并开始调试。

注意，查看堆中数组值跟栈中数组的过程不一样。在 Linux 中使用 cuda-gdb，out[5000]@100 命令将打印从第 5000 个元素开始的 100 个数组元素。在 Visual Studio 中，数组元素不能再通过 Locals 面板（只能查看栈内存）来查看了，而是使用 Watch 窗口。（如果还没有打开 Watch 窗口则选择 DEBUG ⇒ Windows ⇒ Watch ⇒ Watch1。）在 Watch 窗口的 Name 列输入 out+5000,100 来显示从第 5000 个元素开始的 100 个元素。（我们在这里用到了一点指针运算，但是不要陷入这里的细节。）你可以再次单击名字旁的三角形来查看按行显示的元素。

看完这篇附录后你应该了解 C 语言编程的基础了，包括声明和定义变量和函数、简单的控制结构以及基础的数组操作。你应该也对出现在第 1 章中的样例应用 dist_v1 和 dist_v2 有了基本的理解，这些例子也为我们在第 3 章中使用 CUDA 进行并行化提供了初始学习样例。

参考文献

[1] Mike McGrath, *C Programming in Easy Steps, Fourth Edition* (Southam, Warwickshire, UK: In Easy Steps Limited, 2012).

[2] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Zone, CUDA Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#abstract.

[3] David Griffiths and Dawn Griffiths, *Head First C* (Sebastopol, CA: O'Reilly Media, 2012).

[4] Zed A. Shaw, *Learn C the Hard Way* (New York, NY: Addison-Wesley, 2016).

[5] Brian Kernighan and Dennis M. Ritchie, *The C Programming Language, Second Edition* (Englewood Cliffs, NJ: Prentice Hall, 1988).

[6] "C++ Keywords," cppreference.com, 2015, http://en.cppreference.com/w/cpp/keyword.

[7] Free Software Foundation, *GNU Make*, 2013, http://www.gnu.org/software/make/manual/make.html.

附录 D Appendix D

CUDA 实践技巧：计时、性能分析、错误处理与调试

本附录将着重介绍一些 CUDA 实践中的实用技巧，帮助读者学习一些能够在以后的项目开发中用到的技术。我们将着重介绍以下几部分内容：

- 运行计时与性能分析
- 错误处理
- CUDA 调试工具

D.1 运行计时与性能分析

开始创建 CUDA 程序时，你可能会想量化代码的性能（如果你一点都不在意性能，那你的 CUDA 可能还没入门）。虽然标准 C 语言也有相关计时方法，但由于 CPU 与 GPU 之间同步的问题可能造成测时不准确，所以我们也将对基于 CUDA C 和 Nsight 的 CUDA 提供的测时方法进行介绍。

D.1.1 标准 C 语言计时函数

C 语言当前版本中包含一个头文件 <time.h>，该文件中定义了一个时间变量 clock_t，一个获取程序到目前为止的运行时间的函数 clock()，以及一个将 clock() 函数结果转换为

以秒为单位的常量 CLOCKS_PER_SEC。通过一个简单的例子我们就能最直观地了解这种方法。本书第 3 章中有一个示例程序“ dist_v2_cuda”，该程序用标准 C 语言计时方法测量了一个核函数的执行时间以及输入数据从主机端复制到设备端的时间。

代码清单 D.1 罗列了 dist_v2_cuda/kernel.cu 文件中的计时代码。修改 main.cpp 文件中数组大小定义的代码 #define N 256000，可使 kernel.cu 文件中测试的时间产生改变。

代码清单D.1 dist_v2_cuda/kernel.cu插入的cpu计时代码用于测量把输入数据从主机端复制到设备端的耗时

```
#include "kernel.h"
#include <stdio.h>
#include <time.h>
#define TPB 32
#define M 100 // Number of times to do the data transfer

__device__
float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

__global__
void distanceKernel(float *d_out, float *d_in, float ref)
{
  const int i = blockIdx.x*blockDim.x + threadIdx.x;
  const float x = d_in[i];
  d_out[i] = distance(x, ref);
}

void distanceArray(float *out, float *in, float ref, int len)
{
  float *d_in = 0;
  float *d_out = 0;
  cudaMalloc(&d_in, len*sizeof(float));
  cudaMalloc(&d_out, len*sizeof(float));

  // Record the clock cycle count before data transfer.
  clock_t memcpyBegin = clock();
  // Copy input data from host to device M times.
  for (int i = 0; i < M; ++i)
  {
    cudaMemcpy(d_in, in, len*sizeof(float),
               cudaMemcpyHostToDevice);
  }
  // Record the clock cycle count after memory transfer.
  clock_t memcpyEnd = clock();

  clock_t kernelBegin = clock();
  distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
  clock_t kernelEnd = clock();

  cudaMemcpy(out, d_out, len*sizeof(float), cudaMemcpyDeviceToHost);

  // Compute time in seconds between clock count readings.
  double memcpyTime =
    ((double)(memcpyEnd - memcpyBegin))/CLOCKS_PER_SEC;
  double kernelTime =
```

```
49      ((double)(kernelEnd - kernelBegin))/CLOCKS_PER_SEC;
50
51    printf("Kernel time (ms): %f\n", kernelTime*1000);
52    printf("Data transfer time (ms): %f\n", memcpyTime*1000);
53
54    cudaFree(d_in);
55    cudaFree(d_out);
56  }
```

以下是这段代码的一个小结：

- 通过 #include<time.h> 这行代码引用标准计时库函数。
- 常量 M 定义了数据重复传输的次数。
- distanceArray() 中所做修改包括：
  - 第 29 行与第 37 行的代码 memcpyBegin = clock() 与 memcpyEnd = clock() 保存了内存复制的初始 CPU 时间与结束 CPU 时间。
  - 使用 for 循环对复制操作循环执行 M 次。
  - 第 39 行与第 41 行的代码 kernelBegin = clock() 与 kernelEnd = clock() 保存了核函数执行的初始 CPU 时间和结束 CPU 时间。
  - 将记录的 CPU 时间差的单位都转换成秒。

在搭载 NVIDIA GeForce GT 650M 的 OS X 系统上运行该示例程序，得到的结果如下：

```
Kernel time (ms): 0.032000
Data transfer time (ms): 41.790000
```

数据的传输时间是正确的，但核函数的执行时间却是错误的。为什么这个计时方法对内存传输有效，对核函数执行却无效了？这个问题涉及一个的重要概念，在 CUDA 程序中，有两类完全不同的操作：

- 同步操作：对计算流进行阻塞，防止其他操作的执行。
- 异步操作：在异步操作执行的同时允许其他操作的执行。

数据复制默认是同步的，所以当 cudaMemcpy() 执行完毕时 CPU 才允许继续执行如读取时钟和读取结束时间等其他操作。但另一方面，核函数的执行却是异步的。核函数一启动，CPU 就会继续执行其他任务，它会在核函数完全执行完毕之前就读取时钟的值，并将其赋给 kernelEnd。接下来我们将介绍 CUDA 提供计时方法，其主要基于 CUDA 事件，对同步和异步操作均能计时。

D.1.2　CUDA 事件

CUDA 拥有一套自己的计时机制，以防止 CPU 与 GPU 之间同步导致的问题并提供

更精准的测量。在 CUDA 的 API 中，有一个特殊的数据类型 cudaEvent_t，以及如下所示的几个与 CUDA 事件相关的关键函数：

- cudaEventCreate() 与 cudaEventDestroy() 负责创建和销毁事件（命名具有顾名思义的效果）。
- cudaEventRecord() 负责记录时间。
- cudaEventSynchronize() 用于确保异步函数全都执行完毕。
- cudaEventElapsedTime() 负责将两个事件记录转成运行时间（以毫秒为单位）。

再次强调，了解这些函数的最好方法就是实践，因此我们将直接在 distanceArray() 函数中使用 CUDA 事件来计时。代码清单 D.2 为 kernel.cu 文件的升级版，其主要采用 CUDA 事件 API 对数据从主机端复制到设备端以及核函数的执行进行计时。

代码清单D.2 dist_v2_cuda/kernel.cu，使用CUDA事件对内存从主机端复制到设备端以及核函数执行的计时方案进行了修改

```
#include "kernel.h"
#include <stdio.h>
#define TPB 32
#define M 100 // Number of times to do the data transfer

__device__
float distance(float x1, float x2)
{
  return sqrt((x2 - x1)*(x2 - x1));
}

__global__
void distanceKernel(float *d_out, float *d_in, float ref)
{
  const int i = blockIdx.x*blockDim.x + threadIdx.x;
  const float x = d_in[i];
  d_out[i] = distance(x, ref);
}

void distanceArray(float *out, float *in, float ref, int len)
{
  // Create event variables for timing.
  cudaEvent_t startMemcpy, stopMemcpy;
  cudaEvent_t startKernel, stopKernel;
  cudaEventCreate(&startMemcpy);
  cudaEventCreate(&stopMemcpy);
  cudaEventCreate(&startKernel);
  cudaEventCreate(&stopKernel);

  float *d_in = 0;
  float *d_out = 0;
  cudaMalloc(&d_in, len*sizeof(float));
  cudaMalloc(&d_out, len*sizeof(float));

  // Record the event that "starts the clock" on data transfer.
  cudaEventRecord(startMemcpy);
```

```
37   // Copy input data from host to device M times.
38   for (int i = 0; i < M; ++i)
39   {
40     cudaMemcpy(d_in, in, len*sizeof(float),
41                cudaMemcpyHostToDevice);
42   }
43   // Record the event that "stops the clock" on data transfer.
44   cudaEventRecord(stopMemcpy);
45
46   // Record the event that "starts the clock" on kernel execution.
47   cudaEventRecord(startKernel);
48   distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
49   // Record the event that "stops the clock" on kernel execution.
50   cudaEventRecord(stopKernel);
51
52   // Copy results from device to host.
53   cudaMemcpy(out, d_out, len*sizeof(float), cudaMemcpyDeviceToHost);
54
55   // Ensure timed events have stopped.
56   cudaEventSynchronize(stopMemcpy);
57   cudaEventSynchronize(stopKernel);
58
59   // Convert event records to time and output.
60   float memcpyTimeInMs = 0;
61   cudaEventElapsedTime(&memcpyTimeInMs, startMemcpy, stopMemcpy);
62   float kernelTimeInMs = 0;
63   cudaEventElapsedTime(&kernelTimeInMs, startKernel, stopKernel);
64   printf("Kernel time (ms): %f\n", kernelTimeInMs);
65   printf("Data transfer time (ms): %f\n", memcpyTimeInMs);
66
67   cudaFree(d_in);
68   cudaFree(d_out);
69 }
```

注意，在这段代码中我们创建了四个事件：startMemcpy 与 stopMemcpy 用于测量数据复制时间，startKernel 与 stopKernel 用于测量核函数执行时间。在需要计时的每个操作（代码段）执行之前和执行之后运行 cudaEventRecord() 函数，即便是异步的核函数启动也不例外。在使用 cudaEventElapsedTime() 将事件记录转化为时间之前，调用 cudaEventSynchronize() 函数进行同步。在搭载 NVIDIA GeForce GT 650M 的 OS X 系统上运行该示例程序，得到的结果如下：

```
Kernel time (ms): 1.066176
Data transfer time (ms): 42.495968
```

注意，数据传输时间与标准 C 方法计时的结果基本一致，但是核函数的执行时间比标准 C 方法测量的时间更精准，标准 C 方法测量的时间小了 30 倍，丢掉了大部分核函数的执行时间。

D.1.3　使用 NVIDIA Visual Profiler 进行性能分析

NVIDIA Visual Profiler（NVVP）是一款跨平台的可视化性能分析工具，其包含在

CUDA Toolkit 中。此处我们将只对 NVVP 进行简要介绍，读者可以阅读其文档了解详情，文档地址为 https://docs.nvidia.com/cuda/profiler-users-guide/index.html。

启动 NVVP（在 Linux 或 OS X 操作系统中使用命令行输入 nvvp，Windows 操作系统中双击 NVVP 图标即可），NVVP 的主窗口如图 D.1 所示。

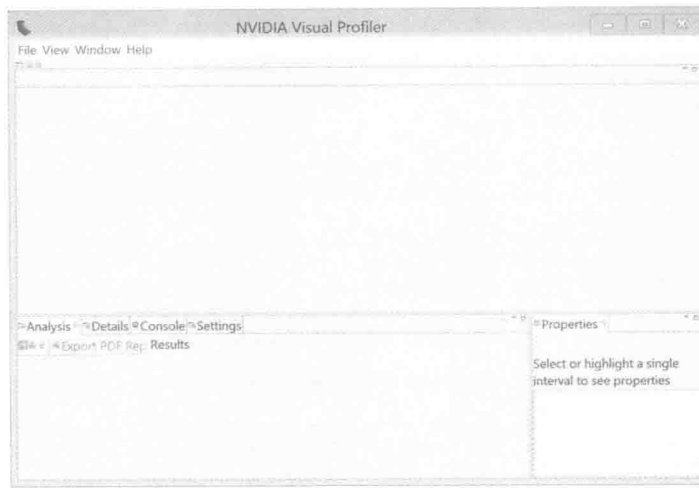


图 D.1 NVVP 主窗口

一旦界面出现，从 File 菜单下单击 New Session，将弹出一个 Create New Session 的对话框，如图 D.2 所示。

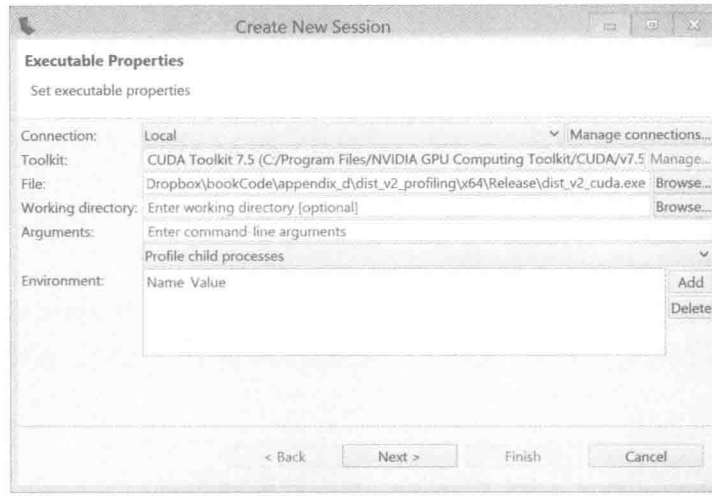


图 D.2 选择可执行程序 dist_v2_cuda.exe（如果你在 Linux 下使用 Makefile 编译，会得到可执行程序 dist_v2_cuda/main.exe）创建新会话的对话框

进入想要进行性能分析的 CUDA 程序的文件夹下，选择相应可执行程序，单击 Next（该对话框也能对那些需要命令行参数的程序进行参数配置）。之后会进入 Executable Properties 页面，保持默认选项，单击 Finish。

除非 Run guided analysis 选项未被选中，否则 NVVP 将自动执行该程序，并生成一个主机端和设备端的时间轴。查看标为 compute 的标签行，在时间轴的右侧可以看到一些信息。使用缩放工具和滚动条可以看清其为一个条形块，代表核函数的执行。单击条形块选择 distanceKernel() 核函数（此时其他条目颜色会黯淡），查看 Properties 标签可以看到核函数的各种状态信息，如核函数的执行时间 Duration，该执行时间单位为毫秒。

除了时间轴，在 NVVP 窗口下方提供了各种标签页，如 Analysis、Details、Console 以及 Settings。查看 Analysis 标签页，可以看到有两个图标，一个是数字列表图标，一个是无序符号列表图标，分别表示 guided analysis 与 unguided analysis。用户可以在 guided 模式下逐步获取不同的性能分析结果，也可以在 unguided 模式下根据自己的需要获取指定的性能分析结果。图 D.3 显示了对 dist_v2_cuda 程序进行性能分析的截图：对应时间轴选中 Default 行、Details 标签页的信息。该标签页提供了在程序进行性能分析时每次内存复制操作的时间以及每个核函数启动的时间。

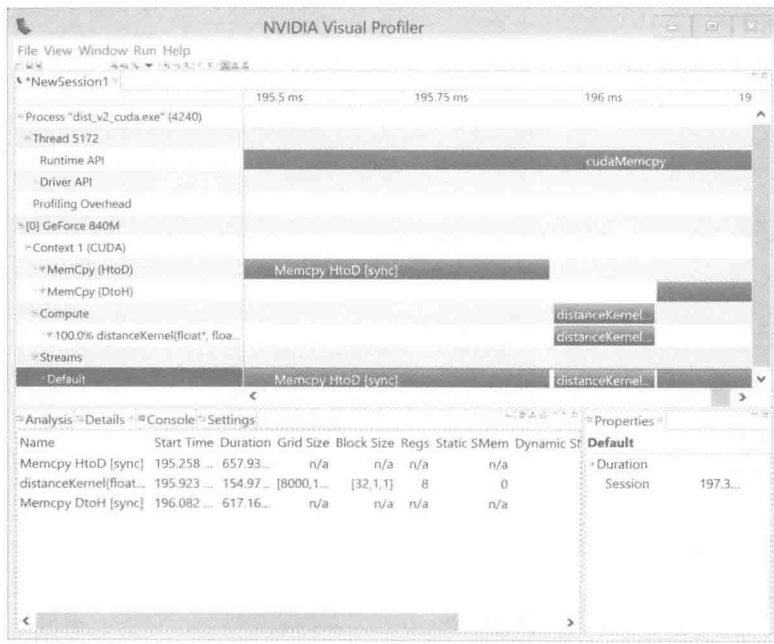


图 D.3　Details 标签页显示了 dist_v2_cuda 程序内存复制操作的时间以及核函数启动的时间

若要查看核函数的性能，可选择 Analysis 标签页下的 guided analysis 图标，单击 Examine Individual Kernels。本例只有一个核函数，其占据了 GPU 全部的计算时间。单击之后会出现一个 Performance Critical Kernel 列表。选定需要分析的核函数，单击 Perform Kernel Analysis。NVVP 窗口中显示 Performance is Bound by Memory Bandwidth，表明 distanceKernel() 核函数只使用少部分计算资源，并且执行时间也只占总体执行时间的一小部分，绝大部分时间花在了内存传输上。另外，NVVP 还提供了 Perform Memory Bandwidth Analysis、Perform Compute Analysis 以及 Perform Latency Analysis 等其他几种性能分析功能。内存带宽分析提供了内存传输的相关统计数据并提供优化性能的一些建议。计算分析也提供了 SM 使用情况的相关统计数据以及高效利用 SM 资源的相关建议。延迟分析提供了与所请求数据的可用性相关的统计信息和建议。

dist_v2_cuda 程序需要传输大量的数据（几十万个浮点数在主机端与设备端之间往返传输），然后每个线程计算一个距离值。内存带宽分析显示执行时间被主机端与设备端内存传输最大速率限制了，程序基本没有可提升的空间，因此 dist_v2_cuda 程序基本无需再进行性能优化。第 5 章中提供了一个示例程序，该程序的主要瓶颈在于内存传输，并通过使用分块与共享内存获得了较大的性能提升。

为了保证完整性，我们也尝试使用 unguided analysis 模式来进行性能分析查看相关信息，如图 D.4 所示。尽管设备内存带宽达到了最大性能的 95%，但内存带宽依然被认为是程序性能的限制瓶颈。

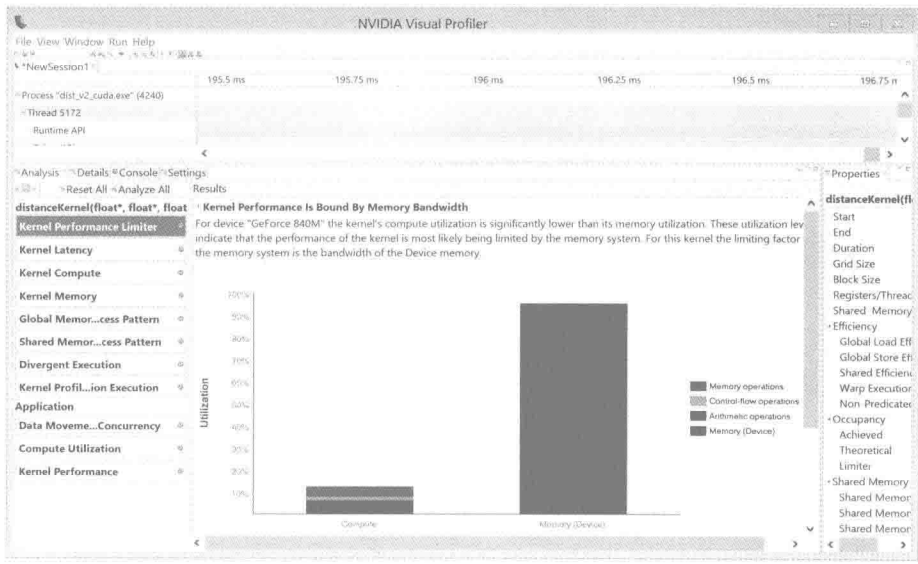


图 D.4　NVVP 在 unguided analysis 模式下对 dist_v2_cuda 程序性能分析的信息

D.1.4　使用 Nsight Visual Studio 进行性能分析

Nsight Visual Studio 也能生成运行时间和性能分析信息（若读者希望在一台机器上开发 CUDA 程序，在另一台机器上运行该程序，则请详细阅读 Nsight 文档[1]）。打开 Visual Sutdio，选择 NSIGHT ⇒ Start Performance Analysis，然后会出现一个 Activity 页面（弹出这个页面之前可能先会出现一个系统授权运行 Nsight Monitor 的对话框），该页面包含若干个窗口，如图 D.5 所示。

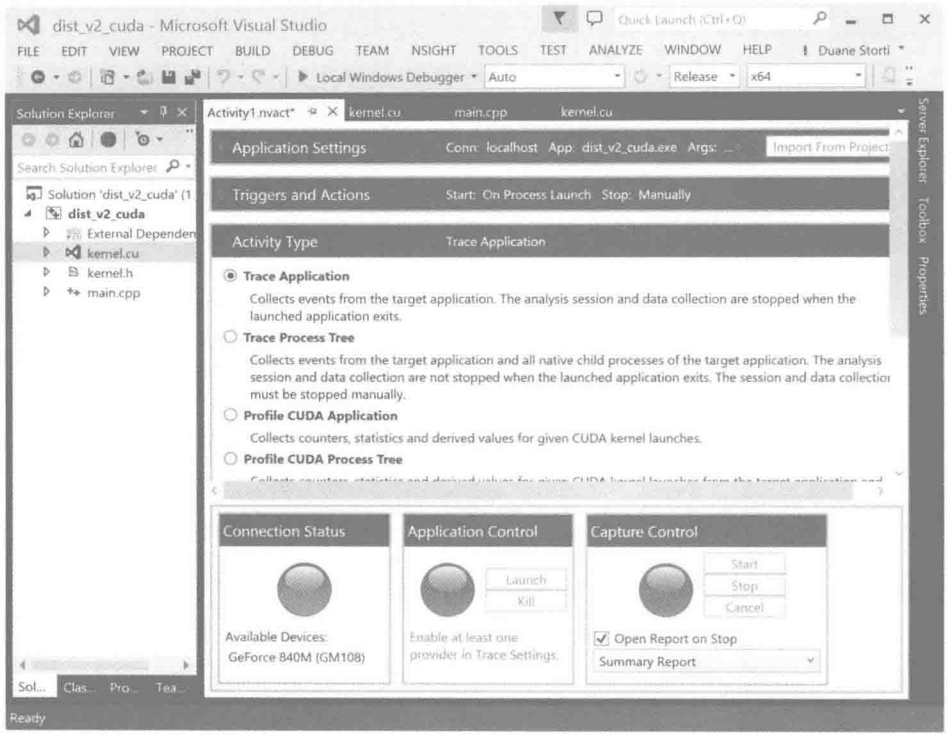


图 D.5　选择 NSIGHT ⇒ Start Performance Analysis 之后出现的 Activity 页面

- 在活动页面的顶部，有一个 Application Settings 窗口，该窗口中的默认设置不需要修改。
- 同样也不要修改 Triggers and Actions 窗口中的选项。
- 重要的选项都在 Activity Type 窗口中，其提供了四个选项：
  - Trace Application
  - Trace Process Tree

- Profile CUDA Application
- Profile CUDA Process Tree

我们将从默认选项 Trace Application 开始介绍，选中该选项后程序执行时 Nsight 会收集相关信息并在程序执行结束时生成一份报告。当选中 Trace Application 或 Trace Process Tree 选项时，可以对下方的 Trace Settings 下拉框中的选项进行设置，选择需要收集的信息，如图 D.6 所示。我们现在的目的是获取 CUDA 性能分析信息，因此需要选中 CUDA 这个复选框（你也可以单击 CUDA 复选框旁边的小三角形，选择 CUDA 这一栏下的子选项，收集指定信息）。

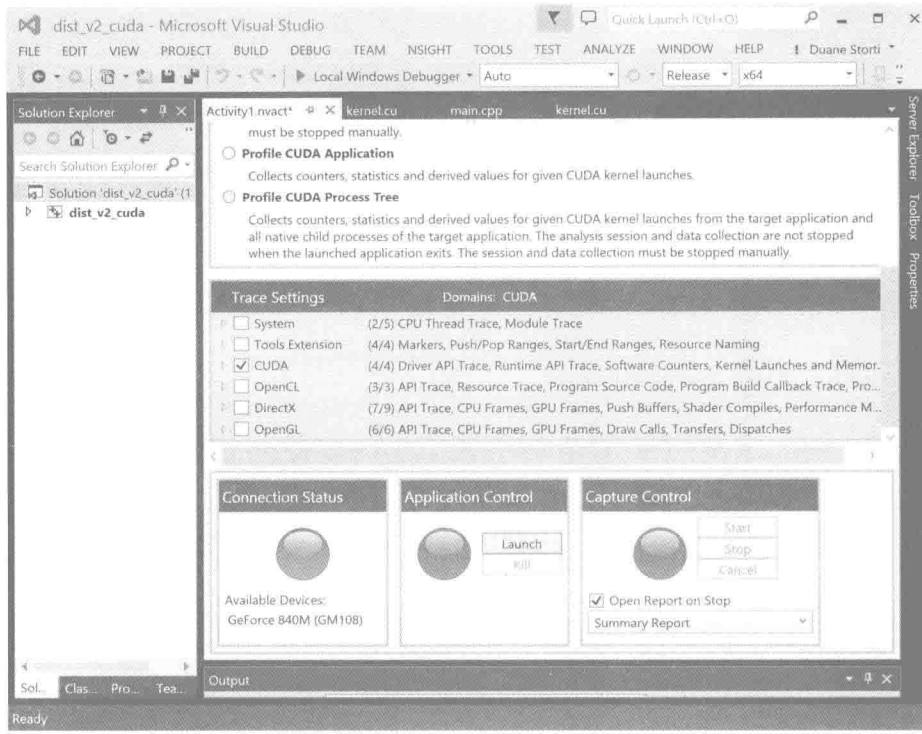


图 D.6 选择 Trace Application 并下拉展开 Trace Settings 选中 CUDA 选项时的 Activity 页面

在 Trace Settings 下方有三个小窗口：

- 左边是 Connection Status 窗口，在一个绿色的圆球下方列出了所有可用的 GPU 设备（假定 Nsight 已经成功地连接上当前 GPU）。
- 中间是 Application Control 窗口，其提供了一个 Launch 按钮控制程序的使用。
- 右边是 Capture Control 窗口，该窗口可以手动控制性能分析的开始与结束。

当点击 Application Control 窗口的 Launch 按钮时，应用程序会开始运行，然后会出现一个活动报告窗口，窗口中的 Summary Report 报告显示了收集到的性能数据。Session Overview 报告了收集到的所有数据的整体运行时间。CUDA Overview 显示了在收集信息这段时间内 CPU 与 GPU 的占用比。在显示 Summary Report 的下拉菜单中还有一些选项可供查看。其中我们最感兴趣的就是 Timeline 了，它通过可视化的方式显示了在程序执行过程中和 CUDA 相关的各种操作，如图 D.7 所示。用户可以通过按住 Ctrl 键滑动鼠标滚轮，拖曳滚动条来查看时间轴上自己感兴趣的时刻。

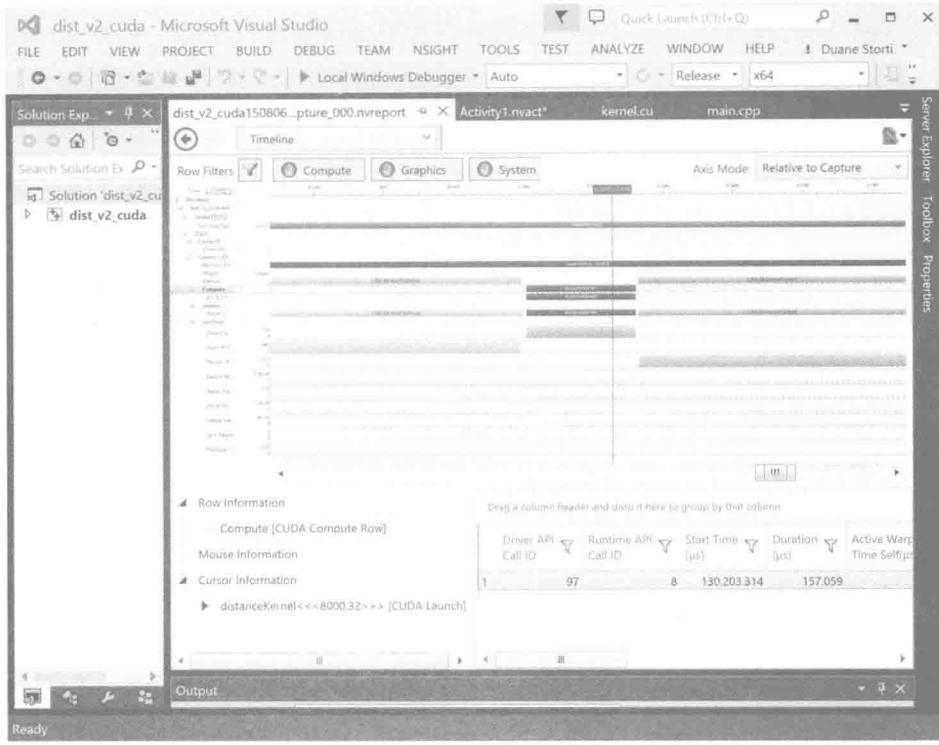


图 D.7　dist_v2_cuda 程序生成的时间轴

创建了 dist_v2_cuda 程序的时间轴之后，你可以看到 cudaMemcpy() 函数的调用，还可通过对时间轴进行缩放，来查找自己需要的核函数。如果单击时间轴上对应的条形块（如 Compute 行中标记为 distanceKernel 的条形块），然后单击下方下拉项 Row Information 的 Compute 子项，即可查看核函数相关的详细信息，包括执行时间等（测量的时间与 NVVP 测量的时间基本相同，只相差几毫秒）。

到目前为止，我们已经能够获取核函数的执行时间，接下来我们将对其他的实践技巧进行介绍。

D.2 错误处理

本书使用的 CUDA 函数属于 CUDA 运行时 API[2]。CUDA 运行时 API 函数会返回一个 cudaError_t 类型的数据，用以检查函数调用是否有错误[3]。与计时部分的讨论一样，同步函数与异步函数的处理方式也有所不同。

同步函数的处理方式比较简单，因为只有在同步函数执行结束之后才能继续执行下一个操作，所以无论可能出现何种错误，都将在其他错误出现之前就已出现，不会定位不准错误。因此，对于同步类型的函数，我们只需将其函数调用的返回值赋给一个 cudaError_t 类型的变量即可。例如，为一个输入数组分配一块设备内存：

```
cudaMalloc(&d_in, len*sizeof(float));
```

为了在调用该操作出现错误时发出警告，可把函数调用作为赋值操作的右侧，赋给一个 cudaError_t 类型的变量（此处该变量名为 err）。然后使用一个 if 条件语句判断 err 是否等于 cudaSuccess，若等于则表示没有错误产生，否则，调用 cudaGetErrorString() 函数，返回该错误的字符串表达方式。这段代码如下所示，代码主要完成了内存分配、错误检查以及获取错误描述（如果存在错误）并输出到屏幕。

```
cudaError_t err = cudaMalloc(&d_in, len * sizeof(float));
if (err != cudaSuccess) printf("%s\n", cudaGetErrorString(err));
```

如果不想在程序发生错误之后不知所措，那么尽量养成习惯，在调用每个 CUDA 函数的时候都加上这段代码。

注意，如果要重复使用一个变量进行错误检查，那么在第一次使用这个变量的时候必须对其类型进行声明，即变量声明，之后使用时就只需要将函数调用的返回值赋给该变量即可，无需再进行变量声明。上面这段代码为 d_in 分配了一块显存并进行了错误检查，同理，d_out 的显存分配和错误检查亦可如此。

```
err = cudaMalloc(&d_out, len * sizeof(float));
if (err != cudaSuccess) printf("%s\n", cudaGetErrorString(err));
```

如果你想让这段代码稍微简洁一些，可以从 CUDA Samples 中包含一个名为 helper_cuda.h 的头文件，该头文件可以在示例文件夹下的 common/inc 子文件夹下找到，每次调用 CUDA 函数时，可以使用 checkCudaErrors() 函数将调用封装起来，这样调用出现错误

时就会在标准输出中打印错误信息。在代码文件的顶部加入 #include <helper_cuda.h> 这行代码，分配显存并进行错误检查的代码就可以写成下面这种形式：

```
checkCudaErrors(cudaMalloc(&d_in, len * sizeof(float)));
```

核函数的错误处理方式稍有不同。首先核函数是 void 类型，没有返回值，也就没有错误值，所以我们只能通过其他函数调用来获得错误值。有两种独立的错误需要我们处理，一种是核函数启动时出现的同步错误（例如，在核函数启动时显存和计算网格的大小超过了可用范围），另一种是在核函数执行期间出现的异步错误。我们可以通过在核函数启动之后立刻调用 cudaGetLastError() 或 cudaPeekAtLastError() 内置函数来获得同步错误。而对于异步错误，我们必须调用 cudaDeviceSynchronize() 确保核函数已经执行完毕。下面这段代码展示了核函数启动时同步错误和异步错误的处理方式：

```
distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
cudaError_t errSync = cudaGetLastError();
cudaError_t errAsync = cudaDeviceSynchronize();
if (errSync != cudaSuccess)
  printf("Sync kernel error: %sn", cudaGetErrorString(errSync));
if (errAsync != cudaSuccess)
  printf("Async kernel error: %sn", cudaGetErrorString(errAsync));
```

使用 helper_cuda.h 头文件中的函数可以让代码更简洁：

```
distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
checkCudaErrors(cudaPeekAtLastError());
checkCudaErrors(cudaDeviceSynchronize());
```

这段代码的第一行代码是核函数的调用，第二行代码提供了核函数启动时可能出现的错误，第三行代码提供了核函数执行时可能出现的错误。虽然错误检查在开发时很有帮助，但进行错误检查也有一定代价，那就是降低了程序的性能，特别是 cudaDeviceSynchronize() 的调用，它会阻塞线程直到核函数执行完毕。为了高效的开发，同时也保证程序的性能，许多开发者只在开发的过程中使用错误检查，而在程序的发布版本中不使用。代码清单 D.3 为 dist_v2_cuda 应用程序的 kernel.cu 文件中的错误检查代码。

代码清单D.3　为dist_v2_cuda/kernel.cu代码加入错误处理

```
 1 #include "kernel.h"
 2 #include <helper_cuda.h>
 3 #define TPB 3200
 4
 5 __device__
 6 float distance(float x1, float x2)
 7 {
 8   return sqrt((x2 - x1)*(x2 - x1));
 9 }
10
```

```
11 __global__
12 void distanceKernel(float *d_out, float *d_in, float ref)
13 {
14   const int i = blockIdx.x*blockDim.x + threadIdx.x;
15   const float x = d_in[i];
16   d_out[i] = distance(x, ref);
17 }
18
19 void distanceArray(float *out, float *in, float ref, int len)
20 {
21   float *d_in = 0;
22   float *d_out = 0;
23   checkCudaErrors(cudaMalloc(&d_in, len*sizeof(float)));
24   checkCudaErrors(cudaMalloc(&d_out, len*sizeof(float)));
25
26   checkCudaErrors(cudaMemcpy(d_in, in, len*sizeof(float),
27                              cudaMemcpyHostToDevice));
28
29   distanceKernel<<<len/TPB, TPB>>>(d_out, d_in, ref);
30   checkCudaErrors(cudaPeekAtLastError());
31   checkCudaErrors(cudaDeviceSynchronize());
32
33   checkCudaErrors(cudaMemcpy(out, d_out, len*sizeof(float),
34                              cudaMemcpyDeviceToHost));
35   checkCudaErrors(cudaFree(d_in));
36   checkCudaErrors(cudaFree(d_out));
37 }
```

注意，由于我们包含了 helper_cuda.h 头文件，所以编译设置必须进行修改，以便于编译器能够找到该头文件。代码清单 D.4 为编译带错误处理的程序的 Makefile 文件的修改方式。对于 Visual Studio 用户，可参考 D.6 节，详细了解如何指定附加包含文件夹的路径。

代码清单D.4　带错误处理的dist_v2_cuda程序的Makefile文件

```
1 NVCC = /usr/local/cuda/bin/nvcc
2 NVCC_FLAGS = -g -G -Xcompiler -Wall
3 INC = -I/usr/local/cuda/samples/common/inc
4
5 all: main.exe
6
7 main.exe: main.o kernel.o
8   $(NVCC) $^ -o $@
9
10 main.o: main.cpp kernel.h
11   $(NVCC) $(NVCC_FLAGS) -c $< -o $@
12
13 kernel.o: kernel.cu kernel.h
14   $(NVCC) $(NVCC_FLAGS) $(INC) -c $< -o $@
```

现在，我们来看一个错误处理的例子。将代码的第 3 行 #define TPB 32 修改为 #define TPB 3200，重新编译工程并运行，会得到以下错误信息：

```
CUDA error at kernel.cu:30 code=9(cudaErrorInvalidConfiguration)
```

在 CUDA 运行时 API 文档的“Data types used by CUDA Runtime”小节找到“CUDA Error Types”查看所有错误编码，可以看到错误编码 9 是这样定义的：“This indicates that a kernel launch is requesting resources that can never be satisfied by the current device. Requesting more shared memory per block than the device supports will trigger this error, as will requesting too many threads or blocks。”（这表明当前设备资源无法满足核函数启动的条件。当每个线程块使用了过多的共享内存或者线程数、线程块数目超出限制时该错误会触发。）通过错误处理机制，我们得知由于每个线程块开启了过多的线程才造成错误。

最后，我们来看看如何避免错误。许多错误（一些错误会产生输出信息，一些错误则只是产生错误的计算结果）只需我们注意下待解决问题的规模和设备计算网格大小与设备性能相符即可避免。典型的，如我们需要处理一个指定规模（数组大小即可表示问题规模）的问题，我们只需确定需要多少个线程块能覆盖整个数组，并且确保在对数组进行读写访问时不会出现数组越界。通常只需要简单地做个整数除法再加个 if 判断就可以解决。

假设我们要对一个长度为 N 的数组进行计算，TPB 表示每个线程块的大小。如果 N 是 TPB 的整数倍，那么 N/TPB 就是需要开启的线程块的数目。但是，如果 N 比 TPB 的整数倍还大一点，那么则需要额外开启一个线程块处理数组末尾剩下的那一部分。

事实上，在计算时无需条件判断也能正确计算出线程块的数目，只需进行如下代码所示的简单整数除法即可：

```
GRIDSIZE = (N + TPB - 1)/TPB
```

注意，如果用标准的公式计算每个线程访问的数组索引，那最后一个线程块部分线程计算的索引将大于 N－1（长度为 N 的数组最大索引为 N－1）。此处就需要用到 if 条件判断。

在核函数里面，将计算部分放入 if(i ＜ N){} 条件判断语句内部执行，这样就能确保计算过程中访问的数据都是我们需要（有效）的数据。另外，也可以使用一种等价的方法来处理超出数组范围的部分，即当索引超出范围核函数直接返回。

```
if(i >= N) return;
```

同样我们也给出了一个具体的例子。调整后的核函数定义和实现代码如下所示：

```
__global__
void distanceKernel(float *d_out, float *d_in, float ref, int len)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if(i >= len) return;
```

```
    d_out[i] = distance(d_in[i], ref);
}
distanceKernel<<<((len + TPB - 1)/TPB, TPB>>>(d_out, d_in, ref, len);
```

确保核函数启动参数的有效性稍微有些复杂，因为不同设备的限制条件（一个线程块能开启的最大线程数）不同。另外，代码的可用性也与设备相关，例如双精度计算、动态并行就受限于运行代码设备的计算能力。当你开发一个程序时，你可以选择要使用的CUDA特性，但你无法决定用户使用何种GPU。为了解决这个问题，CUDA提供了以下几个函数用于检查安装的设备，看其是否满足程序运行的要求：

- cudaGetDeviceCount()函数以一个int类型变量的地址作为输入，将当前可用CUDA设备的数目返回赋给该int变量。
- cudaGetDeviceProperties()函数以cudaDeviceProp结构体变量地址和设备编号的整型值作为输入，将该设备相关属性写入到cudaDeviceProp结构体中。

代码清单D.5为获取CUDA设备数量以及设备属性的相关代码。

代码清单D.5 查询指定CUDA设备属性的代码

```
#include <stdio.h>

int main() {
    int numDevices;
    cudaGetDeviceCount(&numDevices);
    printf("Number of devices: %d\n", numDevices);
    for (int i = 0; i < numDevices; ++i) {
        printf("---------------------------\n");
        cudaDeviceProp cdp;
        cudaGetDeviceProperties(&cdp, i);
        printf("Device Number: %d\n", i);
        printf("Device name: %s\n", cdp.name);
        printf("Compute capability: %d.%d\n", cdp.major, cdp.minor);
        printf("Maximum threads/block: %d\n", cdp.maxThreadsPerBlock);
        printf("Shared memory/block: %lu bytes\n",
               cdp.sharedMemPerBlock);
        printf("Total global memory: %lu bytes\n",
               cdp.totalGlobalMem);
    }
}
```

现在我们就可以检测运行代码的目标机器的相关硬件限制，然后根据需要做出调整或发出警告。关于cudaDeviceProp结构体的成员变量，可参考CUDA运行时API了解更多详细内容，或者阅读CUDA Samples中的deviceQuery示例程序代码了解如何获取你感兴趣的细节（如果使用的是Windows操作系统，你也可以通过Visual Studio的NSIGHT ⇒ SystemInfo ⇒ CUDA Devices查找属性信息）。

D.3　在 Windows 系统中调试

此处我们将对 Nsight 提供的一个特殊工具进行介绍，即 CUDA 代码调试工具。我们仍将采用 dist_v2_cuda 示例程序进行具体介绍。利用该工具查看核函数计算的每个值。首先需要注意的是，Nsight 是专门针对 CUDA 代码的软件，在使用 Nsight 调试工具时，断点必须设置在程序的 CUDA 代码部分。核函数毫无疑问是 CUDA 代码，因此我们可以将断点设置在 distanceKernel() 核函数的第一行（断点的设置与在 Visual Studio 中常规调试时一致）。调试时不是单击 DEBUG 菜单（或按 F5 键），而是单击 NSIGHT 菜单下的 Start CUDA Debugging。

程序会正常执行到核函数开始处的断点，然后会出现包含具体信息的 Locals（局部变量）窗口，类似于图 D.8 所示。

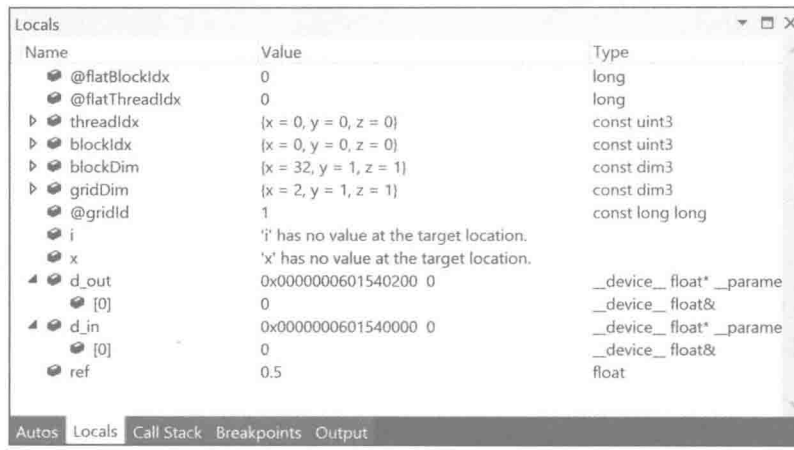


图 D.8　distanceKernel() 核函数第一行触发断点时的 Locals 窗口

但考虑到 CUDA 的计算特性，即能够实现大规模的并行，这迫使我们必须重新考虑调试的方法。由于许多线程同时在执行计算，每个线程都有自己独立计算出的局部值。许多线程同时并行执行，对于人而言，我们无法将每个线程的信息都显示出来进行查看，图 D.8 中 Locals 窗口显示的只是其中一个线程中变量和值的信息。可以看到，此处 blockIdx 为 0，threadIdx 为 0。此外，我们还可以看到列表中显示的其他变量，如输入数据的 d_in[0] 值为 0，还未计算的局部变量 i，以及输出数据 d_out[0]。

接着我们可以通过菜单选项（DEBUG ⇒ Step Into）或按快捷键（F11）进行单步跟踪。更新之后的 Locals 窗口如图 D.9 所示。此时变量 i 已参与了计算（通常是由

blockIdx、blockDim 与 threadIdx 联合算出）并被赋予值 0。之后我们便可查看第 0 到第 63 号这前 64 个线程的信息（注意，Locals 窗口上方的 @flatThreadIdx 是线程编号的另一种表达方式）。

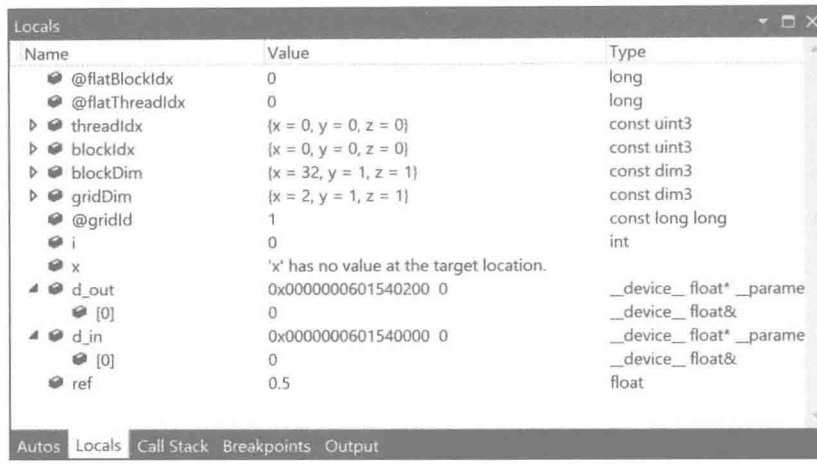


图 D.9　distanceKernel() 核函数单步执行到第 2 行时的 Locals 窗口

按 F11 单步执行一次，可以看到光标移动到了下一行执行代码，即 distance() 设备函数的调用。再按一次 F11 则进入了 distance() 函数内部 sqrt() 函数调用处，继续单步则将跳转至 math.h 文件中 sqrt() 函数原型处。然而调试标准数学库函数并不是我们的目的，所以我们需要从这段代码跳出而并不是继续单步执行。选择菜单选项（DEBUG ⇒ Step Out）或按快捷键（Shift+F11）就能单步跳出。第一次按 Shift+F11 快捷键跳回到 distance() 中，再按一次则会跳回到 distanceKernel() 函数中。对应的 Locals 窗口如图 D.10 所示。

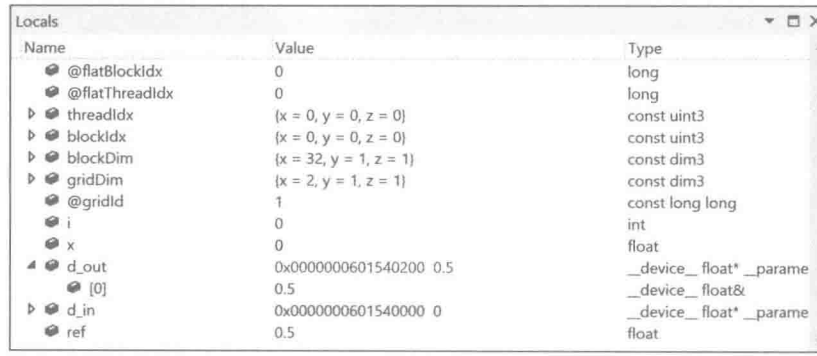


图 D.10　单步跳出到 distanceKernel() 函数的最后一行代码时的 Locals 窗口

我们可以看到目标值 0.5 已经计算出并保存到 d_out[0] 中，但如何查看其他线程中的该值？这时就需要使用 Nsight 的调试窗口了（选择菜单 NSIGHT ⇒ Windows）。

- System Info 窗口（该窗口是唯一一个无需开始 Start CUDA Debugging 就可以查看的窗口）提供的信息与当前任务无关，其主要帮助用户了解当前系统与当前设备的相关信息。
- CUDA Debug Focus 窗口可以允许用户查看指定线程的局部变量信息。例如，重启 CUDA 调试（单击 DEBUG ⇒ Stop Debugging 之后选择 NSIGHT ⇒ Start CUDA Debugging），然后选择 NSIGHT ⇒ Windows ⇒ CUDA Debug Focus，之后会弹出一个窗口，让你输入希望查看的线程块与线程索引。作为一个具体的例子，我们设置需要查看的线程块的索引为 0,0,0，而线程的索引由 0,0,0 变为 1,0,0。（由于我们的计算网格是一维的，索引的三维结构变量里只有第一维有效；对于多维计算网格，请阅读第 4 章和第 7 章。）单步进入计算部分，可以看到 i 值赋为了 1（这样我们的焦点就移到了 64 个线程的第二个线程），但是当我们展开 d_in 与 d_out 时，发现其显示的值仍然是索引 0 的值而不是索引 1 的值，这是因为 Locals 窗口显示的是局部变量的值，但对于每个线程而言，d_in 与 d_out 并不是局部变量。（每个线程只能对以核函数参数传入的指针进行访问，并且能直接访问的只有数组的初始元素，例如 d_in[0] 与 d_out[0]。）

解决这种限制的一种简单方法就是创建一个局部变量，然后将相关数据赋给当前线程的局部变量以便显示。例如，我们可以对 distanceKernel() 函数进行一定修改，用变量 y 和变量 d 分别表示输入位置和输出距离：

```
__global__
void distanceKernel(float *d_in, float *d_out, float ref, int len)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float y = d_in[i];
  float d = distance(y, ref);
  d_out[i] = d;
  printf("distance = %f\n", d_out[i]);
}
```

编译新版核函数，然后开始 CUDA 调试。我们可以通过修改 CUDA Debug Focus 中的线程块索引与线程索引，查看 Locals 窗口中的变量 y 与变量 d 来观察指定线程的输入与输出数据（注意，局部变量只是临时能够查看，若编译器进行过优化，可能无法查看）。到目前为止，我们依然相信计算出的结果 d 值是正确的保存在数组 d_out 中。编程经验告诉我们一个一般规则，相信计算机完全理解了你的意图是危险的，因此我们必须用一

种直观的方法核实存于 d_out 中的结果。让我们继续介绍 Nsight 的窗口，找到一个工具达到这个目的。

- 开始一个新的 CUDA 调试会话（单击 DEBUG ⇒ Stop Debugging 之后选择 NSIGHT ⇒ Start CUDA Debugging），选择 NSIGHT ⇒ Windows ⇒ CUDA Info，然后会出现一个窗口显示了计算相关的各种信息。目前，我们只查看两个最相关的选项。单击窗口中的下拉菜单，选择 Warps，然后会看到两行信息，分别是线程块为 0,0,0 与 1,0,0 的线程束相关信息，如图 D.11 所示。

图 D.11 CUDA Info 窗口中焦点在 threadIdx = {0,0,0} 且 blockIdx = {0,0,0} 项上时线程束相关信息

在最右侧可以看到有一组绿色的如盒子一样的标志，其表示计算单元（每个表示带有一个流处理器的计算单元），线程束中的每个线程都在这些单元中执行。红色盒子表示该单元中的线程执行到了断点处。红色盒子中可以看到一个黄色的箭头，表示当前 CUDA 调试主要集中在 threadIdx = {0,0,0} 且 blockIdx = {0,0,0} 的线程上。与 CUDA Debug Focus 窗口一样，双击该窗口中的绿色盒子，可以改变当前聚焦的线程。图 D.12 显示了改变当前焦点线程为 threadIdx = {3,0,0} 且 blockIdx = {1,0,0} 的线程，单步运行越过断点。可以看到黄色箭头也发生了改变，通过 Locals 窗口查看变量 i 对应的相关值。

```
blockIdx.x * blockDim.x + threadIdx.x = 1*32 + 3 = 35
```

回到下拉菜单，选择 Lanes，将看到当前焦点线程束的 32 个线程的相关信息，包括线程索引与活动状态。该窗口无法让我们查看保存在 d_out 数组的值，但在其他方面提供了有用信息，这些信息将来我们会用到，现在，我们将继续介绍下一个（也是最后一个）窗口。

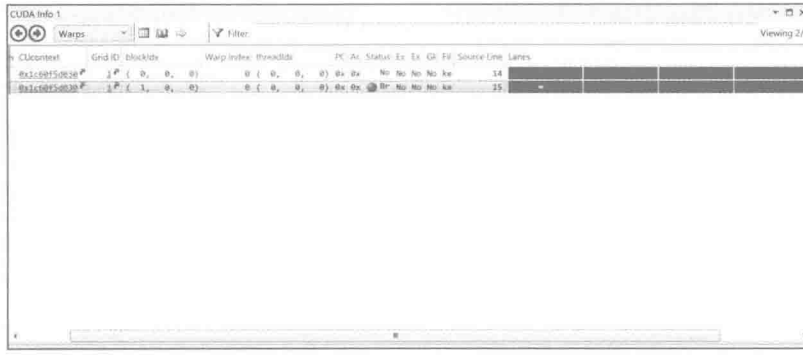


图 D.12　CUDA Info 窗口中焦点在 threadIdx = {3,0,0} 且 blockIdx = {0,0,0} 项上，单步越过断点时的相关信息

- 选择 NSIGHT ⇒ Window ⇒ CUDA Warp Watch ⇒ CUDA Warp Watch 1，将打开一个窗口，窗口左侧一列显示着 Name 与 Type，接着是 0 到 31 这些数。如果你看到 Name 与 Type 就能联想到 C 语言中变量的命名方式，说明你的直觉还不错。如果在 Name 的右列输入一个变量名，则这一列将显示出当前线程束每个线程中该变量的值。如果输入 i、d_in[i] 与 d_out[i] 这三个变量名，然后单步执行到 distanceKernel() 函数的最后一行代码，从 CUDA WarpWatch1 窗口中可以看到当前线程束中每个线程的索引值、输入值以及计算的距离输出值，如图 D.13 所示。

CUDA WarpWatch 1

Name	i	d_in[i]	d_out[i]	<Add Watch>
Type	int	__device__ float&	__device__ float&	
0	0	0	0.5	
1	1	0.015873017	0.48412699	
2	2	0.031746034	0.46825397	
3	3	0.047619049	0.45238096	
4	4	0.063492067	0.43650794	
5	5	0.079365082	0.42063493	
6	6	0.095238097	0.40476191	
7	7	0.11111111	0.3888889	
8	8	0.12698413	0.37301588	
9	9	0.14285715	0.35714287	
10	10	0.15873016	0.34126985	
11	11	0.17460318	0.32539684	
12	12	0.19047619	0.30952382	
13	13	0.20634921	0.29365081	
14	14	0.22222222	0.27777779	
15	15	0.23809524	0.26190478	
16	16	0.25396827	0.24603173	
17	17	0.26984128	0.23015872	
18	18	0.2857143	0.2142857	
19	19	0.30158731	0.19841269	
20	20	0.31746033	0.18253967	

图 D.13　执行到 distanceKernel() 函数最后一行时第一个线程束中 d_in[i] 与 d_out[i] 在 Warp Watch 1 窗口中的显示结果

若要查看其他线程束中的值，可利用之前介绍的 CUDA Debug Focus 或 CUDA Info 窗口进行选择（当切换到新线程束时，程序只执行到断点处，需再次使用单步跟踪执行到输出结果计算且保存的代码处）。

到目前为止，我们已经能从查看单个线程中的所有局部变量发展到同时查看一个线程束中的所有信息。若要一次性查看整个数组中的所有值，则需要直接查看数组存储的内存块。在核函数的最后一行代码设置一个断点，重新开启一个 CUDA 调试会话。选择 DEBUG ⇒ Windows ⇒ Memory ⇒ Memory 1（或使用快捷键 Alt+6）打开一个内存窗口。在地址栏输入 d_out,64，选择 4 列。窗口中第一列为十六进制的内存地址，其中并没有太多有价值的信息。如果窗口中其他列显示的不是直接能读的数，则可以在该窗口中右键单击，选择其他的显示方式，如 32-bit Floating Point（32 位浮点数），然后可以看到计算的距离值大约在 0.008 到 0.5 之间，如图 D.14 所示。

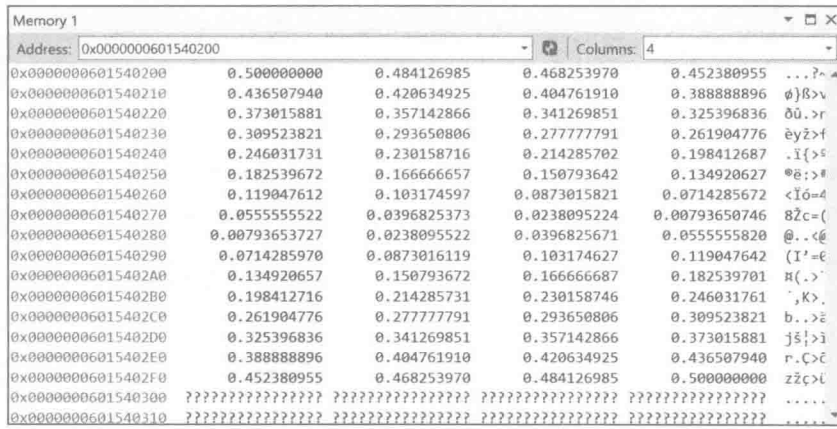


图 D.14 Memory 1 窗口中显示由 distanceKernel() 函数计算且保存在 d_out 中的 64 个值（共有 4 列）

至此，我们已经能够查看变量的值。无论该变量是线程中的局部变量还是保存在数组中，在 CUDA 代码执行的各个阶段我们都能查看。更多详细信息和其他示例，可参考“Nsight Visual Studio Edition”中的“Using the CUDA Debugger”。[4]

D.4 在 Linux 系统中调试

对于 Linux 操作系统，NVIDIA Nsight Eclipse Edition 提供了一个使用方便的 GUI 前端界面。在本节中，我们将采用非 GUI 的方式来介绍 cuda-gdb 这一工具，其实就是

Nsight Eclipse Edition 调用的调试器。cuda-gdb 是 gdb（GNU 调试器）的扩展版本，其拥有 gdb 的所有功能，并且能够调试设备上的 CUDA 代码。

> **OS X 上的 CUDA-GDB**
>
> 随着 CUDA 7.0 的发布，CUDA-GDB 已经从 OS X 中移除。对于 Mac 用户，可通过远程的方式在 Linux 机器上进行调试。

与性能分析不同，调试时程序必须编译且包含调试信息。因此，利用编译命令进行编译时，需向命令传递 -g 与 -G 这两个参数。另外，目前为止 Linux 上单 GPU 调试仍存在一点小问题。在 Linux 系统中，如果 NVIDIA GPU 的 CUDA 计算能力在 3.5 及以上，则可以利用一个处于测试中的新特性进行调试，无需任何其他步骤。若 GPU 的 CUDA 计算能力在 3.5 以下，也能进行单 GPU 调试，只不过在调试时需将桌面管理器关闭，利用控制台进行调试。

接下来，我们将利用 dist_v2_cuda 示例，移除其核函数中的 printf() 函数，然后编译（注意，编译时必须加入参数 -g 与 -G）。程序编译成功之后，使用 cuda-gdb main.exe 命令将可执行程序加载到 cuda-gdb 中。若调试时桌面管理器仍在运行（显卡 CUDA 计算能力至少为 3.5），则输入 set cuda software_preemption on 以确保单 GPU 调试时 GUI 的特性一直为开启状态。图 D.15 显示了 Linux 上调试的具体步骤。

```
cuda-gdb main.exe
→  dist_v2_cuda  cuda-gdb main.exe
NVIDIA (R) CUDA Debugger
7.5 release
Portions Copyright (C) 2007-2015 NVIDIA Corporation
GNU gdb (GDB) 7.6.2
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-unknown-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/duane/Desktop/dist_v2/dist_v2_cuda/main.exe...done.
(cuda-gdb) set cuda software_preemption on
(cuda-gdb) show cuda software_preemption
Software preemption debugging is on.
(cuda-gdb)
```

图 D.15　调试之前 cuda-gdb 的 software_preemption 选项必须开启

输入 break kernel.cu:14 命令，在文件 kernel.cu 中的 distanceKernel() 函数的第一行加入一个断点，另外，输入 break main.cpp:26 命令，在 main.cpp 文件中的最后一行加入另一个断点，接着使用 run 命令运行程序，如图 D.16 所示。

```
cuda-gdb main.exe
(cuda-gdb) break kernel.cu:14
Breakpoint 1 at 0x4028ab: file kernel.cu, line 14.
(cuda-gdb) break main.cpp:26
Breakpoint 2 at 0x4025fa: file main.cpp, line 26.
(cuda-gdb) run
Starting program: /home/duane/Desktop/dist_v2/dist_v2_cuda/main.exe
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[New Thread 0x7ffff6131700 (LWP 3545)]
[Switching focus to CUDA kernel 0, grid 1, block (0,0,0), thread (0,0,0), device 0, sm 0, wa
rp 0, lane 0]

Breakpoint 1, distanceKernel<<<(2,1,1),(32,1,1)>>> (d_out=0x508a80200, d_in=0x508a80000,
    ref=0.5) at kernel.cu:14
14        const int i = blockIdx.x*blockDim.x + threadIdx.x;
(cuda-gdb) next
[Switching focus to CUDA kernel 0, grid 1, block (0,0,0), thread (0,0,0), device 0, sm 1, wa
rp 1, lane 0]
15        const float x = d_in[i];
(cuda-gdb) print i
$1 = 0
(cuda-gdb) cuda kernel block thread
kernel 0, block (0,0,0), thread (0,0,0)
(cuda-gdb) cuda block 1,0,0 thread 5,0,0
[Switching focus to CUDA kernel 0, grid 1, block (1,0,0), thread (5,0,0), device 0, sm 0, wa
rp 1, lane 5]
14        const int i = blockIdx.x*blockDim.x + threadIdx.x;
(cuda-gdb) next
[Switching focus to CUDA kernel 0, grid 1, block (1,0,0), thread (5,0,0), device 0, sm 1, wa
rp 2, lane 5]
15        const float x = d_in[i];
(cuda-gdb) print i
$2 = 37
(cuda-gdb)
```

图 D.16　设置断点并调试程序

程序运行到 kernel.cu 文件的第 14 行停止，此时当前的焦点线程与核函数启动参数一并输出到屏幕。此刻，索引变量还未设置。当执行到当前行时，输入 next 命令，然后输入 print i，返回结果 0，当前处于第一个线程。使用 cuda kernel block thread 命令可以查看当前焦点线程的相关信息，或者使用 print threadIdx 与 print blockIdx 命令，通过查看 CUDA 变量也能获得这些信息。使用 cuda thread 5,0,0 命令查看当前线程块中的另一个线程。现在，如果再输入 print i 命令，返回的结果则是 5，表明当前在第 6 个线程。使用 cuda block 1,0,0 thread 30,0,0 命令查看另一个线程块中的线程。现在，由于查看的线程块发生了变化，第 14 行的断点重新触发，输入 next 命令，然后输入 print i 命令查看索引值，结果显示为 62，这是因为当前焦点在 1 号线程块的 30 号线程（32×1+30）。图 D.17 为焦点在不同线程时的状态。

现在，输入 continue 命令让程序运行到核函数外的另一个断点。此时，程序运行到 main() 函数中，输入 info locals 命令查看当前上下文中的局部变量。程序包含两个主机端数组的指针，in 与 out，输入 print out[0]@64 命令查看 out 数组中的具体内容。可以看到数组的开头为 0.5，结尾也是 0.5，中间的数无异常，说明计算基本正确。至此，程序中我们感兴趣的部分已经结束，输入 continue 命令让程序继续执行直至结束并退出。图 D.18 显示了最终的几步。

```
cuda-gdb main.exe
(cuda-gdb) cuda thread 5,0,0
[Switching focus to CUDA kernel 0, grid 1, block (0,0,0), thread (5,0,0), device 0, sm 0, wa
rp 1, lane 5]
15        const float x = d_in[i];
(cuda-gdb) print i
$4 = 5
(cuda-gdb)
(cuda-gdb) cuda block 1,0,0 thread 30,0,0
[Switching focus to CUDA kernel 0, grid 1, block (1,0,0), thread (30,0,0), device 0, sm 1, w
arp 1, lane 30]
14        const int i = blockIdx.x*blockDim.x + threadIdx.x;
(cuda-gdb) next
[Switching focus to CUDA kernel 0, grid 1, block (1,0,0), thread (30,0,0), device 0, sm 1, w
arp 2, lane 30]
15        const float x = d_in[i];
(cuda-gdb) print i
$5 = 62
(cuda-gdb)
```

图 D.17　聚焦不同线程并单步调试

```
cuda-gdb main.exe
(cuda-gdb) continue
Continuing.

Breakpoint 2, main () at main.cpp:26
26        free(in);
(cuda-gdb) info locals
ref = 0.5
in = 0x664220
out = 0x664330
(cuda-gdb) print out[0]@64
$1 = {0.5, 0.484126985, 0.46825397, 0.452380955, 0.43650794, 0.420634925, 0.40476191,
  0.388888896, 0.373015881, 0.357142866, 0.341269851, 0.325396836, 0.309523821,
  0.293650806, 0.277777791, 0.261904776, 0.246031731, 0.230158716, 0.214285702,
  0.198412687, 0.182539672, 0.166666657, 0.150793642, 0.134920627, 0.119047612,
  0.103174597, 0.0873015821, 0.0714285672, 0.0555555522, 0.0396825373, 0.0238095224,
  0.00793650746, 0.00793653727, 0.0238095522, 0.0396825671, 0.055555582, 0.071428597,
  0.0873016119, 0.103174627, 0.119047642, 0.134920657, 0.150793672, 0.166666687,
  0.182539701, 0.198412716, 0.214285731, 0.230158746, 0.246031761, 0.261904776,
  0.277777791, 0.293650806, 0.309523821, 0.325396836, 0.341269851, 0.357142866,
  0.373015881, 0.388888896, 0.40476191, 0.420634925, 0.43650794, 0.452380955, 0.46825397,
  0.484126985, 0.5}
(cuda-gdb) continue
Continuing.
[Thread 0x7ffff7fd3740 (LWP 3929) exited]
[Inferior 1 (process 3929) exited normally]
(cuda-gdb)
```

图 D.18　继续执行到 main.cpp 文件中的断点，检查最终结果。注意对指针使用 [] 与 @ 操作符可决定打印动态数组值的方式

注意，程序运行结束之后断点并没有自动消除。使用 info breakpoints 命令可以查看当前设置的所有断点，使用 delete breakpoint n 命令可以删除不再需要的断点，其中 n 为要删除的断点的编号，或者使用 delete breakpoints 命令，一次性删除所有断点。

除了以上介绍的 gdb/cuda-gdb 的命令与功能，更多相关信息可参考文献 [5, 6]。

D.5 CUDA-MEMCHECK

最后一个需要介绍的 CUDA 工具为 CUDA-MEMCHECK。CUDA-MEMCHECK 是一个轻量级的运行时错误检测工具，在所有平台上都能使用。其主要用来检测内存泄漏、内存访问错误以及硬件错误。打开一个命令行，切换到 CUDA-MEMCHECK 程序所在文件夹，输入 cuda-memcheck 以及可执行程序的文件名（如在 Linux 下输入 cuda-memcheck main.exe，在 Visual Studio 中输入 cuda-memcheck dist_v2_cuda.exe）。程序将执行并返回存在的内存访问错误报告。若未检测到错误，返回的结果则为 ERROR SUMMARY: 0 errors。更多关于 CUDA-MEMCHECK 的信息请查看在线文档[7]。

D.6 使用 Visual Studio 属性页

Visual Studio 提供了一个相关属性页，可供用户在编译和生成程序时指定各种选项。本节我们不会对属性页的所有细节进行介绍，只会针对与 CUDA 开发相关的一些设置进行介绍。如那些使用了链接库的程序（例如第 4 章中的示例程序 flashlight，该程序使用了 OpenGL 库），需告诉系统如何找到相关头文件（在编译阶段引用）与目标文件（将链接到可执行文件中）。设置它们的步骤基本相同：

1. 在 Solution Explorer 面板中右键单击项目名称，选择弹出菜单中最下方的 Properties 选项（或使用快捷键 Alt+F7）。
2. 依次选择 Configuration Properties ⇒ C/C++ ⇒ Additional Include Directories，将出现如图 D.19 所示的属性页面。

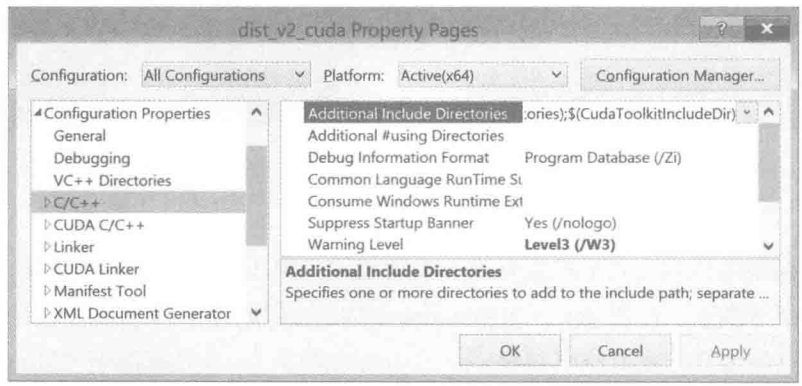


图 D.19 Additional Include Directories 所在的属性页面

3. 单击右侧如“v”形状的下拉图标。

4. 单击 <Edit...>，弹出 Additional Include Directories 对话框，如图 D.20 所示。

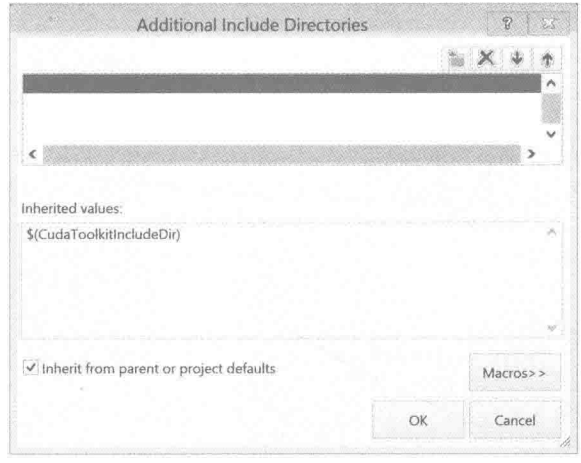


图 D.20　可输入新路径目录的 Additional Include Directories 对话框

5. 双击蓝色条框，输入头文件所在目录的路径，单击 OK 按钮。

6. 依次选择 Configuration Properties ⇒ Linker ⇒ Additional Library Directories，将出现如图 D.21 所示的属性页面。

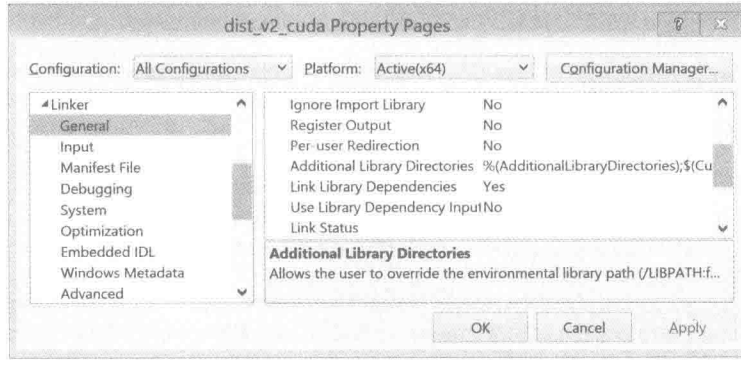


图 D.21　Additional Library Directories 所在的属性页面

7. 单击右侧如“v”形状的下拉图标。

8. 单击 <Edit...>，弹出 Additional Library Directories 对话框，如图 D.22 所示。

9. 双击蓝色条框，输入库文件所在目录的路径，单击 OK 按钮。

10. 依次选择 Linker ⇒ Input ⇒ Additional Dependencies ⇒ Edit，将链接库添加到列表

中，链接库文件如图 8.3 所示。

11. 最后在属性页中应用修改并单击 OK 按钮。

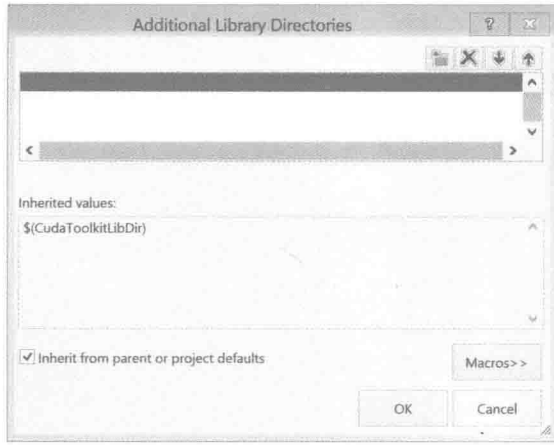


图 D.22　在 Additional Library Directories 页面中输入新的目录路径

参考文献

[1] NVIDIA Corporation, "NVIDIA Nsight Visual Studio Edition 4.7 User Guide," NVIDIA Gameworks, 2015, http://docs.nvidia.com/gameworks/index.html#developertools/desktop/nsight_visual_studio_edition_user_guide.htm.

[2] NVIDIA Corporation, "NVIDIA CUDA Runtime API," NVIDIA Developer Zone, Cuda Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-runtime-api/index.html.

[3] Mark Harris, "How to Query Device Properties and Handle Errors in CUDA C/C++," *Parallel Forall* (blog), November 2012, http://devblogs.nvidia.com/parallelforall/how-query-device-properties-and-handle-errors-cuda-cc/.

[4] NVIDIA Corporation, "NVIDIA Nsight Visual Studio Edition," NVIDIA Gameworks, 2015, https://docs.nvidia.com/gameworks/index.html#developertools/desktop/nvidia_nsight.htm.

[5] Norman S. Matloff and Peter Jay Salzman, *The Art of Debugging with GDB, DDD, and Eclipse* (San Francisco, CA: No Starch Press, 2008).

[6] NVIDIA Corporation, "CUDA-GDB," NVIDIA Developer Zone, Cuda Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-gdb/index.html.

[7] NVIDIA Corporation, "CUDA-MEMCHECK," NVIDIA Developer Zone, Cuda Toolkit Documentation, 2015, http://docs.nvidia.com/cuda/cuda-memcheck/index.html.

CUDA 高性能并行计算

CUDA for Engineers
An Introduction to High-Performance Parallel Computing

本书可帮助读者亲身实践个人高性能计算。在几年之前需要超级计算机才能完成的任务，现在可以在游戏级计算机上轻松完成。

本书以清晰简明的方式介绍了CUDA C编程的核心知识，始于运行CUDA样例程序，快速引导读者构建自己的代码。自始至终，你可以通过亲手创建、运行和修改书中的示例程序进行体验式教学。书中配备的实践项目可用来帮助加深和巩固读者对CUDA的理解。

本书适合科研工作者、工程师和学生阅读。本书面向的读者不需要专门的GPU或并行计算背景知识，只需拥有少量编程经验即可。本书附录提供了简明的C语言编程指南，以备读者查阅。

本书主要内容包括：

- 设置你的计算机，准备好运行CUDA程序的环境
- 理解CUDA的并行模型和扩展CUDA C语言
- 在CPU和GPU间传输数据
- 创建二维计算网格
- 提供支持实时用户交互的CUDA与OpenGL的互操作技术
- 差分方程的基本模拟执行
- 利用模板组织线程间的相关计算
- 探索CUDA共享内存能力，增强性能
- 三维数据交互：切片法、体绘制法与光线投射法
- CUDA函数库实践
- CUDA资源和代码库检索技巧

本书撷取的真实应用案例包括：

- 函数的二维和三维可视化
- 查看和处理图像或图像序列
- 求解线性代数方程系统
- 求解差分方程
- 计算内积和质心
- 蒙特卡罗计算

Pearson
www.pearson.com

投稿热线：(010) 88379604
客服热线：(010) 88379426 88361066
购书热线：(010) 68326294 88379649 68995259

华章网站：www.hzbook.com
网上购书：www.china-pub.com
数字阅读：www.hzmedia.com.cn

上架指导：计算机/程序设计
ISBN 978-7-111-55749-4
9 787111 557494 >
定价：79.00元